OCEANBASE

OBCA V4.0 Certification Training Course

Contents

- **Product Overview**
- Cluster Architecture
- Database Engines
- Installation and Deployment
- Application Development
- Operations and Maintenance Management

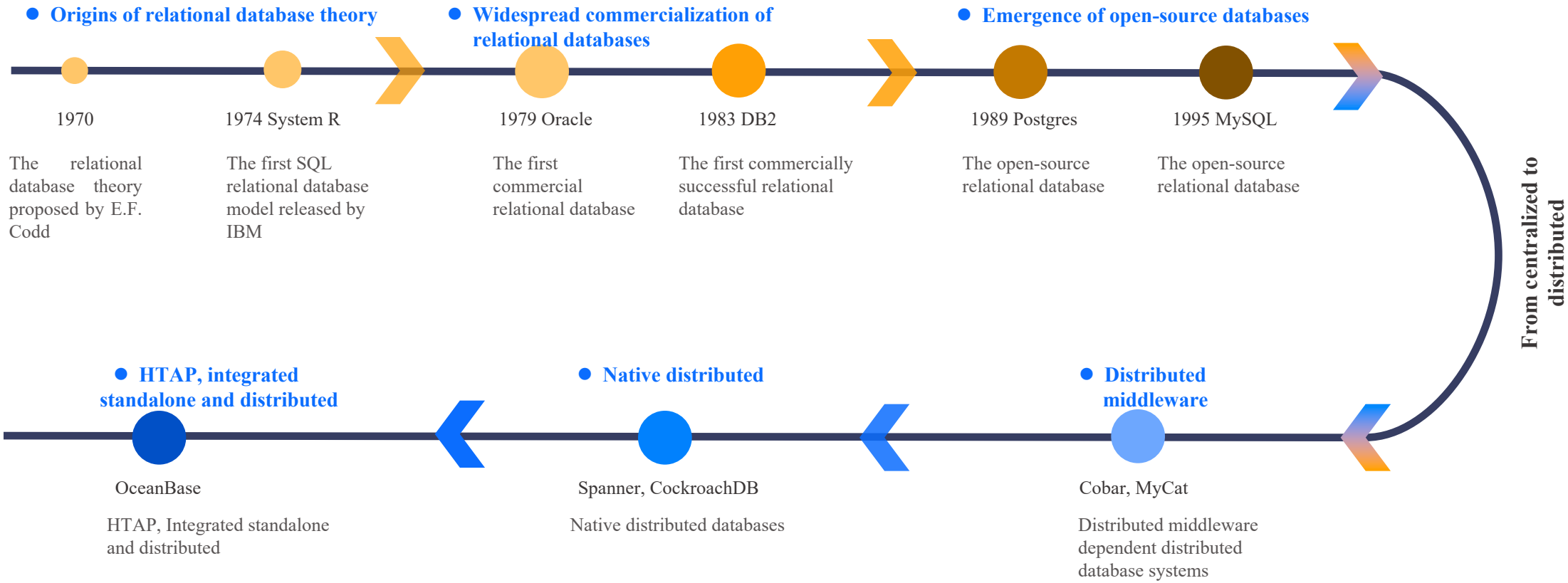
Product Overview

01

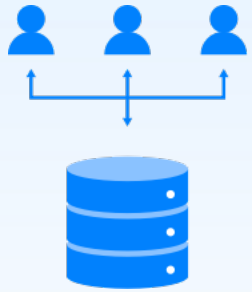
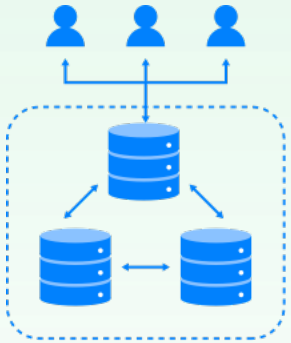
- **Database Development and Evolution**
- About OceanBase Database
- OceanBase Product and Tool System
- OceanBase Certification Training System

Database Development and Evolution

Databases have evolved since the nascent days of computer science, advancing in step with the development of computer hardware, systems software, and the expanding universe of computer applications. Their journey has seen a shift from rudimentary to sophisticated systems and an architectural transition from centralized to distributed models. With the advent of cloud computing, cloud-native databases have emerged as a novel product category, which harness the full spectrum of cloud computing's elastic scalability and automated management, providing users with agile and potent data services.

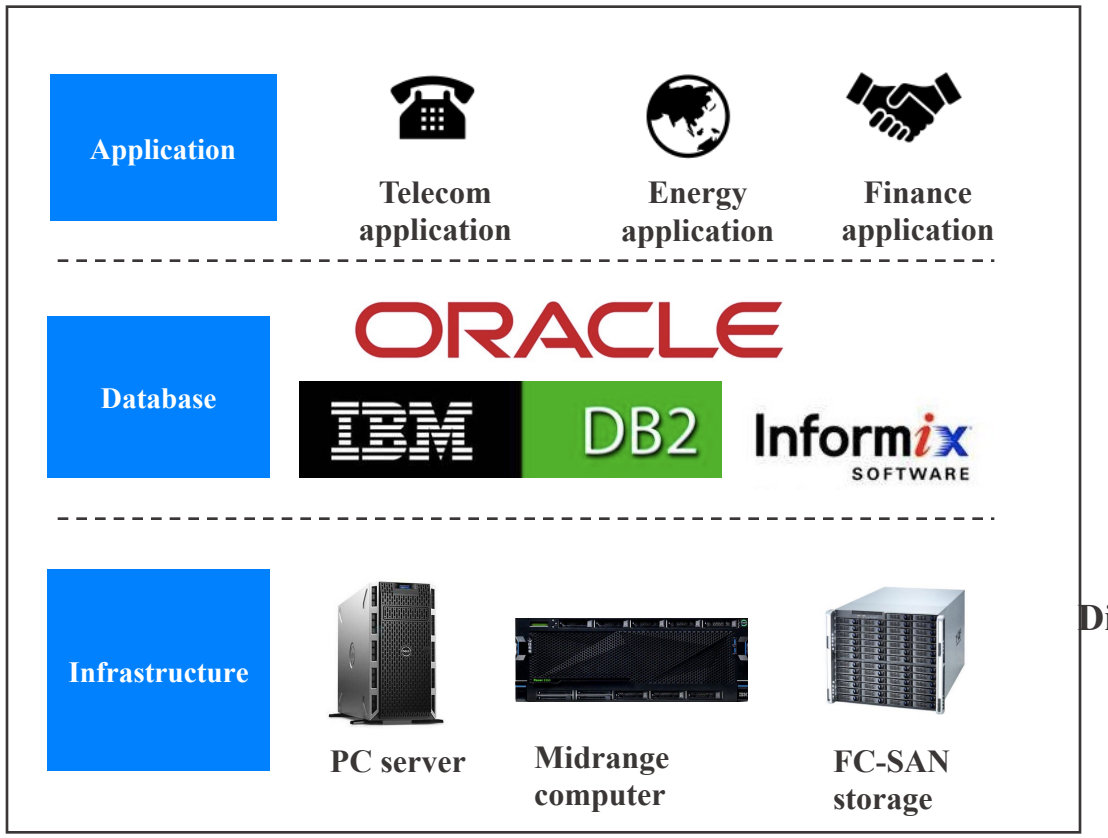


Evolution from Centralized to Distributed

Centralized	Middleware-based distributed	Native distributed	Integrated standalone and distributed + HTAP
1960s	1990s - 2000s	2010s	2020s
			Integrated transaction and analysis
			Integrated standalone and distributed
Typical products	Typical products	Typical products	Typical products
Oracle, DB2	Cobar, MyCAT, DRDS	Spanner, OceanBase	OceanBase

Challenges Faced by Traditional Centralized Databases

Traditional database architecture



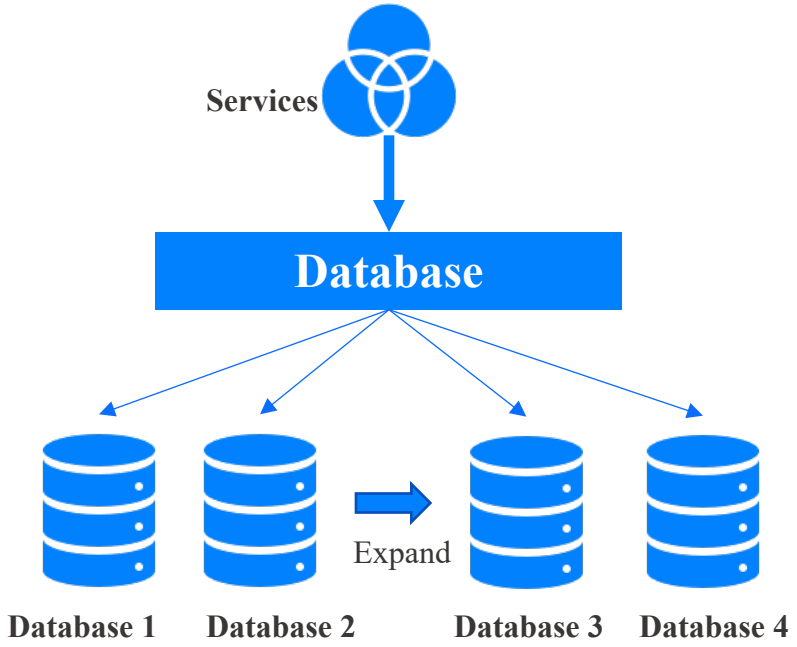
Advantages

- **Mature and stable:** 40-year development, applications in various industries, mature and stable products & technologies
- **Industry adoption:** Adapted to various demands of different industries;
- **Perfect eco-system:** Numerous ISV application developers and technology developers, perfect ecology of technology, industry and talent;

Disadvantages

- **High cost:** High software price, depending on high-end hardware, high CAPEX and OPEX cost;
- **No horizontal scaling:** Capacity scaling can be done only via increase of equipment performance (more CPU/memory/disk or upgrade from PC server to midrange computer), single-point limit exists

DB/Table Sharding Scheme by Middleware - Drawbacks



- The linear scaling can be achieved by generic database model;
- Database server is standalone, no connection in between, unaware of other database existence. Middleware is required to complete cross-database transactions;
- The database middleware connects each database, fulfilling DB shard or table shard.

Advantages

Linear scaling: DB or tableshardingallow quick implementation of horizontal database scaling;

Low technology cost: Few or no transformation of core database engine required;

Disadvantages

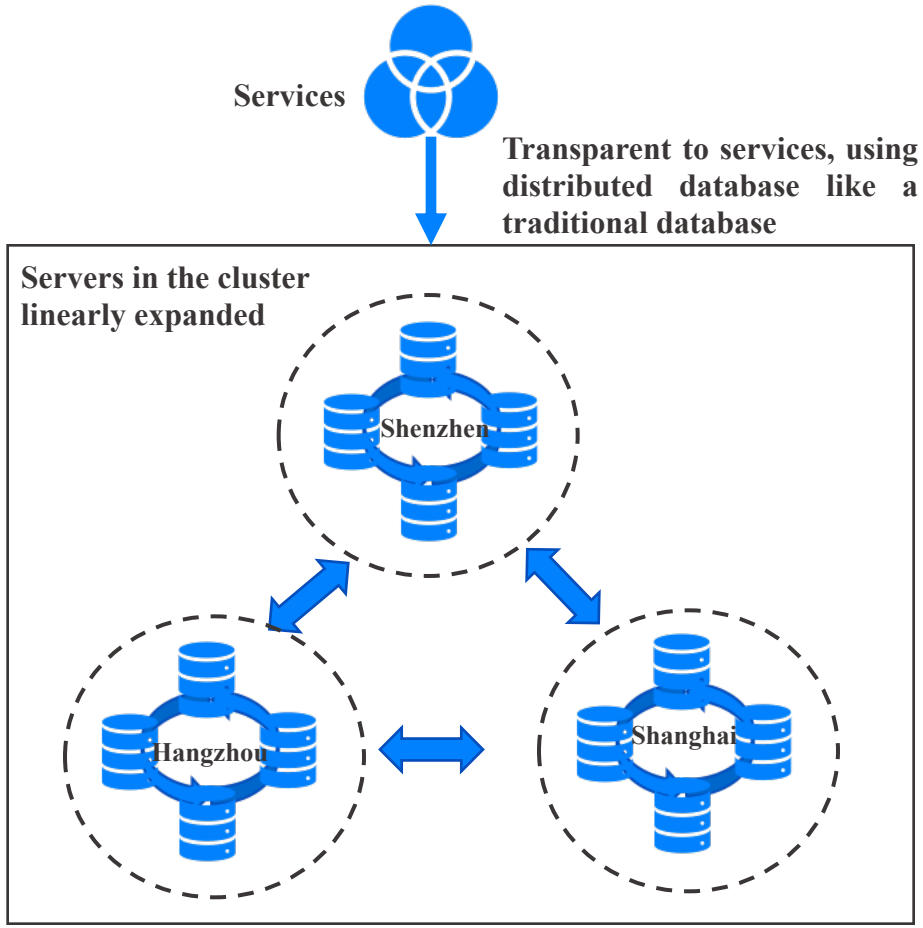
Cross-database distributed transactions: The database core engine has no distributed capability and can only complete distributed processing through middleware, but it is difficult for middleware to achieve RPO=0, so the ACID capability of distributed transactions cannot be guaranteed 100% when exceptions and failures occur;

Global consistency: Because timestamps are inconsistent across multiple database servers, it is difficult to ensure global consistency of data version numbers across multiple databases;

Load balancing: During capacity scaling or reduction, the underlying database engine cannot adjust data distribution rules online. Therefore, services need to be suspended and data needs to be imported/exported again, posing serious challenges to services and O&M;

Cross-database complex SQL: Cross-database complex SQL operations (such as multiple tables to do shard key-independent join query) can only be completed in middleware, which does not have distributed parallel computing capability, and will eventually limit the use of SQL by applications, resulting in business invasion;

Native Distributed Relational Database Architecture



Advantages

High data reliability + High service availability: industrial level implementation of multi-replica consistency protocol Paxos ensures zero data loss (RPO=0) and fast service recovery (RTO<30 seconds) when individual nodes fail.

Linear capacity scaling: Capacity scaling with the increase of business volume (such as during online promotion), capacity reduction with the decrease of business volume (such as after promotion)

Low cost: High availability based on commodity X86 servers without high-end midrange computers and storage.

Global consistency: Supports distributed transactions to ensure global consistency and supports distributed complex queries.

Flexible deployment mode: Supports multiple deployment modes, such as three-center, five-center, and active/standby mode;

Transparent to services: Service systems can use a distributed database just like a standalone database, and the cost of business migration and transformation is low;

Comparison Between Native Distributed Database and Traditional Database

	Traditional centralized database	Distributed database represented by OceanBase
Product architecture	Classic "single node centralized" architecture adopts "share-everything" architecture. Built on high-end hardware, such as IBM high-end servers and EMC high-end storage devices.	Native "distributed" database, using the industry's most stringent Paxos distributed consistency protocol. Design based on ordinary PC hardware, no need for high-end hardware.
Data reliability and high availability of services	Use high-end hardware to ensure data reliability. The “master-slave replication” is adopted, in the event of a primary node failure, there may be data loss (RPO>0). The service cannot be automatically recovered. The service recovery time (RTO) is usually measured in hours.	Based on common PC hardware, Paxos distributed consistency protocol is used to ensure data reliability. In the case of a primary node failure, Paxos can ensure that data is not damaged (RPO=0), and automatically vote and restore services. The service recovery time (RTO) is less than 8 seconds.
Scalability	Data storage can only scale vertically within a single server, and eventually reaches the maximum capacity of the single architecture. Computing nodes usually cannot be extended. In a small number of scenarios (such as RAC and pureScale), computing nodes can be expanded. However, multiple computing nodes still need to access the shared storage, and the number of computing nodes that can be expanded is limited.	Both data nodes and computing nodes can be scaled horizontally under the MPP architecture. There is no limit on the number of data nodes and computing nodes. You can expand the number of data nodes and computing nodes to any number if the network bandwidth is sufficient.
Applications	Core systems focused on corporate customers (finance, telecommunications, government and enterprise, etc.). Unable to cope with Internet business scenarios, few application cases.	Core component of Alipay, core of MYBank, many businesses of Alibaba, and many external commercial banks. Gradually penetrating traditional business.
Use cost	More expensive. There are high-end infrastructure costs, high software licensing fees, and product and service costs.	Relatively low. PC based hardware design reduces hardware costs, software licensing costs and service costs also have advantages.

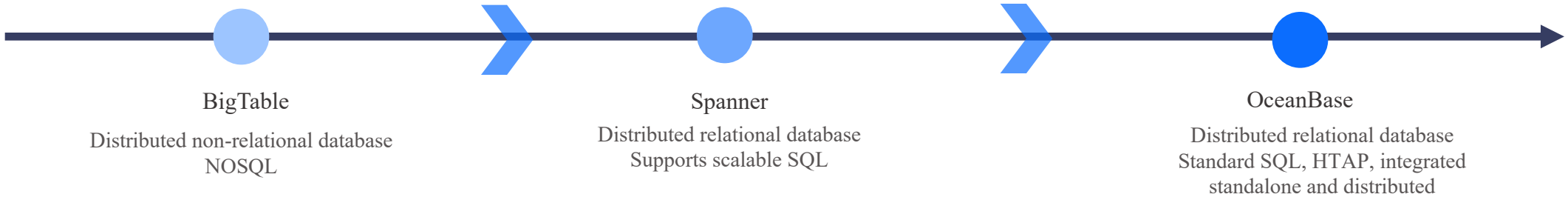
Product Overview

01

- Database Development and Evolution
- **About OceanBase Database**
- OceanBase Product and Tool System
- OceanBase Certification Training System

OceanBase: Gen3 Native Distributed Database

OceanBase is a **fully self-developed native distributed database** designed to offer a stable and reliable data foundation for enterprise core systems. It innovatively develops its own distributed technologies, achieving financial-grade high availability on standard PC servers, thereby addressing issues related to scalability, ease of use, and the high costs of databases. OceanBase is the third generation of native distributed databases, offering support for the functionalities and SQL syntax of enterprise-level databases. It has pioneered an integrated standalone and distributed architecture that supports HTAP hybrid loads within a single system, making it a versatile database.



Gen1 distributed database

- Non-relational database, NOSQL
- Supports only single-row transactions, compromising consistency
- Supports high availability and scalability
- Compromises enterprise-level features such as security

Gen2 distributed database

- Supports most SQL functions, but not compatible with SQL standards
- Supports full-featured transactions to ensure high consistency
- Focuses on basic SQL functionality and scalability
- Compromises enterprise-level features such as performance, cost and security

Gen3 distributed database

- Transparent and scalable enterprise-level databases
- Supports standard SQL, and compatible with enterprise-level database features
- HTAP hybrid load
- Integrated standalone and distributed

OceanBase Milestones and Architecture Evolution

1.0 Era

Distributed engine

Distributed KV storage

2010

Project initiation

OceanBase acquired its inaugural customer

2013

Expanded use

Serving business systems of dozens of e-commerce platforms within Alibaba Group

2014

Core transactions go-live

Supported Alipay's core transaction system

Handled 10% of transaction traffic for "Double 11"

Achieved RPO=0 and RTO<30s for the first time

2.0 Era

Native distributed databases

SQL Engine, multi-replica high availability

2016

Full service coverage

Launched Alipay's core accounting and payment system

Supported 120,000 payment peaks per second and 175,000 transaction peaks per second

2017

Multiple financial clients

Completed the last Oracle core replacement for Ant Group's core system

Bank of Nanjing became the first operational user

Set a record of 42 million database processing peaks per second in "Double 11"

2019

World record breaking

Oracle was compatible with public cloud services

Achieved a TPC-C benchmark of 60.88 million tpmC, topping the leaderboard

Recorded 61 million database processing peaks per second in "Double 11".

3.0 Era

HTAP hybrid engine

Compatibility, transparent scaling, multi-active high availability

2020

Independent commercialization

Achieved a TPC-C score of 7.07 billion tpmC, breaking its own world record and surpassing Oracle by 23 times

Launched the core system for top-tier clients.

2021

Scaling up

Released HTAP engine

Achieved a TPC-H result of 1,526 tpmC, ranking first and becoming the only distributed database in the world to top both the TPC-C and TPC-H benchmarks

Released the community edition, opening up 3 million lines of core code

Conducted pilot with overseas clients

4.0 Era

Integrated architecture with multi-cloud deployment capabilities

Batch processing, enterprise-level features, HTAP

2022

Public cloud launch and international scaling

Integrated standalone and distributed architecture

Launched public cloud services, with the North American data center operational.

launched AWS Marketplace

2023

Integrated standalone and distributed products

Number of clients exceeded 1,000

Held No.1 market share in financial industry

OceanBase Core Technical Capabilities

Fully self-developed	Over a decade of self-development, with code-level control and extensive experience in large-scale financial core scenarios, ensuring 10 years of proven reliability
Native distributed	Full-volume data validation ensures robust data consistency, no data loss, high availability, and smooth scaling
Integrated standalone and distributed	The self-developed integrated architecture surpasses performance and availability benchmarks, enabling boundless application scaling and perpetual service availability
HTAP hybrid transactional and real-time analytical processing	Data can serve both transactional and real-time analytical purposes, broadening its potential with the capabilities of HTAP.
Oracle/MySQL smooth migration	Smooth migration of Oracle/MySQL: quick and cost-effective migration of applications and data
Advanced compression technology	The LSM-Tree-based high compression engine harmonizes the trade-offs between "performance" and "compression," significantly reducing storage costs by 70% to 90%.
Native multi-tenant	Native multi-tenant, resource isolation, on-demand utilization, and flexible management

OceanBase, a General-purpose Database Serving Thousands of Industries

Large-sized clients

Hyper-scale enterprises, leading internet companies

OceanBase, with a decade of financial scenario experience within Ant Group, has become the first Chinese database in the financial and carrier sectors. Its distributed architecture, high transaction consistency, transparent scalability, and proven capability to support the core systems of hyper-scale enterprises have established its leading position.

Mid-sized clients

SMEs, growing internet companies

OceanBase has been progressively improved in compatibility, cost-effectiveness, high reliability, and miniaturization, which satisfies the core needs of medium-sized clients regarding TCO and data sovereignty, leading to an increasing number of clients opting to grow alongside OceanBase.

Small and micro clients

Startups, e-commerce individual proprietorship, developers

The OceanBase public cloud database offers low-cost, architecture advanced, high performance, and maintenance-free database services and products to a broader range of small and medium-sized clients. Additionally, the open-source community entices more database developers to engage with OceanBase.

Product Overview

01

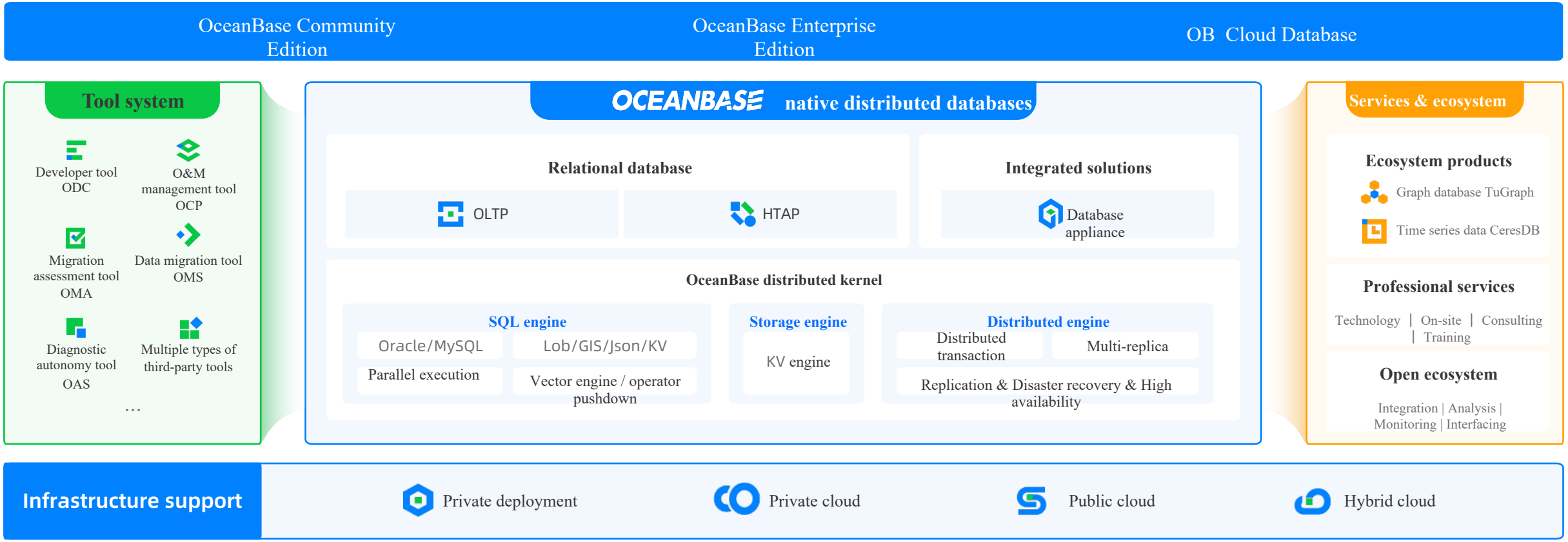
- Database Development and Evolution
- About OceanBase Database
- **OceanBase Product and Tool System**
- OceanBase Certification Training System

OceanBase Product Family

Full-scenario, full-form data management solutions

OceanBase provides clients with a full-scenario, full-form enterprise-level database solution. The Product suite encompasses the Enterprise Edition, OB Cloud, and Community Edition of distributed database solutions. It caters to diverse deployment and delivery forms, including standalone deployment, cloud-based services, and database appliances.

OceanBase comes with a comprehensive suite of database tools that support the entire data lifecycle management for clients, including data development, evaluation, migration, O&M, and diagnostics. Additionally, OceanBase offers specialized consulting and deployment services to address the varying business requirements of enterprises at different developmental stages and for diverse database product application scenarios.



OceanBase Tool System

OceanBase tool system offers a full suite of productized support for OceanBase databases from pre-migration, replacement, to post-migration, facilitating a steady growth of your business.

01	02	03	04	05	06	07	08
Renovation assessment	**Real-time migration**	**Development management**	**Production O&M**	**Disaster recovery replication**	**Security control**	**Lifecycle management**	**Autonomy service**
Automatic collection	Object migration	Connection management	Deployment upgrade	Primary/Standby management	Privilege management	Data collection	End-to-end monitoring
Automatic conversion	Full migration	Object management	Resource management	midrange computer	Operational auditing	Data archiving	SQL diagnostics
Replay and stress testing	Incremental migration	Development debugging	System administration	Binlog subscription	Security collaboration	Partition plan	Capacity management
Database profiling	Data validation	Import and export	Backup recovery	Data synchronization	Change control	SQL plan	Automatic recovery
Compatibility assessment	Incremental writeback	Data visualization	Monitoring and alerting	Data exchange	Data masking	Data cleaning	Automatic optimization
Distributed transformation	Data correction	Analog data	Automatic inspection	Data integration	Security governance	Scaling	Artificial intelligence for IT operations (AIOps)

Migration assessment tool OMA	**Data migration tool OMS**	**Developer tool ODC**	**O&M management tool OCP**	**OAS** Diagnostic autonomy tool	**Binlog** Binlog subscription	**OBloader/ OBdumper** Data import/export tool	**OAT** Deployment tool	**…**

Data Migration and Assessment Tools

Migration assessment tool OMA

OceanBase offers database migration assessment tools that deliver precise compatibility assessments, efficient performance assessments, and recommendations for transforming application logic during data migration.

- It supports compatibility assessment for databases such as Oracle and DB2 LUW with OceanBase, offering profile analysis and automatic conversion solutions.
- Supports application SQL replay to help clients predict potential performance risks after migration and to offer optimization solutions.
- Supports compatibility assessment for C and Java business code and drivers.

Online migration tool OMS

OceanBase provides a one-stop online data transfer and synchronization product, which is a data transfer service integrating data migration, real-time data synchronization and incremental data subscription.

- Supports migration between OceanBase and various databases, including structural, full, incremental data migrations, as well as data reverse flowback.
- Supports real-time data synchronization between OceanBase and self-built Kafka and RocketMQ.
- Supports real-time subscription to OceanBase MySQL tenant logs through services compatible with MySQL Binlog.
- Provides data consistency check and protection.

Offline migration tool DataX

DataX is the open-source version of Aliyun DataWorks data integration, serving as an extensively utilized tool/platform for offline data synchronization within Alibaba Group.

- Supports a variety of data sources including popular relational databases, NoSQL databases, file systems, and message queues.
- Supports full-scale migration of data from various sources and data files to the OceanBase database.
- Does not support incremental synchronization and data validation.

Data import/export tool OBDumper/OBLoader

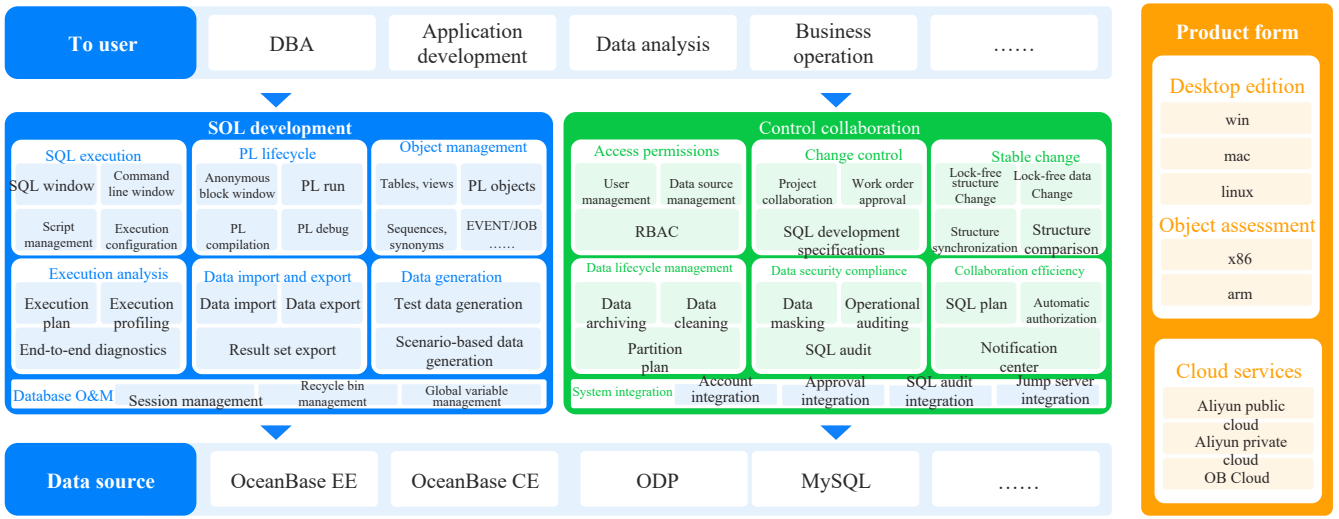
OBDumper and OBLoader are specialized data import/export tools for OceanBase database.

- OBDumper is mainly used to export objects and data from OceanBase database. It supports the export of data in multiple formats, partition export, global filter conditions export, and other functionalities, making it highly suitable for database migration or backup scenarios.
- OBLoader is mainly used to import database object definitions and table data into OceanBase. It supports flexible command line options and various import features to ensure the stability and reliability of data import.

OceanBase Developer Center (ODC)

The OceanBase developer tool (OceanBase Developer Center, ODC), as an enterprise-level database development platform tailored for OceanBase database, is designed to help enterprises use the database safely and efficiently. With ODC, you can create and manage over 10 types of database objects, such as tables and views in the database. Based on WebSQL, ODC provides a SQL window and an anonymous block window as workspaces for database developers to develop and diagnose SQL and PL/SQL. You can also allocate resources and grant access permissions to specific roles, making development collaboration between different roles in your organization easy and controllable.

Product architecture



Product advantages

■ Efficient SQL development at anytime and anywhere

Based on a modern web architecture, enables development of databases at anytime and anywhere, as long as you have a browser. Boasts comprehensive and easy-to-use development features that cover all common desktop development tools, including PL debugging.

■ Guarding every change during SQL development

Provides project-based collaboration and change approval process, and customizes the control rules to align with actual business scenarios through SQL checking rules, SQL window specification, risk level identification and other modules.

■ Automatic data lifecycle management

Supports data cleaning or archiving based on the retention time, facilitating the quick establishment of a hot and cold data separation system. Also provides automatic maintenance of partition table and SQL scheduled execution capability to ensure smooth execution of periodic tasks.

■ Full-scenario sensitive data protection

Provides data masking capability, covering all development scenarios such as table data view, SQL query, result set export, data export, etc.

■ Smooth integration experience

The currently available enterprise control integration capabilities include SSO integration, approval integration, SQL audit integration, jump server integration, and audit integration.

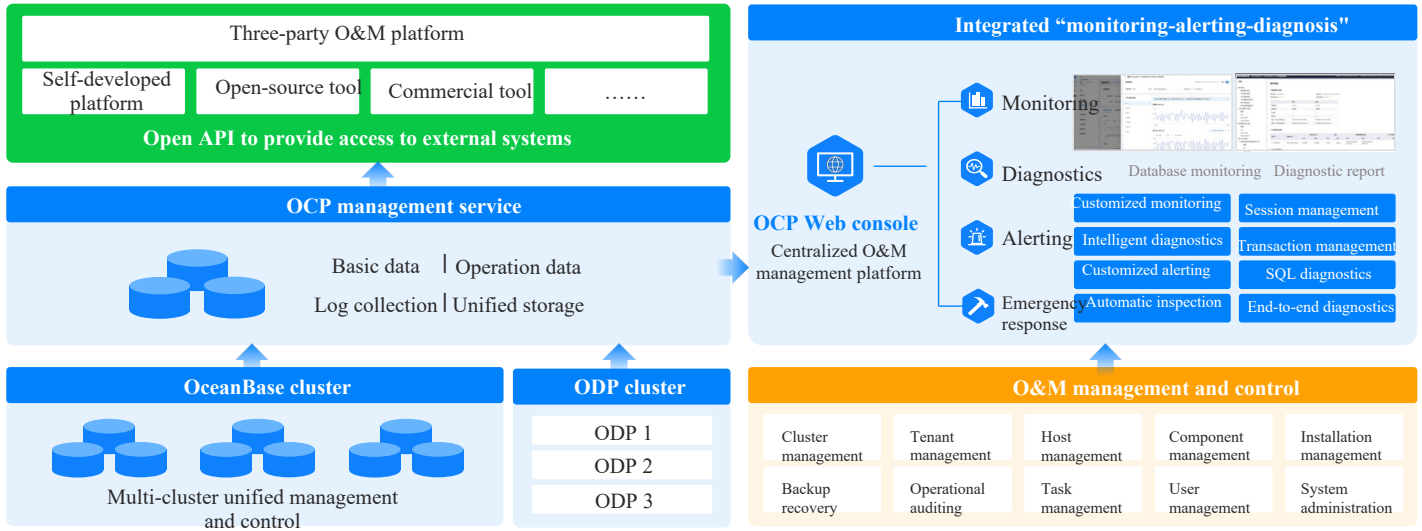
■ Full-scenario collaboration for database development and management

Provides two working modes: personal space and team space. In the personal space, you can freely manage data sources and use various windows and tools provided by the platform to develop databases without being constrained by control rules. In the team space, ODC provides users with one-stop service of database development and control through a series of functions, such as project collaboration, stable change, data security, and hot/cold data separation.

OceanBase Cloud Platform (OCP)

OceanBase O&M management tool (OceanBase Cloud Platform, OCP) is an enterprise-level management platform tailor-made for OceanBase database clusters, which is compatible with all major versions of OceanBase. OCP provides graphical management of OceanBase cluster, including full lifecycle management of database components and related resources, monitoring and alerting, performance diagnosis, disaster recovery, backup recovery, etc. It is designed to assist customers to manage OceanBase database more efficiently and effectively, and to reduce the IT O&M cost of the enterprise and the learning cost of the users.

Product architecture



Product advantages

■ Simple and easy to use

OCP's web-based visualization facilitates multi-cluster and 1000+ OBServer O&M management capabilities. It standardizes and automates the complex and highly challenging tasks of database operation and maintenance through its product features. Users can achieve expert-level database O&M management with just simple page operations, significantly reducing management costs.

■ Ecosystem interfacing

OCP offers open APIs, enabling ecosystem users to leverage OCP resources via standard API interfaces. Additionally, OCP facilitates the transmission of monitoring data to Prometheus and supports the capability to push alerting data to tools such as DingTalk, Lark, and WeCom, thereby enabling seamless connectivity with the user's unified O&M platform and simplifying the user's O&M processes.

■ High availability

OCP supports the deployment mode across multiple clusters and availability zones. It enhances the O&M management capabilities for OceanBase primary and backup cross-city clusters, ensuring the stability of OceanBase clusters in the event of a disaster and achieving high availability in O&M management.

■ Multi-form

Two product forms: OCP Enterprise Edition and OCP Express are available. OCP Enterprise Edition is designed to offer a unified platform for enterprise-level database management and control, while OCP Express meets the needs of individual users and small businesses for lightweight monitoring and management of individual OceanBase clusters, enabling them to experience OceanBase at minimal cost.

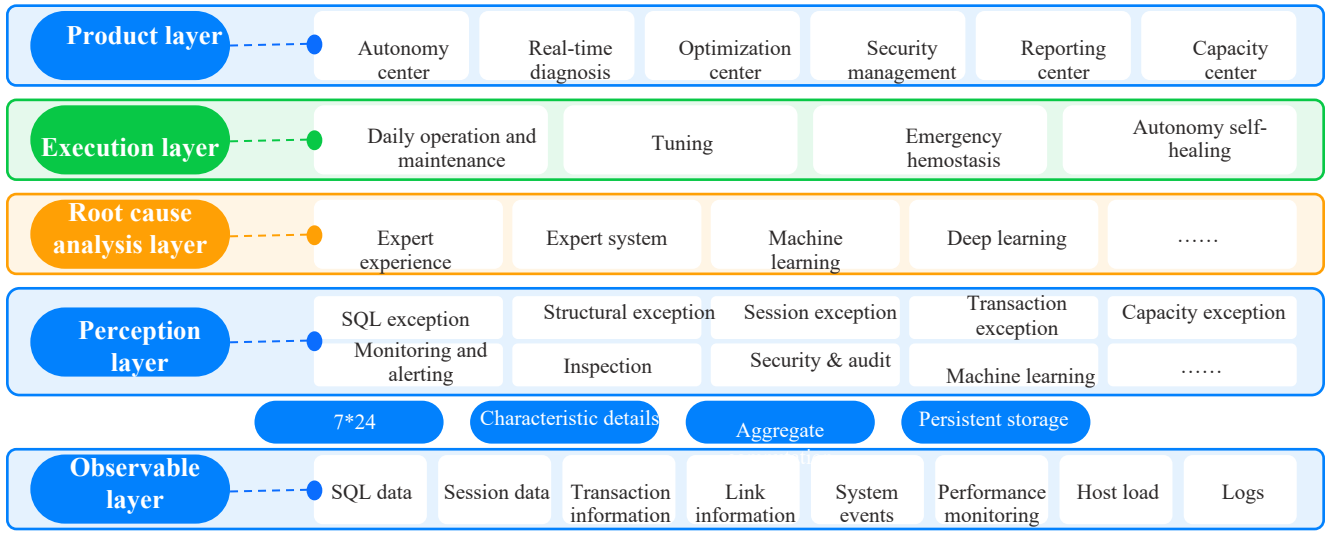
■ Secure

OCP provides a user role-based privilege isolation mechanism to ensure the secure utilization of resources, and enables enterprise-level operation auditing. OCP also provides comprehensive enterprise-level monitoring, alerting, inspection, and autonomy functions for the managed resources, guarding the operation of the cluster in real time to effectively protect database security.

OceanBase Autonomy Service (OAS)

OceanBase Autonomy Service (OAS) is a comprehensive, intelligent diagnostic, and autonomic service designed for developers, operators, and DBAs. It offers users visual monitoring, performance optimization, troubleshooting, security management, and capacity management, facilitating the use of the OceanBase database in a simpler, more stable, and cost-effective manner.

Product architecture



Product advantages

■ Comprehensive diagnostic foundation based on panoramic monitoring

Provides comprehensive monitoring of all database operation metrics and kernel events, drivers and data links, as well as the database runtime environment, including network, storage, and computing resources, ensuring full visibility across the entire system. Provides data basis for comprehensive diagnosis and analysis through the collection and monitoring across multiple dimensions.

■ Experience of OceanBase experts

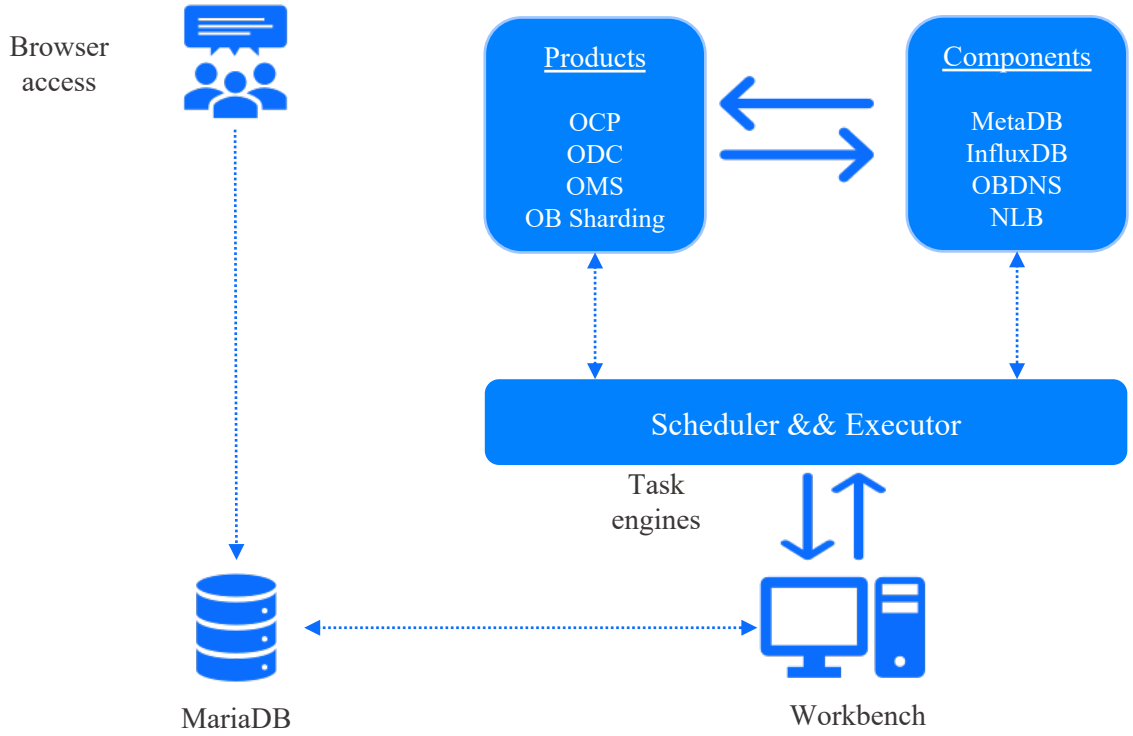
In a large number of businesses within Ant Group as well as rich business scenarios in finance, carriers and other industries, best practices are constantly verified, refined and precipitated to help users quickly identify and locate database and system exceptions, which are seamlessly integrated with the kernel diagnostic metrics and data models of OceanBase, and are continuously iterated and optimized.

■ Multi-form support

OAS currently offers ready-to-use solutions tailored for OceanBase Enterprise Edition and OB Cloud database, equipping clients with an intelligent and efficient diagnostic platform.

OceanBase Admin Toolkit (OAT)

OceanBase Admin Toolkit (OAT) serves as a visualization platform for the installation and management of OceanBase ecosystem products and components.



Product features

Installation and management of OceanBase ecosystem products

- OceanBase Cloud Platform (OCP)
- OceanBase Developer Center (ODC)
- OceanBase Migration Service (OMS)
- OceanBase Proxy (OB Sharding)

Installation and deployment of OceanBase peripheral tools

- DataX offline data migration tool
- OceanBase data import/export tools (OBLoader/OBDumper)
- OBLB load balancing tool
- Other self-developed programs

Installation and management of OceanBase components

- MetaDB: Container-based OceanBase database services, used for metadata storage in products such as OCP, ODC, and OMS.
- InfluxDB: Used to store monitoring data for OMS's data transport links.
- NLB/OBDNS: DNS and load balancing services provided by OceanBase.

Product Overview

01

- Database Development and Evolution
- About OceanBase Database
- OceanBase Product and Tool System
- **OceanBase Certification Training System**

Join OceanBase Certification Program and Become an Expert of Distributed Database !

Certification Benefits

01

Embrace the trend of distributed database development to update knowledge system

02

Explore the core technology of the industry-leading distributed database

03

Join OceanBase's extensive industrial chain

Certification Path

OceanBase Certified Associate

OceanBase Certified Professional

OceanBase Certified Expert

Online

Academy

Documents

Community

Trial software

+

Offline

Select professional database training institutions as training partners

Experts on-site guidance to answer questions

Complete experimental environment

Summary

- Database development history
  - Origin: The relational database theory originated in the 1970s and was successfully commercialized by IBM and Oracle.
  - Evolution: In step with the industry's shifting requirements, the database technology has undergone a significant evolution from relational to non-relational and back to enhanced relational models, alongside a shift from centralized to distributed systems.
  - Native distributed: Enterprise-level native distributed databases offer better high-availability solutions than traditional centralized databases, particularly for applications with high concurrent processing demands.
- History and current status of OceanBase
  - Native distributed: OceanBase is a pioneer in the field of distributed databases and represents the Gen3 native distributed databases.
  - Innovative development: 100% self-developed. OceanBase pioneers the "Five IDCs across Three Regions" city-level automatic lossless disaster recovery solution; the first database to achieve an RTO of less than 8 seconds; and the first to introduce an integrated standalone and distributed architecture. .
  - Proof of strength: OceanBase is the only distributed database in the world to set new records in both TPC-C and TPC-H benchmark tests.
- OceanBase core technical capabilities
  - Fully self-developed: OceanBase is fully self-developed, with code-level control and extensive experience in large-scale financial core scenarios, ensuring 10 years of proven reliability.
  - Native distributed: Full-volume data validation ensures robust data consistency, no data loss, high availability, and smooth scaling
  - Integrated standalone and distributed: The self-developed integrated architecture surpasses performance and availability benchmarks, enabling boundless application scaling and perpetual service availability.
  - HTAP and real-time analytical processing: Data can serve both transactional and real-time analytical purposes, broadening its potential with the capabilities of HTAP.
  - Smooth migration: Smooth migration of Oracle/MySQL: quick and cost-effective migration of applications and data.
  - Advanced compression technology: Self-developed coding compression technology harmonizes the trade-offs between "performance" and "compression," significantly reducing storage costs by 70% to 90%.
  - Native multi-tenant: Resource isolation between tenants, on-demand utilization, and flexible management.

Summary

- OceanBase product and tool system
  - Full-form product system: OceanBase offers an Enterprise Edition, an OB Cloud Edition, and an open-source Community Edition.
  - Full-scenario deployment: OceanBase can be deployed independently, utilized as a cloud service, and also offers OceanBase appliances. OceanBase supports both standalone and distributed deployments.
  - Comprehensive tool system: It offers a full suite of productized support for OceanBase databases from pre-migration, replacement, to post-migration, facilitating a steady growth of your business.
    - OMA: Migration assessment tools
    - OMS: Data migration and synchronization tools
    - ODC: Development tools
    - OCP: Cloud platform O&M management tools
    - OAS: Intelligent autonomy service tools

Self-test

- [True/false] OceanBase is a 100% self-developed Chinese database, independent of open-source database adaptations. ( )
- [True/false] OceanBase is the only database in the world to set new records in both TPC-C and TPC-H benchmark tests. ( )
- [True/false] OceanBase is a native distributed database and cannot be deployed as a standalone system like a standalone database. ( )
- [True/false] OceanBase is a general-purpose database that is neither tied to specific hardware nor bound to any particular platform, and can be deployed in a private environment or in the cloud. ( )
- [True/false] OceanBase is a transactional database that excels in handling high concurrency transactionsm, but it is not specialized in large-scale analytical operations and is not an HTAP database. ( )
- [Single answer] Regarding OceanBase core technology, the false statement is ( )
    - OceanBase relies on distributed middleware to achieve distributed access and transaction processing on a standalone database.
    - OceanBase's self-developed high-compression technology effectively addresses the trade-off between "performance" and “compression”, which can significantly reduces data size and disk space usage without imposing a substantial performance burden.
    - OceanBase's distributed architecture delivers ultimate high availability, ensuring zero data loss (RPO = 0) even in the event of a failure in a minority of nodes, and minimizing the impact on business operations with an extremely short recovery time (RTO < 8 seconds).
    - OceanBase supports HTAP mixed loads, ensuring that different types of tenants do not interfere with each other through multi-tenant resource isolation. Additionally, it leverages multi-replica technology, allowing a single data set to be used for both transaction processing and real-time analysis.

Self-test

- [Single answer] Regarding OceanBase product system, the false statement is ( )
  - OMS is a data synchronization tool provided by OceanBase, which can synchronize data from other databases to OceanBase database, such as synchronizing data from Oracle database to OceanBase's Oracle-compatible tenant.
  - OMA is a data migration and assessment tool provided by OceanBase database, which enables one-click database migration.
  - OCP is a cloud platform management tool provided by OceanBase that allows multiple OceanBase clusters to be unified into one OCP for management and monitoring.
  - OAS is an autonomy service tool provided by OceanBase that provides automated optimization recommendations and intelligent troubleshooting capabilities.
- [Multiple answers] Regarding OceanBase’s core features, the false statement is ( )
  - OceanBase is inherently designed for hyperscale business processing systems and may not be suitable for small and micro enterprises.
  - The OceanBase distributed database offers high concurrency capabilities that scale linearly, capable of supporting tens of millions of business processing transactions per second.
  - The OceanBase distributed architecture is compatible with both MySQL and Oracle databases, facilitating a seamless migration of applications and data from MySQL and Oracle to the OceanBase database.
  - The OceanBase distributed database has to keep multiple replicas of data, which results in higher storage costs.
- [Multiple answers] Regarding the development and architectural evolution of databases, the false statement is ( )
  - One of the main reasons for transitioning from centralized to distributed is to overcome the performance limitations of the standalone architecture.
  - Compared to centralized databases, distributed databases primarily offer higher concurrent processing capability, but demonstrate no particular advantage in high availability capability.
  - Distributed database systems, utilizing distributed middleware, are capable of efficiently managing SQL queries and transactions across databases.
  - Native distributed databases do not rely on distributed middleware and provide transparent scalability.

Contents

I. Product Overview

II. **Cluster Architecture**

III. Database Engines

IV. Installation and Deployment

V. Application Development

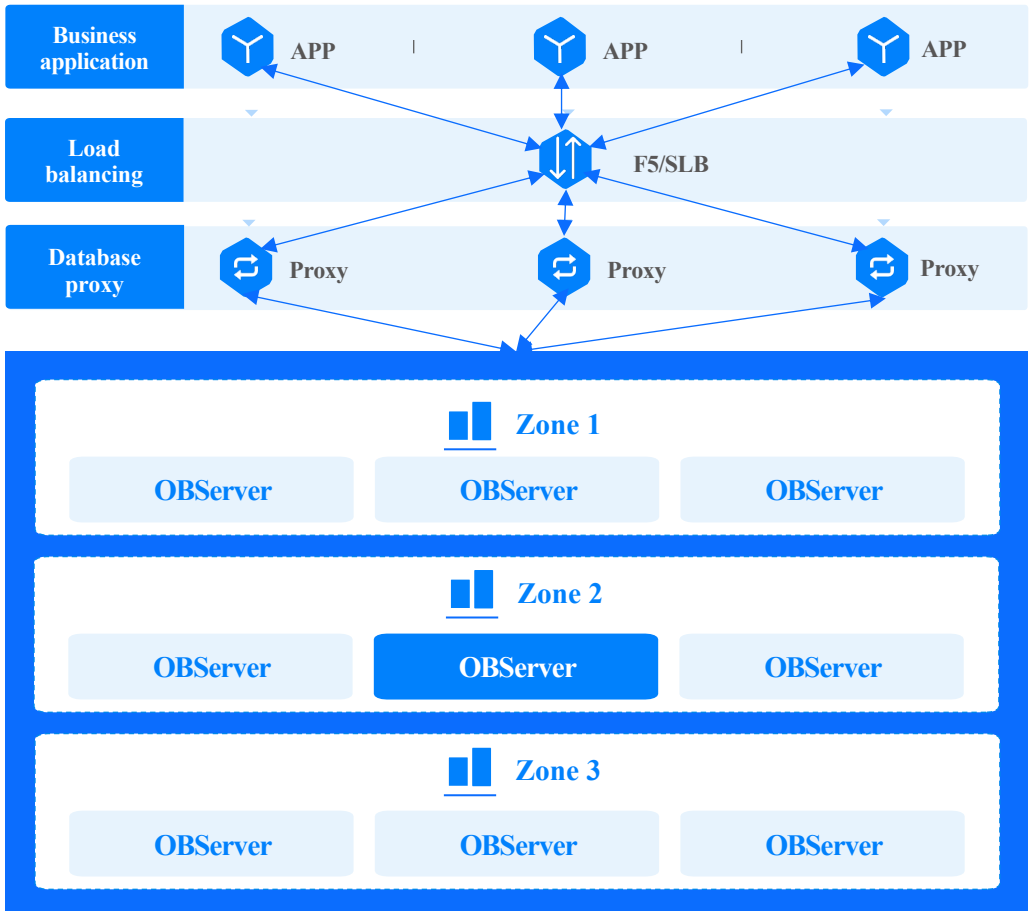
VI. Operations and Maintenance Management

Cluster Architecture

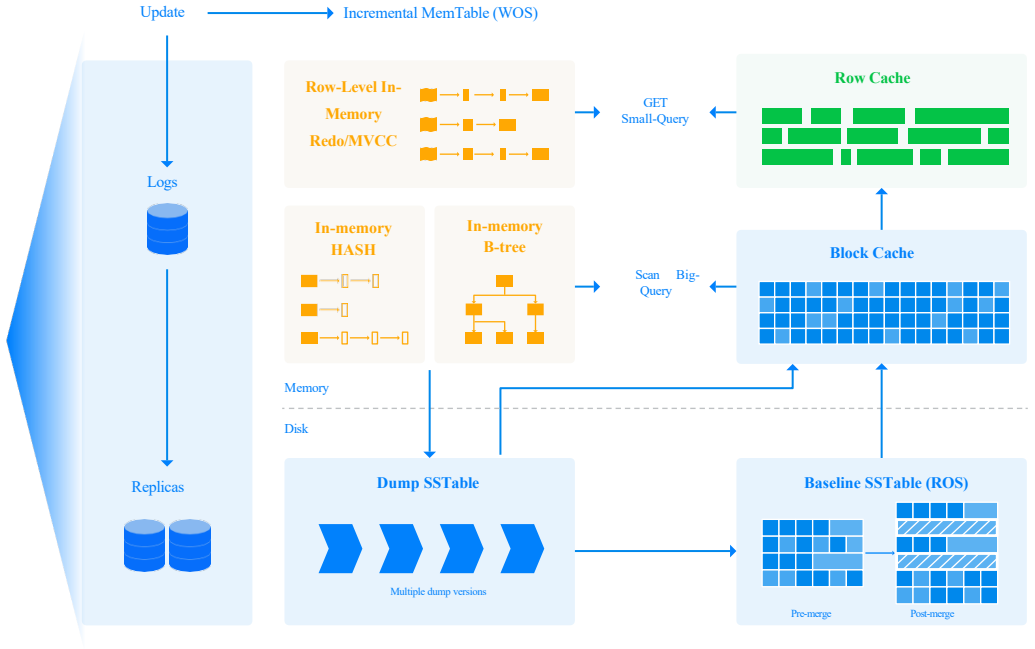
02

- **Basic Concepts of Cluster**
- Routing and Load Balancing
- High-Availability Deployment Mode

Overall System Architecture

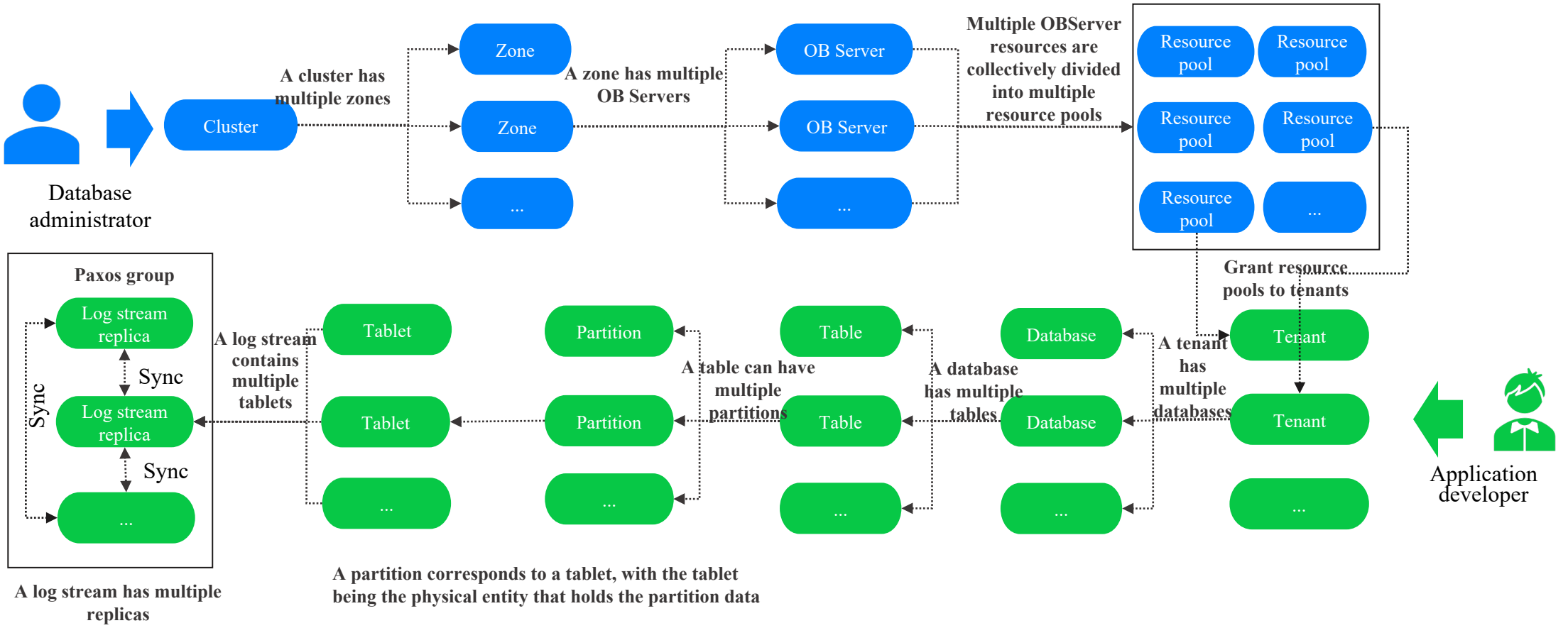


Multi-active, multi-region, cross-city high availability deployment



Introduction to OceanBase Basic Concepts

From the system management and application development perspectives, it is essential to understand the basic concepts of OceanBase cluster, Zone, OB Server, Resource Pool, Tenant, Partition, Tablet, Log Streams and Replicas, and Paxos.



Cluster and Availability Zones

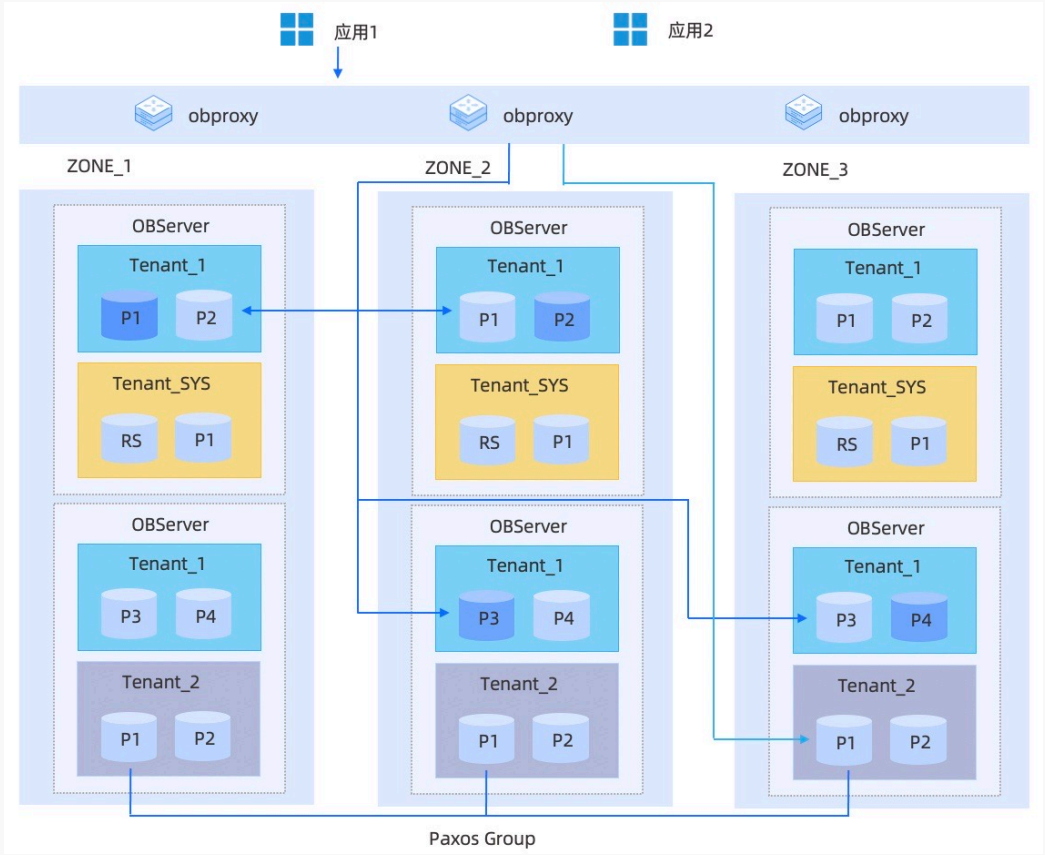
OceanBase leverages commodity servers with reliance on local storage. Servers for distributed deployment are fully equivalent without special hardware requirements. OceanBase employs a Shared Nothing architecture for its distributed database processing, featuring an SQL execution engine with inherent distributed execution capabilities.

Cluster

- **Cluster:** A cluster contains multiple tenants that are served by multiple OBServer hosts working together.
- **Tenant:** OceanBase features built-in multi-tenant, where each tenant appears as an independent database instance to users, with the capability to configure the tenant's distributed deployment methods at the tenant level.
- **OBServer:** OceanBase runs a single-process program called observer on the server as a running instance of the database, utilizing local file storage for data and transaction Redo logs.

Availability zone

- **Concept:** Availability zone is a logical concept, with each zone hosting a singular, complete replica, and the OceanBase cluster achieves distributed high availability through multiple zones.
- **Deployment:** Zone can consist of different cities, distinct data centers within a single city, or separate racks within the same data center. When the cluster is distributed in multiple data centers, each availability zone can be aligned with a single data center.
- **Number of zones:** The number of zone is 1 for standalone deployment; the number of zone is >= 2 for distributed deployment, the number of zone is recommended to be 2 or 4 when configuring arbitration service, and the number of zone is recommended to be odd when there is no arbitration service, so that minority failures do not disrupt operations.

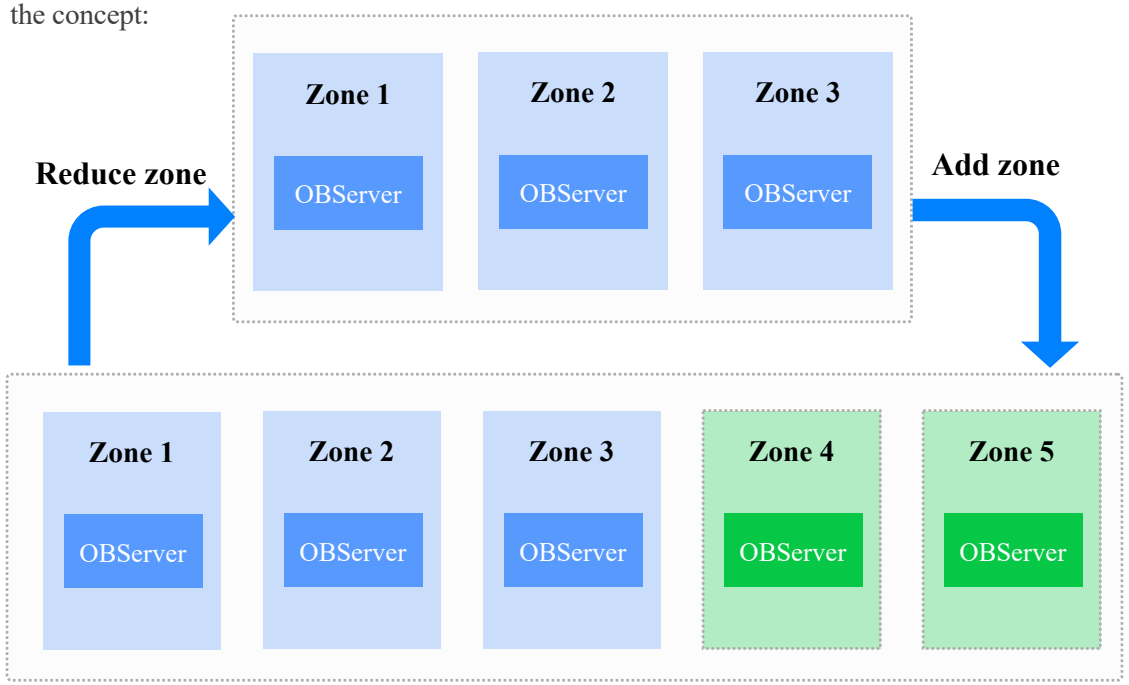


Cluster Scaling

Compared with traditional standalone databases, OceanBase databases based on distributed architecture provide **flexible online scalability**. When the expectation of the disaster recovery level changes, the disaster recovery capability of the cluster can be increased or decreased by adjusting the number of availability zones (adding or removing Zones). When the external load of the cluster changes, the load capacity of the cluster can be changed by adjusting the number of physical machines within the Availability Zone (adding or removing OBServers).

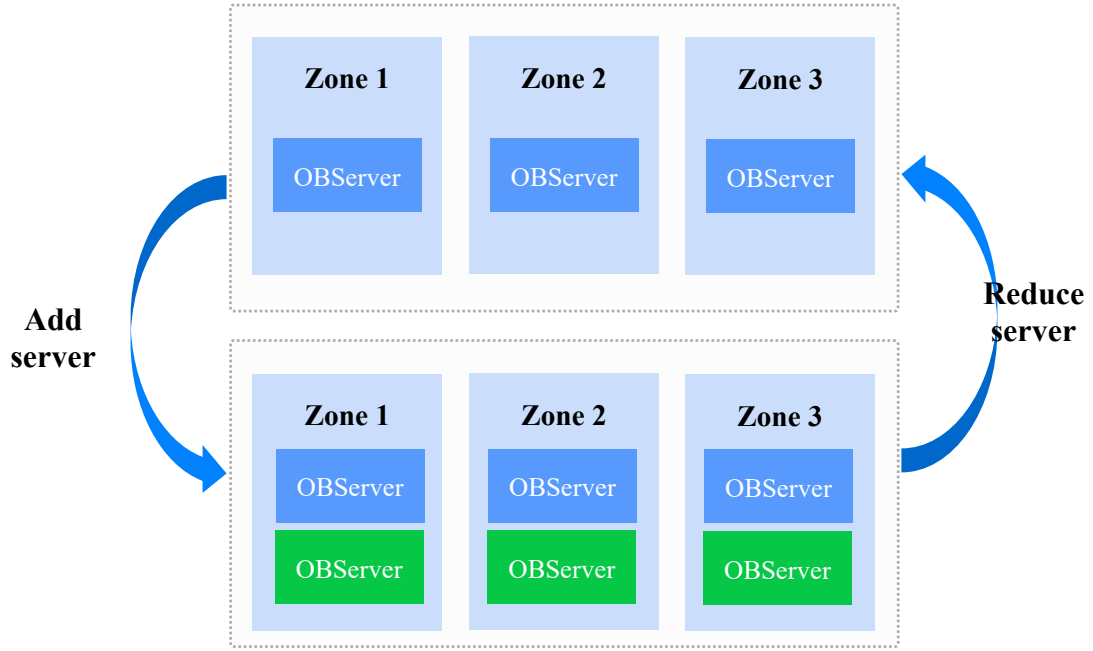
Dynamic adjustment of availability zones

OceanBase cluster has multiple availability zones, and each availability zone contains a complete replica of the data. By increasing the number of zones, the number of replicas of data can be increased, thereby improving system availability. Here's a specific example to illustrate the concept:



Physical machine scaling

OceanBase database supports dynamic scaling of physical machines in an availability zone, i.e., adjusting the number of physical machines in an availability zone to expand service capacity, OceanBase database realizes linear expansion of service capacity, as shown in the following example.



Multi-tenant Architecture: Resource Isolation and Data Isolation

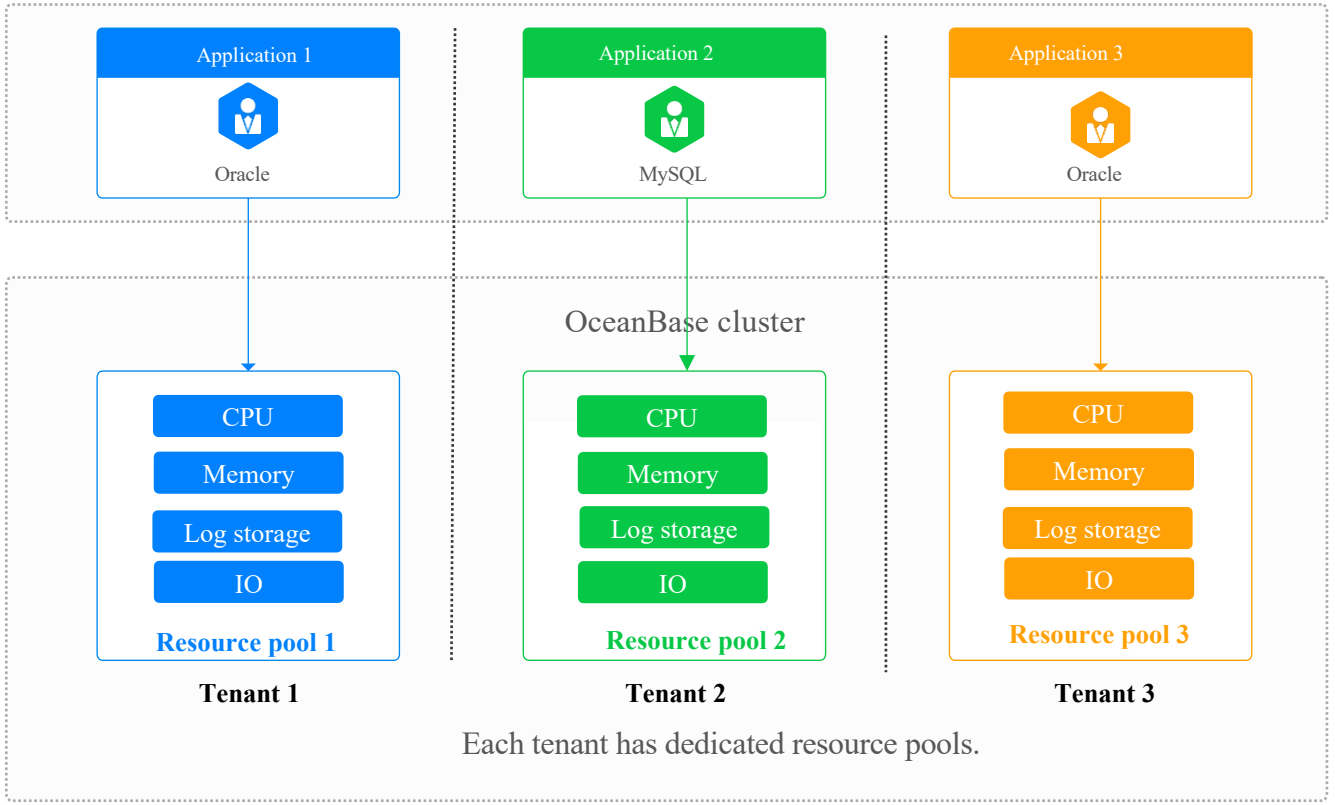
OceanBase database employs a single-cluster multi-tenant design, inherently supporting cloud database architecture, as well as various deployment forms such as public cloud, private cloud and hybrid cloud. OceanBase database achieves resource isolation through tenants, ensuring that each database service instance is unaware of the existence of others. It also guarantees the security of tenant data through access control, in conjunction with the strong scalability of OceanBase database, it is able to provide secure and flexible DBaaS services.

Tenant definition

- Tenant is a logical concept that, to a certain extent, is equivalent to the concept of an "instance" in traditional databases.
- Tenant is a unit of resource allocation. OceanBase divides the cluster into multiple resource pools according to the specifications (CPU, memory, storage, IO) and allocates them to different tenants.
- Tenants have "exclusive" use of their resource quotas. **CPU, memory, and log storage** between tenants are isolated.
- Overall, a tenant serves as a container for various database objects as well as for resources.

Tenant characteristics

- Within a tenant, you can create your own users and manage permissions independently.
- Within a tenant, you can create databases, tables, and all other database objects, all while maintaining isolation from other tenants.
- Tenants have their own independent information_mode and other system databases and management views.
- Tenants have their own independent system variables and system parameters.
- When creating a tenant, you can choose the compatibility mode for the tenant: Oracle mode or MySQL mode.



Tenant Types

Prior to version V4.0.0, there are two types of tenants: system tenant and user tenant. Starting from version V4.0.0, the concept of Meta tenants was introduced. Therefore, in the current version, there are three types of tenants visible to users: system tenant, user tenant and Meta tenant.

System tenant

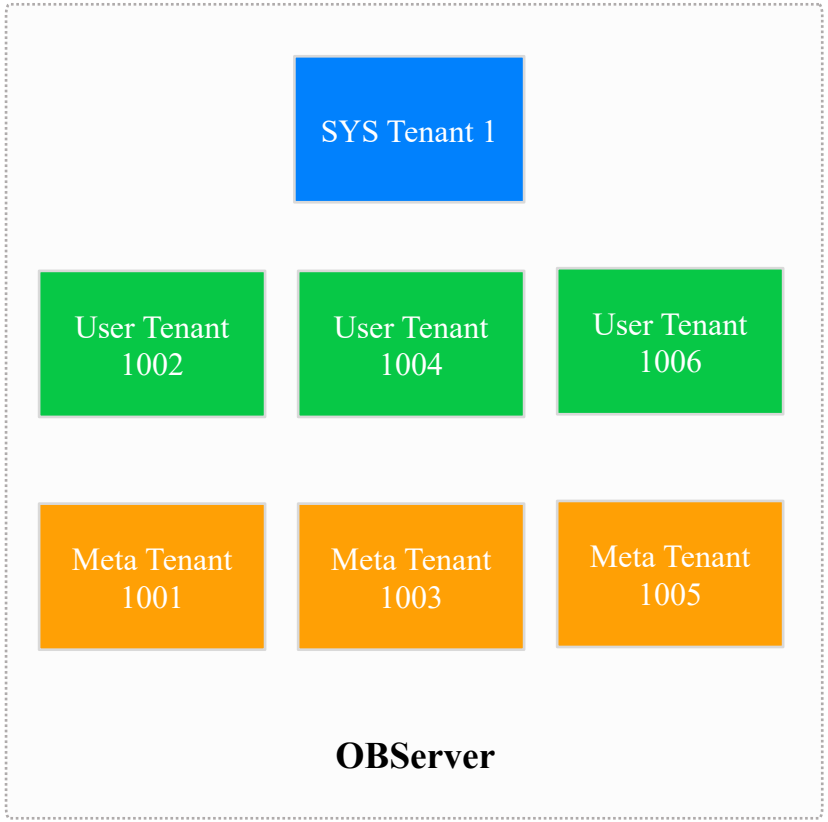
- System tenant is the default tenant created by the cluster, synchronized with the cluster's lifecycle, and is responsible for managing the lifecycle of the cluster and all tenants.
- The data of the system tenant is privately owned by the cluster and does not support physical synchronization, as well as the physical backup and recovery between primary and backup clusters.

User tenant

- User tenants are tenants created by users, providing complete database functionalities and supporting both MySQL and Oracle compatibility modes.
- The data of the user tenants has stronger requirements on data protection and availability, and can be physically synchronized and backed up/restored across clusters. Typical data includes: Schema data, user table data, transaction data, etc.

Meta tenant

- Meta tenants are self-managed tenants within the OceanBase database. For every user tenant created, the system will automatically create a corresponding Meta tenant, and its lifecycle is consistent with that of the user tenant.
- Meta tenants are used to store and manage cluster-private data for user tenants. This data does not require cross-database physical synchronization or physical backup and recovery. Such data include: Configuration items, location information, replica information, log stream status, information related to backup and recovery, merge information, etc.



Tenant Comparison

Attribute	System tenant	User tenant	Meta tenant
Tenant ID	1 Immutable	Min: 1002 Automatically assigned by the system	Min: 1001 Meta Tenant ID + 1 = User Tenant ID
Naming conventions	SYS	User-defined	META${user_tenant_id}
Data attribute	Cluster-private data, does not support cross-cluster physical synchronization or physical backup and recovery	Tenant non-private data, support cross-cluster physical synchronization and physical backup and recovery	Tenant-private data, does not support cross-cluster physical synchronization or physical backup and recovery
Tenant O&M	Create: Not support Delete: Not support Rename: Not support User login: Support	Create: Support Delete: Support Rename: Support User login: Support	Create: Not support Delete: Not support Rename: Not support User login: Not support

* *Tenant ID:* A unique identification number that OceanBase automatically assigns to a tenant when creating a tenant.

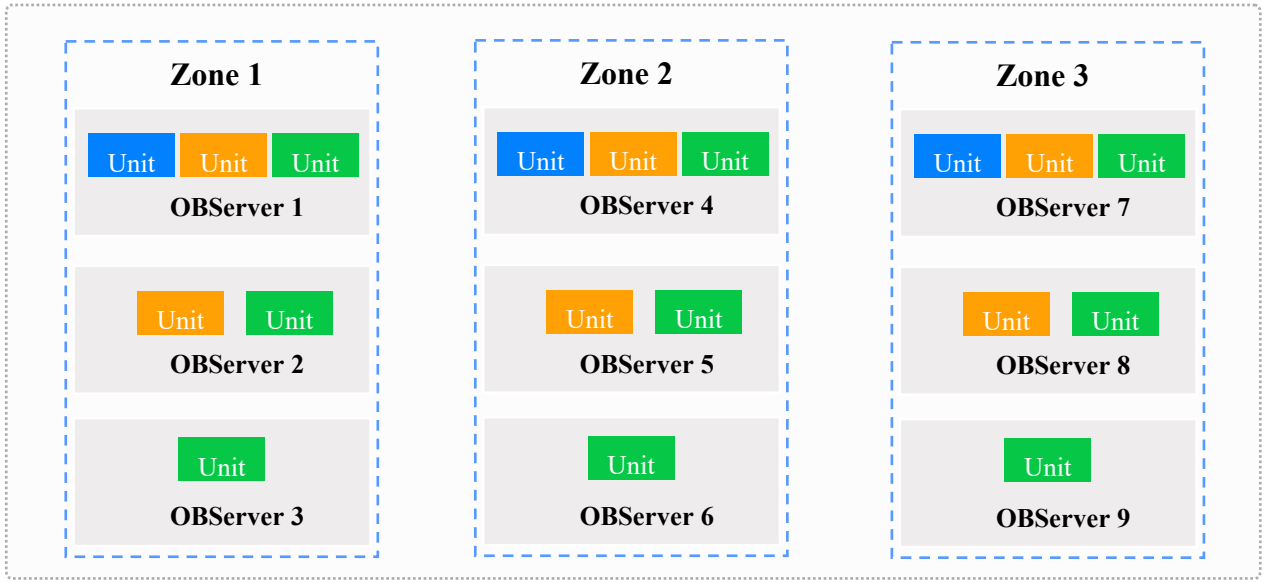
Tenant Resource Allocation: Resource Pool and Resource Unit

Tenants are associated with resources through resource pools, thereby exclusively occupying a certain amount of resource quotas. OceanBase allocates the resources of OBServer to different resource units according to the resource specifications (resource config). The resource units on different OBServers then form a resource pool that is assigned to a specific tenant. OceanBase can assign different specifications and number of Units to tenants to meet different service requirements. Resources are not static; they can be continuously adjusted in response to the evolving needs of the services.

- **Unit config:** Unit configs define the size of common physical resource items, including CPU, memory, disk space, IOPS, etc. When creating a resource pool, its unit configs are designated, thus creating resource units according to the definition.
- **Resource unit:** It refers to the tenant's container on the OBServer, which can be regarded as a lightweight virtual machine, detailing the available resources for tenants on OBServer. OceanBase manages physical resources on a per-unit basis, where each unit is a collection of physical resources such as CPU, memory, storage space and IOPS.
- **Resource pool:** Each unit belongs to a resource pool, and each resource pool is composed of several units. The resource pool is the basic unit of resource allocation. All units within the same resource pool have the same unit configs, meaning that the physical resources of the units in that resource pool are all of the same size.
- A resource unit can only belong to one tenant; a tenant can only have one resource unit on an OBServer at the same time.

First define the specification (CPU and memory) of each Unit, e.g. U1=2C8G;

- Tenant 1: grant a resource pool, unit=U1, unit Num=1
- Tenant 2: grant a resource pool, unit=U1, unit Num=2
- Tenant 3: grant a resource pool, unit=U1, unit Num=3

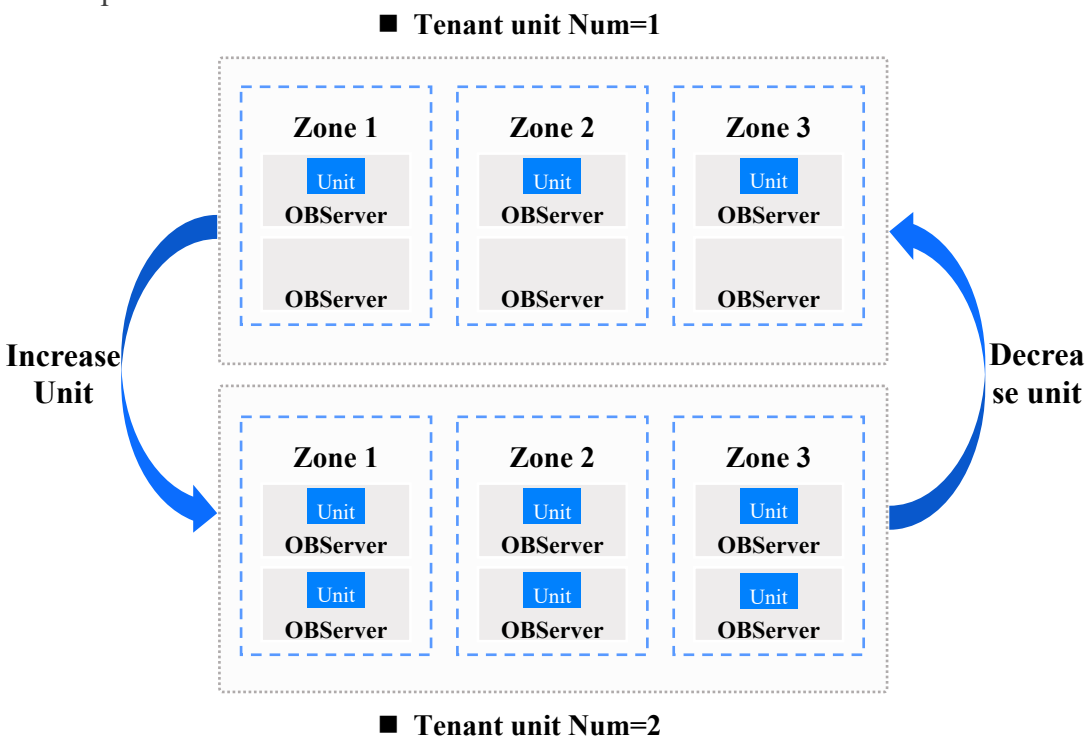


Tenant Scaling

OceanBase implements a resource pooling strategy for tenants, enabling **online elastic scaling** of tenant resources by adjusting the specifications and quantities of resource units within the pool, all while ensuring the continuous availability of the tenants.

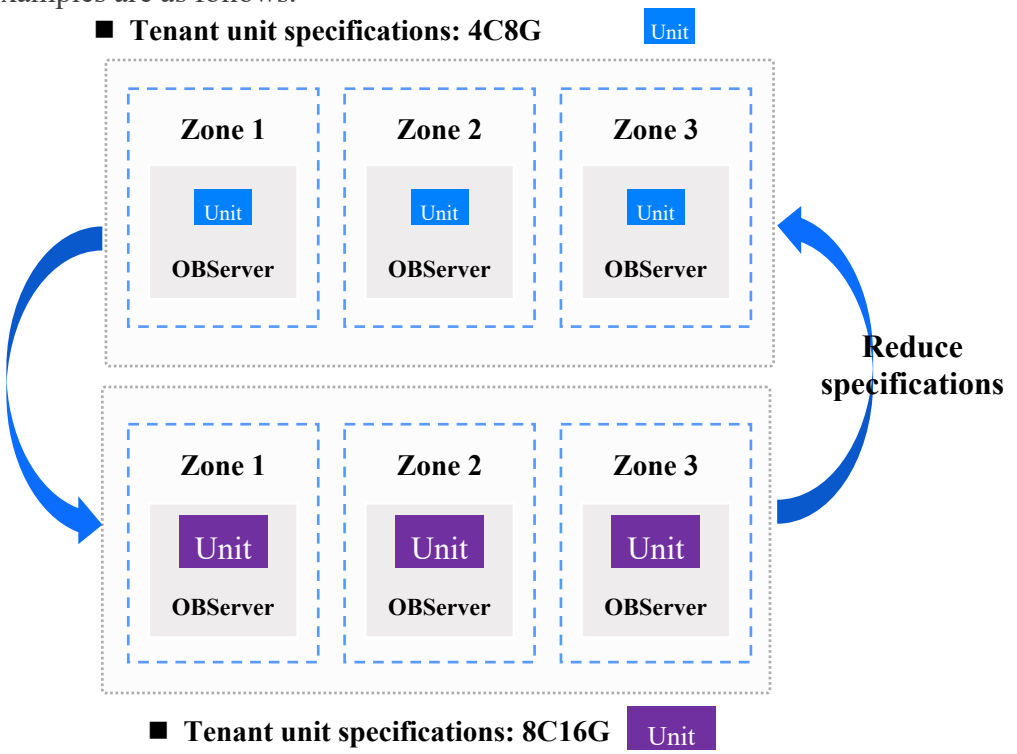
Horizontal scaling of tenants

Horizontal scaling of tenants refers to dynamically adjusting the tenant's available resources by changing the UNIT_NUM of the resource pool, thereby enhancing or reducing the service capacity of the tenant, and some specific examples are as follows.



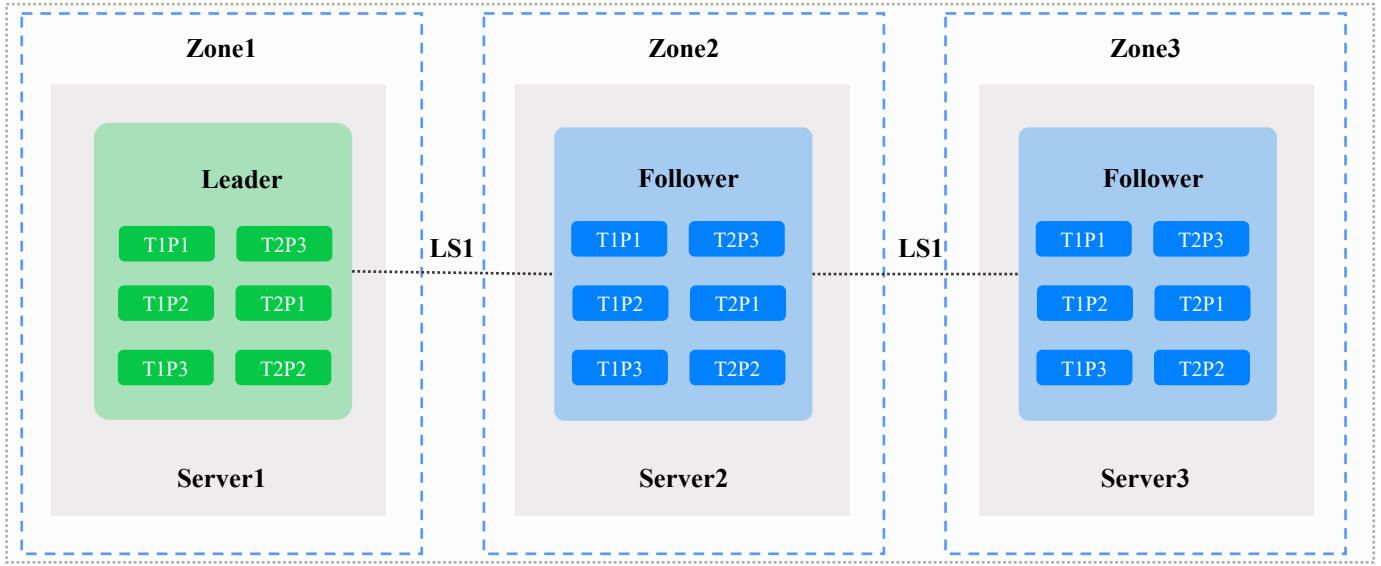
Vertical scaling of tenants

Vertical scaling of tenants refers to dynamically adjusting the tenant's available resources by changing the unit configs of the resource pool, thereby enhancing or reducing the service capacity of the tenant, and some specific examples are as follows.



Replica, Log Stream and Paxos Protocol

- **Replica:** In a distributed architecture, to ensure high availability of data read and write services, OceanBase database will copy the same data across multiple machines, and each copy of the data is referred to as a replica. In a set of replicas, there is exactly one replica accepts strong consistency read and write operations, known as the leader replica; the other replicas only support weak consistency read operations, known as the follower replica.
- **Log stream:** OceanBase synchronizes the transaction logs among multiple replicas to complete the synchronization of the replicas. The logs that are synchronized among different replicas are referred to as the log stream. Similar to a standalone database, OceanBase writes one log file for all leader partitions of a tenant under the same OBServer and synchronizes this log across multiple replicas, which is called as standalone log stream.
- **Paxos protocol:** OceanBase employs the Paxos distributed consistency protocol to synchronize the log stream among multiple replicas. Transaction logs are considered committed only after they have been successfully synchronized across a majority of the replicas, which is called Majority Rule.
- **Replica types:** A replica that participates in log synchronization based on Majority Rule is a full-featured replica, it has complete logs and data, and can also participate in Leader elections or be elected as a Leader. There is another type of replica that, although it also has complete logs and data, does not participate in log synchronization and election based on Majority Rule, and thereby cannot be elected as a Leader. This type of replica can only provide weakly consistent reads, which we call a read-only replica. Only full-function replicas have a distinction between the roles of Leader and Follower.



LS: log stream

Architecture Introduction

02

- Basic Concepts of Cluster
- **Routing and Load Balancing**
- High-Availability Deployment Mode

OceanBase Database Proxy (ODP)

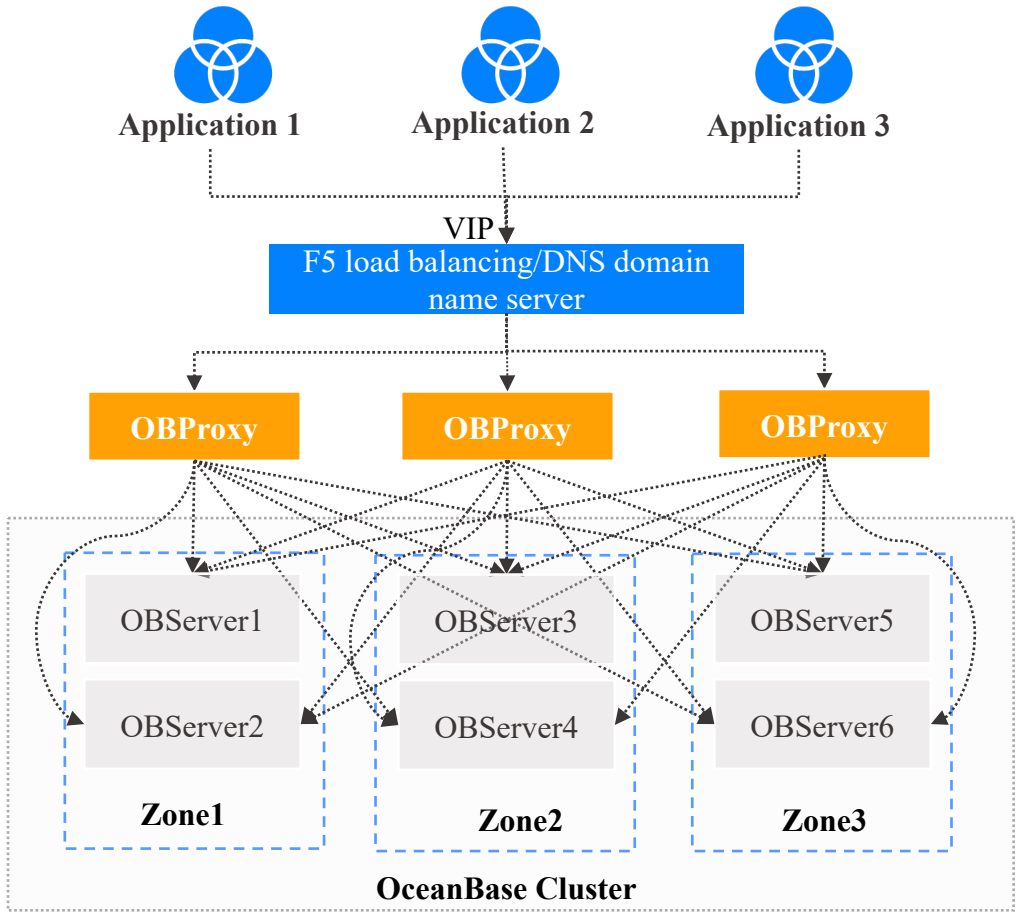
OceanBase Database Proxy (ODP), also known as OBProxy, is a dedicated proxy service for the OceanBase database. User data is stored in multiple copies on different OBServer nodes. ODP receives SQL requests issued by users and forwards these SQL requests to the optimal target OBServer node, and finally returns the execution results back to the user.

Distributed database access requirements

- Simple, the application does not need to connect to multiple IPs.
- Unified, different clusters/tenants don't need to differentiate connection methods.
- High availability, the application IP is unnecessary to change during database switching and node failures.

Capabilities of OceanBase data proxy

- **Reverse proxy:** Perform basic parsing of SQL to determine the machine where the corresponding Leader replica is located, and route the request to that Leader; when the Leader replica's location cannot be determined, select an OBServer according to the routing rules.
- **Status Sensing:** Proactively probing the liveness of OBServers, sensing node status, and automatically blocking problematic nodes.
- **High performance:** Lightweight processing, without data computation, a single OBProxy can forward millions of requests per second.
- **High Availability:** Each OBProxy is a "stateless" service process, no data persistence, no connection between multiple OB Proxies, high availability and load balancing with the help of load balancing VIP.



OBServer Inter-node Routing

In an OceanBase cluster, each OBServer node functions as both a computing node and a storage node. Upon receiving a SQL request, the OBServer activates the distributed optimizer for SQL parsing. If the data that a SQL require to access is stored on remote nodes, the OBServer will automatically forward the request for remote data to the appropriate node where the data is located. Once the remote node has completed the data processing request, it will send the results back to the node that initiated the SQL query, thus enabling intelligent routing for distributed data access.

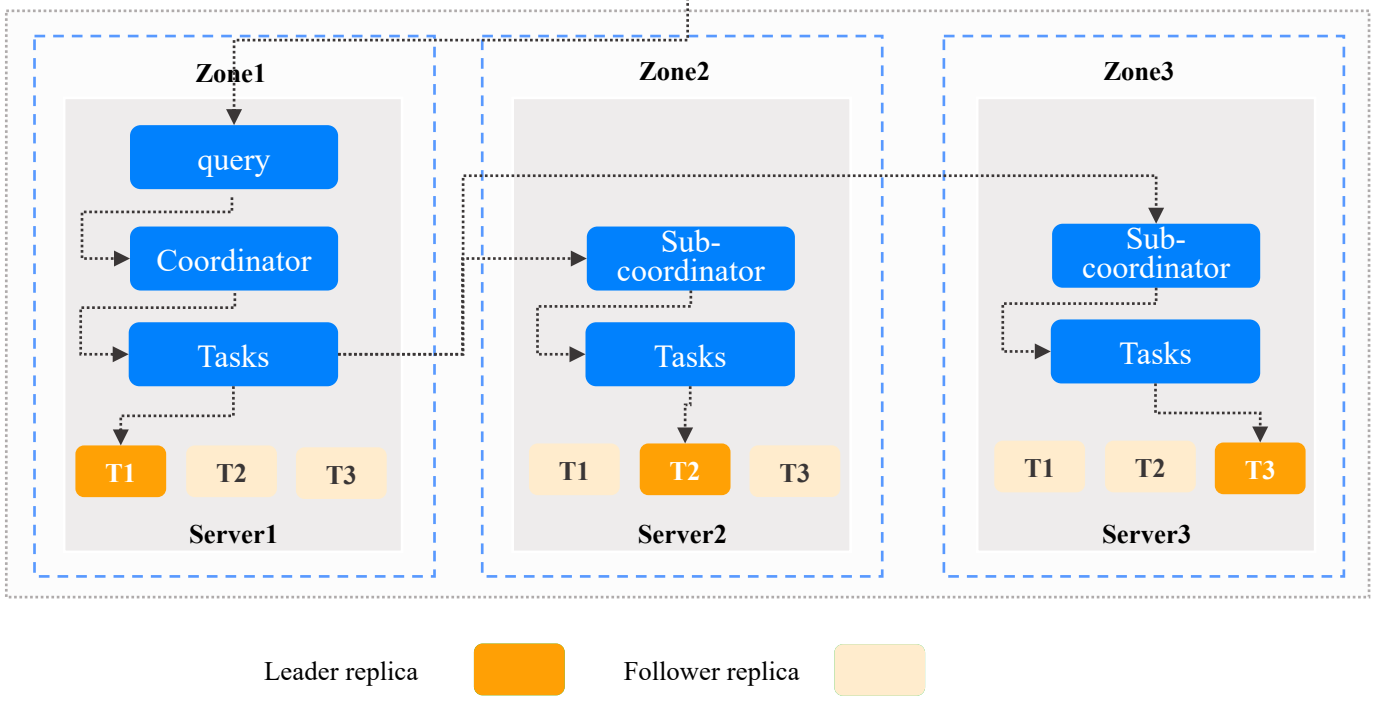
Remote access

- OBServer OBS1 receives and parses the SQL, identifying that the required data is located on another OBServer, OBS2.
- OBS1 sends the SQL execution to OBS2;
- OBS2 receives the SQL execution and sends the execution result back to OBS1;
- OBS1 returns the SQL execution results to the application (or ODP).

Distributed access

- OBServer OBS1 receives and parses the SQL, identifying that the required data is located on multiple OBServer nodes: OBS1, OBS2 and OBS3;
- OBS1 sends the corresponding SQL executions to all participating node individually. OBS1, OBS2, OBS3;
- The participating execution nodes, upon receiving the corresponding SQL executions, send the execution results back to OBS1;
- OBS1 receives the results returned by the participating execution nodes, processes the results locally (correlation, summarization, sorting, etc.), and generates the final execution results;
- OBS1 returns the final SQL execution result to the application (or ODP).

```
SELECT COUNT(*) FROM T1,T2,T3 WHERE T1.C1=T2.C1 AND
T1.C1=T3.C1;
```



Set Primary Zone to Route Services to a Specific Zone

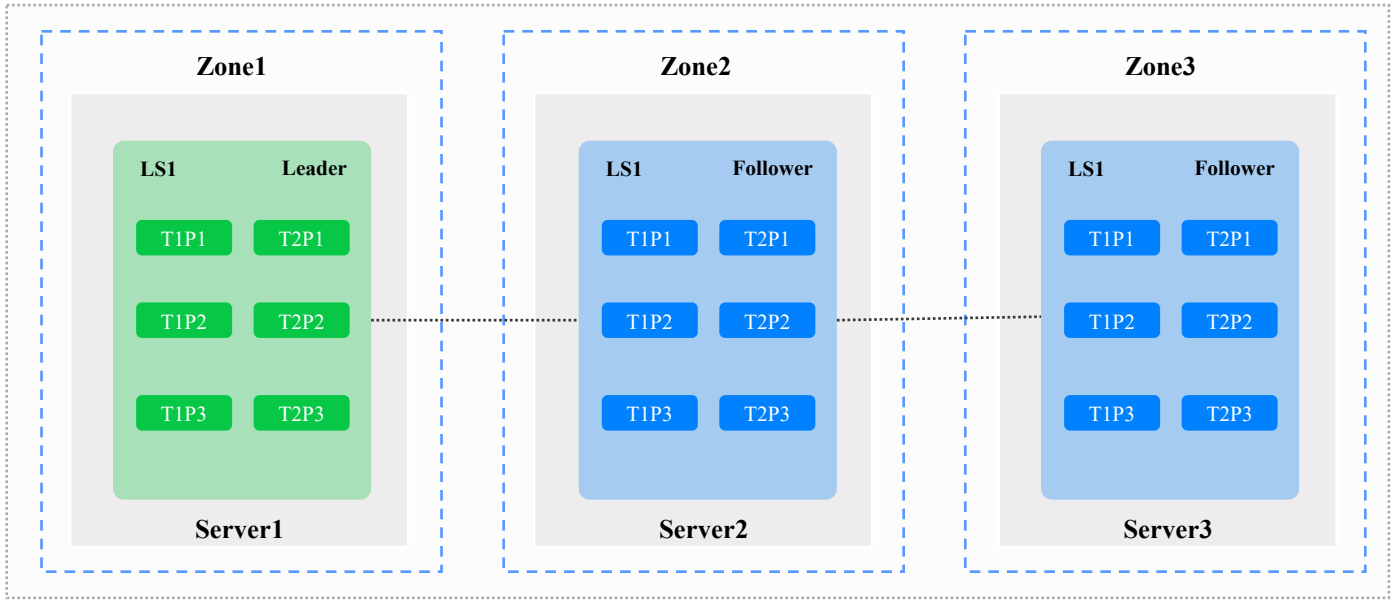
In a multi-center architecture, for performance optimization, we may sometimes need to consolidate service traffic to specific zones. This requires to set the primary zone for the tenant.

- OceanBase distributes Leader replicas in the Primary Zone. Primary Zone is a collection of zones, divided by semicolon (;) to indicate different priorities, separated by comma (,) to indicate the same priority, both sides of the comma have the same priority, and the left side of the semicolon has a higher priority than the right side.
- By configuring different Primary Zones for different tenants, you can converge different service traffic to different data centers, to reduce cross-center access and optimize resource utilization.

Aggregate to one Zone

Primary_Zone=(zone1;zone2;zone3)

- Priority: zone1 > zone2 > zone3.
- The leader replica exists only in Zone1, and the follower replicas in both Zone and Zone3.
- Suitable for latency-sensitive online processing services with small volume, you can attempt to avoid cross-server access to thereby reduce latency.

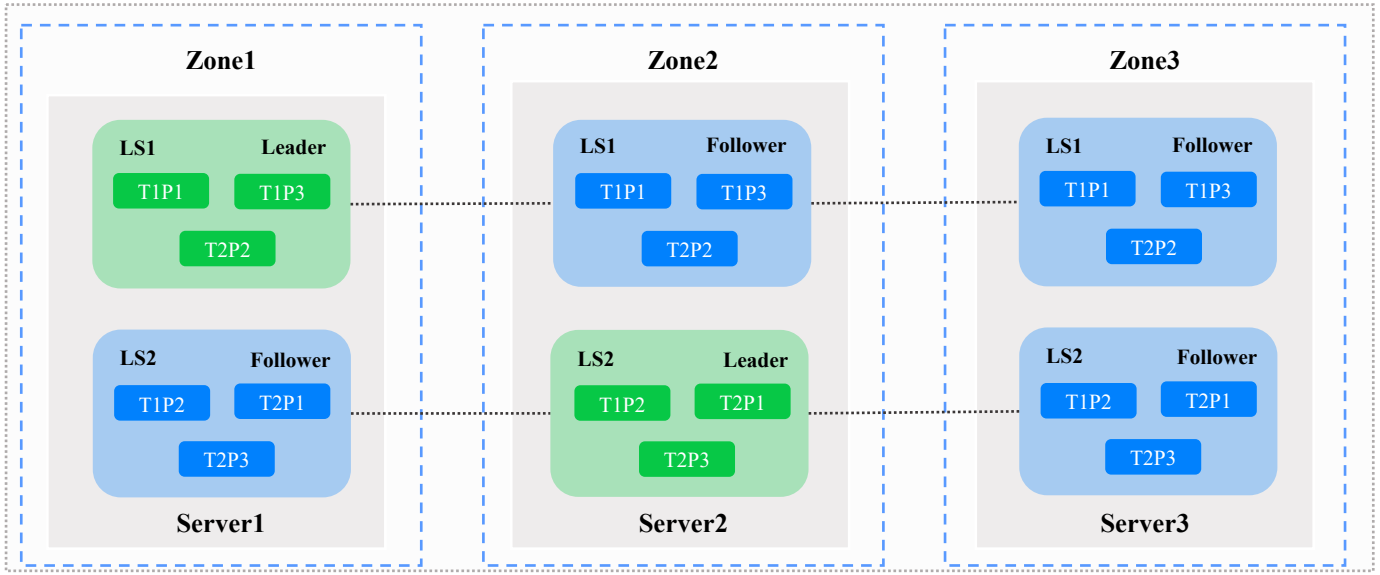


Set Primary Zone to Route Services to Multiple Zones

Aggregate to two Zones

Primary_Zone=(zone1,zone2;zone3)

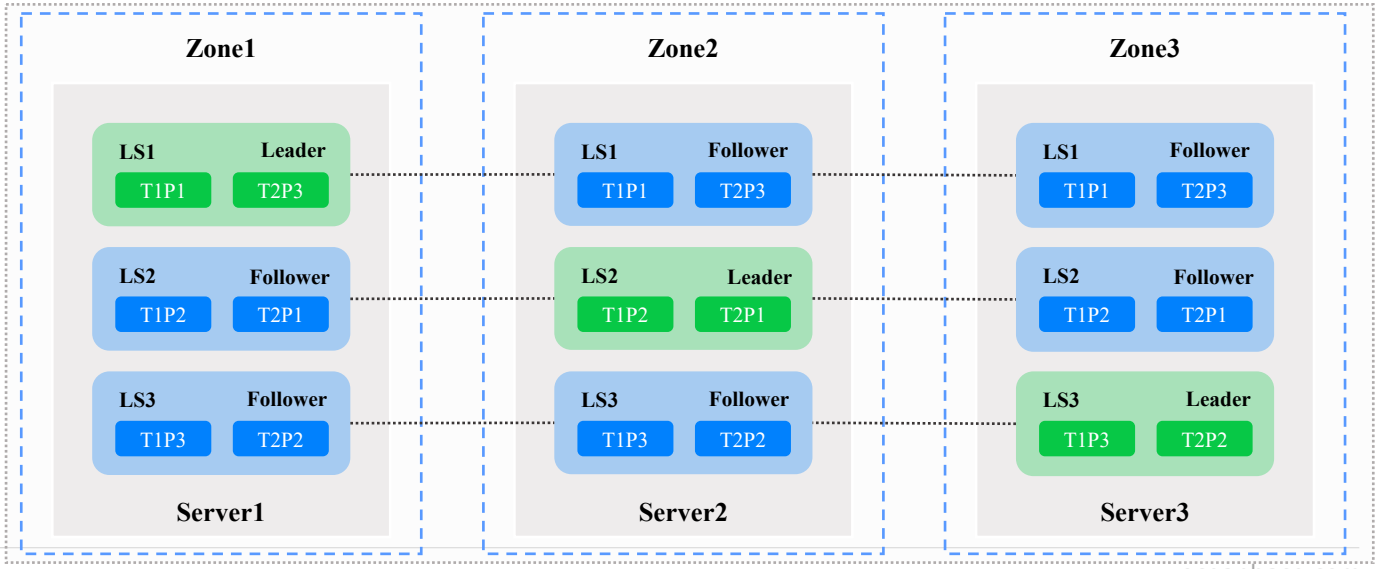
- Priority: zone1 = zone2 > zone3.
- Leader replicas are evenly distributed to Zone1 and Zone2, and the follower replicas are in Zone3.
- Suitable for scenarios with high service volume, where the closer two Zones share the service and the more distant Zone serves solely as a follower replica.



Decentralize to three Zones

Primary_Zone=(zone1,zone2,zone3)

- Priority: zone1 = zone2 = zone3.
- The leader and follower replicas are evenly distributed to each machine.
- Suitable for batch processing environments, the focus is on completing the process as soon as possible, without concern for the execution time of individual SQL. The goal is to expedite the completion of the entire task.



Aggregate Service-related Table Partitions to the Same Node through Tablegroup

Table Group is a logical concept, representing a collection of tables. By default, the data among different tables is randomly distributed with no inherent relationship. By defining a table group, you can control the proximity of a group of tables in physical storage. In the V3.x version, the table group is one with partitions defined, and the requirements for joining tables are exactly the same as the way the table group is partitioned, which restricts tables from joining the table group, exhibiting strong constraints. Starting from V4.2.0 version, the table group only needs to define the SHARDING attribute, enabling a highly flexible approach to incorporating tables with various partitioning methods.

SHARDING=NONE

- Aggregation rules: All partitions of all tables within a table group are aggregated on the same machine.
- Partitioning requirements: There are no restrictions on the partitioning type and method for the tables within the table group.

SHARDING=PARTITION

- Aggregation rules: Within a table group, data from each table is dispersed according to the first-level partition, The partitions with the same first-level partition ID are aggregated on the same machine; for tables with second-level partitioning, all second-level partitions under the same first-level partition are aggregated together.
- Partitioning requirements: All tables within a table group must either be partitioned tables or non-partitioned tables. If all are partitioned tables, only the first-level partitions are required to be partitioned in the same way.

SHARDING=ADAPTIVE

- Aggregation rules: In a table group, the data of each table is dispersed based on an adaptive approach. If all are non-partitioned tables, they are aggregated on the same machine; if all are first-level partitioned tables, they are dispersed according to the first-level partition, with the same first-level partition ID being aggregated on the same machine; if all are second-level partitioned tables, they are dispersed according to the second-level partition under each first-level partition, with the same second-level partition ID being aggregated on the same machine.
- Partitioning requirements: The tables within a table group are either partitioned or non-partitioned. If they are partitioned tables, they are required to be partitioned in exactly the same way.

```
CREATE TABLEGROUP TG1 SHARDING = 'NONE' ;
CREATE TABLE T1 (C1 NUMBER) TABLEGROUP = TG1 ;
CREATE TABLE T2 (C1 VARCHAR(50)) TABLEGROUP = TG1 ;
CREATE TABLE T3 (C1 NUMBER, C2 VARCHAR(50)) TABLEGROUP = TG1
PARTITION BY HASH(COL1) PARTITIONS 3;

+-------------+------------+----------------+----------+
| SVR_IP      | TABLE_NAME | PARTITION_NAME | ROLE     |
+-------------+------------+----------------+----------+
| 172.20.1.23 | T1         | NULL           | LEADER   |
| 172.20.1.23 | T2         | NULL           | LEADER   |
| 172.20.1.23 | T3         | P0             | LEADER   |
| 172.20.1.23 | T3         | P1             | LEADER   |
| 172.20.1.23 | T3         | P2             | LEADER   |
| 172.20.1.27 | T1         | NULL           | FOLLOWER |
| 172.20.1.27 | T2         | NULL           | FOLLOWER |
| 172.20.1.27 | T3         | P0             | FOLLOWER |
| 172.20.1.27 | T3         | P1             | FOLLOWER |
| 172.20.1.27 | T3         | P2             | FOLLOWER |
| 172.20.1.29 | T1         | NULL           | FOLLOWER |
| 172.20.1.29 | T2         | NULL           | FOLLOWER |
| 172.20.1.29 | T3         | P0             | FOLLOWER |
| 172.20.1.29 | T3         | P1             | FOLLOWER |
| 172.20.1.29 | T3         | P2             | FOLLOWER |
+-------------+------------+----------------+----------+
```

RootService (RS)

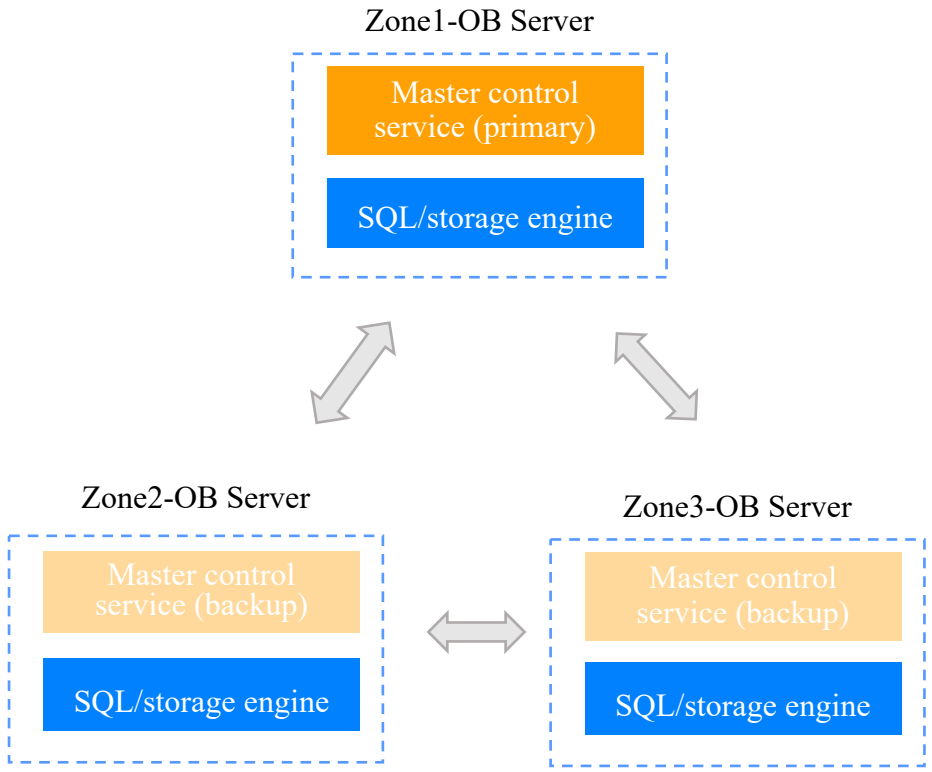
RootService is the cluster-level root service of OceanBase database cluster, **mainly responsible for functions such as resource management, disaster recovery, load balancing, and schema management**. RootService also employs a multi-replica high-availability architecture. In the event of a failure on the machine hosting RootService, the remaining OBServers will elect a new RootService.

OceanBase's "root service"

- The core module of OceanBase manages the whole cluster.
- Cluster built-in services, no need to deploy additional hardware and software.
- High availability, with no risk of single point of failure.

Core functions

- **Resource management:** Resource allocation and management, scaling of clusters and tenants, and so on.
- **Load balancing:** Deciding the distribution of Unit among multiple machines.
- **Disaster recovery:** Through automatic replication, migration and other means to ensure that the distribution and type of log streams are ultimately consistent with these user-specified.
- **Schema management:** Responsible for processing DDL requests and generating new Schema.



Load Balancing

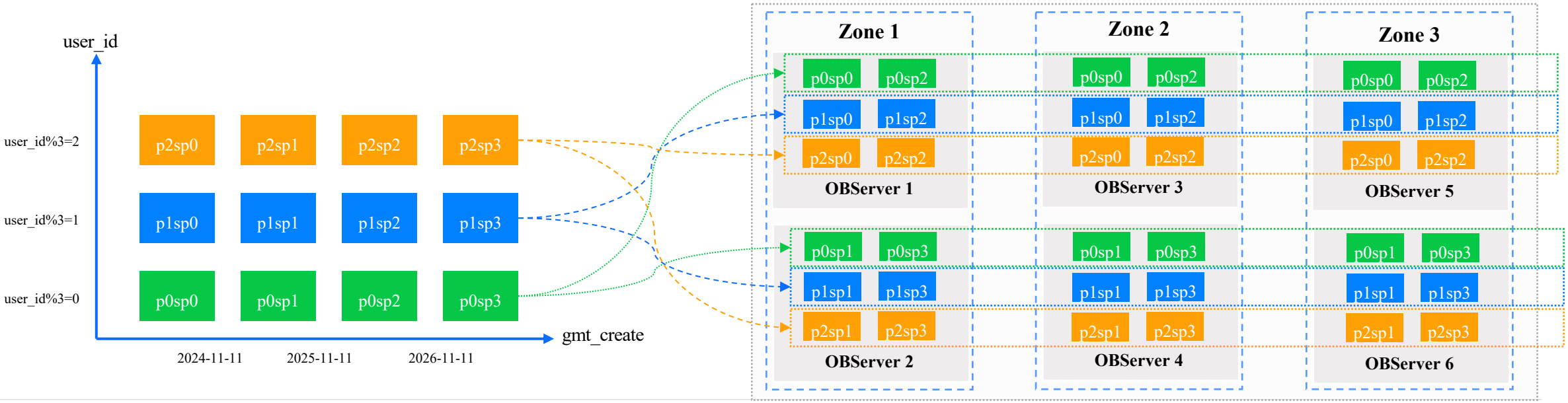
OceanBase database provides intelligent load balancing capability through RootService, which can balance the number of partitions and storage capacity on each service node, and balance the leader replicas among different zones under horizontal scaling out/in scenarios and dynamic data change.

Load balancing under horizontal scaling out/in scenarios

- When performing horizontal scaling out/in for tenants, RS will automatically perform load balancing operations according to the changes in the number of resource units of the tenant, so that the number of partitions and the amount of data carried in each resource unit reaches a balanced state.
- When the user modifies the Primary Zone settings of the tenant, the load balancing feature will adaptively adjust the distribution of log streams and partitions, achieving horizontal scaling out/in of the tenant's read-write service capabilities both within and across Zones.

Load balancing under dynamic data change scenarios

- When creating or deleting partitions within a tenant, RS will dynamically adjust the distribution of partitions to equalize the number of partitions and storage space among service nodes.
- After extended use, when the storage capacity or the number of partitions is no longer balanced across machines, the load balancing will periodically generate a balancing plan to rebalance data and services across multiple servers.



Cluster Architecture

02

- Basic Concepts of Cluster
- Routing and Load Balancing
- **High-Availability Deployment Mode**

Database Disaster Recovery Capability Indicator

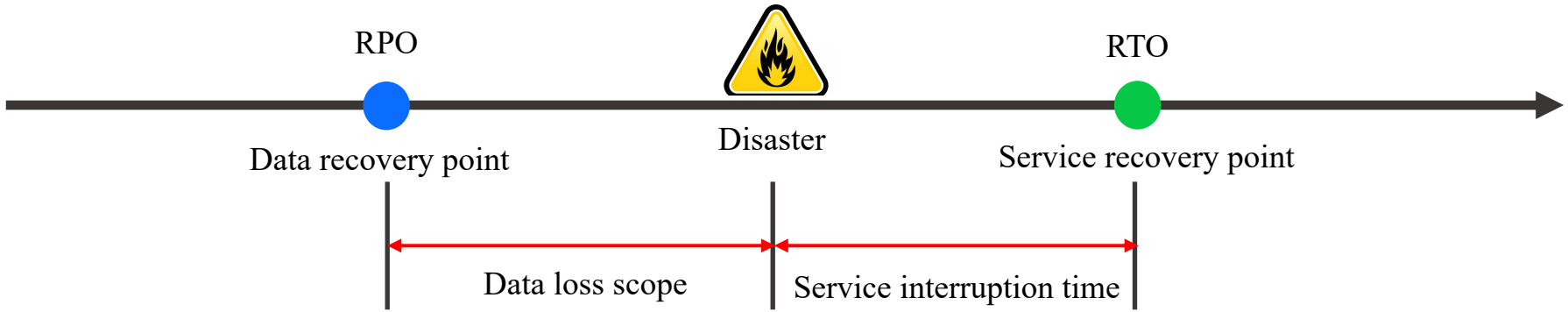
The disaster recovery capability of a database system is measured by the recovery point objective (RPO) and the recovery time objective (RTO).

RTO (Recovery Time Objective)

- **RTO refers to the recovery time objective and represents the reliability of the service.**
- RTO refers to the maximum acceptable period during which a database can remain inoperative following a failure or disaster. It is a critical threshold within which data must be restored to ensure the continuity of service.
- A shorter RTO implies that in the event of a failure, normal operations can be resumed more quickly, minimizing the disruption to services.

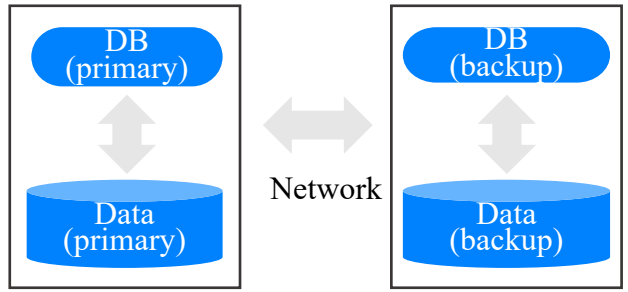
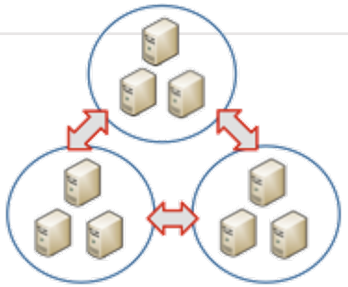
RPO (Recovery Point Objective)

- **RPO refers to the recovery point objective and represents the availability of the service.**
- RPO refers to the point in time to which data can be recovered in the event of a disaster or emergency. It represents the amount of data loss that a service system can tolerate.
- A smaller RPO implies that in the event of a failure, the closer the point at which data can be recovered and the smaller the scope of data loss.



Comparison of High Availability Solutions between Traditional Database and OceanBase

OceanBase meets the financial industry's Level 6 disaster recovery standard (the highest standard), with zero data loss: RPO=0,RTO< 8s.

	Technical implementation and features	Scenario: Backup fault	Scenario: Primary fault	System architecture
Traditional database	■ Primary database+ backup database, synchronize with redo-log between leader and follower replicas. Typically, this setup operates in “Maximum Availability Mode”, with RPO > 0 and RTO in minutes. ■ Relying on "high-end" equipment, such as mainframe or midrange computer, the system ensures high availability through the use of dedicated storage and RAID technology. ■ High hardware cost.	Under normal circumstances, the system operates in "maximum protection mode”. If a failure occurs with the backup database or the network, the Redo-Log of the primary database cannot be synchronized to the backup database, resulting in the application being unable to execute normally, which is unacceptable to users. In such cases, the system typically switches to "maximum performance mode", where the Redo-Log adopts an asynchronous synchronization mode, but at this point, RPO>0.	In the event of a failure for the primary database, the backup database cannot automatically switch to the primary database (to avoid a split-brain scenario where two primary databases might conflict). It is necessary to manually change the backup database to the primary database, taking at least half an hour or more, RTO>30 minutes.	
OceanBase	■ Multi-replica deployment based on Paxos protocol: Strong data consistency, continuous availability, and automatic switching between leader and follower replicas, RPO=0, RTO in seconds. ■ Utilizing a common PC server and its own hard disk, relying on software to ensure high availability. ■ Low hardware cost.	In the event of a failure on the machine hosting a minority of the Follower replicas, although the Leader replica cannot receive responses from these Follower replicas, it can still receive responses from other Follower replicas, thus maintaining the strong consistency of the majority, without affecting the services, RPO=0.	In the event of a failure of the machine hosting the Leader replica, the remaining replicas can automatically elect a new Leader replica. The newly elected Leader replica automatically assumes control of the services without the need for manual intervention, with RTO < 8 seconds.	

OceanBase Arbitration-based High Availability

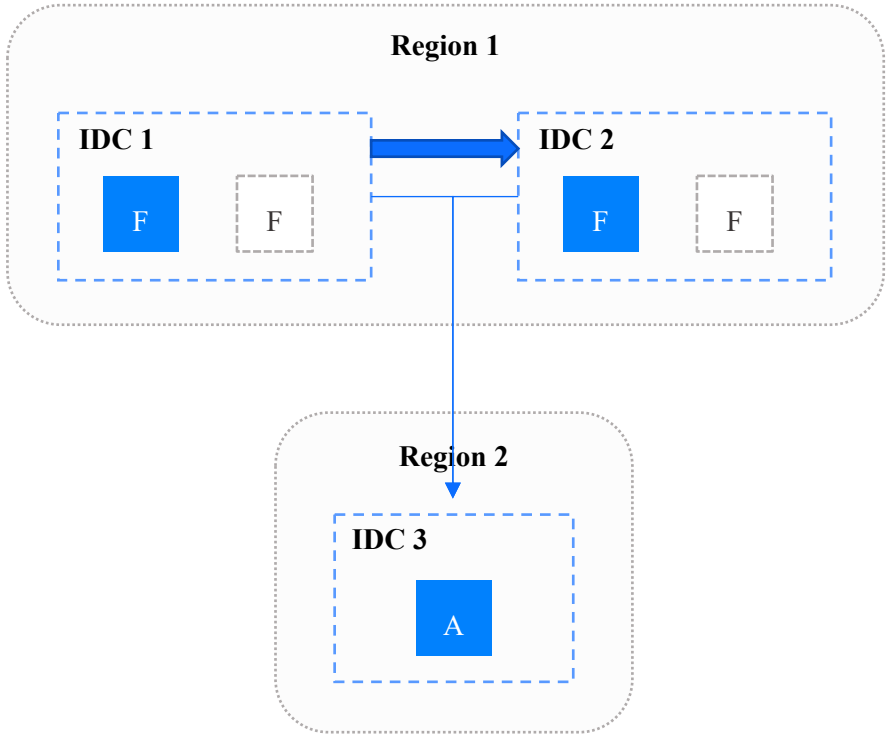
Arbitration typically refers to a method for parties to resolve disputes by selecting an independent and impartial third party (arbitration tribunal). Starting from version V4.1.0, OceanBase database supports Arbitration Service. The Arbitration Service is deployed independently from the OceanBase cluster and is a lightweight observer process started in a special mode, carrying arbitration members at the log stream level.

Characteristics of the arbitration service

- Participates only in the voting related to the Leader replica election, and does not participate in the processing of transaction logs.
- Does not store log and table data, with minimal resource (bandwidth/memory/disk/CPU) overhead.
- When a tenant enables arbitration service, RS will automatically add an arbitration replica for each log stream member.
- Cannot be elected as a Leader replica to provide read/write services.

Advantages of arbitration service

- Arbitration members can replace the previous log-based replica usage scenario, reducing deployment costs with almost the same availability.
- When half of the full-featured replicas fail, arbitration provides an automatic degradation service (removing the faulty node from the Paxos group), and the business response time (RT) is unaffected after the degradation (no need for cross-city log synchronization).
- Arbitration members do not need to synchronize logs, replay logs, and do not store Redo logs, baseline data, so the resource (CPU/memory/disk/bandwidth) overhead of arbitration service is significantly reduced.
- Compared to the primary and backup database solutions, the arbitration solution utilizes very low additional resource overhead to achieve lossless dual-active capability on two full-featured replicas, supports automatic upgrade and downgrade, with no data loss (RPO=0).



Automatic Leader Replica Switching in OceanBase Failure Scenarios

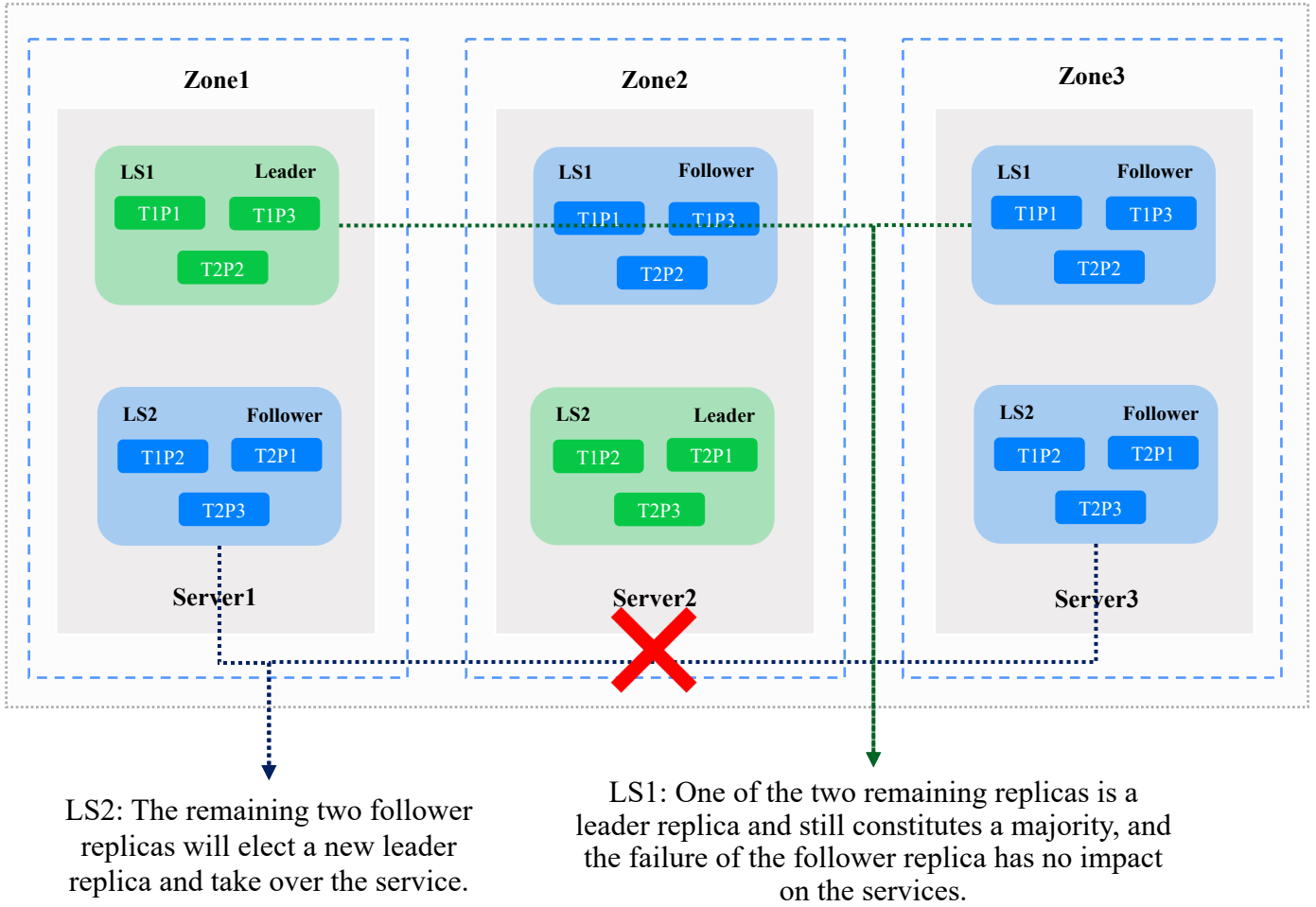
If less than half of the nodes fail, the election protocol ensures automatic recovery with RPO = 0. If more than half of the nodes fail, manual intervention is required, and the service can be pulled up by a single replica, which may lose the last part of the data since the minority may not contain the latest data.

Leader replica failure

- Assuming a failure occurs to Server2 in Zone2, its LS2 Leader replica cannot provide services normally.
- The LS2 Follower replica located in Zone1 and Zone3 will re-elect a new Leader replica and take over the services.
- When Server2 recovers, its LS2 replicas will first synchronize up the data and then continue to participate in multi-replica synchronization. In general, OceanBase load balancing function will switch the LS2 Leader replica from Zone1 to Zone2.
- The entire process is fully automated and requires no manual intervention.

Follower replica failure

- Assuming a failure occurs to Server2 in Zone2, its LS1 Follower replica cannot participate in replica synchronization and voting, but the remaining two LS1 replicas still fulfill the majority requirement.
- When Server2 recovers, its LS1 replicas first have to synchronize up the data and then continue to participate in multi-replica synchronization.
- Follower replicas are not designed to handle services, hence the failure of a Follower replica has no impact on services.
- The entire process is fully automated and requires no manual intervention.



LS2: The remaining two follower replicas will elect a new leader replica and take over the service.

LS1: One of the two remaining replicas is a leader replica and still constitutes a majority, and the failure of the follower replica has no impact on the services.

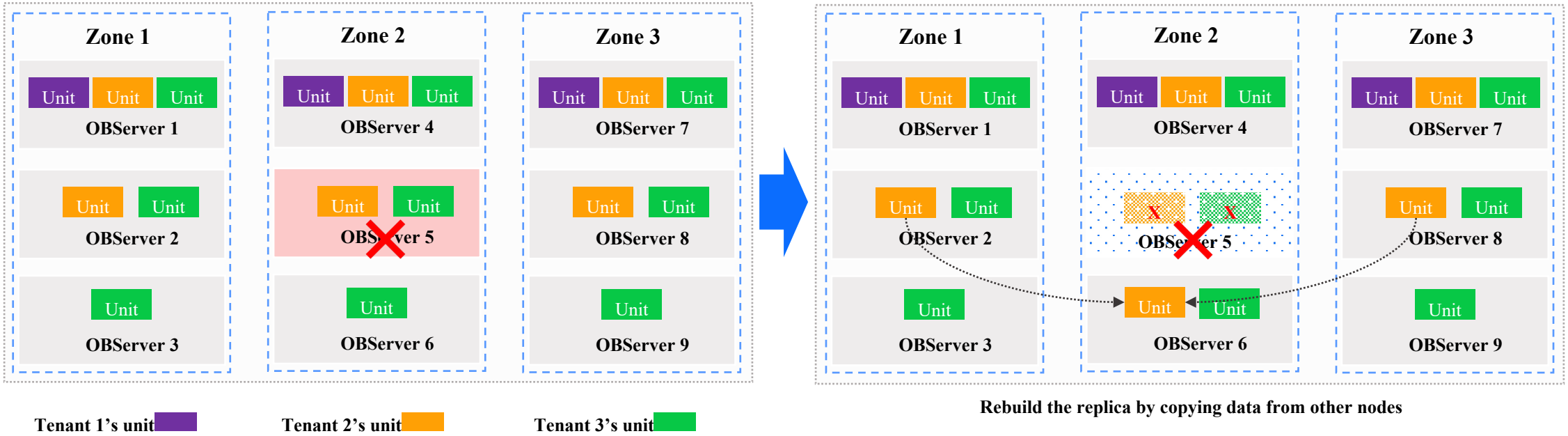
Automatic Re-Replication in OceanBase Failure Scenarios

When an OB Server node's abnormal duration exceeds the permanent offline time threshold (server_permanent_offline_time, with a default value of 1 hour), OceanBase will remove the replicas on the abnormal node from the Paxos group and automatically attempt to replenish replicas on other nodes that have sufficient resources within the same Zone, thereby replenish the high availability of the multi-replica system.

Assuming a 3-3-3 cluster (3 Zones, each with 3 OBServers) has 3 tenants: Unit Num=1 for Tenant 1, unit Num=2 for Tenant 2, and unit Num=3 for Tenant 3.

If OBServer 5 in Zone 2 fails and goes offline, OceanBase performs the following actions after the downtime exceeds the permanent offline time:

- **Permanent offline:** Perform a permanent offline process for OBServer 5, that is, remove the replicas on OBServer 5 from the Paxos group.
- **Assign Unit:** Attempt to allocate the 2 Units from OBServer 5 (for Tenant 2 and Tenant 3) onto OBServer 4 and OBServer 6. Since a single OBServer cannot be allocated two Units for the same tenant, it is not possible to allocate a Unit for Tenant 3 on OBServer 4 and OBServer 6. Ultimately, one Unit for Tenant 2 is allocated on OBServer 6.
- **Replenish replica:** Since it is not possible to allocate a Unit for Tenant 3, the operation to replenish replicas for Tenant 3 cannot be carried out. In the end, only Tenant 2 undergoes the replenishment replica.

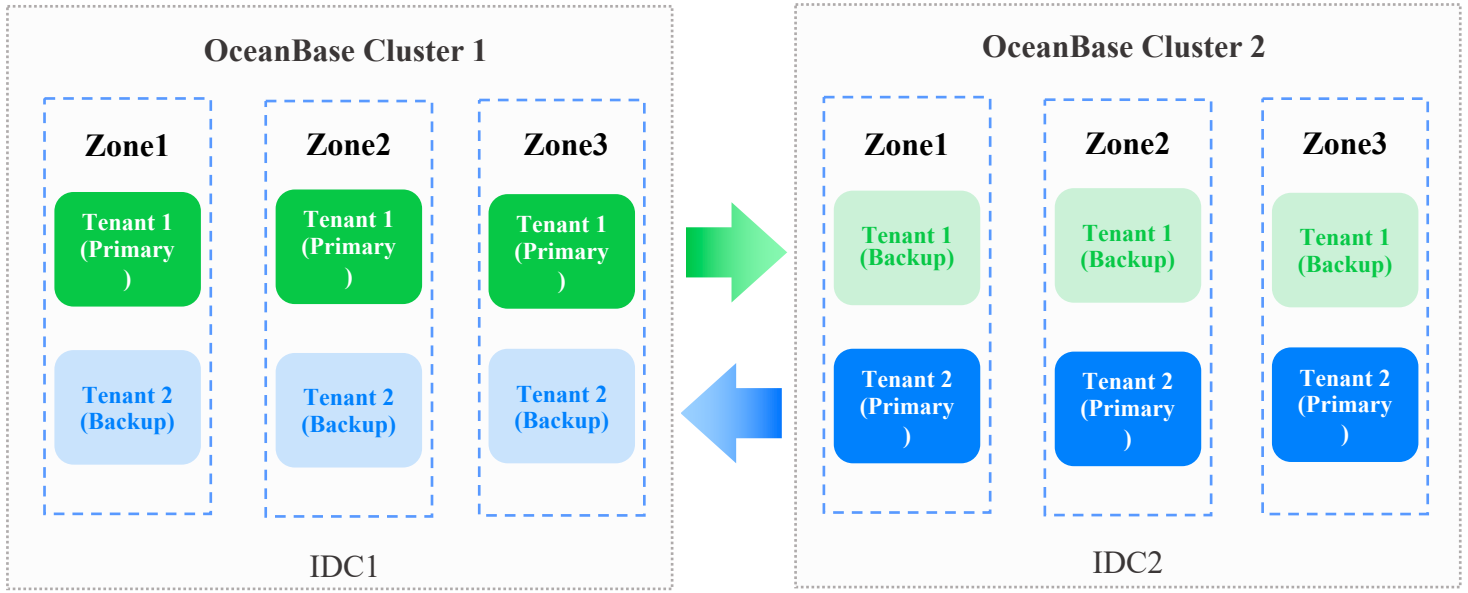


OceanBase Disaster Recovery Deployment: Primary/Standby Deployment of Two IDCs in the Same Region

Features: Deploy two sets of clusters in two local data centers that can provide external services, with the tenants in each cluster serving as the primary database and standby database for each other, achieving dual-activity and disaster recovery in the same city.

Disaster recovery scenarios

- For scenarios involving two data centers within a single city, the network latency between the data centers typically ranges from 0.5 to 2 ms.
- In each data center, an independent three-replica cluster is deployed, with the three replicas of each cluster forming an independent Paxos group. Tenants in both clusters serve as primary database and standby database for each other, both being capable of providing services externally.
- In the event of a minority failure to either cluster, the remaining replicas still constitute the majority and can still synchronize the Redo-Log, ensuring RPO = 0.
- In case of a majority failure in any cluster, another cluster can take over the service through a primary-standby database switchover.



Intra-city dual-IDC

Disaster recovery capabilities

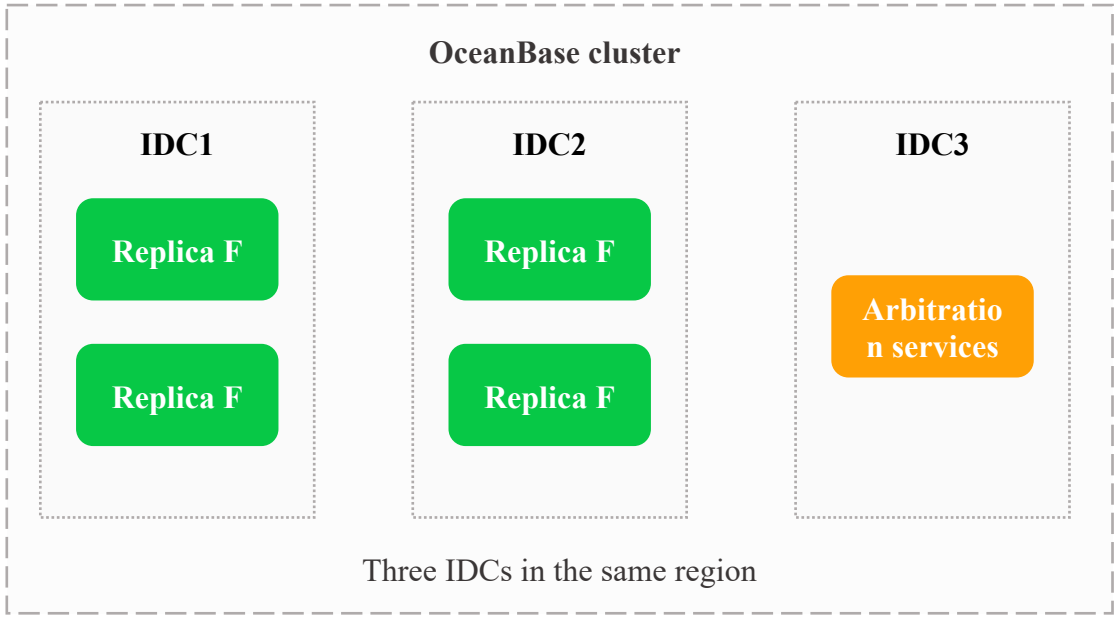
- Capable of safeguarding against minority node failures.
- Use intra-city standby database to prevent single data center failure.
- Unable to protect against city-level failures.

OceanBase Disaster Recovery Deployment: Three IDCs in the Same Region with the Arbitration Service

Features: Utilize intra-city three-data centers to build a cross-data center high-availability capability. Two data-centers provide business services (intra-city dual-activity), while the third data-center is equipped with arbitration services to reduce costs.

Disaster recovery scenarios

- For scenarios involving three data centers within a single city, the network latency between the data centers typically ranges from 0.5 to 2 ms.
- Three intra-city data centers form a cluster, where two of the data centers respectively deploy two full-featured replicas, and the third data center hosts the arbitration service.
- In the event of a minority failure to the cluster, the remaining replicas still constitute the majority and can still synchronize the Redo-Log, ensuring RPO = 0.
- In the event of a data center-level failure in either of the two primary data centers, the arbitration service will automatically perform a replica downgrade operation (5->3), allowing the two replicas in the other primary data center to meet the majority requirement and continue to provide services externally.



Disaster recovery capabilities

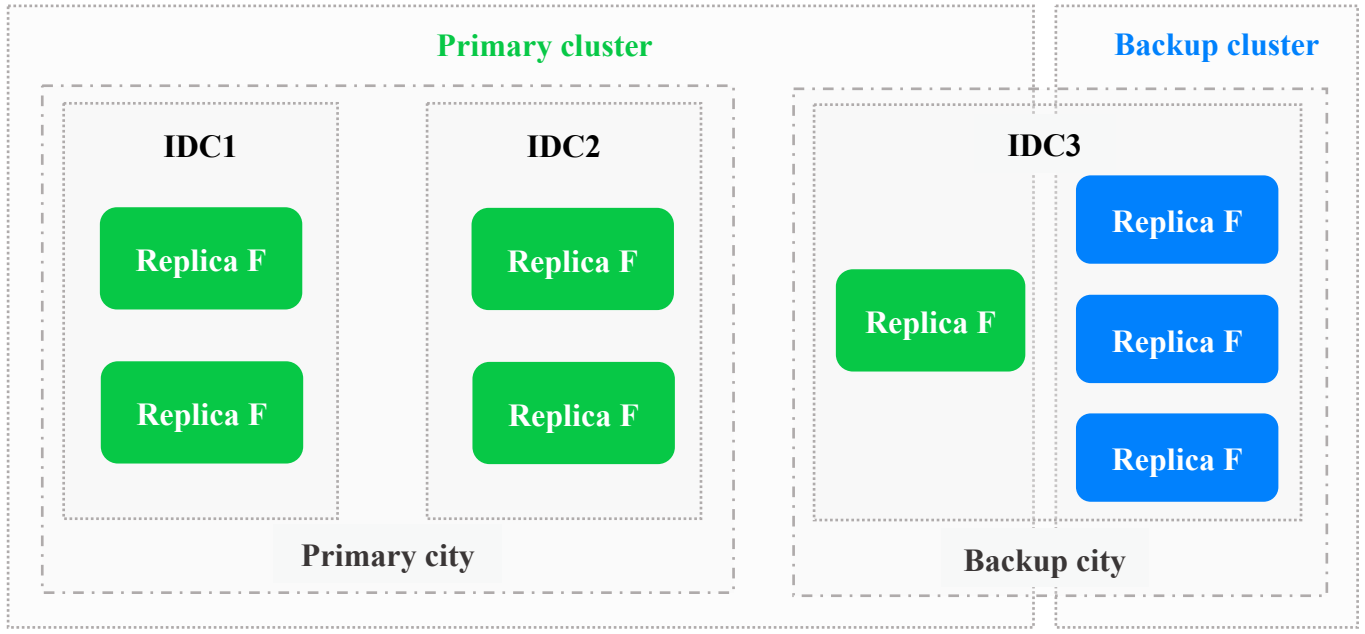
- Capable of safeguarding against minority node failures.
- Be capable of protecting against single data-center failures.
- Unable to protect against city-level failures.

OceanBase Disaster Recovery Deployment: Primary/Standby Database Solutions of Three IDCs across Two Regions

Features: Build cross-city high availability capability. Two IDCs in one city serve as the primary IDC, jointly providing services; one IDC in another city acts as the backup IDC, hosting the fifth replica and backup cluster, offering high availability with an RPO ≈ 0.

Disaster recovery scenarios

- This setup is tailored for situations where there are two data centers in the primary city and one in the backup city.
- The primary city and the backup city form a 5-replica cluster, with 4 replicas in the primary city distributed among two IDCs and 1 replica in the backup city. If any IDC fails, at most 2 replicas will be lost and the remaining 3 replicas will still satisfy the majority requirement.
- The backup city simultaneously deploys a backup cluster, which synchronizes data with the primary cluster using an asynchronous synchronization mode. In the event of a city-level failure in the primary city, it is able to switch to the backup cluster to continue to provide services (RPO = 0 is not possible).



Disaster recovery capabilities

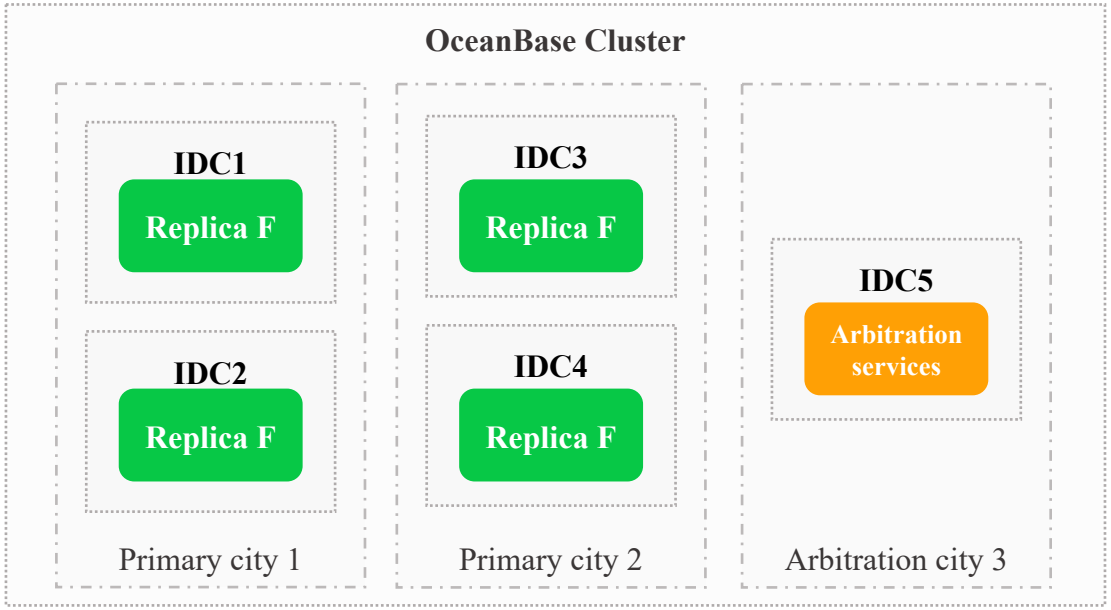
- Capable of safeguarding against minority node failures.
- Be capable of protecting against single data-center failures.
- Use an off-site standby database to protect against failures in the primary city.

OceanBase Disaster Recovery Deployment: Five IDCs Across Three Regions with the Arbitration Service

Features: Building cross-city ultra-high availability capability. (RPO = 0, RTO < 8 seconds). Four data-centers in two primary cities are the primary IDC to jointly provide services; the fifth IDC in another city is deployed for arbitration services, reducing hardware costs.

Disaster recovery scenarios

- Tailored for scenarios requiring city-level disaster recovery capabilities.
- Three cities form a 4F-1A cluster. Each of the two primary cities has two full-featured replicas, and the third city is deployed for arbitration services.
- In the event of a minority failure to the cluster, the remaining replicas still constitute the majority and can still synchronize the Redo-Log, ensuring RPO = 0.
- In the event of a city-level failure in either of the two primary cities, the arbitration service will automatically perform a replica downgrade operation (5->3), allowing the two replicas in another city to meet the majority requirement and continue to provide services externally.



Disaster recovery capabilities

- Safeguarding against minority node failures.
- Protecting against single data-center failures.
- Protecting against minority city failures.

Summary

Cluster Architecture and Distributed Concepts

- **Availability zone:** Each Zone has one and only one complete copy, multiple Zones work together to achieve distributed high availability.
- **Tenants:** Tenant is an independent database instance, serving as the unit of resource allocation.
- **Resource unit:** It is a container for tenants, which can be regarded as a lightweight virtual machine, detailing the available resources for tenants on OBServer.
- **Resource pool:** The resources allocated to a specific tenant, composed of several Units.
- **Log stream:** Represents a collection of data, including several tablets and an ordered Redo log streams.
- **Replica:** Copies of the data from the same log stream across different machines are known as replicas.
- **Paxos**: The distributed consistency protocol on which OceanBase distributed database multi-replica synchronization depends.
- **RootService:** Root service, mainly providing resource management, disaster recovery, load balancing, schema management, and so on.
- **Arbitration service:** Only involving in special services of the distributed election.

Summary

Distributed access routing and load balancing

- **ODP:** OceanBase Database Proxy, also known as OBProxy, is a dedicated proxy service for OceanBase database.
- **Remote access and distributed access:** SQL needs to access data stored on other nodes, and OBServer automatically forwards access to remote data to the corresponding node.
- **Load balancing:** OceanBase database provides intelligent load balancing capabilities such as horizontal scaling out/in and dynamic data balancing, in order to achieve the balance of the number of partitions and storage capacity on each service node, as well as the balance of Leader replicas among different Zones.
- **Primary Zone:** The Zone that hosts the Leader replica of the log stream for a tenant.
- **Tablegroup:** It can control the physical storage adjacency of a group of tables/partitions, allowing tables and partitions within the tablegroup to be moderately aggregated according to certain rules.

Summary

High-Availability Deployment Mode

- **RTO:** Recovery time objective, representing the reliability of the service.
- **RPO:** Recovery point objective, representing the availability of a service.
- **OceanBase high availability capability:** Meet the financial industry's Level 6 disaster recovery standard (the highest standard), with zero data loss: RPO=0,RTO< 8s.
- Typical high-availability deployment mode:
  - Primary/standby deployment of two IDCs in the same region
  - Three IDCs in the same region with four replicas + arbitration scheme
  - Three IDCs across two regions, five replicas + standby database solution
  - Five IDCs across three regions, four replicas + arbitration solution

Self-test

1. [True/false] Zone is a logical concept, within which all OBServers collectively host a complete replica. ( )
2. [True/false] A zone may contain different cities, different data centers within the same city, or different racks within the same data center. ( )
3. [True/false] OceanBase V4 performs data management and log synchronization among multiple replicas on a partition basis. ( )
4. [True/false] From the application perspective, a tenant is a database instance; from the OceanBase perspective, a tenant is a container of resources and data objects. Each tenant exclusively occupies a certain amount of resources, and different tenants are isolated from each other in terms of resources and data. ( )
5. [True/false] In OceanBase V4, Leader replicas of different partitions in the same log stream can be distributed on different OBServers. ( )
6. [Single answer] If a cluster has three Zones, and each Zone has five OBServers. How many full-featured replicas can a single log stream have at most? ( )

   A. 3　　B. 5　　C. 8　　D. 15

7. [Single answer] If a cluster has 3 Zones, each Zone has 5 OBServers, and one of the tenant's resource pool has Unit Num=3, how many servers in the cluster have that tenant's resource units? ( )

   A. 3　　B. 5　　C. 9　　D. 15

8. [Single answer] What type of resource isolation between tenants cannot OceanBase achieve? ( )

   A. CPU　　B. Memory　　C. IOPS　　D. data disks

Self-test

9. [Single answer] RootService is the cluster-level service of OceanBase, which of the following is not a function of RootService? ( )

A. Resource management B. Routing management C. Load balancing D. Schema management

10. [Single answer] Which of the following statements about the arbitration service is incorrect? ( )

A. Participate only in election-related voting and does not participate in the synchronization of transaction logs

B. No MemTable or SSTable, and the resource overhead is minimal.

C. Enabling the arbitration service will add an arbitration replica, which can be elected as the Leader replica to provide services.

D. When half of the full-featured replicas fail, the arbitration provides automatic downgrade services (removing the failed nodes from the Paxos group).

11. [Multiple answers] Regarding the log stream replicas of OceanBase V4, which of the following statements is correct? ( )

A. There are generally two types of log stream replicas: full-featured replicas and read-only replicas.
B. A full-featured replica can be read and written, while a read-only replica can only be read, and both types of replicas support strong-consistency reads.
C. The Leader replica is both readable and writable, serving as the full-featured replica; whereas the Follower replica is read-only, serving as the read-only replica.
D. In a 5-replica cluster, transaction logs need to be strongly synchronized with only 3 replicas, not all 5.

12. [Multiple answers] What is the correct statement about the load balancing mechanism of OceanBase V4? ( )

A. OceanBase offers methods for manual intervention and load balancing, and users can specify the OBServer node for storing a Database, Table, or Partition during their creation.
B. Users can use Tablegroup to influence load balancing, so that Tables and Partitions with business relationships have affinity in their physical distribution, thereby reducing the overhead of distributed access.
C. OceanBase has automatic load balancing capabilities to evenly distribute Tables and Partitions across multiple OBServer nodes within a Zone.
D. OceanBase has automatic load balancing capability, which can evenly distribute Leader replicas to multiple specified Primary Zones.

Contents

I. Product Overview

II. Cluster Architecture

III. Database Engines

IV. Installation and Deployment

V. Application Development

VI. Operations and Maintenance Management

Database Engines

03

- **SQL Engine**
- Storage Engine
- Transaction Engine

OceanBase SQL Engine

SQL

SQL (Structured Query Language) is the standard computer language used for accessing and working with the databases. It primarily offers the following functions:

- **Data definition:** DDL (Data Definition Language), used to create and modify the database structure. For instance, CREATE TABLE is used for creating new tables, ALTER TABLE for modifying the structure of the table, and DROP TABLE for deleting the table.
- **Data query:** Query, which retrieves specific data from a database via a SELECT statement.
- **Data operation:** DML (Data Manipulation Language), used for adding, modifying or deleting records in a database, such as INSERT, UPDATE and DELETE statements.
- **Data control:** DCL (Data Control Language), used to control the user's access to database objects, thus achieving data security management. For instance, GRANT is used for authorization, and REVOKE for the recovery of privileges.
- **Transaction management:** TCL (Transaction Control Language), used to control the execution of transactions and ensure data consistency and integrity. Users can use the BEGIN, COMMIT, and ROLLBACK statements to manage transactions.

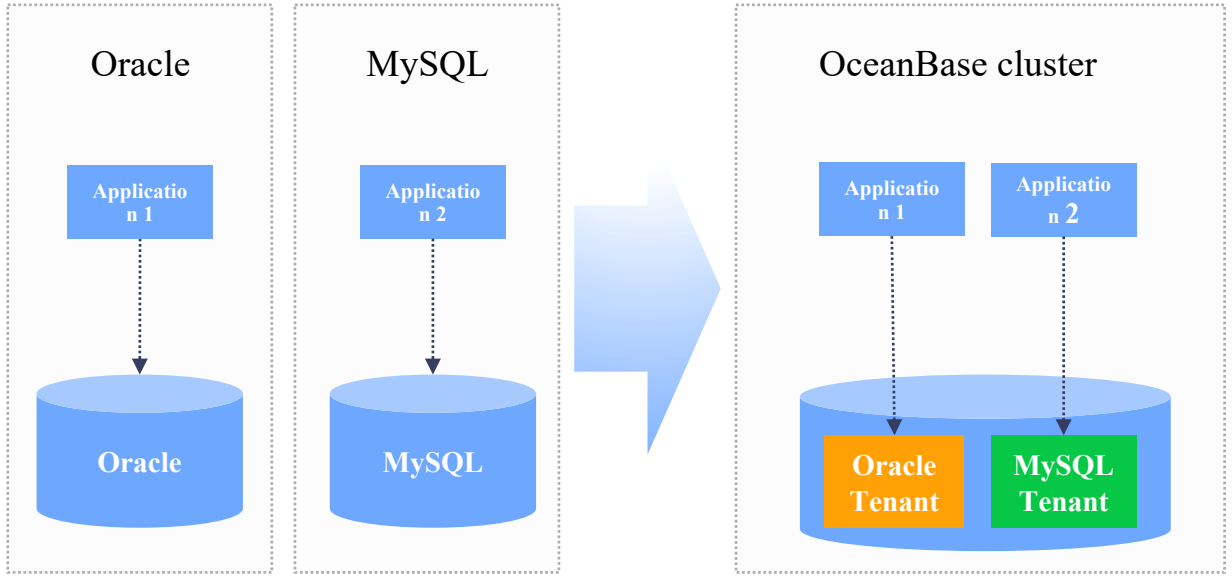
SQL engine

The SQL engine is one of the core components of a database, which is responsible for handling SQL queries and data manipulation. OceanBase SQL engine has the following features:

- **High compatibility:** The OceanBase SQL engine supports standard SQL syntax, foreign key constraints, view column information, DML triggers, self-incrementing columns and SEQUENCE, and stored procedures, achieving a high degree of compatibility with MySQL and Oracle databases.
- **High concurrency and high performance:** OceanBase SQL engine is designed for highly concurrent distributed processing, which supports large-scale data processing and transaction processing. Offering high-performance SQL processing, it is the world's only distributed database that has set world records in TPC-C and TPC-H tests.
- **Data consistency:** Coupled with OceanBase's memory management and distributed architecture, the SQL engine is able to efficiently handle complex transactional and analytical queries while ensuring data consistency and integrity.
- **Online DDL:** The OceanBase SQL engine also has the online DDL capability, which allows table structure to be changed without stopping the service, thus enhancing the availability and flexibility of the system.

Compatibility

OceanBase database can support tenants of both MySQL schema and Oracle schema in one system. When creating a tenant, you can choose to create one in MySQL compatibility mode or Oracle compatibility mode. The compatibility mode of the tenant cannot be changed once it is determined, and all the data types, SQL functions, views, etc. are consistent with the MySQL database or Oracle database.



One cluster, two compatibility modes

- One cluster, supporting both MySQL and Oracle modes.
- Tenants need to be configured in MySQL compatibility mode or Oracle compatibility mode when being created.
- The DBA's work changes from maintaining multiple database products to maintaining a unified database product. DBA can create tenants in different compatibility modes based on application requirements.

OceanBase Database Community Edition offers only the MySQL schema.

DDL and Database Object

Data Definition Language (DDL) statements are used to define, change, and delete database objects.

DDL

The DDL statements enable you to do the following:

- Create, change, and delete Schema objects and other database object structures, including the database itself and database users. Most DDL statements begin with the such keywords as CREATE, ALTER, or DROP.
- Delete all data in Schema objects without deleting the structure of those objects (non-TRUNCATE command).
- Turn on and off the auditing options (keywords: AUDIT and NOAUDIT).
- Add comments to database objects.

Database objects

The database objects in different compatibility modes are slightly different:

- Database objects in MySQL schema mainly include: tables, views, indexes, partitions, sequences, triggers, and storage procedures.
- Database objects in Oracle schema mainly include: tables, views, indexes, partitions, sequences, synonyms, triggers, and various PL objects (PL subprograms and packages).

DATABASE/SCHEMA

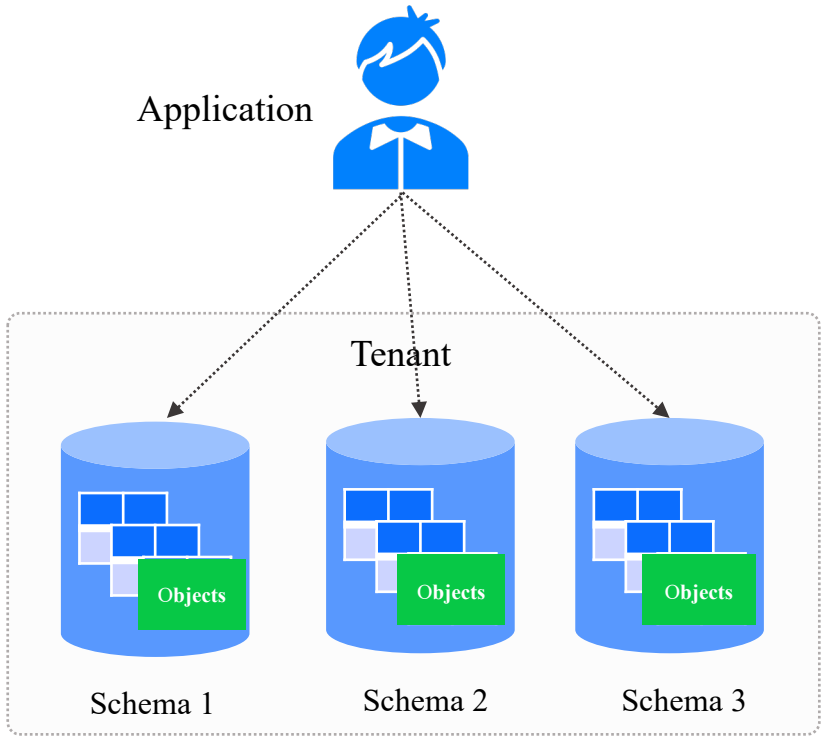
In Database, Database or Schema is a logical concept, i.e., a collection of Database Objects that is used for privilege management and namespace isolation.

MySQL schema: Database

In MySQL schema, Database is a collection of Database Objects that is used for privilege management and namespace isolation. Schema is a synonym of Database. In SQL, the keyword Schema can substitute the keyword Database. For instance, CREATE SCHEMA is used to substitute CREATE DATABASE.

Oracle schema: Schema

In Oracle schema: Schema is a collection of Database Objects that is used for privilege management and namespace isolation. In Oracle schema, Schema can only be created by creating users. Users are created with a default Schema with the Schema name identical to the username.



TABLE

A database table is a collection of two-dimensional arrays used to represent and store relationships between data objects. In a database, a table is the basic unit of data organization, which consists of vertical columns and horizontal rows.

Row, column and primary key

Column

In a database, a Column is used to record the value of the field of a certain attribute in a table. The name the user gives to an attribute is the column name. In addition to the column name, the column has such information as the data type and the maximum length (precision) of the data type.

Row

In a database, a Row represents a collection of data from all the columns in a single record in the table. In a nutshell, a database can be regarded consisting of columns and rows. Each row in the table, of an identical structure, represents a set of related data.

Primary key

A primary key is a collection of columns that uniquely identifies a row in a data table. The primary key needs to meet the following rules:

- The value cannot be NULL or an empty string
- Unique values for a table-wide collection of primary key columns

Take a data table related to student information as an example. Each row represents a student, and its corresponding column may represent such information as the student number, name, gender, and age. Since the student number is non-empty and unique in the table, it is the primary key column.

Table students

S_ID	Name	Sex	Age
2023010101	TOM	M	20
2022020202	AMY	F	21
2023030103	BETTY	F	20
2024040303	JACK	M	19

Row

Primary key column Ordinary column

Design Database Tables with ER Model

The ER Model, or Entity-Relationship Model, is a conceptual data model for designing databases that describes information systems in the real world through three main elements: entities, attributes, and relationships.

ER model

The ER model is a typical approach to requirements analysis and conceptual design. The ER model abstracts the real-world information into entities, attributes, and relationships:

- **Entity:** An entity represents an individual object that can be distinguished in the real world, usually corresponding to a table in a database. For example, in a school information system, students, courses and teachers can be considered as entities.
- **Attribute:** The attribute is a characteristic of an entity that delivers basic information about the entity and corresponds to a column (field) in a database table. For example, a student entity might contain attributes such as the name, student number, and date of birth.
- **Relationship:** Relationships define the way in which entities are related to each other, and reflect the associations between entities. There may be 1-to-1, 1-to-many or many-to-many relationships. For example, there is a course selection relationship between a student and a course, where a student can take more than one course, and a course can be selected by more than one student.

Concept

Foreign key: Used to maintain the referential integrity between two tables, thus ensuring that the values of the data columns in the child table match the values of the keys that already exist in the parent table.

Let's use a school information system as an easy example to show the process of creating a table using the ER model. This system needs to manage students, courses and the relationship between them in terms of course selection.

- Table students: Student Number (S_ID, Primary Key), Name (S_Name), Sex (Sex), and Age (Age).
- Table courses: Course Number (C_ID , Primary Key), Course Name (C_Name), and Teacher (Teacher).
- Table enrollment: Course Number (C_ID), and Student Number (S_ID). Table Enrollment defines the associations between the student and the course.

```
Table 1: Students
+------------+------+-----+------+
| S_ID       |S_Name| Sex | Age  |
+------------+------+-----+------+
| 2023010101 | TOM  | M   | 20   |
| 2022020202 | AMY  | F   | 21   |
| ...        | ...  |     | ...  |
+------------+------+-----+------+
Primary Key: S_ID
```

```
Table 2: Courses
+-----------+----------+---------+
| C_ID      | C_Name   | Teacher |
+-----------+----------+---------+
| CS101     | Math     | Glen    |
| EE202     | Music    | Mat     |
| ...       | ...      | ...     |
+-----------+----------+---------+
Primary Key: C_ID
```

```
Table 3: Enrollment
+------------+-----------+
| S_ID       | C_ID      |
+------------+-----------+
| 2023010101 | CS101     |
| 2023010101 | EE202     |
| 2022020202 | CS101     |
| ...        | ...       |
+------------+-----------+
Primary Key: (S_ID, C_ID)
Foreign Key 1: S_ID ->
Student.S_ID
Foreign Key 2: C_ID ->
Course.C_ID
```

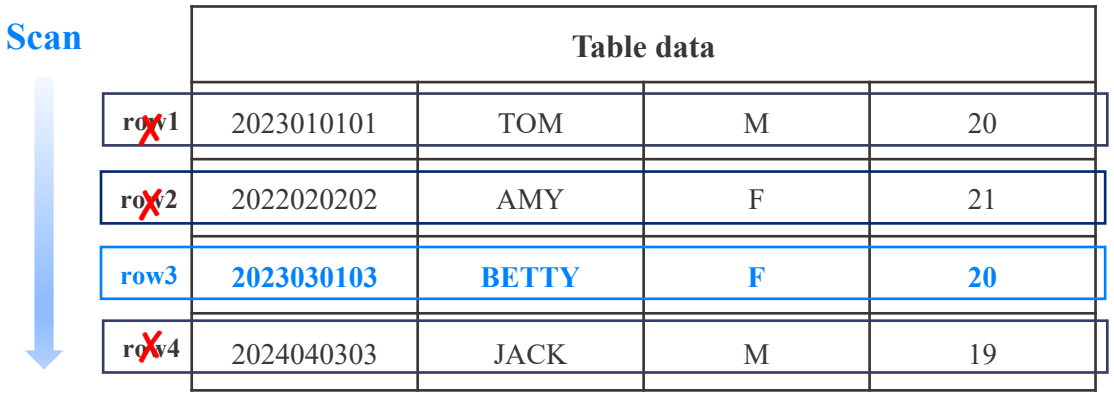
TABLE FULL SCAN

TABLE FULL SCAN is a data retrieval operation that involves scanning an entire table (or partition) for records that meet specific criteria. When the query does not utilize any indexes, it is common to execute the Table Full Scan.

TABLE FULL SCAN

- When executing an SQL query, the database system performs Table Full Scan if the query condition does not use the table's indexes, or if the query needs to return all the records in the table.
- Table Full Scan involves reading each row of data in a table and checking if each row satisfies the query conditions. If the condition is met, the row is returned as the query result.
- Table Full Scan usually is not the most efficient way to retrieve data, especially when working with large tables. Because it needs to read and check every row of data in the table, this can lead to higher I/O overhead and longer query time.
- In contrast, it is often more efficient to retrieve data using index retrieval if the query can utilize an index.

For example, to query for information about a student named BETTY, if you use the Table Full Scan, you need to scan every record in the table to find all student records with the name BETTY.

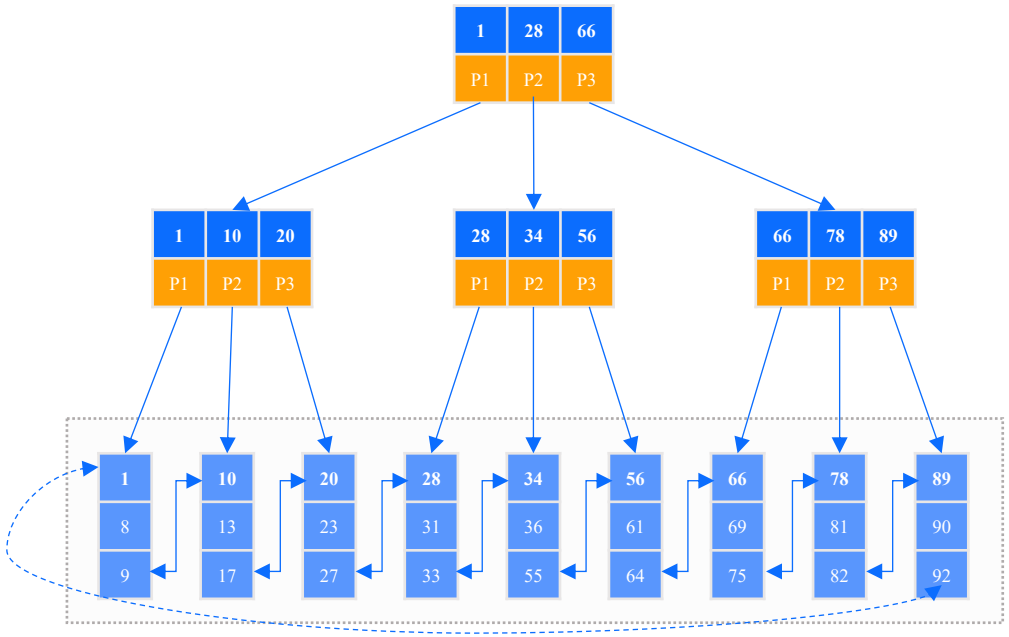


INDEX

Concept of indexed tables

An index is an ordered data structure, similar to a book's table of contents, that allows specific rows in a table to be quickly located without scanning the entire table.

- **What is a database index?**
  - Index is a data structure used to quickly retrieve data in databases. OceanBase uses B+ tree structure to organize indexes.
  - An index is similar to a book's table of contents, which helps the database system quickly locate the data it needs instead of scanning the entire table.
- **Function of index**
  - Raise the querying speed: With indexes, the database can quickly locate the target data, thus greatly reducing the query time.
  - Reduce disk IO: Indexes typically store fewer columns, thus saving query IO.
  - Improve sorting and grouping efficiency: For data that needs to be sorted or grouped, the operation will be more efficient if a suitable index exists.
- **Index types supported in OceanBase**
  - Global index: A global index is an index created for the entire table that contains data from all partitions of the table. Global indexes are suitable for scenarios where queries are performed across partitions.
  - Local index: A local index is an index created in each partition that contains only the data for that partition. Local indexes are suitable for scenarios where queries are performed inside partitions.
  - Spatial index: It is a special type of index for accelerating the querying of spatial data, mainly used in the geographic information systems (GIS) and spatial data processing field.



Tree structure of the index

TABLE RANGE SCAN

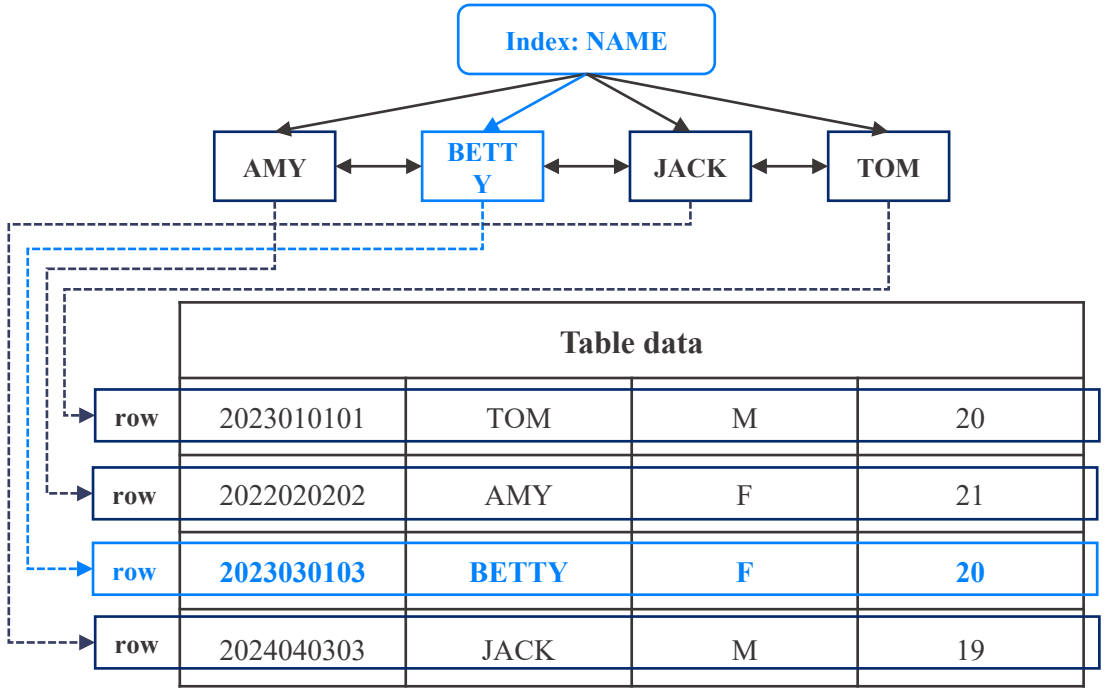
The OceanBase database is accessed in a similar way to traditional databases, either by using an index to find an exact record, or by using an index to find multiple records within a range.

In an OceanBase database, tables use the storage structure of the Index Organized Table (IOT). Tables resemble indexes in terms of storage. In OceanBase, querying or scanning with indexes is collectively referred to as **TABLE RANGE SCAN**, with the same index access with other databases.

- For equivalent queries on index keys, OceanBase can quickly locate a specific position in the index.
- When the query condition on an index key contains a range, such as BETWEEN, >, and <, the optimizer will scan the corresponding range of the index and filter the eligible records from it.

Example: Query the information about a student named BETTY:

```
SELECT *
FROM students
WHERE name = 'BETTY';
```



Matching Rules for Indexes

Because of the data structure characteristics of the B+ tree index, the combined key values of the index are ordered. Matching queries using the index needs to follow the following principles.

Principles of index matching and scanning

- **Left-most matching principle:** For composite indexes (i.e., indexes that contain multiple fields), the fields in the query condition must match in the order of the index fields. Since the B+ tree is sorted first by the leftmost key, subsequent keys can participate in the match only if the leftmost key is an equivalent query. If you skip the leftmost key and query based on the subsequent keys, the index will not be utilized.
- **Match range values:** Indexes are available with range queries, i.e., fields in a query condition can match a range of values. In this case, the index queries the records that satisfy the conditions in order, thus improving querying efficiency.
- **Prefix of the matching column:** Indexes can also work if the prefix of the column can be converted into a range. (For example, create an index on character-type field C1 with query condition "C1 LIKE 'ABC%' ")
- **Avoid sorting:** If the query statement needs to be sorted, and the sorting order is the same as the index key, you can use an index to avoid sorting. (For example, if an index is created on fields C1 and C2, the sorting statement "ORDER BY C1,C2" can use the index to avoid sorting.)

For example: A combined index is created on fields C1, C2, and C3 in order, and indexes matched with the query statement are shown in the following table:

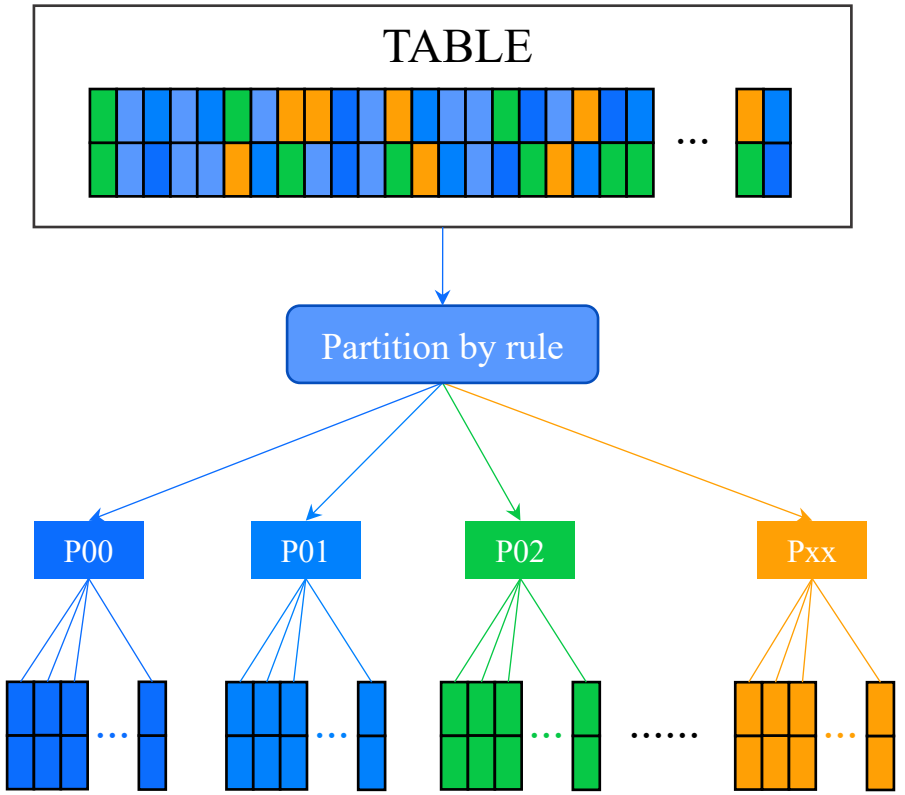
Query condition	Index matching conditions
C1=? AND C2=? AND C3=?	C1=? AND C2=? AND C3=?
C1=? AND C3=?	C1=?
C1>? AND C2=? AND C3=?	C1>?
C2=? AND C3=?	None (Note 1)

Note 1: In some cases, the query condition of the right column may be used for matching even though the equivalent condition of the left column of the index is not specified.

PARTITION

Partitioning is the process of breaking a table into smaller, more manageable parts based on certain rules. Each partition is an independent object with its own name and optional storage characteristics. In OceanBase distributed database, different partitions of one table can be distributed on different server nodes.

- **Partitions are transparent to the application**
  - Each partition is a separate object that can handle access alone or as part of a table.
  - From the application's perspective, there is only one Schema object. No SQL statement modifications are required to access the partition table.
- **Benefits of using partitions**
  - Manage objects more easily: DDL statements can operate on partitions rather than entire tables or indexes. For example, you can TRUNCATE a partition to avoid a large amount of data being DELETE.
  - Better load balancing: Different partitions can be stored in different nodes. OceanBase automatically distributes the different partitions of a table evenly across different service nodes, thus evenly distributing the table data and access to the table across the entire cluster.
  - Reduce resource contention of trading system: Applications can batch process data by partition range to avoid lock conflicts and contention for other database resources between concurrent batch processes.
  - Enhance query performance of the warehouse system: In AP scenarios, partitioning can speed up the processing of immediate queries. The partition key has an inherent filtering function. Take querying sales data for a quarter as an example. When sales data is partitioned by the time of sale, querying just one partition or several partitions, not the entire table, is enough.



Query Data by Partition

OceanBase database supports querying data by partition, allowing users to directly specify the desired partitions in SQL statements. The database optimizer can also calculate the partitions to access based on the query range of the partition key in the WHERE condition and perform corresponding **partition pruning**, (excluding non-conforming partitions from the scanning range). The partition SQL eventually accesses is the intersection of both the specified partition and the partition pruning.

Example

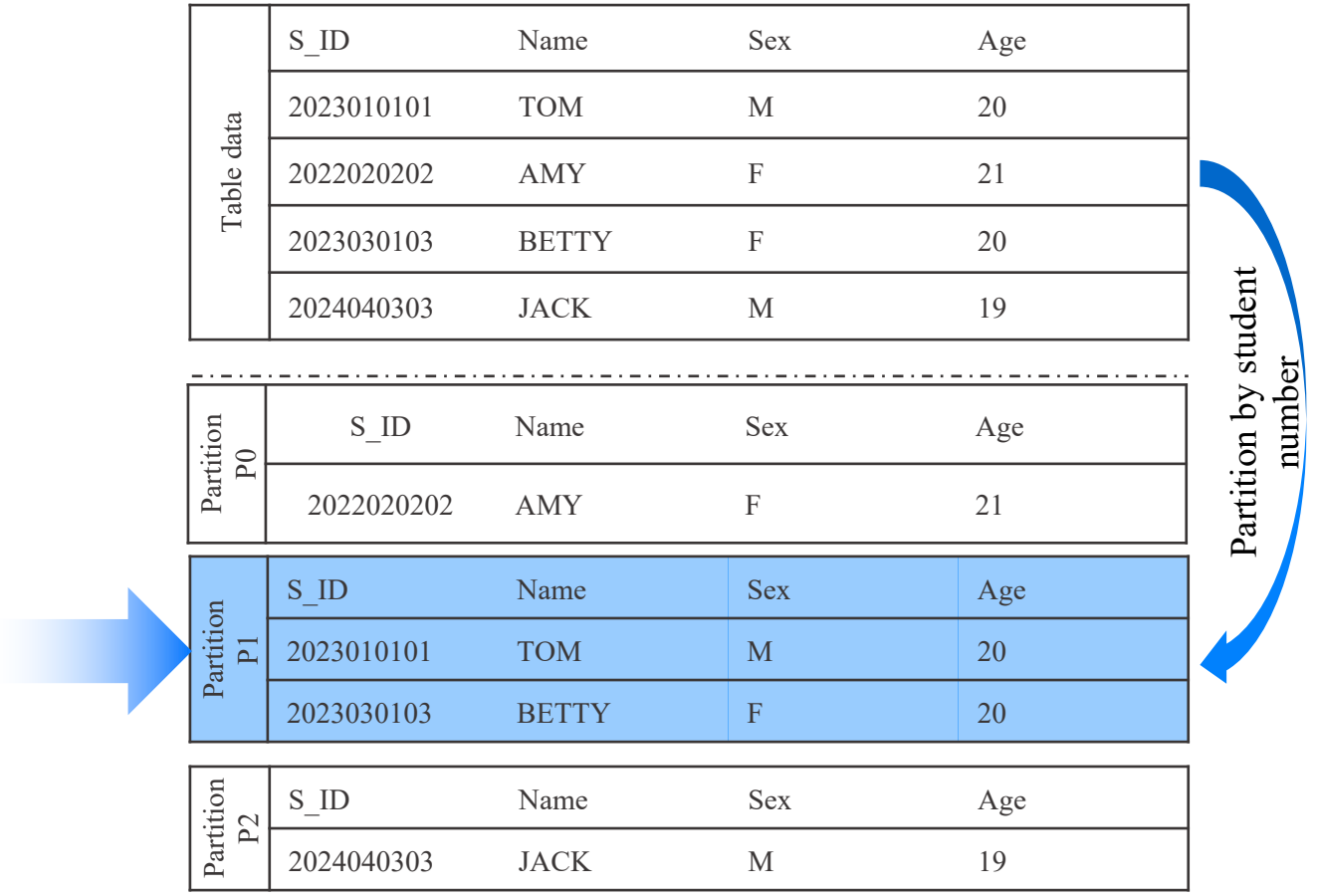
- Partition by student number:

Partition	Student number range
P0	[,2022999999)
P1	[2022999999, 2023999999)
P2	[2023999999, 2024999999)

- Query all data in Partition P1, in one of the two equal ways below

```
#Specify partitions directly in SQL#
SELECT * FROM students PRTITION(P1);

#Perform partition pruning through the database#
SELECT * FROM students;
WHERE s_id>2022999999 AND s_id<= 2023999999;
```



Create an Index on a Partition Table

In the OceanBase distributed database, the data of the Partition Table is divided into several partitions. Each partition is an independent data structure, and different partitions may be distributed on different service nodes. Thus, the single data structure on which the index relies no longer exists. Then, how does the index cope with it? - This introduces the concepts of "local index" and "global index".

Local index

Local indexes, also known as partitioned indexes, are ones created for data on a single partition; thus, the index key values of local indexes are in one-to-one correspondence with the data in the table.

- When creating indexes for Partition Table, specifying LOCAL keyword means creating a Local Index.
- A partition on a local index must correspond to a table partition; the number of partitions on the local index is equal to the number of table partitions.
- The partition key of a local index is equivalent to the partition key of a table, sharing the same partitioning rules.
- The partition of the local index is in the same physical location as the corresponding table partition, and the Leader replica is in the same location.

Global index

Unlike local indexes, global indexes no longer maintain a one-to-one relationship with the partitions of the master table; instead, global indexes are created by synthesizing the data of all master table partitions into a whole.

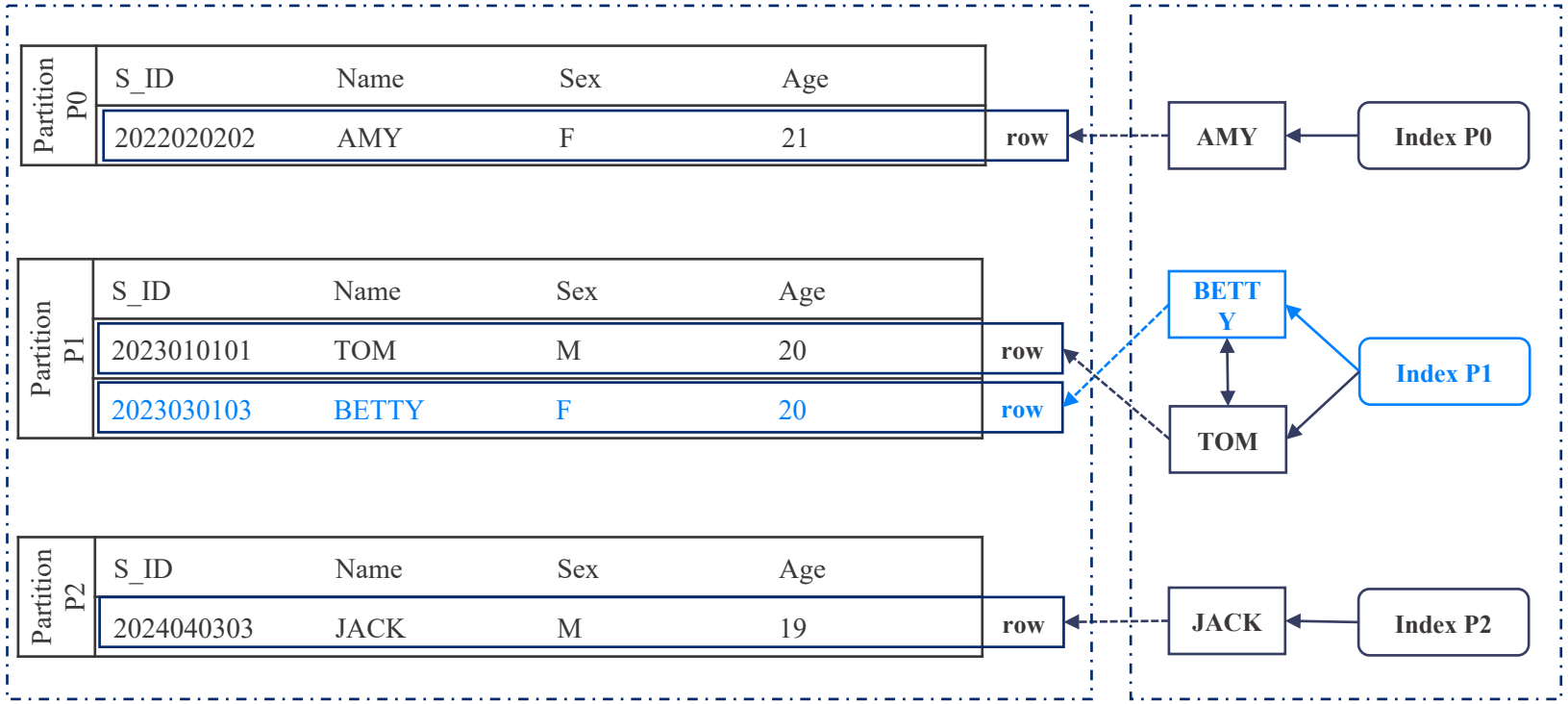
- When creating indexes for Partition Table, specifying GLOBAL keyword means creating a global index.
- The partitioning rules for global indexes are independent of those for tables. For global indexes, it’s allowed to specify their own partitioning rules and number of partitions, and there’s no need to be consistent with partitioning rules for tables.
- Global indexes can be partitioned or non-partitioned.
- The physical location of the global index is independent of the distribution of table partitions.
- A globally unique index ensures global uniqueness of table data.

MySQL schema creates local indexes by default, and Oracle schema creates global indexes by default.

Local Index

The local index of a partition table is similar to the index of a non-partition table, the data structure of the index still maintains a one-to-one relationship with the data structure of the master table; however, because the master table has been partitioned, "each partition" of the master table will have its own separate index data structure.

Local Index: With local indexes, partition key in the query condition should be specified to do partition pruning, otherwise all index partitions will be accessed, thus affecting the performance.



Partition the table by S_ID

Create a local index on NAME

Example

- Create a local index on NAME:

```
CREATE INDEX students_ix1
ON students(name)
LOCAL;
```

- Query information about a student named BETTY and specify partition conditions

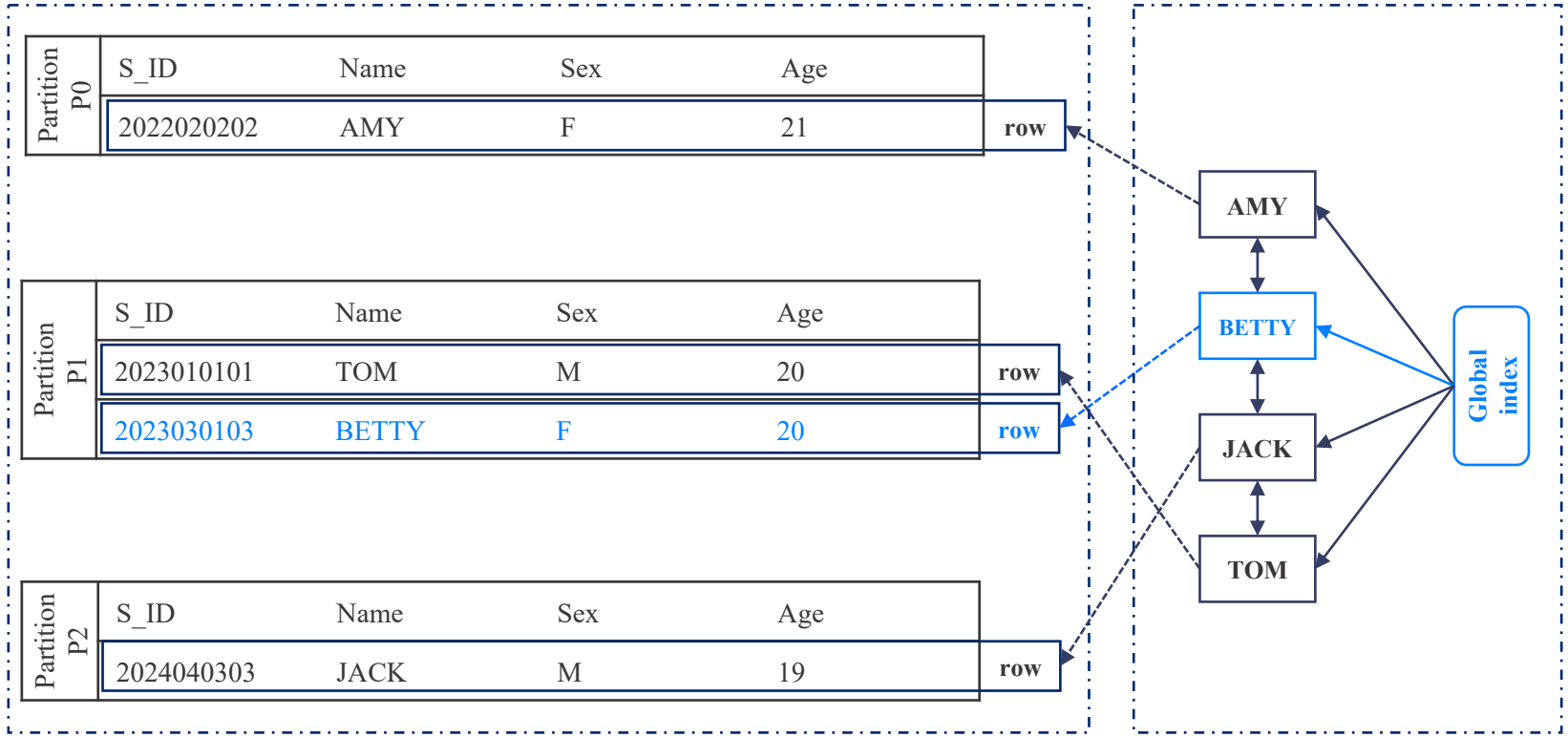
```
#Specify partitions directly in SQL#
SELECT *
FROM students PRTITION(P1)
WHERE name = 'BETTY';

#Perform partition pruning through
the database#
SELECT *
FROM students
WHERE name = 'BETTY'
AND s_id> 2022999999
AND s_id< 2023999999;
```

Global Index - Non-partitioned

The global index of partition table no longer maintains a one-to-one relationship with the partitions of the master table; instead, global indexes are created by synthesizing the data of all master table partitions into a whole. Furthermore, the global index can define its own independent data distribution model, either a non-partitioned or a partitioned one

Non-partitioned global index



Partition the table by S_ID

Create global non-partitioned index on NAME

Example

■ Create global non-partitioned index on NAME:

```
CREATE INDEX students_ix1
ON students(name)
GLOBAL;
```

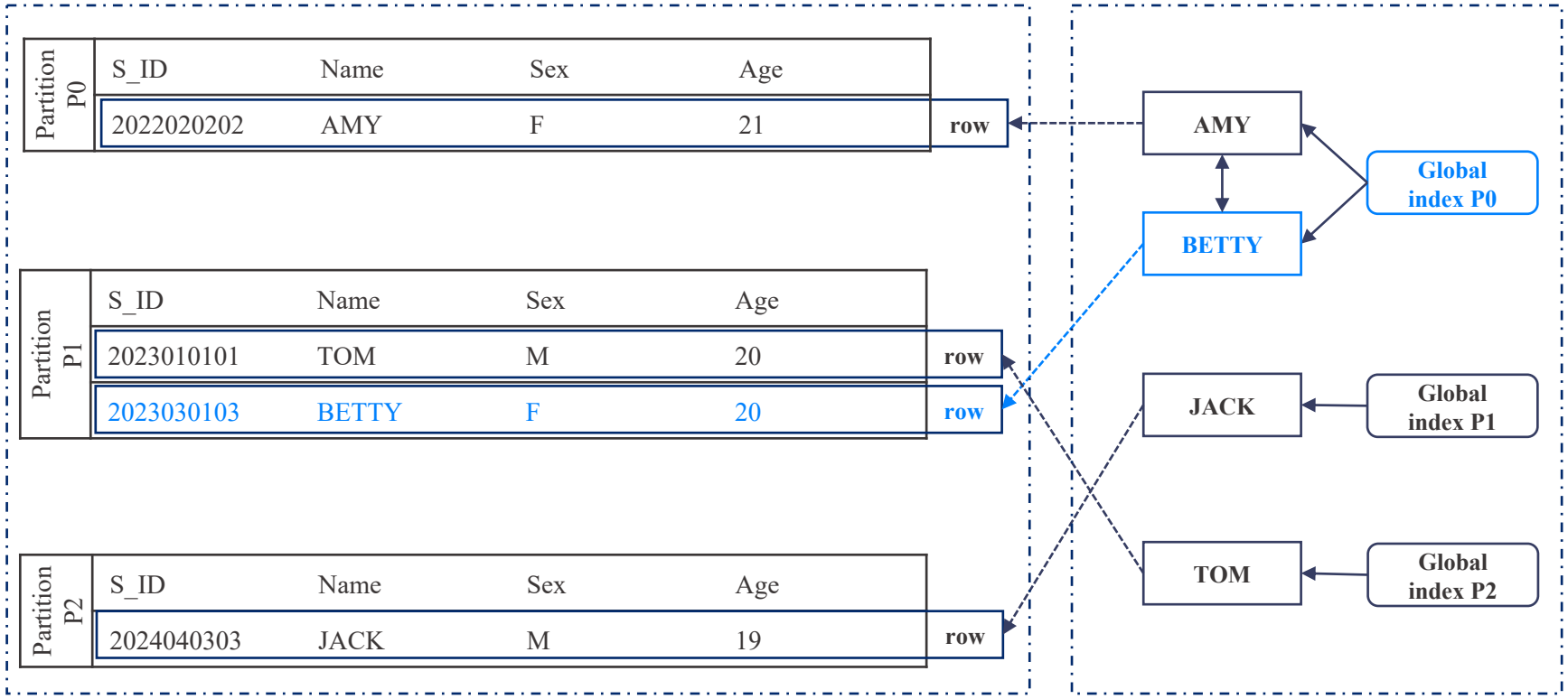
■ Query information about a student named BETTY and specify no partition conditions:

```
SELECT *
FROM students
WHERE name = 'BETTY';
```

Global Index - Partitioned

The global index of partition table no longer maintains a one-to-one relationship with the partitions of the master table; instead, global indexes are created by synthesizing the data of all master table partitions into a whole. Furthermore, the global index can define its own independent data distribution model, either a non-partitioned or a partitioned one

Global Partitioned Index: The partition key for a global index is the index key or its subset. When querying with a global partitioned index, you need to specify the partition key in the query condition to do the partition pruning of the global index.



Partition the table by S_ID

Create global indexes on NAME partitioned by NAME

Example

- Create a global index on NAME and partition the global index by NAME:

```
CREATE INDEX students_ix1
ON students(name)
GLOBAL PARTITION BY RANGE(name)
(PARTITION p0 VALUES LESS THAN('H'),
PARTITION p1 VALUES LESS THAN('N'),
PARTITION p2 VALUES LESS
THAN(MAXVALUE));
```

Index partition	Name range
P0	[,'H')
P1	['H', 'N')
P2	['N',MAXVALUE)

- Query information about a student named BETTY

```
SELECT *
FROM students
WHERE name = 'BETTY';
```

Execution Plan

Execution Plan, also known as Access Path, describes the methods for searching matching records and computing query results when a database executes a query. For access to a single table,the execution plan selects to find data either by using an index or by full table scanning. Execution plan analysis is one of the most important things for single-table querying. For execution plans that use master table scanning, the execution time is generally proportional to the amount (range) of data to be scanned.

Selection of optimizer and execution plan

Database optimizer

- **Optimizer:** The optimizer is the core component of the SQL engine, which is responsible for parsing SQL and selecting the optimal execution plan for SQL.
- **Distributed optimizer:** Taking into account challenges unique to distributed databases, such as partitioning of data, replicas, network latency, and parallel execution, Optimizer of the OceanBase Distributed Database is able to select an execution plan that functions most efficiently in a distributed environment.

Selection of Execution Plan

- **Minimum cost:** The optimizer calculates the execution costs of different execution plans and selects the plan with the lowest cost as the final execution plan.
- **Cost calculation:** When calculating the execution cost, the optimizer takes into account multiple sources of information, such as query conditions, available indexes, and statistical information about the data in the table, to comprehensively calculate the execution cost of each available execution plan.
- **Execution plan hint:** When selecting an execution plan, the optimizer takes into account the execution plan hint (HINT) specified in the SQL and tries to generate the execution plan according to the hints as far as possible.

Common execution plan for single-table querying

TABLE FULL SCAN

- **Execution mode:** Table Full Scan needs to scan the entire table (or partition)and read each row of data in the table,thus checking if the query condition is met.
- **Selection condition:** When the query does not utilize any indexes, it is common to execute the Table Full Scan.
- **Execution efficiency:** Table Full Scan is usually the less efficient way of executing single-table queries.

TABLE RANGE SCAN

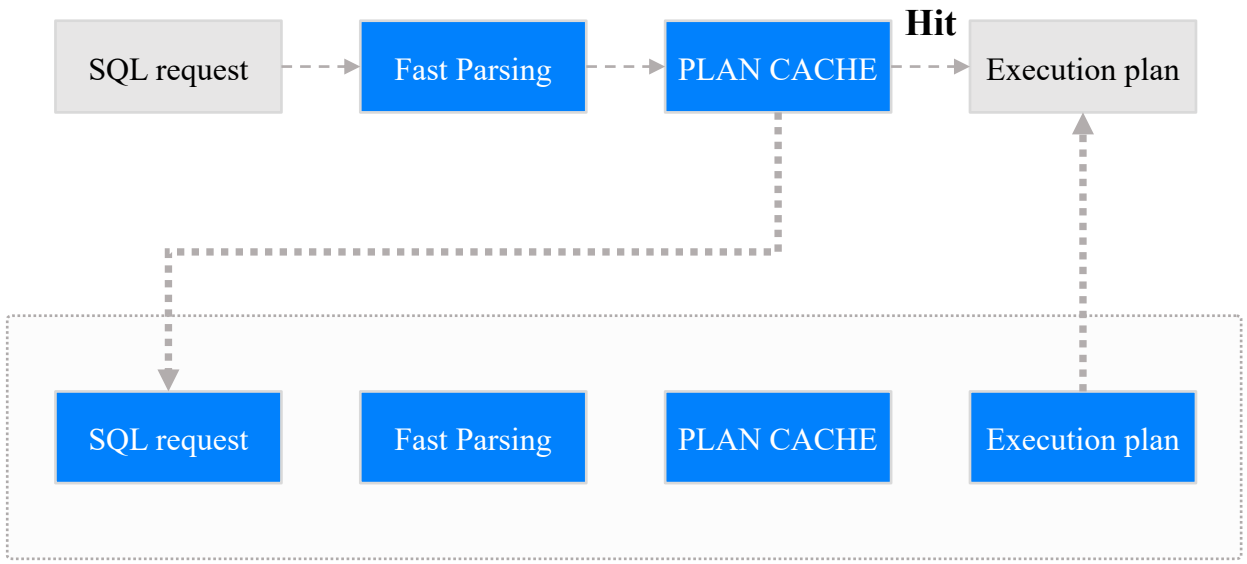
- **Execution mode:** Perform exact query or range scan of indexed records by index key value. If the query requires access to fields not contained in the index, further access to the complete records in the table is required.
- **Selection condition:** When a query condition matches an index key, it typically chooses to use the index for the query.
- **Execution efficiency:** Table Range Scan is usually the more efficient way of executing single-table queries.

TABLE GET

- **Execution mode:** Precisely find unique records by the primary key.
- **Selection condition:** When a query condition matches the primary key, the database chooses to use the primary key for the query.
- **Execution efficiency:** Table Get is usually the most efficient way of executing single-table queries.

PLAN CACHE

For SQL with high execution frequency and short execution time, if an execution plan is generated for each execution, the time spent on SQL optimization will seriously affect the SQL performance. OceanBase puts the generated plan into the Plan Cache and gets the plan directly from Plan Cache upon execution again.



Features of OceanBase plan cache:

- The plan cache is in a typical Key-Value structure, where Key is the parameterized SQL string and Value is the execution plan for the SQL.
- Each tenant has a separate plan cache on each server for SQL plans processed on that server.
- In the plan cache of OceanBase database, the execution plan of SQL can be categorized into three types: local plan, remote plan, and distributed plan. Dependent upon the data to access, an SQL may have three execution plans.

Refreshing and Eviction of Execution Plan

Refreshing of plan cache

The following scenarios will cause failure of the execution plan, where the execution plan shall be refreshed. Delete the execution plan, and re-optimize the generated plan before adding it to the plan cache:

- If the Schema involving the table(s) in the SQL changes (e.g., adding an index, and deleting or adding a column), the execution plan the SQL corresponds to in the Plan Cache will be refreshed.
- When the SQL engages re-collecting table statistics, the execution plan the SQL corresponds to in the PLAN CACHE will be refreshed.

Eviction of plan cache

Plan cache eviction refers to the removal of execution plans from the plan cache to reduce memory usage.

- Automatic eviction: Automatic eviction is to automatically obsolete the plans in Plan Cache, when memory occupied by the Plan Cache reaches the upper limit that requires elimination of plans. When Plan Cache eviction is triggered, execution plans that have not been used for the longest period of time shall first be eliminated. After eliminating some of the execution plans, the elimination shall stop when the memory used by the plan cache is at the low water mark of the elimination plan set by this tenant.
- Manual eviction: To forcibly delete the plans in Plan Cache with commands.

Database Engines

03

- SQL Engine
- **Storage Engine**
- Transaction Engine

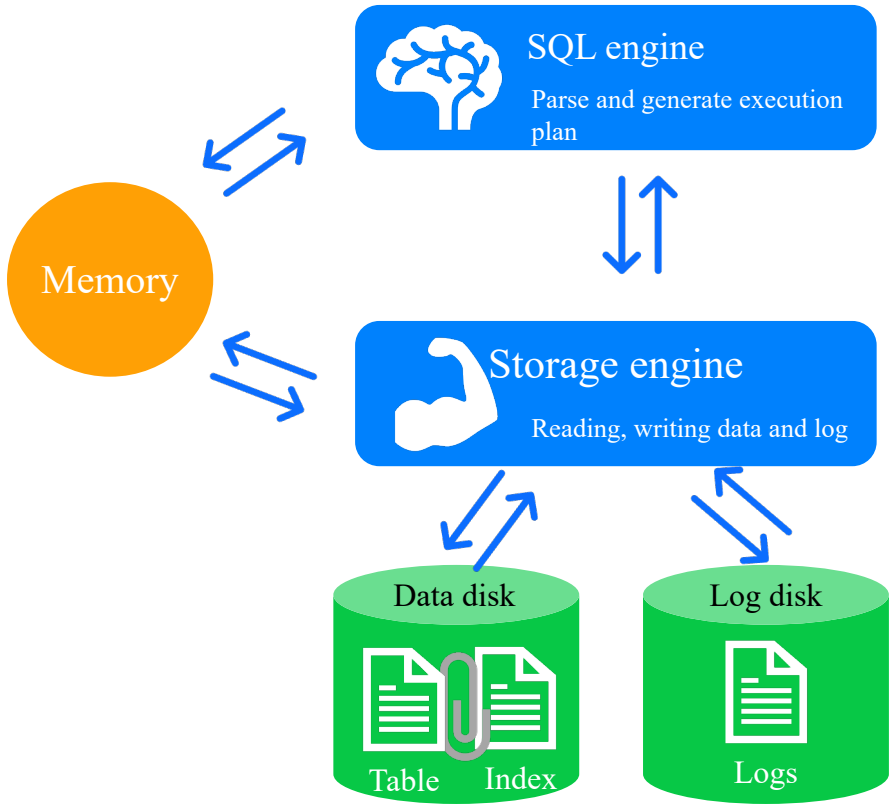
Storage Engine

Database Storage Engine is a core component of the database management system,which is responsible for saving data to files or data blocks on the hard disk and providing fast query and manipulation capabilities. Different database systems use different storage engines. Different storage engines provide different data storage mechanisms, caching mechanisms, indexing techniques, and other features, and each storage engine has its unique advantages and applicable scenarios.

Function of storage engine

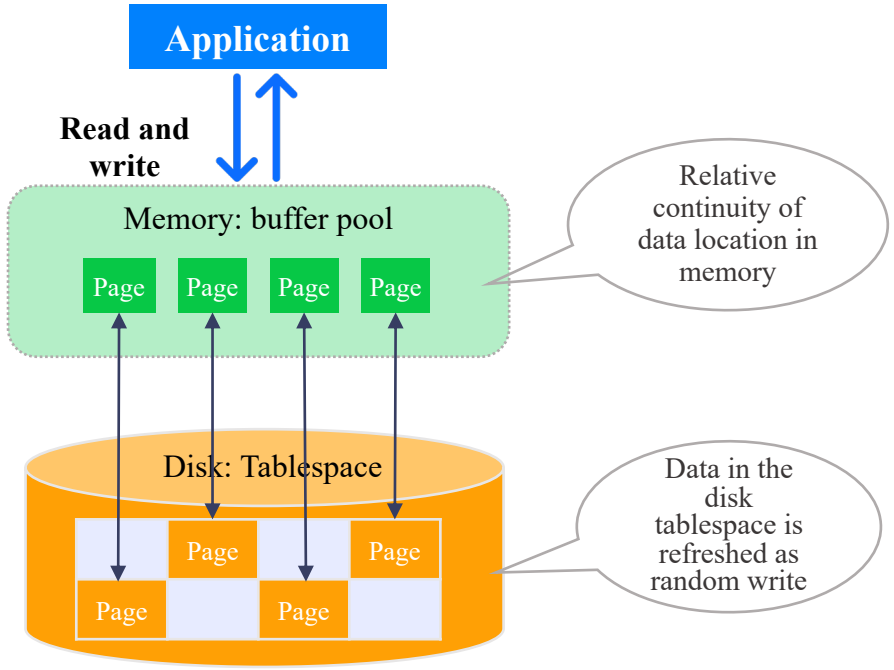
The database storage engine is responsible for storing, retrieving, and managing data. The main functions of the database storage engine are specified below:

- **Data storage:**
  - The storage engine defines how to store data on disk, including the physical storage format of the data, storage location, and data organizing mode.
  - The storage engine is responsible for Write operation of the data, with memory management, data compression, and other features.
- **Data read:**
  - The storage engine is responsible for the data read operation, with its database read performance enhanced by optimizing the data retrieving methods.
  - The storage engine provides a data caching mechanism that uses memory to cache frequently accessed data and indexes, thus reducing frequent access to the disk and improving the data access speed.
- **Index management:**
  - The storage engine is responsible for creating, maintaining, and using indexes to speed up the retrieval of data.
  - Different storage engines may provide different index types (e.g., B-Tree, and Hash), as well as different indexing strategies, to meet different query requirements.
- **Data security:**
  - A data persistence mechanism is provided to ensure that data will not be lost after a database system crash or restart.
  - It provides a data recovery mechanism to restore the database to a consistent state when the database fails or data is damaged.



Storage Engine of Traditional Database

Traditional databases (MySQL, Oracle, etc.) typically use the storage mode of Heap File Organization, which is characterized by an in-place update strategy and maintains the logical order of data.



Data write

- Data is stored in a logical order according to specified rules, for example, in the order of indexes.
- The data is updated with the in-place update strategy and the logical order of the data is maintained.
- When writing data, the data is first written to the buffer pool, and the dirty data is flushed to the hard disk through the check point.

Data read

- There is only one copy of the data on the hard disk, which is kept as organized as possible. Relatively good read performance can be provided through indexing and other methods.
- When reading data, if there's data in the buffer pool, it is read directly from memory; if there's not, it is extracted from the hard disk into the buffer pool.

Advantages and disadvantages

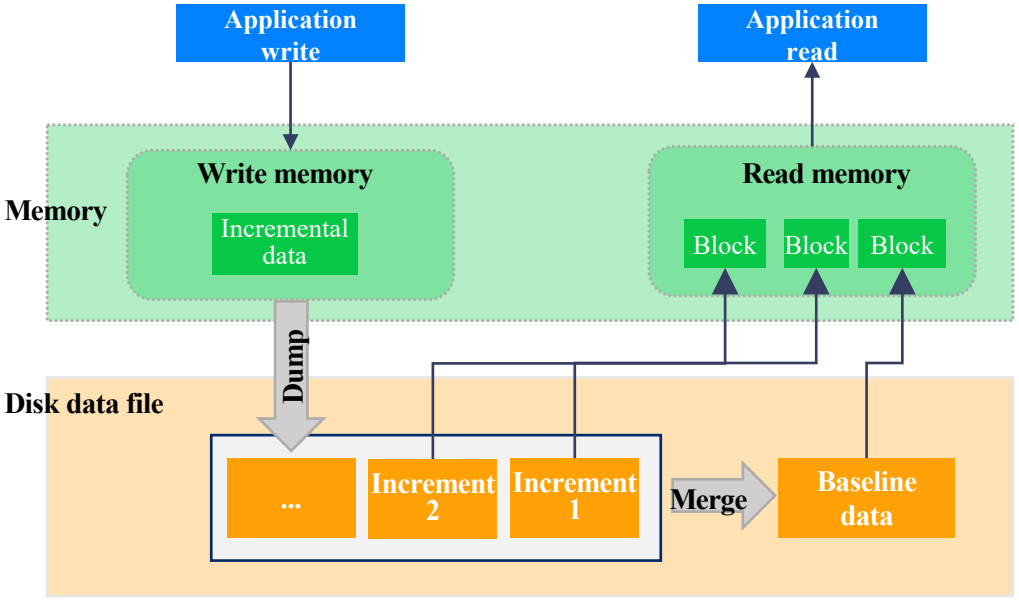
Advantages: Query performance is better when data is kept in only one copy and as organized as possible.

Disadvantages: There are problems of random write and write amplification. The write performance is relatively low, which affects the disk's service life.

- **Abundant random write: Data update generates frequent random write on the disk, and the performance of data write is not high.**
- **Write amplification: Write operation is done in pages or segments, resulting in write amplification. Random write on SSD can lead to severe write amplification, thus significantly shortening the service life of the SSD.**

Storage Engine for New-generation Database

The new-generation databases (LevelDB, RocksDB, etc.) usually adopt the LSMTree (Log-Structured Merge-Tree) storage method. Leveraging the disk's characteristic that sequential write operation is much faster than the random write operation, all the modifications are first sequentially written to the log structure, and then through the background merge operation, many minor modifications are combined into major modifications and eventually written into the disk.



Data write

- For data storage, multiple versions of the data can be stored.
- For data update, the strategy of append update (append write) is adopted, but the original version of the data is not overwritten or updated.
- In writing data, the data is firstly written to the “write memory” area (MemStore), then dumped to the disk through the compaction mechanism, lastly the background thread is responsible for merging the incremental data with the baseline data.

Data read

- When reading data, if there’s data in the "read memory" (Cache), it is read directly from the "read memory"; if there’s not, you need to check the "write memory" as well as the incremental and baseline data in the hard disk to extract the data into the "read memory".
- The disk retains multiple versions of the data, and multiple versions of the data may be combined during data reading, with a long read path.

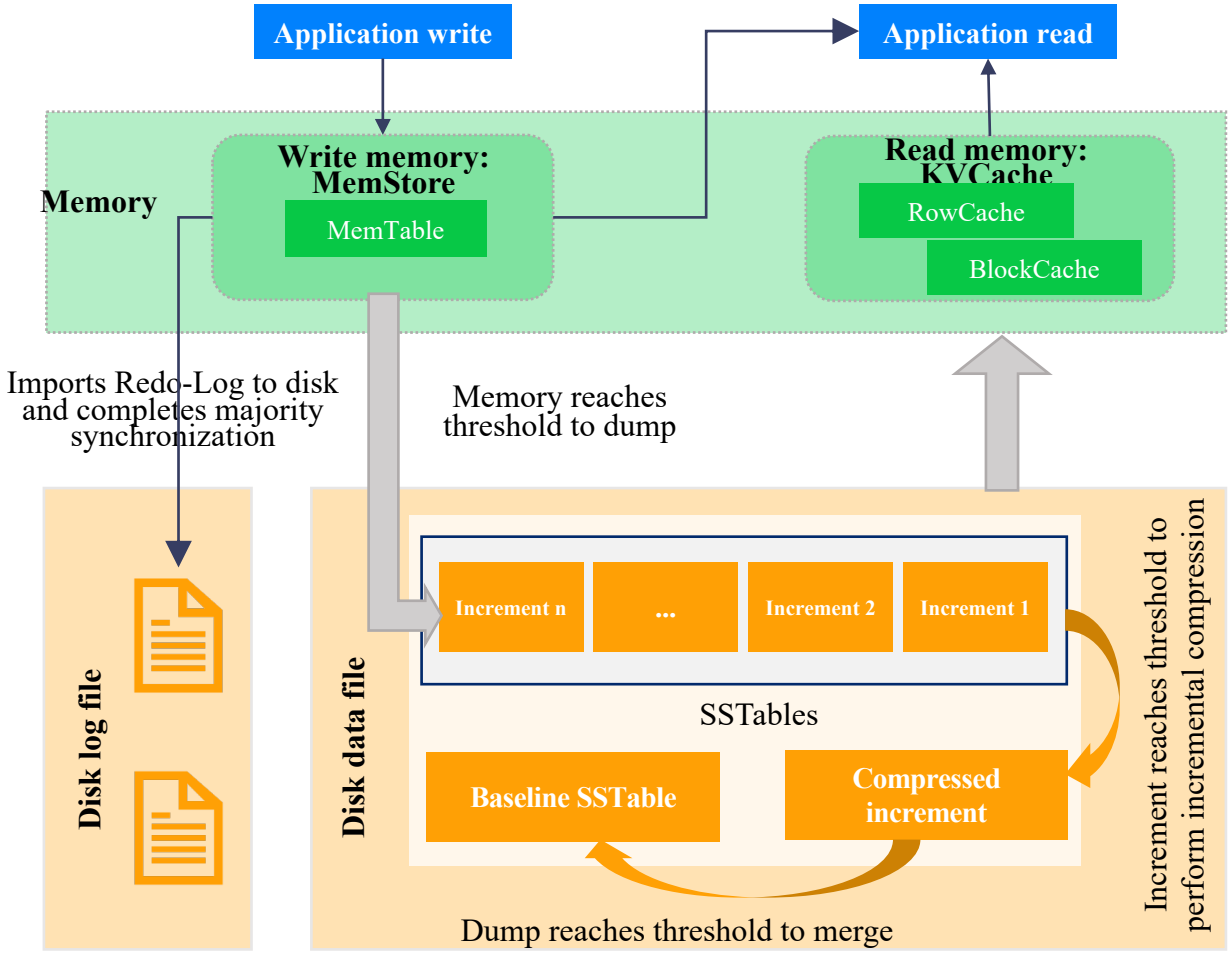
Advantages and disadvantages

Advantages: The problems of random write and write amplification are solved by appending writes, which is suitable for scenarios with abundant data updates.

Disadvantages: The path to data reading is long and the performance is poorer than traditional storage engines.

OceanBase Storage Engine: LSM Tree Storage + Quasi-Memory Database

The OceanBase storage engine utilizes the LSMTree storage structure and incorporates technologies from traditional relational databases and main memory databases to achieve the ultimate in read and write performance.

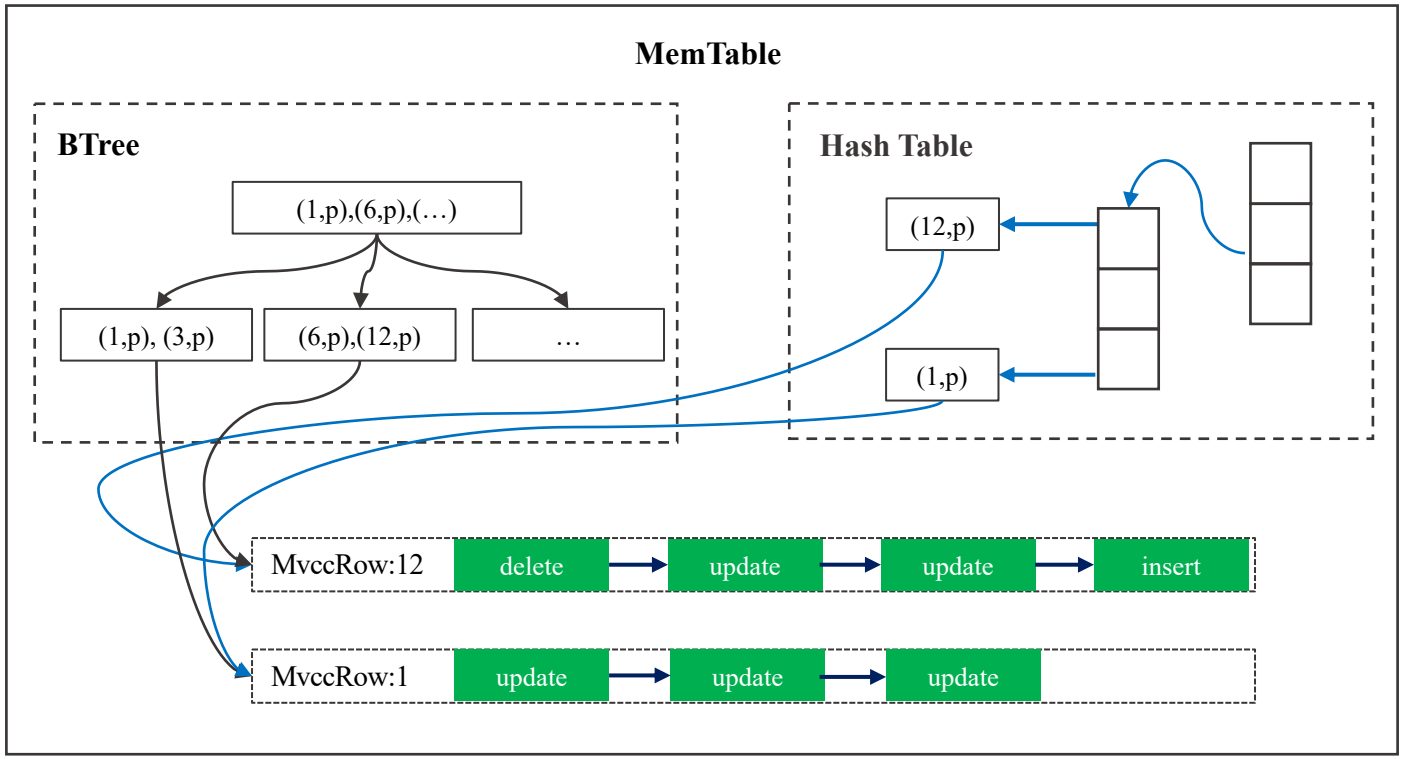


Technical characteristics

- **Read-write separation:** Writes are done in a Memory Table (MemTable) in the Write Memory (MemStore) and reads are done using the Read Cache (KVCache).
- **Sequential append write:** When incremental data is dumped to disk, the system does not overwrite the original data version, which greatly improves the write performance and extends SSD's service life.
- **Multi-level cache:** Read memory (KVCache) is designed to cache hot data in memory as much as possible with multi-level read cache, which greatly improves the read performance of LSMTree.
- **Data ordering:** Data is stored on disk in the order of the primary key during data dumping and merging.
- **Data durability:** To avoid data loss in memory, redo-log uses the WAL mechanism to import data to the disk in real time to ensure data persistence; meanwhile, distributed multiple replicas are forcibly consistent when importing logs to the disk to ensure high availability of the distributed replicas.
- **Data consistency:** When reading data, you need to read data from the hotspot cache, MemTable, and dump SSTable to ensure data consistency.
- **Lower disk costs:** Disk data is arranged in the order of the primary key, with less disk fragmentation, and a large percentage of data compression enabled by self-developed compression coding, thus significantly reducing disk costs.

Dynamic Data (MEMTABLE)

MemTable is the organizational structure of incremental data in write memory in the LSMTree architecture,known also as the dynamic data. Each partition corresponds to a MemTable, which consists of the BTree and the Hash Table. When inserting/updating/deleting data, the data is written to a block of memory. What are stored in Hash Table and BTree are pointers to corresponding data.



MemTable

To improve the query performance, MemTable uses a dual index structure, Hash Table and BTree, where pointers to MVCCRow are stored.

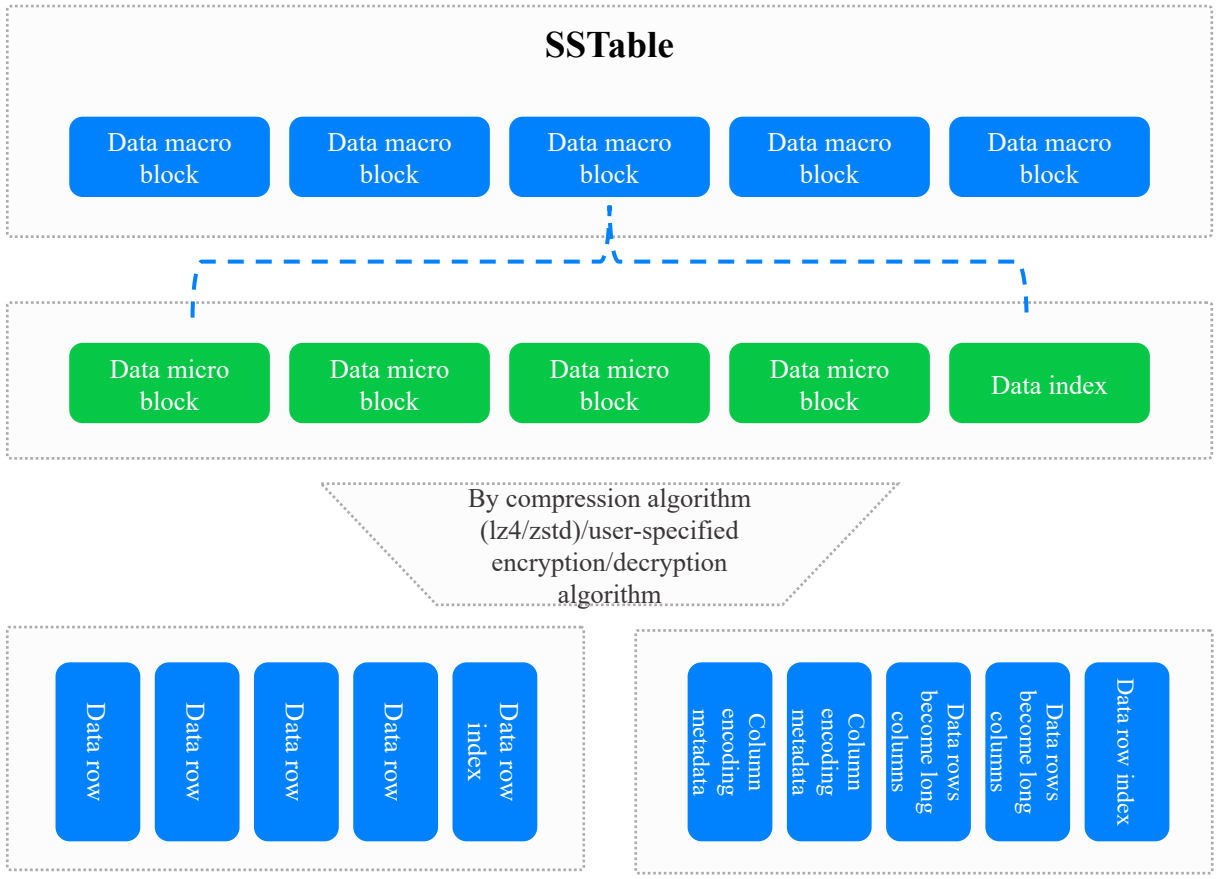
- **Hash Table:** Fast query performance is enabled for KV type point select.
- **BTree:** The data is orderly stored according to BTree, which is suitable for range query.
- **MVCCRow:** Multiple versions of recorded data are linked together in sequential order, and multiple versions of data are all written to the disk.

Concept

MVCC: Its full name is Multi-Version Concurrency Control. When data is modified, the system creates a new version of it and keeps the old version, stringing different versions of the data together through a chained table, which means that multiple versions of the data are kept. By saving historical versions of the data, MVCC enables read and write operations to be performed based on different versions of the data, thus avoiding direct read/write conflicts.

Static Data (SSTABLE)

SSTable is how table data is stored on disk in the LSMTree architecture. When data is dumped to disk from memory and written to SSTable, modifications to the data by the application will not change the version of the data that has already been on disk. Thus, we call the data in SSTable the static data.



SSTable

SSTable is composed of macro block and micro block of data.

Macro Block

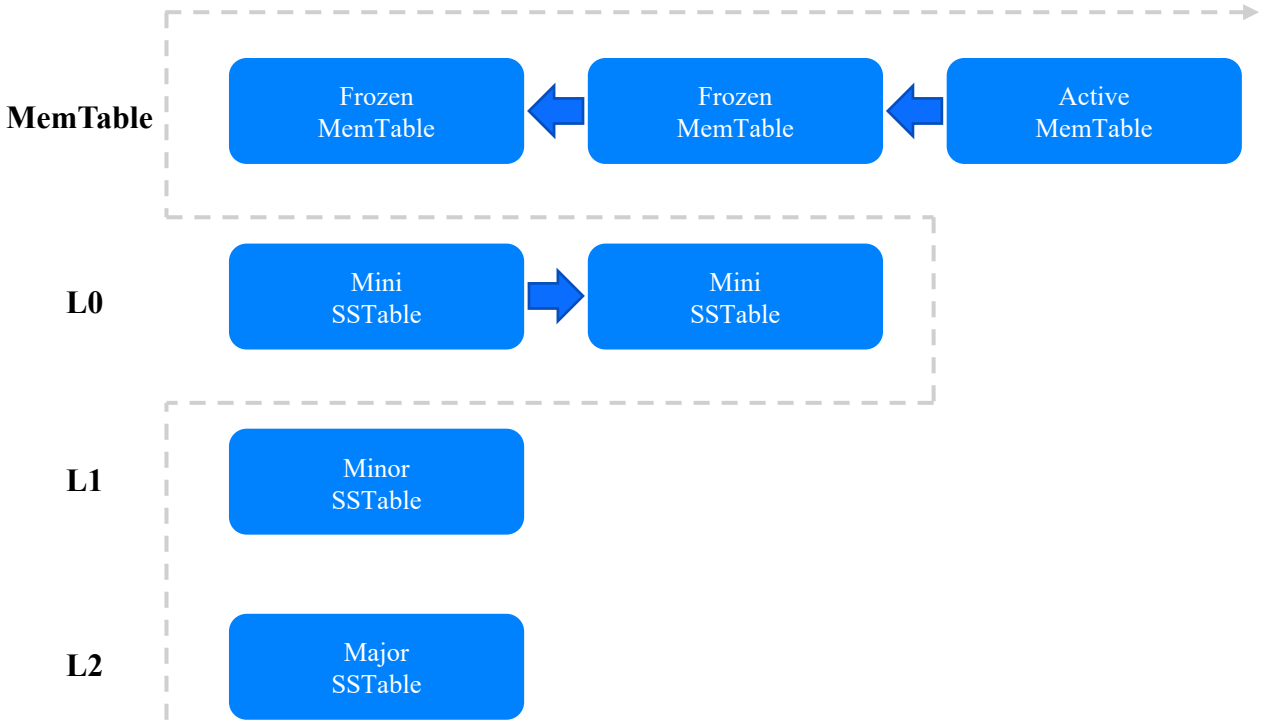
- Each SSTable consists of several macroblocks, namely fixed-length blocks of data on disk sized 2MB.
- Macroblock is the basic unit of write I/O operations.

Micro Block

- Each macroblock is internally composed of several micro blocks, namely variable-length data blocks sized about 16KB containing several rows of data.
- Micro blocks are compressed with a user-specified algorithm when they are constructed. Thus, what is actually stored on the macro block is a compressed micro block of data.
- Micro Block is the minimum unit of read I/O operations.

Flush Data to Disk (Minor Compaction)

When the MemTable is sized beyond a specified threshold, it is necessary to dump the data in the MemTable to the SSTable to free up memory. This process is called the "Minor Compaction".



Layered minor compaction

OceanBase adopts a hierarchical minor compaction strategy:

- **Layer L0:** The SSTable generated by dumping from memory is called the Layer L0.
- **Layer L1:** When the data volume in Layer L0 reaches the threshold, it automatically compacts multiple SSTables in Layer L0 into a single SSTable, called Layer L1.
- **Layer L2:** All the dynamic and static data are merged to form a new baseline SSTable, called Layer L2.

Trigger mechanism for minor compaction:

- **Automatic trigger:** A minor compaction is automatically triggered when a tenant's usage of the MemTable memory reaches the threshold pre-specified.
- **Manual trigger:** A minor compaction can also be initiated proactively through O&M commands.

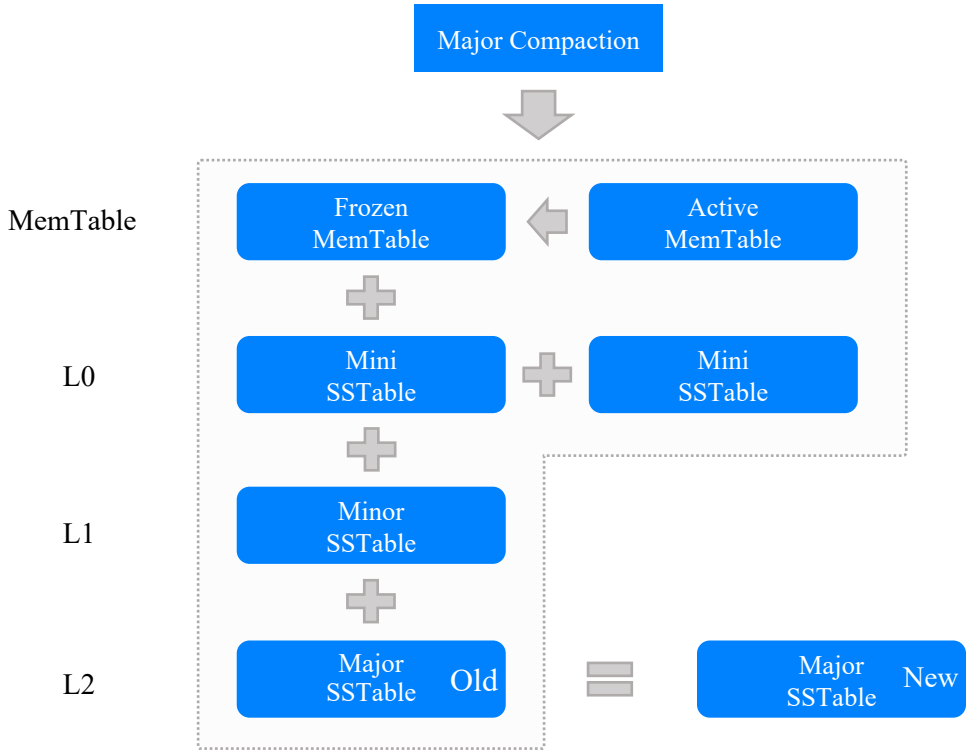
Concept

Sequential append writes are adopted to avoid random writes and write amplification and improve write performance.

The system automatically compacts and merges incremental data through background tasks using the layered minor compaction strategy, which improves the dump efficiency and data read performance.

Major Compaction

Each minor compaction writes incremental data into a new SSTable. The data of a table or partition will be distributed in the baseline SSTable and multiple dumped(mini/minor) SSTables, prolonging the read path of the data. Since the mini/minor SSTable contain a large amount of multi-versions of the data that have been invalidated, it is necessary to normalize all of the dynamic and static data through major compaction.



Role and trigger mechanism of major compaction

Role of major compaction:

- Major compaction will put the dynamic and static data together, delete the invalid multi-versions of data, and generate a new baseline SSTable sorted according to the primary key, which is conducive to improving the query performance.
- Major compaction encodes and compresses data when generating a new baseline SSTable, thus reducing the disk space.
- During the merging process, OceanBase validates the data and induces the data changes corresponding to the Schema changes.

Trigger mechanism of merge:

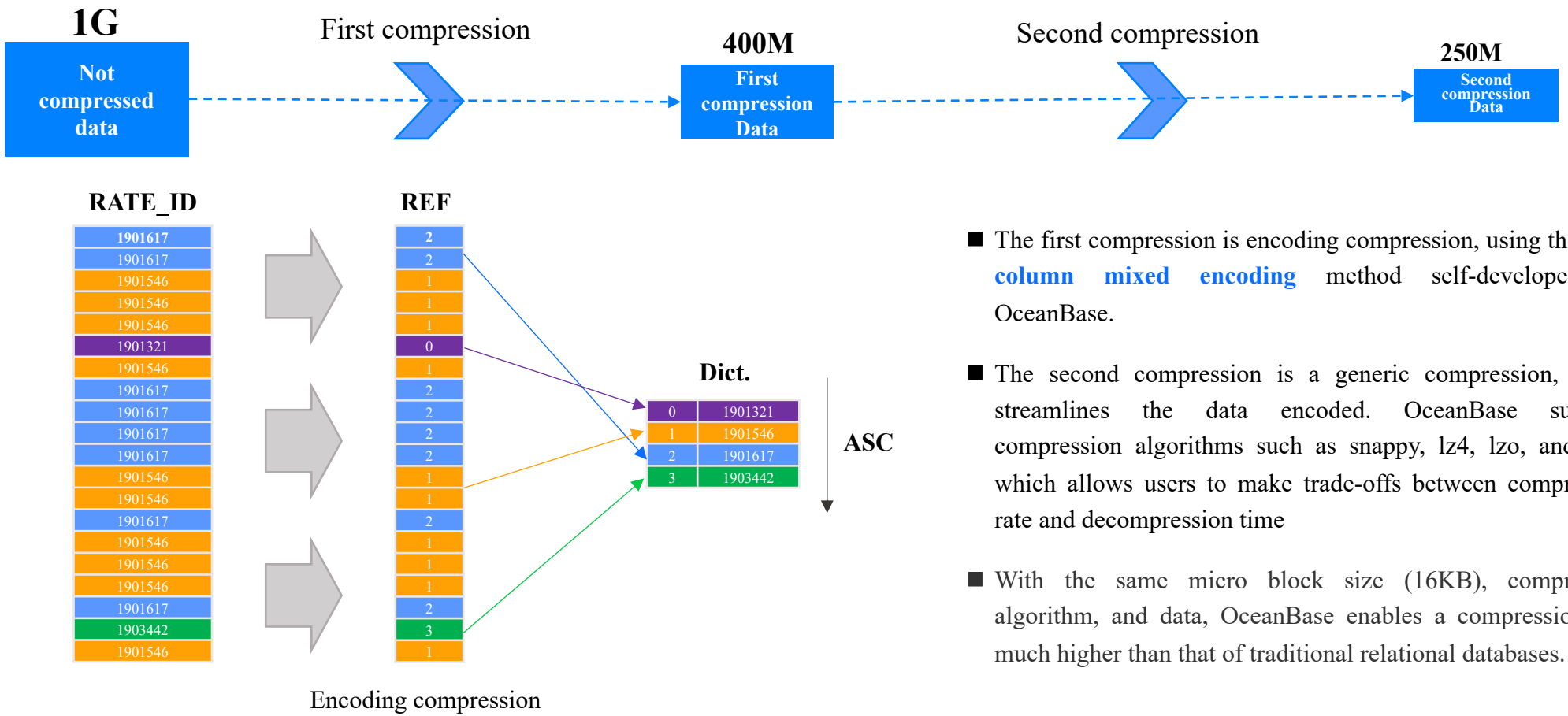
- **Automatic trigger:** Major compaction is automatically triggered when the number of minor freezes of a tenant exceeds the limit specified.
- **Manual trigger:** Major compaction can also be initiated proactively through O&M commands.
- **Scheduled trigger:** Major compaction is a relatively time-consuming operation, which can also be set to perform at a fixed time each day.

Features

- Consolidate all dynamic and static data into one, eliminate invalid multi-versions of data, and arrange disk data in the order of primary key to improve query performance.
- Consolidation is a relatively heavy operation, and it is expected to be done only once a day, controlled to be done during off-peak hours.

Data Compression

During the major compaction, OceanBase database will compress and encode the data in SSTable to save the storage space on hard disk and reduce the IO generated when querying SSTable. Counting up LSMTree's storage architecture and the self-developed coding algorithm, OceanBase obtains a remarkable compression rate, materially lowering the storage cost at a low price of performance sacrifice.



- The first compression is encoding compression, using the **row-column mixed encoding** method self-developed by OceanBase.
- The second compression is a generic compression, which streamlines the data encoded. OceanBase supports compression algorithms such as snappy, lz4, lzo, and zstd, which allows users to make trade-offs between compression rate and decompression time
- With the same micro block size (16KB), compression algorithm, and data, OceanBase enables a compression rate much higher than that of traditional relational databases.

Database Engines

03

- SQL Engine
- Storage Engine
- **Transaction Engine**

Transaction

The database transaction is a series of operations performed as a single logical functioning unit to ensure the correctness and integrity of data operations, either all or none of which are performed.

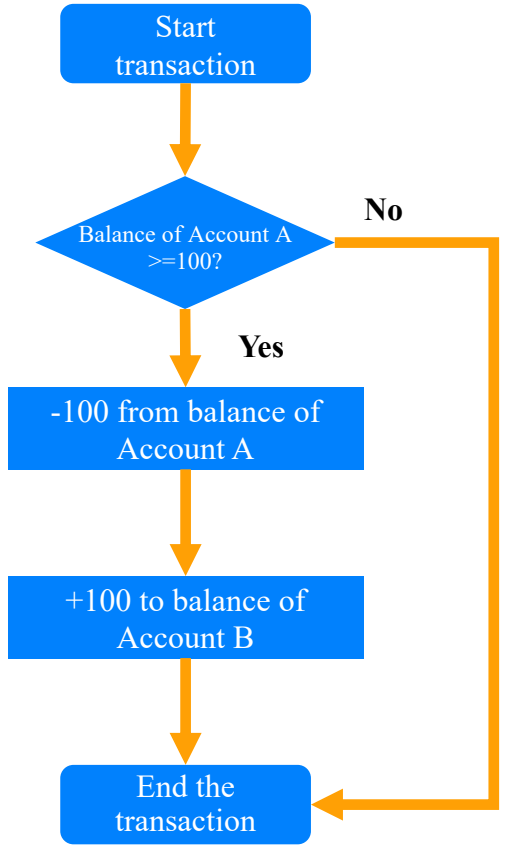
The Database Transaction serves two purposes:

- It provides a means for a sequence of database operations to recover from a failure to the normal state, as well as a means for the database to maintain consistency even in an abnormal state.
- It provides a means to isolate multiple concurrent accesses to a database to prevent multiple concurrent operations from causing the database to enter an inconsistent state.

Example: A database transfer transaction that transfers RMB100 from Account A to Account B.

1. Start a database transaction;
2. Check if the balance of Account A is greater than RMB100;
3. If it is less than RMB100, it reports an error and ends the task;
4. Deduct RMB100 from the balance of Account A;
5. Add RMB100 to the balance of Account B;
6. The transfer is successful and the transaction is closed.

In a transfer transaction, the modification to the two accounts must be both successful or failed. It's disallowed if an amount is deducted from Account A but Account B receives nothing.



ACID Characteristics of Transactions

Database transaction has four features: Atomicity, consistency, isolation, and durability are generally referred to as the ACID characteristics.

A

Atomicity

- All operations in a transaction must succeed or fail as a whole.
- Atomicity of transactions is achieved through database logs.

C

Consistency

- The database is consistent before and after execution of transactions.
- Consistency is achieved through the data integrity constraint and transaction rollback mechanism.

I

Isolation

- Transactions are isolated from each other and do not interfere with each other during concurrent execution.
- Isolation is achieved through locks, MVCC, and transactional isolation levels.

D

Durability

- Once a transaction is committed, its changes to the database are permanent.
- Persistence is guaranteed by redo log and WAL mechanisms.

In a transfer transaction, the ACID characteristics of a transaction are manifested in the following ways:

- Atomicity: The debit to Account A and the credit to Account B must be both successful or rolled back.
- Consistency: Before and after the transfer transaction,the balances of Account A and Account B are logically consistent.
- Isolation: Before the transfer transaction is committed: Through the locking mechanism, others can't perform modification operations on both Account A and Account B; through the isolation level and MVCC mechanism, others can't see the balance changes of Accounts A and B that have not been committed.
- Durability: Upon completion of transfer transaction,the database adopts WAL mechanism to ensure that transaction logs are imported to disk. Even if the machine fails and restarts, the database can still recover the data state after transaction submission through logs.

Concurrent Control of Transactions

Concurrently executing transactions may read and write the same records. If concurrent control is not applied to transactions, they may interfere with each other, resulting in data inconsistencies or other exceptions. OceanBase controls concurrent operations of multiple transactions through locking mechanism, Multi-Version Concurrency Control (MVCC), and Isolation Level of transactions to guarantee the ACID properties of database transactions.

Problems that may result from transaction concurrency

- **Dirty read:** Transaction 1 reads the uncommitted data of Transaction 2.
- **Unrepeated read:** During the execution of a transaction, the data that has just been read has been modified by another transaction, and the transaction reads the same batch of data again and finds that the data is not the same.
- **Phantom read:** Two queries are performed during one transaction, and the results of the second query contain data that did not appear in the first query or lack data that appeared in the first query (during which data was inserted or deleted by another transaction).

Dirty read	The bank made a mistake in testing the account and transferred RMB1 million to Wang's account, but did not commit the transaction, at which point Wang inquired about the account and found an extra RMB1 million. The bank reviewed the account before committing the transaction and realized that it had made a mistake and did a rollback of the transaction, at which point Wang checked again and found that the RMB1 million had flown away. Wang found the extra RMB1 million in his account because of the dirty read.
Unrepeated read	Wang checked his bank account and found a balance of RMB1,000, but when he tried to recharge RMB500 for video games from the balance, it prompts insufficient balance. Wang checked again and found only RMB100 in his account balance. His wife just spent RMB900 buying makeups from this account. This is unrepeatable read that Wang saw different balances the two times.
Phantom read	Wang checked the list of pending approvals in his name before the work off and found that only one approval had not been completed. After Wang had completed this approval, he checked the list of pending approvals under his name again, only to find that there was one more item. Wang thought he had blurred vision. It turned out that while he was dealing with the final approval, his colleague Li transferred another approval to his name. This is phantom read that Wang saw different records the two times.

ISOLATION LEVEL

Isolation level is used to describe the degree to which transactions interfere with each other when they are executed concurrently. The ANSI/ISO SQL standard defines four isolation levels based on exceptions that must be circumvented during transaction execution. The degree of interaction between transactions varies at different isolation levels; the higher the isolation level, the less interaction between transactions and the fewer exceptions allowed. OceanBase currently supports three of the four isolation levels.

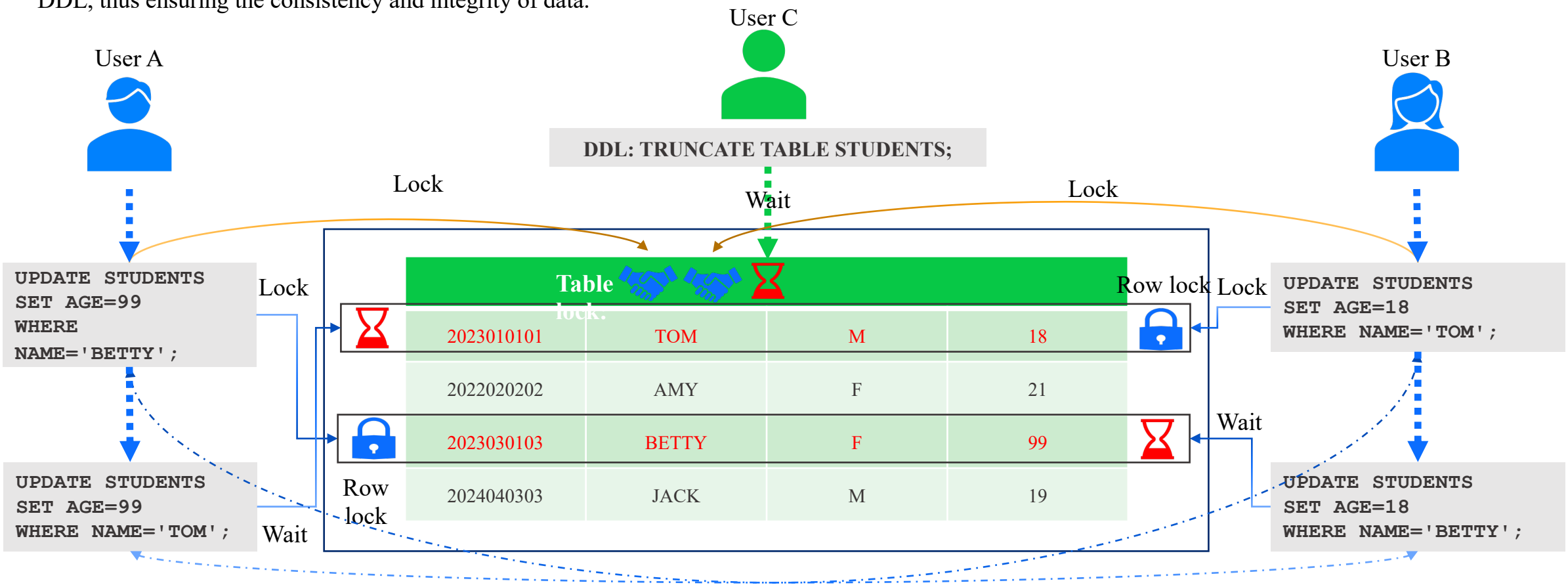
Isolation level

- **Read Committed**: The query executed by a transaction can only see data committed before the start of this query. Read Committed does not prevent the two exceptions of unrepeatable reads and phantom reads.
- **Serializable**: The query executed by a transaction can only see data committed before the start of this transaction. This is the strictest isolation level that prevents the three exceptions of dirty reads, unrepeatable reads, and phantom reads. The transaction appears to be executed serially.
- **Repeatable Read:** A single batch of data read at different time within a transaction is consistent. **Only MySQL schema supports repeatable reads.**
- **Read Uncommitted**: A transaction reads data that another transaction has not yet committed. **OceanBase doesn't support Read Uncommitted.**

Isolation level	Dirty read	Unrepeated read	Phantom read
Read uncommitted	Possible	Possible	Possible
Read committed	Impossible	Possible	Possible
Repeatable read	Impossible	Impossible	Possible
Serializable	Impossible	Impossible	Impossible

LOCK

The OceanBase V4 release implements two levels of locking: table locks and row locks. Table locks are used to lock the entire table.Different levels of table locks are available, which may be compatible or incompatible with each other;row locks are intended to lock a row, which are mutually exclusive in data updates. DML (except SELECT) operations put not only exclusive row locks on rows, but also compatible table locks on tables, whereas DDL operations usually put mutually exclusive table locks on tables. Through table locks and row locks, OceanBase takes concurrent control over DML and DDL, thus ensuring the consistency and integrity of data.

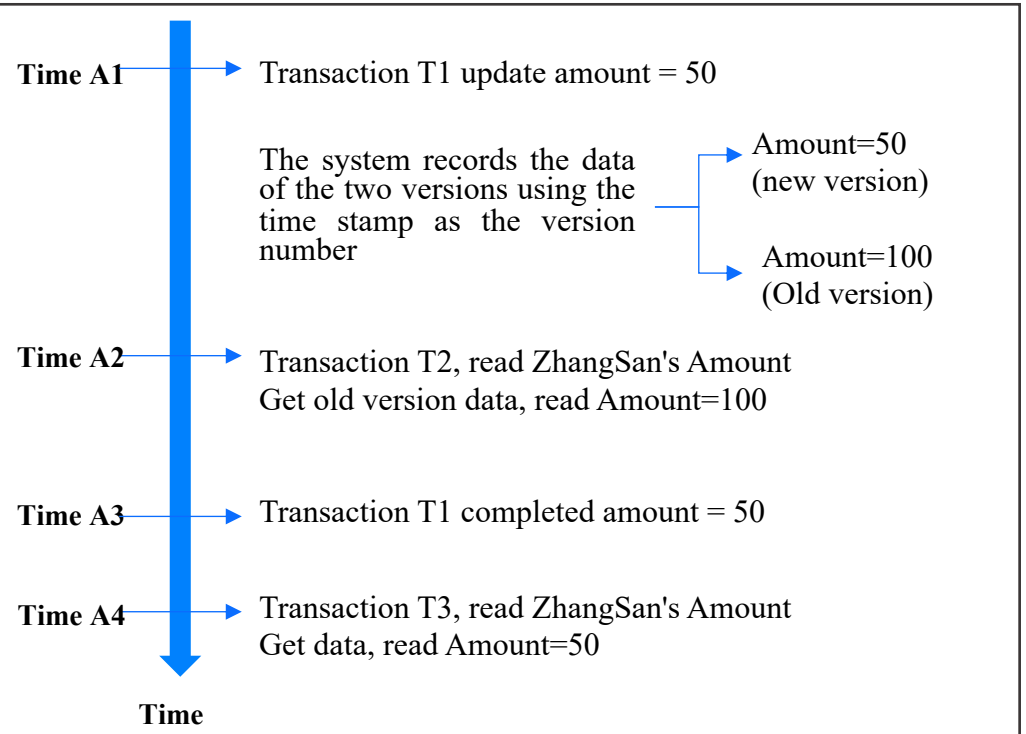


Users A and B wait for each other and get into a deadlock

Multi-Version Concurrency Control (MVCC)

The Multi-Version Concurrency Control (MVCC) mechanism for databases is a technique used to improve the concurrency performance of databases by allowing multiple users to access the database at the same time without causing data conflicts and inconsistencies. The OceanBase database is initially designed to store multiple versions of data and enable transaction-consistent snapshot reads via the transaction's commit version number and read version number.

Name	Amount
ZhangSan	100



With the MVCC function, the write transaction of T1 and the read transaction of T2 can be executed concurrently.

Core functions of OceanBase MVCC

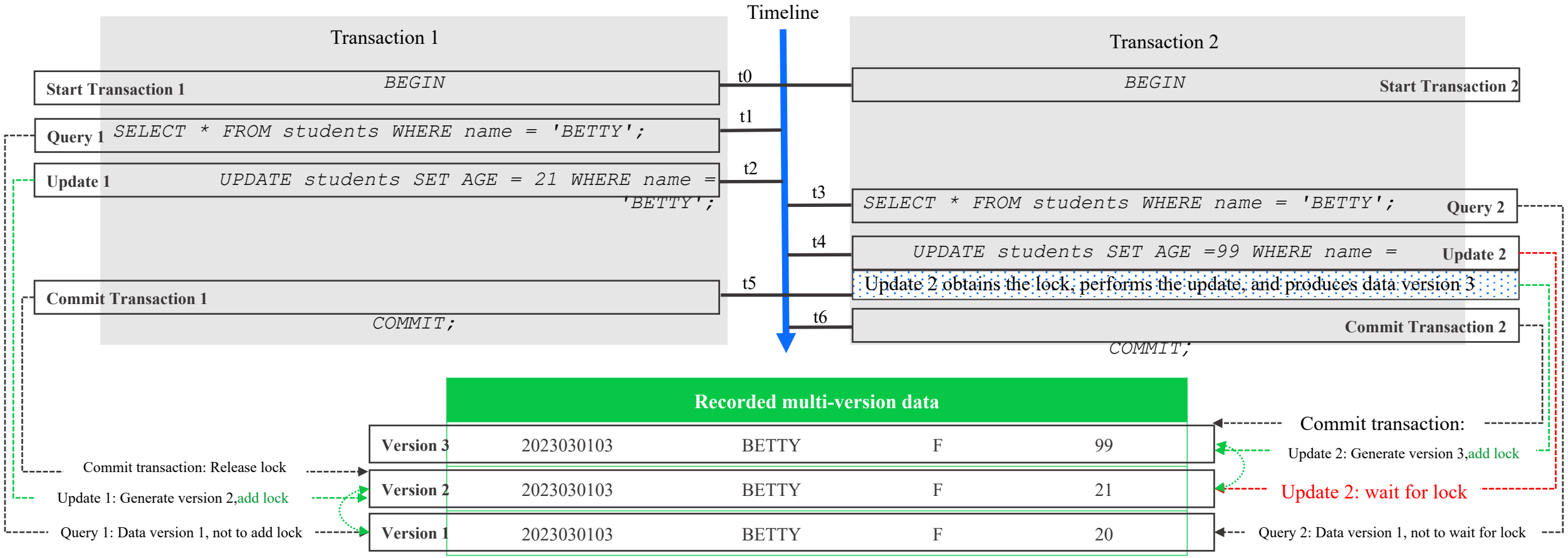
- **Support concurrent reads:** Multiple transactions can read data from the database at the same time.
- **Avoid lock contention:** When reading data, there is no need to lock the records, thus avoiding lock contention and further improving concurrency performance.
- **Ensure data consistency:** With version control, MVCC ensures that data consistency is guaranteed in case of concurrent reads.

Working Principles of OceanBase MVCC

- MVCC controls concurrent access by assigning a unique transaction ID to each transaction, based on the version number of the data and the start time of the transaction as each transaction accesses or modifies the data.
- When a transaction modifies a piece of data, the system creates a new version of that data, stores the modified data in the new location, and keeps the old version of the data available for other transactions to access.
- When a transaction wants to access a piece of data, the system checks the version number of that data and the start time of the transaction to ensure that the transaction can access the correct version of the data.

Combination of Locks and MVCC

OceanBase controls concurrent read and write operations through locks and MVCC mechanism, thus realizing read without locking, avoiding read/write conflicts, and ensuring data integrity and consistency.



Challenges of Distributed Transactions

Distributed transactions are ones that span multiple network nodes or between multiple databases, and ensure that data operations on all participating nodes have the ACID characteristics. In a distributed architecture, transaction processing faces challenges different from those of centralized databases, which stem from network latency, data consistency, fault handling, and performance.

Unreliability of network communication: Since nodes in a distributed system are usually distributed in different geographical locations, communication between nodes may be subject to network latency, thus prolonging the transaction execution time. Network unreliability can lead to message losses, timeouts, and inconsistent states, further affecting the proper execution of transactions.

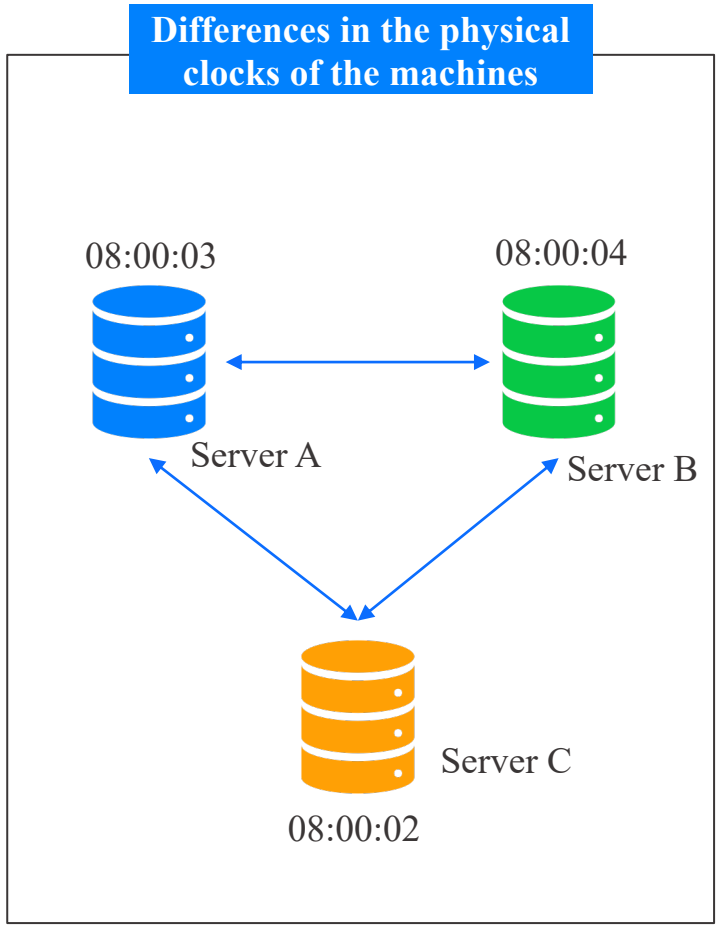
Impact of node failures: Nodes in a distributed system may fail, causing a process crash, a network failure, etc. In the event of a failure, the distributed transaction processing system must be able to detect the failure and recover to the correct state to ensure the transaction consistency.

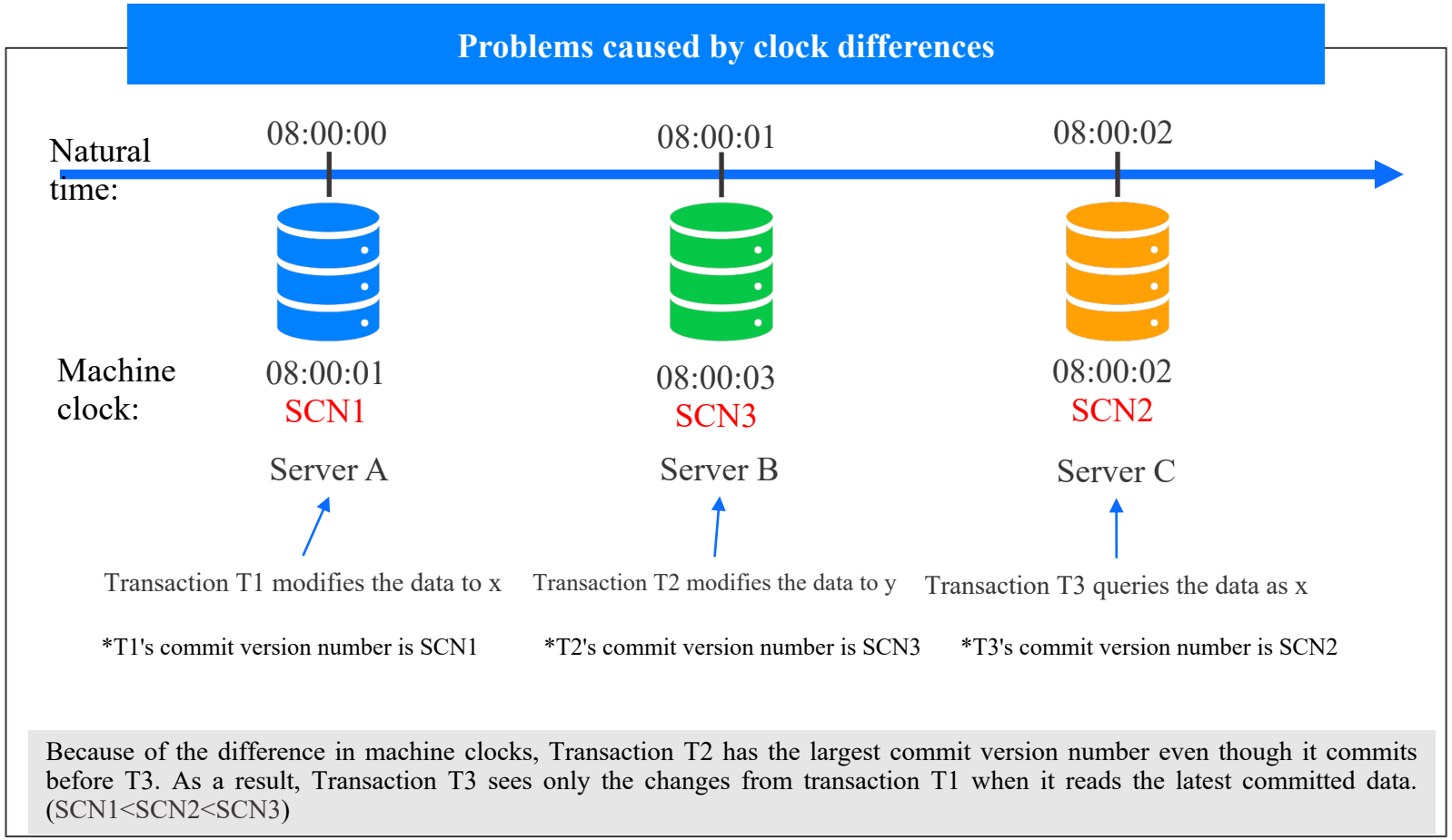
Challenges of data consistency: When multiple nodes are processing transactions at the same time, data on each node must remain consistent, and distributed databases must be able to cope with the challenges of data timing and consistency brought about by concurrent operations, e.g., network latency, node failures, and clock differences.

Challenges of performance degradation: In distributed systems, the commit of a distributed transaction shall interact with participants at different nodes, and the commit result of the transaction shall be synchronized across multiple replicas. Network latency will affect the speed of transaction commit.

Problems Caused by Clock Differences

Unlike the Shared-Everything architecture of traditional databases, OceanBase is a native distributed architecture with the Shared-Nothing architecture. It will face the technical challenges posed by distributed architectures when realizing globally (cross-machine) consistent snapshot isolation and multi-version concurrency control.

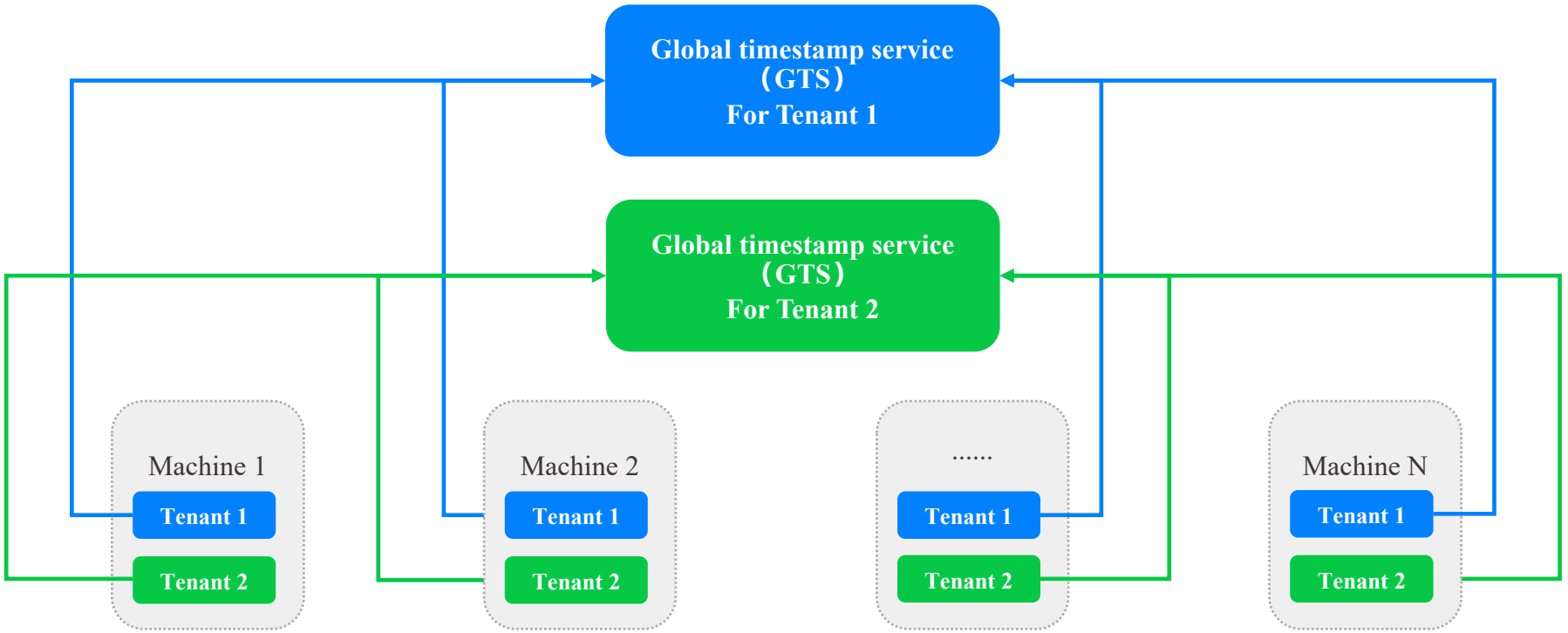




Because of the difference in machine clocks, Transaction T2 has the largest commit version number even though it commits before T3. As a result, Transaction T3 sees only the changes from transaction T1 when it reads the latest committed data. (SCN1<SCN2<SCN3)

Global Timestamp Service (GTS) of OceanBase

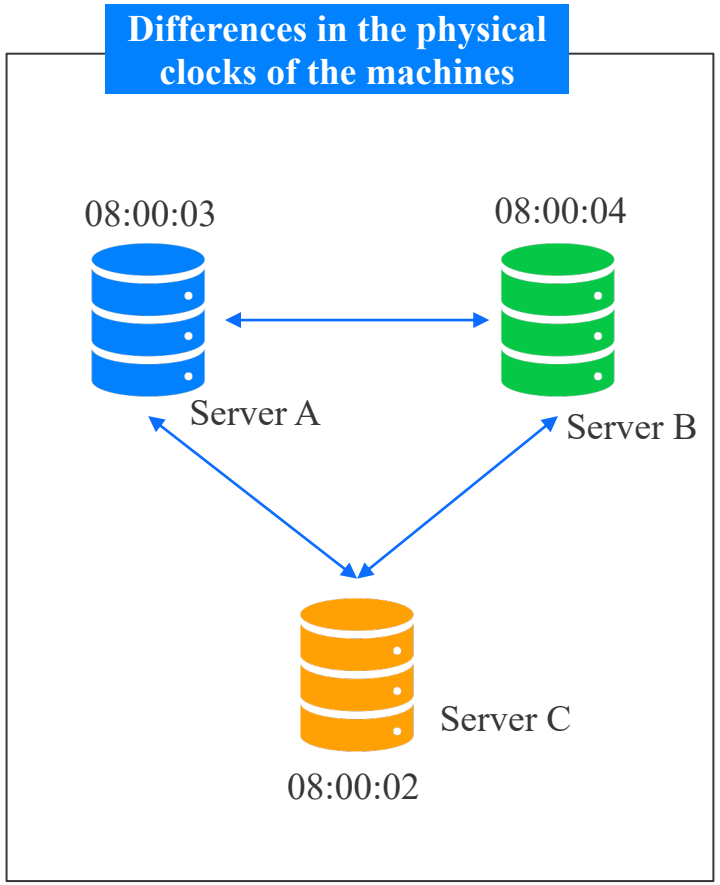
OceanBase's Global Timestamp Service (GTS) is a core component of its distributed database. Each OceanBase tenant starts a GTS service.When the transaction is modifying data or querying data, regardless of the physical machine from which the request originates, it will get the version number from this centralized service, thus ensuring all version numbers are monotonically forward and consistent with the real-world chronology.

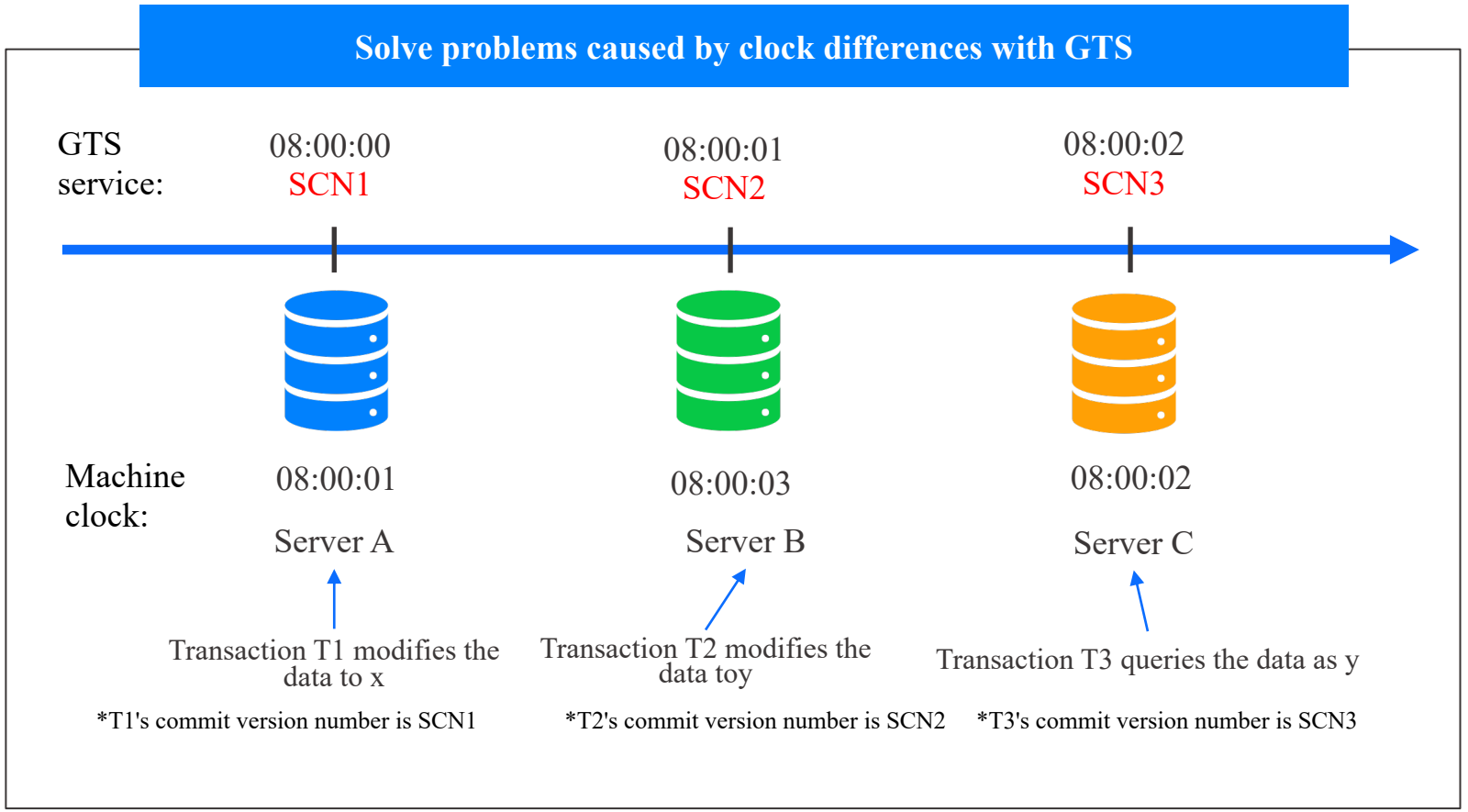


OceanBase database cluster

Solve Problems Caused by Clock Differences with GTS

GTS assigns a globally unique, monotonically incrementing timestamp to each transaction, to ensure transactions are committed in the same order in a multi-replica, multi-node environment. Meanwhile, GTS can ensure that the sequence of data reading and transaction commit between different nodes of the transaction is consistent, and the MVCC mechanism is paired with the isolation level to ensure the consistency and isolation of data.





Standard Two-phase Commit

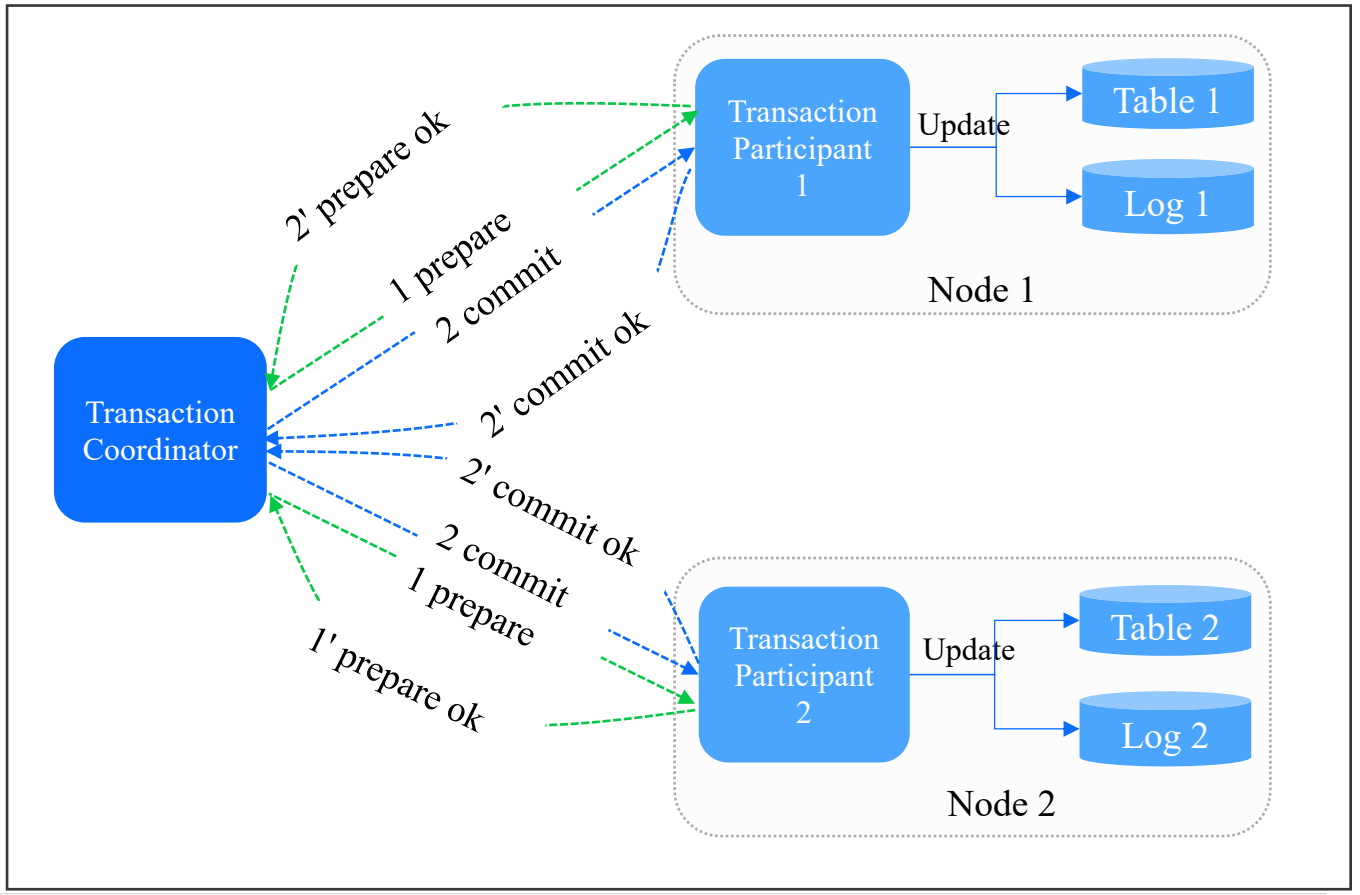
Two-Phase Commit is to commit a transaction in two phases, the Prepare Phase and the Commit Phase. A transaction consists of a Coordinator, which is responsible for coordinating the execution of the entire transaction, and a Participant, which is a database or node that participates in the update of the transaction. Participants in an OceanBase database are log streams, and tables or partitions that belong to a log flow are one participant.

Two-phase commit protocol

In a distributed environment, the participants of a transaction may be distributed in different nodes, and the commit status of each participant must be known upon the transaction commit. Only when all participants commit successfully can the final transaction commit be performed. If any participant fails to commit, all participants have to roll back. Thus, the two-phase commit protocol divides the transaction commit process into the following two phases:

1. **Prepare phase**: At this phase, the coordinator sends a preparation request to all participants and waits for their response. Participants are required to record the results of their operations in the transaction log at this phase, but not to commit the transaction.

2. **Commit phase**: If all participants acknowledge prepare phase completion, the coordinator sends a commit request to all participants. The participant receiving the commit request will formally commit the operation result recorded in the transaction log and release the transaction resources. However, if there is a failure or a participant unable to commit at any stage, the coordinator will issue a rollback request and the participant has to undo the previous operation to return to the state before the transaction starts.

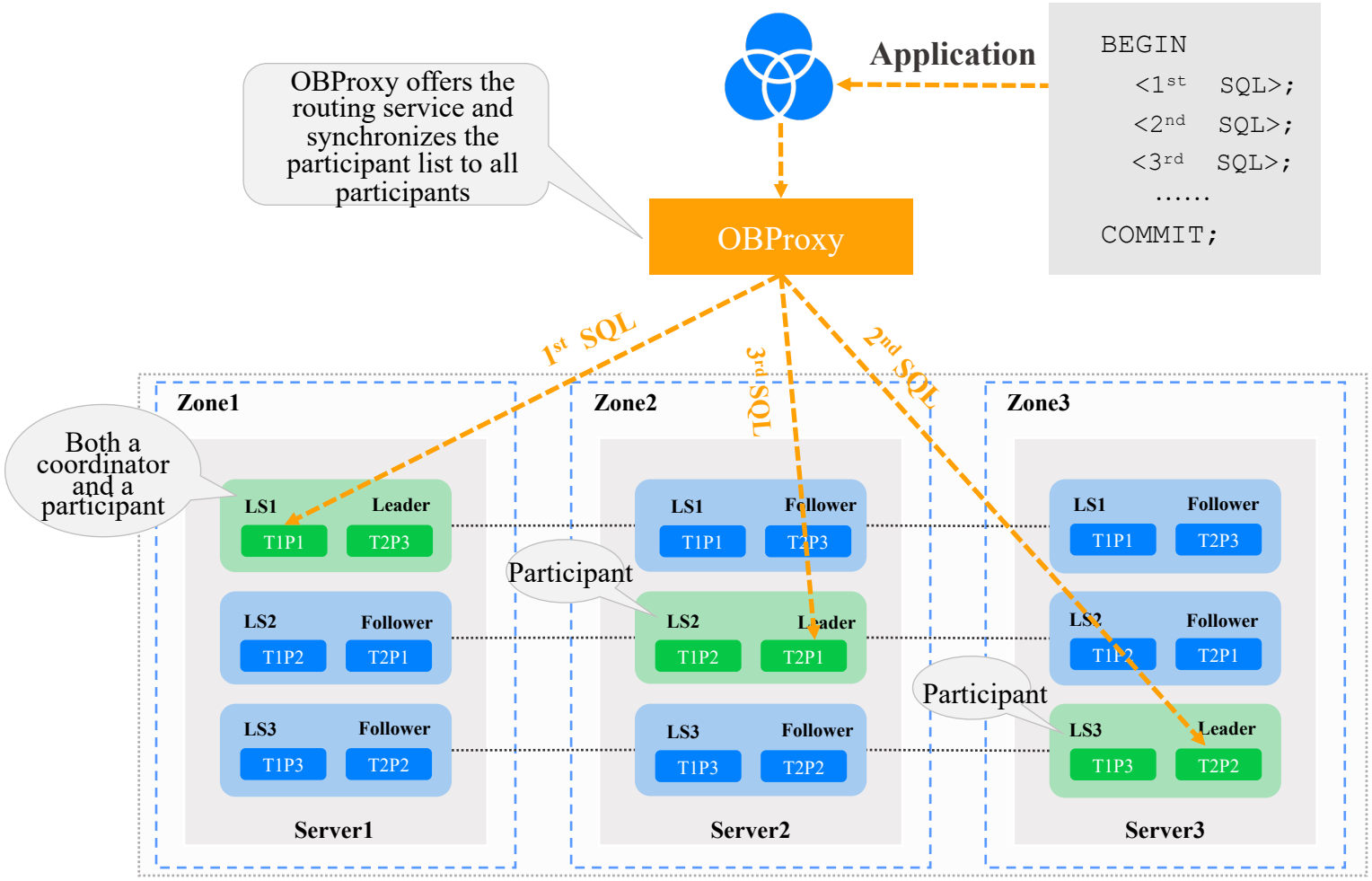


OceanBase's Innovative Two-phase Commit

Based up the original two-phase commit protocol, OceanBase Database optimizes the transaction commit process and lowers the latency of distributed commit according to features of the distributed architecture. Moreover, OceanBase uses the improved Multi-Paxos protocol to support concurrent transactions in multi-replica synchronization with unordered commit, which significantly improves the concurrency performance of distributed transactions.

Features of OceanBase's two-phase commit

- **Atomicity:** OceanBase ensures the atomicity of transactions through the two-phase commit protocol, i.e., the commit of a transaction either succeeds in its entirety or is rolled back in its entirety.
- **High availability:** OceanBase database ensures both the coordinator and participant are multi-replica highly available through Paxos consistency protocol.
- **High performance:** OceanBase optimizes the execution process of the two-phase commit protocol to minimize the performance overhead due to coordination and wait. Paxos supports unordered log synchronization and highly concurrent execution of transactions.
- **Fault tolerance:** OceanBase prevents communication problems caused by network latency or node failures with timeout and retry mechanisms during the transaction commit.
- **Disaster recovery:** In case of node failure, OceanBase will automatically reelect the Leader within 8 seconds, and the new Leader will automatically advance the unfinished transaction execution after reelected.



OceanBase’s Distributed Transaction Engine

OceanBase's distributed transaction engine solves the challenges in transaction processing in a distributed architecture, supporting highly concurrent, low-latency transaction processing while ensuring data consistency and integrity. It employs the multi-replica replication and consistency algorithms to obtain ACID transaction characteristics in a distributed environment.

Core mechanisms of OceanBase’s distributed transaction engine

To realize efficient and reliable distributed transactions, OceanBase uses the following core mechanisms:

- **Global timestamp service:** OceanBase initiates the global timestamp service for each tenant. When a transaction is committed, the transaction version number is obtained from the timestamp service of the tenant, to ensure the global transaction order.
- **Optimized two-phase commit protocol:** The optimized two-phase commit protocol is adopted to ensure all nodes involved in a transaction can successfully commit or roll back the transaction.
- **Consistency algorithm:** Consistency of multiple replicas of the log stream (logs and data) across a distributed environment is ensured by the Paxos consistency algorithm. Multiple replicas of the log stream are replicated among different availability zones to enable fault tolerance and high availability.
- **Automatic disaster recovery:** OceanBase can automatically detect and recover from failures, to ensure correct transaction execution and data consistency. It uses multi-replica synchronization technology to prevent data loss, and maintains a high system availability through automatic leader switching and load balancing mechanisms.

OceanBase transaction categorization

There are various types of transactions in the OceanBase database, which can be categorized into the following main types by the number and location of participants:

- **Single log stream transaction:** If the transaction involves only one log stream, and the leader of this log stream is located on the same Server with the transaction Session, it is a single log stream transaction.
- **Distributed transaction:** If the transaction operation involves data distributed on multiple Log Streams, or even if only one Log Stream is involved, but its leader is not on the same Server as the transaction Session, it is considered a distributed transaction.
- **XA transaction:** XA transactions are global transactions that are executed with the participation of multiple database resource managers. OceanBase's XA transactions maintain their atomicity and consistency by allowing two-phase commits (2PC) to be performed across multiple database instances.

Summary

SQL engine

The OceanBase SQL engine is responsible for receiving SQL statements from the client, and parsing, optimizing and transforming them into plans suitable for distributed execution.

- **High compatibility:** Supports standard SQL syntax and is highly compatible with MySQL and Oracle databases.
- **High concurrency and high performance:** Designed for highly concurrent distributed processing, supporting large-scale data processing and transaction processing.
- **Data consistency:** The SQL engine efficiently handles complex transactional and analytical queries, while ensuring data consistency and integrity.

Storage engine

The OceanBase storage engine employs a distributed architecture that distributes and manages data across multiple nodes by using technologies such as LSM-Tree.

- Minor compaction and major compaction: Effectively solves random write and write amplification problems by minor and major compaction, to improve write performance.
- High compression ratio: With self-developed encoding algorithms and common compression algorithms, OceanBase enables a high compression ratio and a lower disk cost.
- **Data durability:** Real-time writing business data updated to disk is achieved through the WAL mechanism to ensure data persistence, and high availability of data through multi-replica synchronization.

Summary

Transaction Engine

The OceanBase Transaction Engine has powerful distributed transaction processing capabilities to ensure the atomicity, consistency, isolation and durability (ACID) of transactions.

- **Global timestamp service:** Guarantees global transaction order and logic consistency through the global timestamp service.
- **Two-phase commit protocol:** The optimized two-phase commit protocol is adopted to ensure all nodes involved in a transaction can successfully commit or roll back the transaction.
- **Consistency algorithm:** Guarantees consistency in a distributed environment through the Paxos consistency algorithm.

The SQL engine works seamlessly with the storage engine and the transaction engine to ensure that each SQL operation is executed under the constraints of distributed transactions in strict compliance with the ACID principles. It prevents data loss by synchronizing and persisting data changes through distributed transaction logs, using the Paxos protocol for multi-replica synchronization. OceanBase can automatically detect and recover from failures, and maintain high system availability through automatic leader switching and load balancing mechanisms, thus ensuring ultimate data consistency and correct execution of transactions even upon failure of network partitions or nodes.

Self-test

1. [True/false] OceanBase database supports both MySQL schema and Oracle schema tenants in the same system. ( )

2. [True/false] OceanBase's SQL engine does not support foreign key constraints, views, DML triggers and other features. ( )

3. [True/false] In OceanBase database, it is possible to create tables without specifying a primary key. ( )

4. [True/false] OceanBase storage engine uses LSM Tree structure to solve the problems of random write and write amplification. ( )

5. [True/false] OceanBase database will merge incremental data with static data to form a complete SSTable during minor compaction. ( )

6. [Single answer] What mechanism is employed in OceanBase database to solve the problem of data inconsistency caused by clock differences in distributed scenarios? ( )

   A. MVCC B. Two-phase commit C. Table lock D. Global Timestamp Service

7. [Single answer] Which of the following is NOT the main function of the storage engine? ( )

   A. Data storage B. Data read C. Index management D. Transaction control

8. [Single answer] Which consistency algorithm is used by the transaction engine in OceanBase database to ensure consistency in a distributed environment? ( )

   A. Raft B. Paxos C. Quorum D. Zab

9. [Single answer] In OceanBase database, if a transaction involves multiple log streams, what type is such a transaction called? ( )

   A. Single log stream transaction B. Distributed transaction C. XA transaction D. Unit transaction

10. [Single answer] How many times of compression does OceanBase usually perform to achieve better compression effects? ( )

    A. 1 time B: 2 times C: 3 times D: 4 times

Self-test

11. [Multiple answers] Which of the following statements are true about compaction operations in an OceanBase database? ( )

A. Minor compaction is the process of writing data from a MemTable to an SSTable.

B. Major compaction is the process of combining multiple SSTables, as well as baseline data, to form a new baseline SSTable.

C. Minor compaction can be triggered manually, or automatically by the system.

D. Major compaction reduces disk space usage because it removes invalid multiple versions of data.

12. [Multiple answers] Which isolation levels are supported by OceanBase database? ( )

A. Read committed B. Serializable C. Repeatable read D. Read uncommitted

13. [Multiple answer] Which of the following is a characteristic of OceanBase database transactions? ( )

A. Atomicity B. Consistency C. Isolation D. Durability

14. [Multiple answers] Which of the following are challenges for transaction processing in a distributed architecture compared to a centralized database? ( )

A. Unreliability of network communication B. Impact of node failure C. Challenge of data consistency D. Performance degradation

15. [Multiple answers] What mechanisms are used in the OceanBase database to control the concurrent execution of transactions? ( )

A. Consistency algorithm B. MVCC C. Isolation level D. Lock

Contents

I. Product Overview

II. Cluster Architecture

III. Database Engines

IV. Installation and Deployment

V. Application Development

VI. Operations and Maintenance Management

Installation and Deployment

04

- **Introduction to Deployment Form**
- Resource Configuration Requirements
- Standalone Deployment of OceanBase
- Cluster Deployment of OceanBase
- Verification and Use

Overview of OceanBase Database Software Deployment

OceanBase database provides two forms, namely enterprise edition and community edition.

Enterprise edition

- OceanBase Enterprise Edition is a completely self-developed enterprise-grade native distributed database, which achieves financial-grade high availability on commodity hardware;
- OceanBase pioneered new standard for automatic lossless disaster recovery of city-wide failure known as "Five IDCs across Three Regions". As the world's first distributed database to pass the TPC-C benchmark test, it supports over 1,500 nodes in a single cluster, featured by cloud-native, strong consistency, and highly compatible with Oracle/MySQL databases.

Community edition

- The standalone distributed integrated database of the compatible MySQL has the same native distributed architecture as the Enterprise Edition, which supports the enterprise-grade characteristics such as financial-grade high availability, transparent horizontal scaling, distributed transaction, multiple tenants and syntactic compatibility.
- The kernel of OceanBase Database Community Edition is open-source. Moreover, the Community Edition provides open interfaces and rich ecological capabilities to support enterprises or individuals to better achieve customized service requirements.
- Based on Mulan Permissive Software License, it is free for use.

Deployment Methods of OceanBase Database Software

The OceanBase database can be deployed in various modes in different environments. Both the OceanBase Enterprise Edition and Community Edition support standalone centralized deployment and high-availability cluster development.

Deployment methods	Usage methods	Application scenarios
OCP deployment	Cluster deployment function on OCP	OCP offers the capability of deployment and maintenance for OceanBase cluster, suitable for enterprise-grade standalone deployment or distributed cluster deployment in a privatized production environment; and in addition, OCP of the Community Edition provides the OceanBase cluster deployment of the Community Edition.
Installation of observer command line	Manually complete the server initialization by oatcli, environment setting and starting of observer command.	Manually configure and start an observer cluster by using the command line. Due to the lack of O&M capability for OB clusters like OCP, this mode is suitable for testing and learning in a non-production environment. Both the Enterprise Edition and the Community Edition can be installed by command line.
Installation of OBD command line	Use the OBD tool command line for one-click deployment.	OBD is short for the installation and deployment tool OceanBase Deployer. It provides common O&M capabilities such as cluster deployment, package manager, pressure test software, and cluster management. It is mainly used for the one-click deployment of OceanBase clusters in the community edition.
Docker container deployment	Deploy with Docker images	It is suitable for deploying the OceanBase clusters in the community edition under the non-native supported operating systems (such as MAC and Windows).
Ob-operator container deployment	Use ob-operator to realize the containerized deployment of OceanBase clusters.	ob-operator is built on the basis of the Kubernetes Operator framework and is suitable for deploying the OceanBase clusters in the community edition in a Kubernetes environment.

Introduction to OceanBase Deployment Mode

OceanBase database has two deployment modes, namely standalone centralized deployment and high-availability cluster deployment.

Standalone centralized deployment

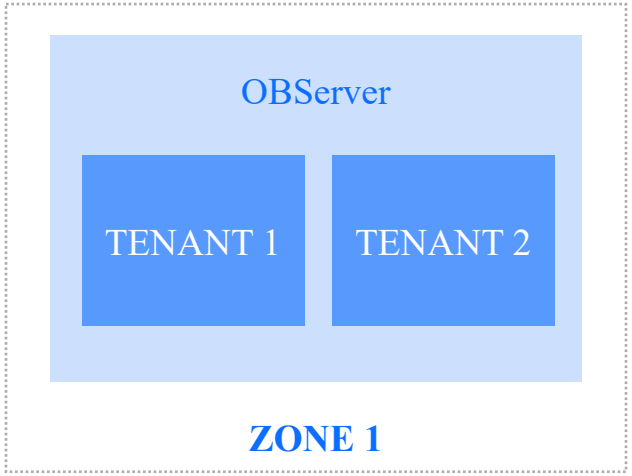
- The standalone centralized deployment only needs a host. All required components can be deployed on a machine, saving resources.
- The standalone centralized deployment is applicable to the service systems that do not require extreme high availability . The high availability mode of traditional centralized databases is available to improve the high availability capability (RAID and primary and standby databases) on oceanbase database.
- For personal study and research, the small-size standalone centralized deployment of OceanBase database can be performed by virtual machine on the personal computer.

High-availability cluster deployment

- OceanBase database adopts a multi-replica architecture based on Shared-Nothing, so that the entire system doesn’t have a single point of failure to ensure the continuous availability of the system.
- OceanBase supports single-center (deployment of OceanBase clusters in a single center), multi-center (deployment of OceanBase clusters in multiple centers in the same city. The internet data center is hereinafter referred to as IDC), multi-city (deployment of OceanBase centers in multiple cities) high availability and disaster recovery. It can be deployed in a single center, two centers, three IDCs across two regions as well as five IDCs across three regions, and supports the deployment of arbitration services to reduce costs.

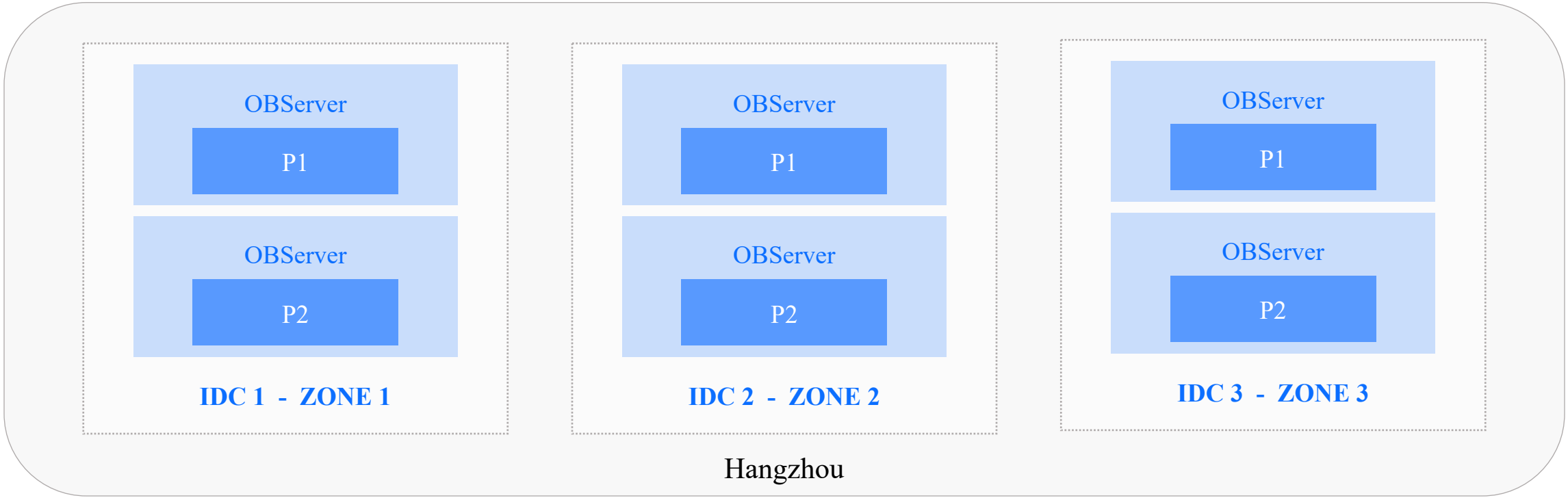
Deployment Mode 1: Standalone Centralized Deployment

- Standalone centralized deployment is an extremely simple database architecture launched by OceanBase for the users. Compared with the distributed cluster deployment, the standalone centralized deployment only needs a host, which is simple and ready to use. The standalone deployment contains a Zone in which there is only one OBServer node.
- Application scenarios:
  - Small-scale business processing without requiring high availability with zero data loss (RPO=0)
  - Study and research, suitable for the equipment with small resource specifications (the memory for the deployment of community edition is not lower than 6 GB)



Deployment Mode 2: High-availability Cluster Deployment

- Three internet data centers in the same city form a cluster (each center is a zone). The network delay in the internet data center is generally within 2 ms.
- In case of a disaster at the level of internet data center, the remaining two replicas still constitute the majority to provide normal external services and ensure RPO=0.
- It is unable to deal with the city-level disasters.



Installation and Deployment

04

- Introduction to Deployment Form
- **Resource Configuration Requirements**
- Standalone Deployment of OceanBase
- Cluster Deployment of OceanBase
- Verification and Use

Requirements for Operating System (OAT/OCP OS)

Operating system	Supported version	Server type
RHEL	Version 7.2 or later	x86_64
CentOS	Version 7.2 or later	x86_64
Alibaba Cloud Linux	2,3	x86_64,ARM aarch64
openSUSE	12SP3 or later	x86_64
Alibaba Cloud Linux	2,3	x86_64,ARM aarch64
Kylin Operating System	V10	ARM aarch64
Huawei EulerOS	2.0 SP8	ARM aarch64

Requirements for Operating System (For OceanBase Database)

Linux OS	Supported version	Server architecture
Alibaba Cloud Linux	2,3	x86_64,ARM_64
Anolis AnolisOS	8.6 or later	x86_64 (including HYGON), ARM_64 (Kunpeng, Phytium)
KylinOS	V10	x86_64 (including HYGON), ARM_64 (Kunpeng, Phytium)
UnionTech UOS	V20	x86_64 (including HYGON), ARM_64 (Kunpeng, Phytium)
NFS-China	4.0 or later	x86_64 (including HYGON), ARM_64 (Kunpeng, Phytium)
Inspur kos	5.8	x86_64 (including HYGON), ARM_64 (Kunpeng, Phytium)
CentOS / Red Hat Enterprise Linux	7.2 or later	x86_64 (including HYGON), ARM_64 (Kunpeng, Phytium)
SUSE Enterprise Linux	12SP3 or later	x86_64 (including HYGON)
Debian	8.3 or later	x86_64 (including HYGON)

■ Note: The operating system needs to be configured with network and software manager (yum or zypper source).

Requirements for Standalone Configuration in a Personal Learning Environment

The minimum configuration requirements for servers are as follows:

Number of servers	Deployed product	Resource configuration			Disk Type
		CPU	Memory	Disk	
1	**ODP (optional)**	1C	2G	500M	
	OCP-express (optional)	1C	2G	Shared OceanBase storage space	Suggested SSD
	OceanBase	Enterprise Edition: 4C Community Edition: 2C	Enterprise Edition: 16G Community Edition: 6G	50G	Suggested SSD
	OBD	It can be ignored as there are low resource requirements.			

- The standalone environment has no special requirements for network card.
- OceanBase V4 supports small-size deployment. Its minimum specification can meet the needs for OBServer function verification and is only used for study and research.
- According to the deployment and installation of components, the resource configuration can be increased for deployed servers as appropriate.

Requirements for Server Configuration in a Production Environment

The minimum configuration requirements for servers are as follows:

Deployed product	Number of servers	Minimum configuration for production			Disk Type
		CPU	Memory	Disk	
OAT	1				
OCP	1	32C	128G Remark 1	1.5T	SSD
OceanBase Enterprise Edition	1/n	32C	256G	Remark 2	SSD
OceanBase Community Edition	1/n	At least four cores are required. 32 cores or more are recommended.	At least 16 GB is required. 256 GB or more is recommended.	Remark 2	SSD
ODP	3	2C	4G	10G	

- Remark 1: The resource requirements of OCP are related to the objects it manges, including the number of OBServers, the number of OceanBase clusters and tenants, and the business volume of the OceanBase cluster.
- Remark2: The log disk in the OBServer disk needs to be 3 times as large as the memory, and the data disk must be sufficient for storing the target data size.

Description of Requirements for Servers in a Production Environment

- The OceanBase Database cluster consists of at least three nodes in high availability deployment. Each node is corresponding to one OBSERVER process, and multiple OBSERVER processes on different nodes form a cluster to provide external services.
- If the OCP management service is required to provide the high availability capability, the high-availability deployment needs to be performed on multiple management servers. After the deployment of an OceanBase cluster, the deployment way of ODP mainly depends on the application scenarios and requirements.
- If the operations such as high availability, load balancing and connection pool management need to be performed for the OceanBase cluster according to the application requirements, the deployment of ODP can be considered. ODP can effectively solve the problems such as excessive number of connections and load balancing, and improve the stability and reliability of applications in OceanBase cluster.
- If an application is simply connected to the OceanBase cluster for query and operation without the requirements for the operations such as high availability, load balancing and connection pool management, it is not a must to deploy ODP. The client library provided by OceanBase can also be directly connected to the cluster so as to meet the application requirements.

Requirements for Configuration of Arbitration Server in a Production Environment

The arbitration server shall meet the minimum machine specification as shown below:

Version	Minimum specification			Bandwidth	Remark
	CPU	Memory	Disk		
V4.1.0 BP1 or later	2C	4G	5 GB	The transmission rate of the inputted and outputted data shall be at least 20 Mbps.	No need for the data disk to store business data; the log directory used for storing the log files requires a separate space from software directory.

For the resource planning of arbitration server, refer to the following rules:

- The machine of minimum specification (2C4G) can support 32 user tenants and sys tenants by default to simultaneously enable arbitration service (2F or 4F, primary_zone=RANDOM, unit_num=1).
- Whenever 32 single unit tenants (2F or 4F, primary_zone=RANDOM, unit_num=1) are increased, the resources of arbitration server shall also be accordingly increased: 1C1G, log disk: 2 GB, network bandwidth: 20 Mbps.
- In the scenario where unit_num=N (N>1), it can be simply treated as N single unit tenants.
- The one-way network delay between the arbitration server and the OceanBase cluster node is less than or equal to 800 ms.

Remarks: If arbitration service needs to be configured for the cluster, the arbitration server shall be configured according to the configuration requirements (the Community Edition doesn't support arbitration service).

Disk Requirements in a Production Environment

The requirements for the disk mount point of OCP server are shown as follows:

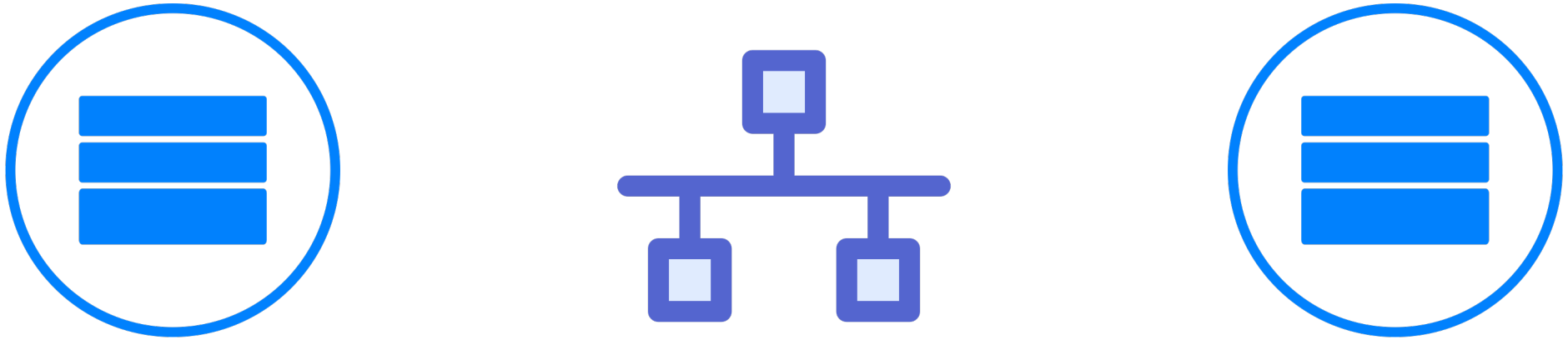
Mount point	Size	Purpose	Disk format
/home	100 GB - 300 GB	Operation log disk of each component	ext4 (recommended)
/data/log1	3 to 4 times as large as the memory	OCP Metadb log disk	ext4 (recommended)
/data/1	Depend on the size of data to be stored	OCP Metadb data disk	ext4 (recommended). xfs is used if greater than 16 TB
/docker	200 GB - 500 GB	Docker root directory	ext4 (recommended)

The requirements for disk mount point of OBSERVER node are shown as follows:

Mount point	Size	Purpose	Disk format
/home	100 GB - 300 GB	Operation log disk of observer process	ext4 (recommended)
/data/log1	3 to 4 times as large as the memory	Log disk of observer process	ext4 (recommended)
/data/1	Depend on the size of data to be stored	Data disk of observer process	ext4 (recommended). xfs is used if greater than 16 TB

- SSD is recommended for disk;
- RAID is not suggested for NVME SSD disk, and LVM is configured at the OS level;
- RAID 5 is suggested for SATA SSD disk. Cache WriteBack With BBU and auto_rebuild are enabled.

Network Requirements in a Production Environment



- It is recommended to configure two 10 gigabit network cards to ensure redundancy.
- eth0 or eth1 is recommended as the name of network card.
- Each server is configured with dual optical port aggregation. Bond is made for dual optical port to start mode4. The switch is configured with 802.3ad protocol and fixed IP. The IP is accessible and the network is stable without high latency.

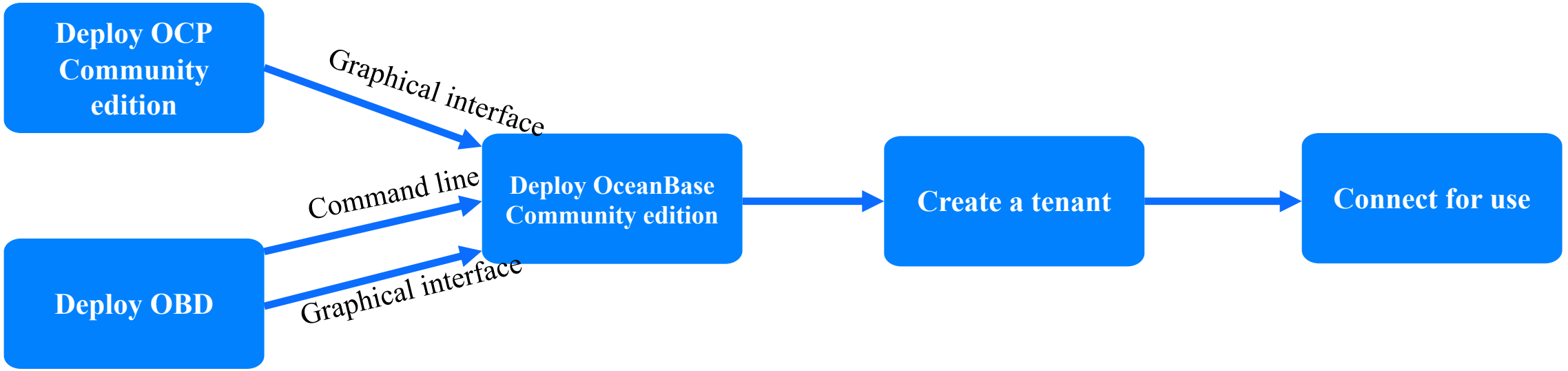
Installation and Deployment

04

- Introduction to Deployment Form
- Resource Configuration Requirements
- **Standalone Deployment of OceanBase**
- Cluster Deployment of OceanBase
- Verification and Use

Deployment Process (By Taking OceanBase Community Edition as an Example)

- Both the Community Edition and the Enterprise Edition support standalone deployment and cluster deployment.
- The deployment method supports graphical interface and also command line.
- The deployment of OceanBase Community Edition can be completed by either OBD or OCP Community Edition.



Step 1: Obtain the Installation Package

■ Online installation

If the machine can be connected to the network, the following commands can be executed to carry out online installation.

```
bash -c "$(curl -s https://obbusiness-private.oss-cn-
shanghai.aliyuncs.com/download-center/opensource/oceanbase-all-in-
one/installer.sh)"

source ~/.oceanbase-all-in-one/bin/env.sh
```

■ Offline installation

If the machine cannot be connected to the network, the following steps can be referred to for offline installation.

Download the latest all-in-one installation package from the OceanBase software download center, and replicate it to any directory on the central control computer.

```
tar -xzf oceanbase-all-in-one-*.tar.gz
cd oceanbase-all-in-one/bin/
./install.sh
source ~/.oceanbase-all-in-one/bin/env.sh
```

Step 2: Configure OBD

The full name of OBD is OceanBase Deployer which is a deployment tool for the installation of OceanBase cluster. The complex configuration process is standardized through the deployment of CLI or GUI to reduce the difficulty in cluster deployment.

Configuration steps:

1. Disable remote repositories `obd mirror disable remote`
2. After installing the all-in-one installation package, the remote repository will be closed by default. You can confirm it by the command "obd mirror list", if the status for Type=remote changes from "Enabled" into "False", it indicates that the remote mirror source has been closed.

```
[admin@OBserver rpms]$ obd mirror list local
+-------------------------------------------------------------------------------------------------------------+
|                                          local Package List                                                 |
+-------------------------+----------+----------------------+--------+----------------------------------+
| name                    | version  | release              | arch   | md5                              |
+-------------------------+----------+----------------------+--------+----------------------------------+
| grafana                 | 7.5.17   | 1                    | x86_64 | 1bf1f338d3a3445d8599dc6902e7aeed4de4e0d6 |
| obagent                 | 4.2.2    | 100000042024011120.el7 | x86_64 | 19739a07a12eab736aff86ecf357b1ae660b554e |
| ob-configserver         | 1.0.0    | 2.el7                | x86_64 | feca6b9c76e26ac49464f34bfa0780b5a8d3f4a0 |
| ob-deploy               | 2.6.2    | 2.el7                | x86_64 | a18d1db136eaf424bec324b08fb8fcd7c7cd61ca |
| ob-deploy               | 2.8.0    | 4.el7                | x86_64 | cd73c47e348ccdb71455b1a671e95f77ef818e0c |
| obproxy-ce              | 4.2.1.0  | 11.el7               | x86_64 | 0aed4b782120e4248b749f67be3d2cc82cdcb70d |
| obproxy-ce              | 4.2.3.0  | 3.el7                | x86_64 | 0490ebc04220def8d25cb9cac9ac61a4efa6d639 |
| oceanbase-ce            | 4.2.1.4  | 104000052024022918.el7 | x86_64 | 736ac0f3379032dd41436c1b5a229f8d42b44e21 |
| oceanbase-ce            | 4.2.1.6  | 106000012024042515.el7 | x86_64 | 66bbd1b4683296d99470ea922b6a17b14b0124a2 |
| oceanbase-ce-libs       | 4.2.1.4  | 104000052024022918.el7 | x86_64 | ed5c776874a7b6f6ef9c67e39be86d3a6d304441 |
| oceanbase-ce-libs       | 4.2.1.6  | 106000012024042515.el7 | x86_64 | 348130264773b11a52774ec9f2fa10d7e05d993c |
| oceanbase-diagnostic-tool | 1.5.2  | 22024010415.el7      | x86_64 | 4c0f0551be4ad04bc40a66763f66f76fb8bdf10c |
| oceanbase-diagnostic-tool | 2.0.0  | 32024041111.el7      | x86_64 | 5deeab8e3359ebf933898136090f72ebb3729f8a |
| ocp-express             | 4.2.2    | 100000022024011120.el7 | x86_64 | 09ffcf156d1df9318a78af52656f499d2315e3f7 |
| openjdk-jre             | 1.8.0_322 | b09.el7             | x86_64 | 051aa69c5abb8697d15c2f0dcb1392b3f815f7ed |
| prometheus              | 2.37.1   | 10000102022110211.el7 | x86_64 | 58913c7606f05feb01bc1c6410346e5fc31cf263 |
+-------------------------+----------+----------------------+--------+----------------------------------+
Trace ID: a09a6a5c-2597-11ef-9d1a-00163e268411
If you want to view detailed obd logs, please run: obd display-trace a09a6a5c-2597-11ef-9d1a-00163e268411
```

3. Add the installation package to the local mirror `obd mirror clone *.rpm`
4. View the package list in the local mirror `obd mirror list local`

Step 2: Configure OBD (II)

Configuration steps:

5. Select a configuration file.
6. View the example of configuration file provided by OBD under the directory of ~/.oceanbase-all-in-one/obd/usr/obd/example. Select the corresponding configuration file according to the resource conditions.
7. Modify the configuration file.

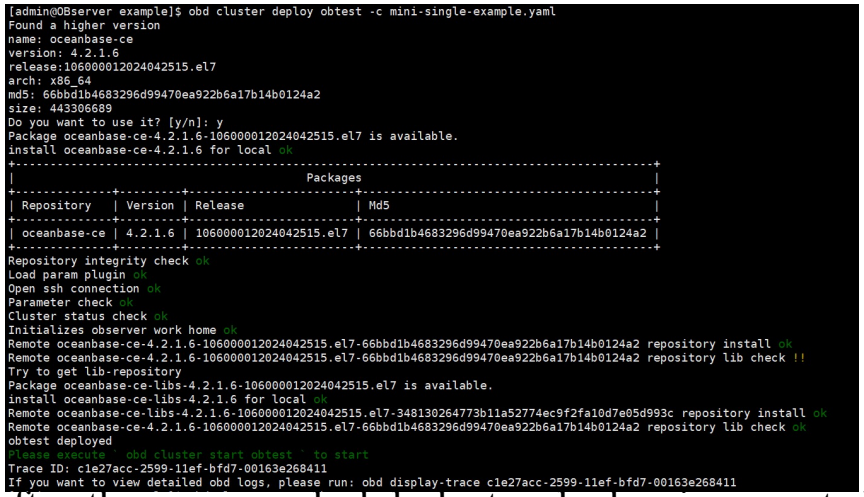
```
## Only need to configure when remote login is required
user:
   username: admin
   password: admin@123
#   key_file: your ssh-key file path if need
#   port: your ssh port, default 22
#   timeout: ssh connection timeout (second), default 30
oceanbase-ce:
  servers:
    # Please don't use hostname, only IP can be supported
    - 172.20.178.75
  global:
    #  The working directory for OceanBase Database. OceanBase Database is started under this directory. This is a required field.
    home_path: /home/admin/observer
    # The directory for data storage. The default value is $home_path/store.
    data_dir: /data/1
    # The directory for clog, ilog, and slog. The default value is the same as the data_dir value.
    redo_dir: /data/log1
    # Starting from observer version 4.2, the network selection for the observer is based on the 'local_ip' parameter, and the 'devname' parameter is no longer mandatory.
    # If the 'local_ip' parameter is set, the observer will first use this parameter for the configuration, regardless of the 'devname' parameter.
    # If only the 'devname' parameter is set, the observer will use the 'devname' parameter for the configuration.
    # If neither the 'devname' nor the 'local_ip' parameters are set, the 'local_ip' parameter will be automatically assigned the IP address configured above.
    # devname: eth0
    mysql_port: 2881 # External port for OceanBase Database. The default value is 2881. DO NOT change this value after the cluster is started.
    rpc_port: 2882 # Internal port for OceanBase Database. The default value is 2882. DO NOT change this value after the cluster is started.
    obshell_port: 2886 # Operation and maintenance port for Oceanbase Database. The default value is 2886. This parameter is valid only when the version of oceanbase-ce is 4.2.2.0 o
r later.
    zone: zone1
    cluster_id: 1
    # please set memory limit to a suitable value which is matching resource.
    memory_limit: 6G # The maximum running memory for an observer
    system_memory: 1G # The reserved system memory. system_memory is reserved for general tenants. The default value is 30G.
    datafile_size: 2G # Size of the data file.
    datafile_next: 2G # the auto extend step. Please enter an capacity, such as 2G
    datafile_maxsize: 20G # the auto extend max size. Please enter an capacity, such as 20G
    log_disk_size: 14G # The size of disk space used by the clog files.
    cpu_count: 16
    production_mode: false
    enable_syslog_wf: false # Print system logs whose levels are higher than WARNING to a separate log file. The default value is true.
    enable_syslog_recycle: true # Enable auto system log recycling or not. The default value is false.
    max_syslog_file_count: 4 # The maximum number of reserved log files before enabling auto recycling. The default value is 0.
    root_password: root@123
```

Here the small-size development mode - standalone deployment (mini-single-example.yaml) is taken as an example to introduce how to modify the configuration file. It is necessary to modify the user information as well as the IP address, port, and related directory of the machine, and configure memory related parameters and password.

Step 3-1: Deploy OceanBase via OBD CLI

Configuration steps:

1. Deploy OceanBase database `obd cluster deploy obtest -c mini-single-example.yaml`



- When the network is connected, after the command obd cluster deploy is executed, OBD will check whether the target machine has been configured with required installation package. If there is no installation package, OBD will obtain it from YUM automatically.
- This command will check whether the directory pointed by home_path and data_dir is empty. If the directory is not empty, an error message will be displayed. If it is confirmed that the contents under this directory can be deleted, the option -f can be added to compulsively clear the directory.

Step 3-1: Start OceanBase via OBD CLI

Configuration steps:

2. Start OceanBase database `obd cluster start obtest`
3. View OBD cluster list `obd cluster list`

```
[admin@OBserver example]$ obd cluster start obtest
Get local repositories ok
Search plugins ok
Load cluster param plugin ok
Open ssh connection ok
Check before start observer ok
[WARN] OBD-1007: (172.20.178.75) The recommended number of stack size is unlimited (Current value: 10240)

Start observer ok
observer program health check ok
Connect to observer 172.20.178.75:2881 ok
Initialize oceanbase-ce ok
Wait for observer init ok
+-----------------------------------------------+
|                   observer                    |
+--------------+---------+------+-------+-------+
| ip           | version | port | zone  | status |
+--------------+---------+------+-------+-------+
| 172.20.178.75 | 4.2.1.6 | 2881 | zone1 | ACTIVE |
+--------------+---------+------+-------+-------+
obclient -h172.20.178.75 -P2881 -uroot -p'root@123' -Doceanbase -A

cluster unique id: a119539e-be76-50f8-b9a2-a71e2d75117d-18ff80355e4-06010204

obtest running
Trace ID: f11e4a32-2599-11ef-a23e-00163e268411
If you want to view detailed obd logs, please run: obd display-trace f11e4a32-2599-11ef-a23e-00163e268411
```

4. View obtest cluster status `obd cluster display obtest`

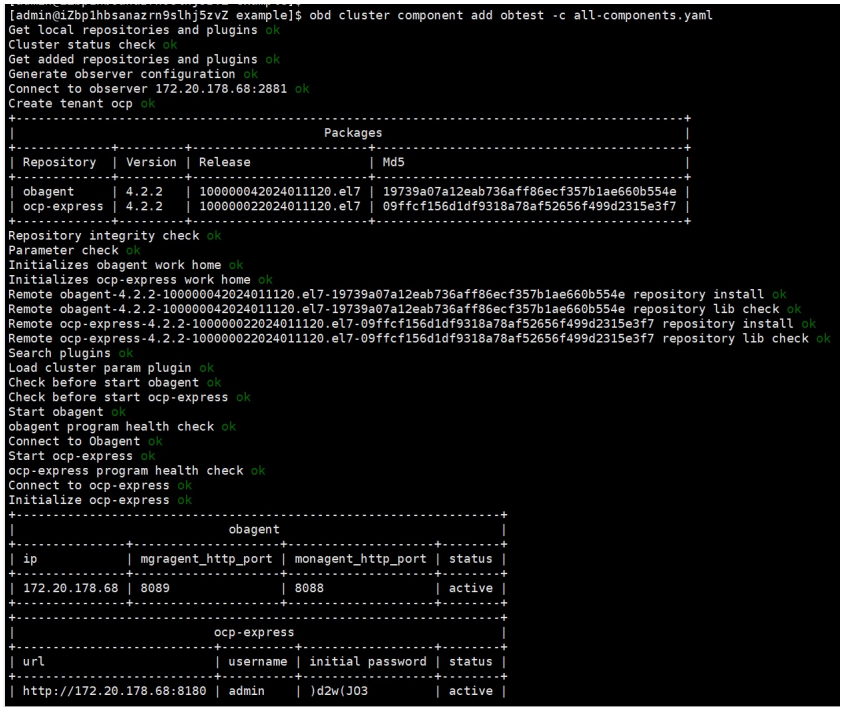
Step 3-1: Deploy OCP Express via OBD CLI

OCP Express is a Web-based OceanBase 4.x management tool. It is integrated in an OceanBase Database cluster to view the key performance indicators of the database cluster and manage the underlying database. It is a lightweight OCP.

Configuration steps:

5. OCP Express is deployed by adding components. Components can be added to the running cluster, such as oceanbase-ce, obproxy-ce, obagent, ocp-express, prometheus, grafana and ob-configserver. Before adding a component, you must ensure that this component does not exist in the cluster.

```
obd cluster component add obtest -c all-
components.yaml
```



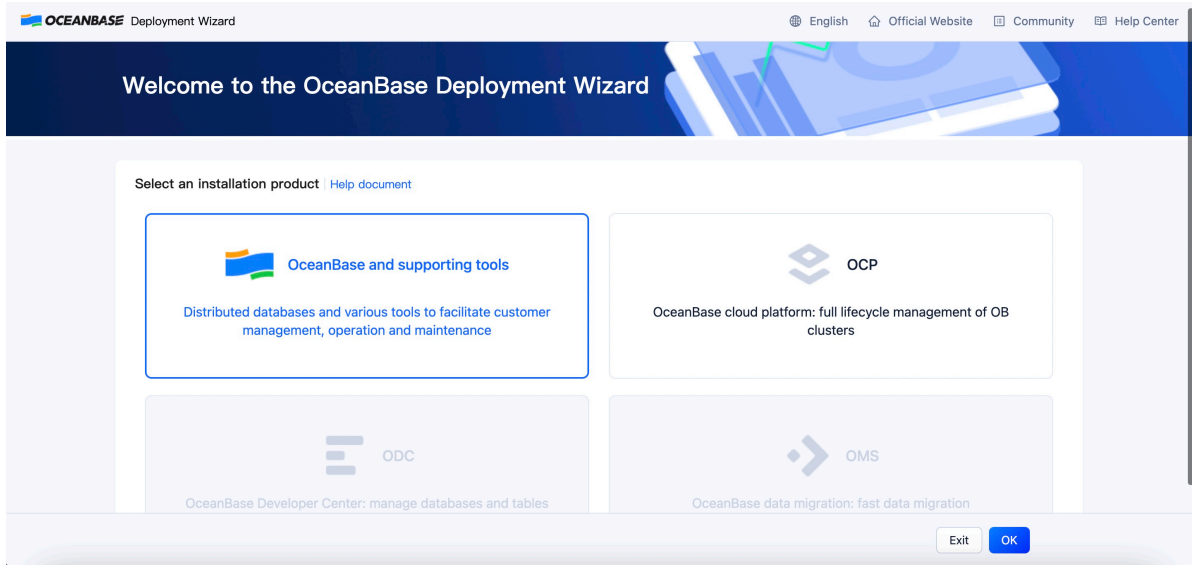
Step 3-2: Use the OBD Web Command to Start the OBD GUI

- Enter the obd web command to start the OBD GUI;
- http://<ip address>:8680 is the access address. Please note that this CLI window cannot be closed during deployment.

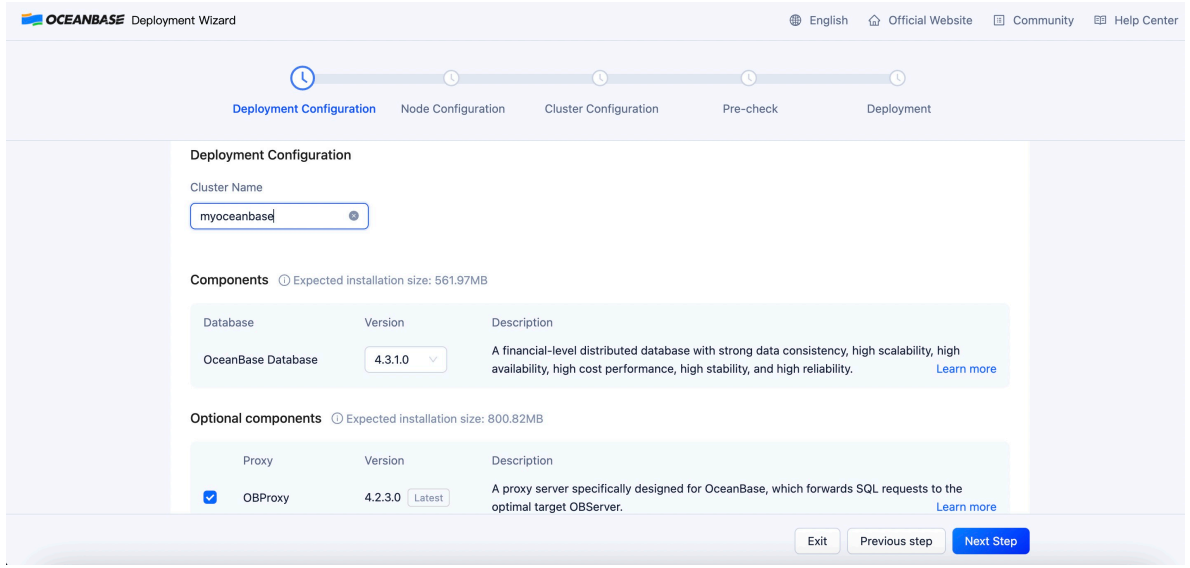
```
[admin@iZbp1hbsanazrn9slhj5zvZ ~]$ obd web
start OBD WEB in 0.0.0.0:8680
please open http://172.20.178.68:8680
2024-04-08 08:47:25,489 INFO dispatch (request_response_log.py:40) [7965ded01a3240569ba5447d1e3b719a] app receive request, method: GET, url: http://172.20.178.68:8680/favicon.ico, q
uery_params: , body: , from: 192.168.111.22:57545
2024-04-08 08:47:25,490 INFO dispatch (idle_shutdown.py:36) [7965ded01a3240569ba5447d1e3b719a] dispatch request and update last request time
2024-04-08 08:47:25,493 INFO dispatch (request_response_log.py:43) [7965ded01a3240569ba5447d1e3b719a] app send response, code: 404
2024-04-08 08:47:26,095 INFO dispatch (request_response_log.py:40) [c578dddeb4bd4e3da2db3497a8ae5a12] app receive request, method: GET, url: http://172.20.178.68:8680/favicon.ico, q
uery_params: , body: , from: 192.168.111.22:57545
2024-04-08 08:47:26,095 INFO dispatch (idle_shutdown.py:36) [c578dddeb4bd4e3da2db3497a8ae5a12] dispatch request and update last request time
2024-04-08 08:47:26,098 INFO dispatch (request_response_log.py:43) [c578dddeb4bd4e3da2db3497a8ae5a12] app send response, code: 404
2024-04-08 08:47:26,737 INFO dispatch (request_response_log.py:40) [12dd6dcdbabc46328edfe1c072fce915] app receive request, method: GET, url: http://172.20.178.68:8680/, query_params
: , body: , from: 192.168.111.22:57547
2024-04-08 08:47:26,737 INFO dispatch (idle_shutdown.py:36) [12dd6dcdbabc46328edfe1c072fce915] dispatch request and update last request time
2024-04-08 08:47:26,741 INFO dispatch (request_response_log.py:43) [12dd6dcdbabc46328edfe1c072fce915] app send response, code: 200
2024-04-08 08:47:26,807 INFO dispatch (request_response_log.py:40) [97111b983d6e4b3db6c6a6fbb77a5e67] app receive request, method: GET, url: http://172.20.178.68:8680/umi.js, query_
params: , body: , from: 192.168.111.22:57548
2024-04-08 08:47:26,807 INFO dispatch (request_response_log.py:40) [8ed1c4c27d814c3981a47a6e821c096a] app receive request, method: GET, url: http://172.20.178.68:8680/umi.css, query
_params: , body: , from: 192.168.111.22:57547
2024-04-08 08:47:26,808 INFO dispatch (idle_shutdown.py:36) [97111b983d6e4b3db6c6a6fbb77a5e67] dispatch request and update last request time
2024-04-08 08:47:26,809 INFO dispatch (idle_shutdown.py:36) [8ed1c4c27d814c3981a47a6e821c096a] dispatch request and update last request time
2024-04-08 08:47:26,811 INFO dispatch (request_response_log.py:43) [97111b983d6e4b3db6c6a6fbb77a5e67] app send response, code: 200
2024-04-08 08:47:26,976 INFO dispatch (request_response_log.py:43) [8ed1c4c27d814c3981a47a6e821c096a] app send response, code: 200
2024-04-08 08:47:27,528 INFO dispatch (request_response_log.py:40) [4616bf871f2d465ea5c588baf65e1645] app receive request, method: GET, url: http://172.20.178.68:8680/898.async.js,
query_params: , body: , from: 192.168.111.22:57548
2024-04-08 08:47:27,529 INFO dispatch (request_response_log.py:40) [8cbac609bc9e44f2bb0ae01c67f338b0] app receive request, method: GET, url: http://172.20.178.68:8680/771.async.js,
query_params: , body: , from: 192.168.111.22:57547
2024-04-08 08:47:27,529 INFO dispatch (idle_shutdown.py:36) [4616bf871f2d465ea5c588baf65e1645] dispatch request and update last request time
2024-04-08 08:47:27,530 INFO dispatch (idle_shutdown.py:36) [8cbac609bc9e44f2bb0ae01c67f338b0] dispatch request and update last request time
2024-04-08 08:47:27,533 INFO dispatch (request_response_log.py:43) [8cbac609bc9e44f2bb0ae01c67f338b0] app send response, code: 200
2024-04-08 08:47:27,536 INFO dispatch (request_response_log.py:43) [4616bf871f2d465ea5c588baf65e1645] app send response, code: 200
2024-04-08 08:47:27,558 INFO dispatch (request_response_log.py:40) [ce409aab54e848d3b12d4e540a4b1771] app receive request, method: GET, url: http://172.20.178.68:8680/p__Layout__ind
ex.chunk.css, query_params: , body: , from: 192.168.111.22:57551
2024-04-08 08:47:27,559 INFO dispatch (idle_shutdown.py:36) [ce409aab54e848d3b12d4e540a4b1771] dispatch request and update last request time
2024-04-08 08:47:27,560 INFO dispatch (request_response_log.py:40) [fdfbacbf3e0445879fb4e4a920f33e42] app receive request, method: GET, url: http://172.20.178.68:8680/648.async.js,
query_params: , body: , from: 192.168.111.22:57552
2024-04-08 08:47:27,561 INFO dispatch (request_response_log.py:40) [535c2ad03eed493eae640b5003dbbd39] app receive request, method: GET, url: http://172.20.178.68:8680/p__Layout__ind
ex.async.js, query_params: , body: , from: 192.168.111.22:57553
```

Step 3-2: Deploy OceanBase via OBD GUI

- Click “Start Experience” in the operation interface and select OceanBase and its supporting tool modules in the displayed wizard interface for OceanBase deployment. Click OK and then enter the wizard interface for deployment.

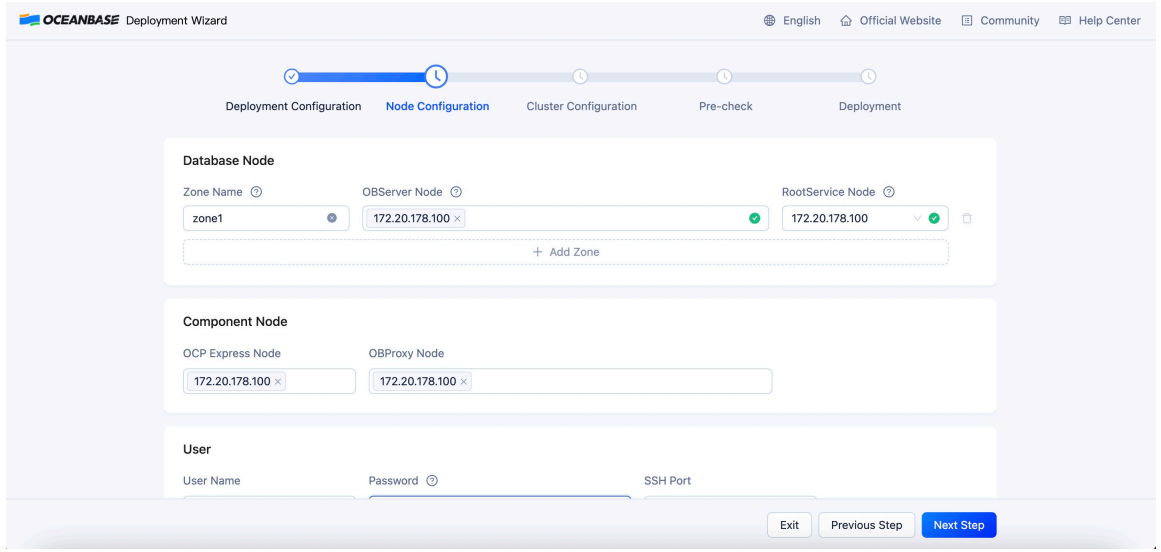


- Deployment configuration: Select cluster name, deployed component, product component as well as product version, etc.

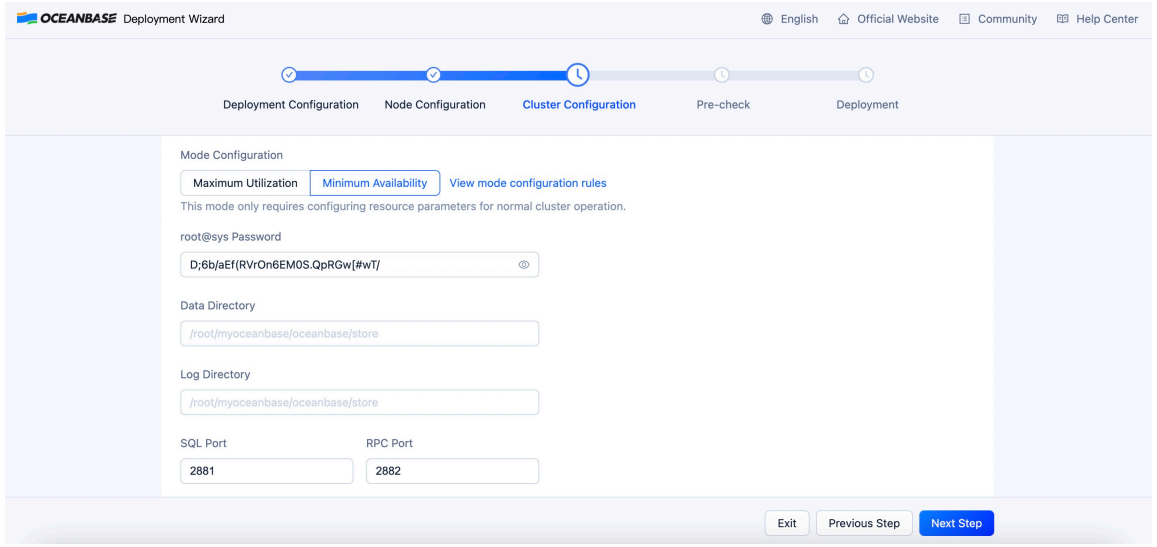


Step 3-2: Deploy OceanBase via OBD GUI

- Node configuration: Carry out the selection of deployment node and the configuration of deployment user.

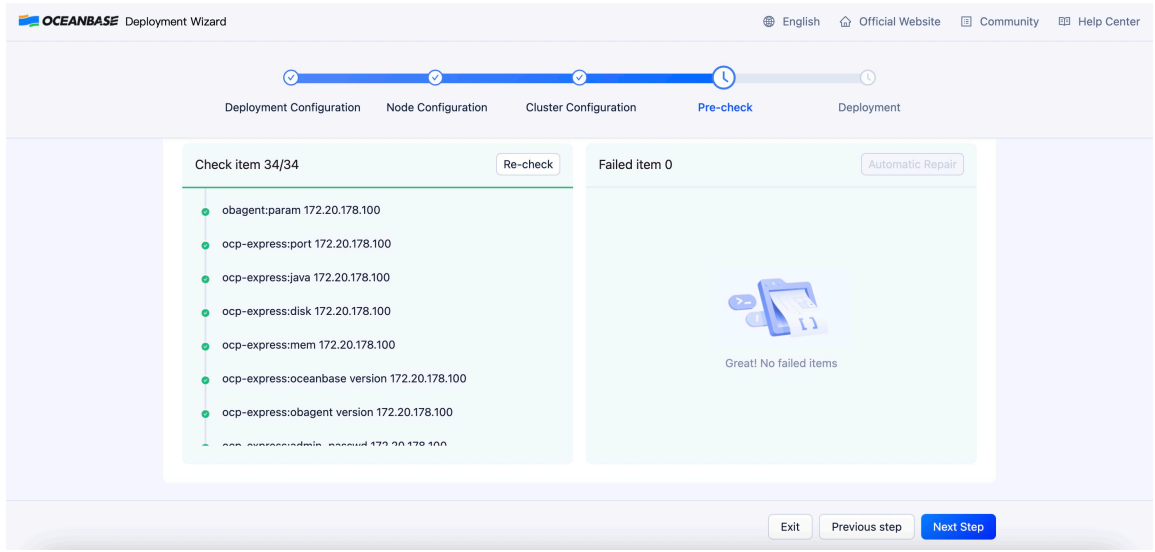


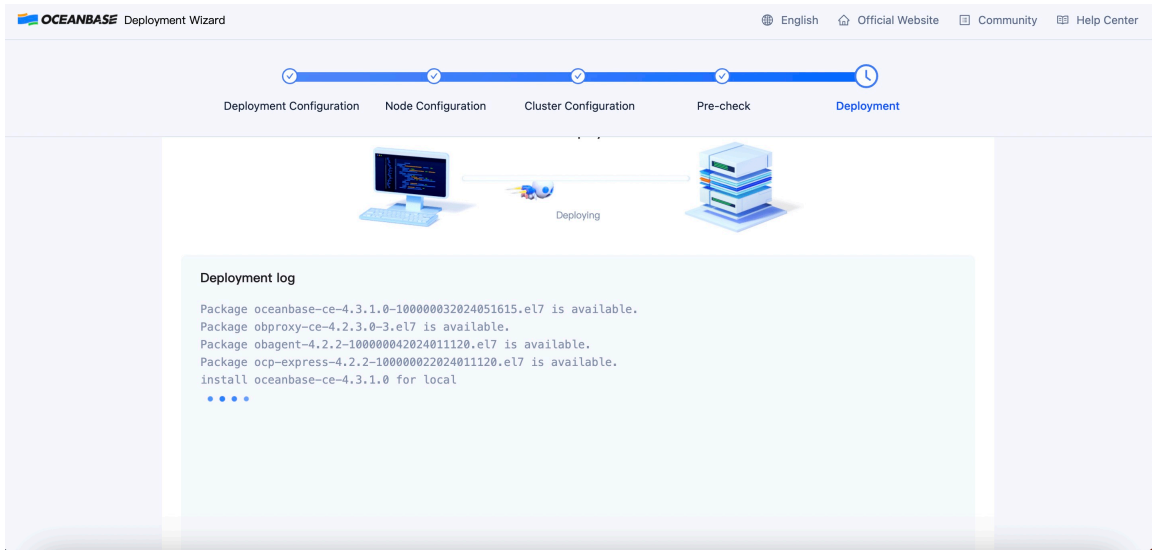
- Cluster configuration: The module is configured with the root@sys password, directory, port and other information of MetaDB (used for storing the ocp – express metadata information).



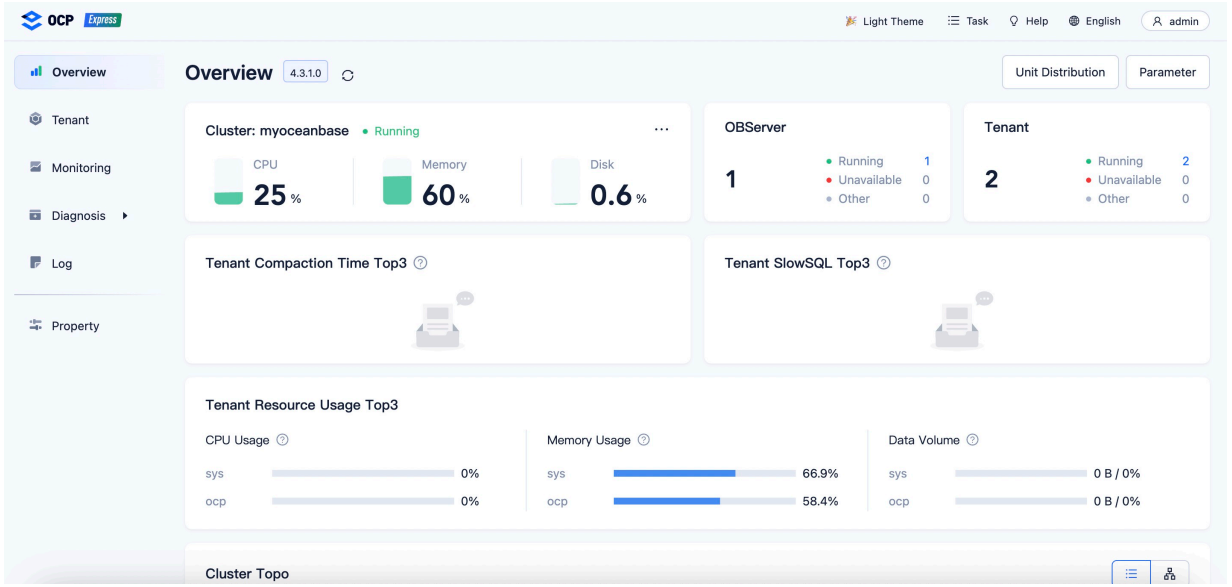
Step 3-2: Deploy OceanBase via OBD GUI

- Pre-check: The pre-check system will verify whether the environment meets all requirements for installation and configuration of MetaDB and other components.
- Deployment: The deployment log will be displayed on the page during deployment, by which the detailed installation log can be viewed.





Step 4: Log in to OCP Express to Monitor and Manage OceanBase



1. **Cluster overview:** View the core information such as cluster topology, status information, resource consumption, and slow SQL.
2. **Tenant management:** Manage the tenants under a cluster, including the creation or deletion of tenants and the management of databases and parameters, etc.
3. **Monitoring center:** Provide the monitoring of cluster and user dimensions and cover dozens of monitoring indicators such as performance monitoring and host monitoring.
4. **Diagnosis center:** Effectively manage the tenant sessions and slow SQL, such as closing sessions and limiting SQL traffic.
5. **Log service:** Analyze the related logs of OBServer node, Agent and host in a cluster on the platform, and locate the faults more conveniently and simply.

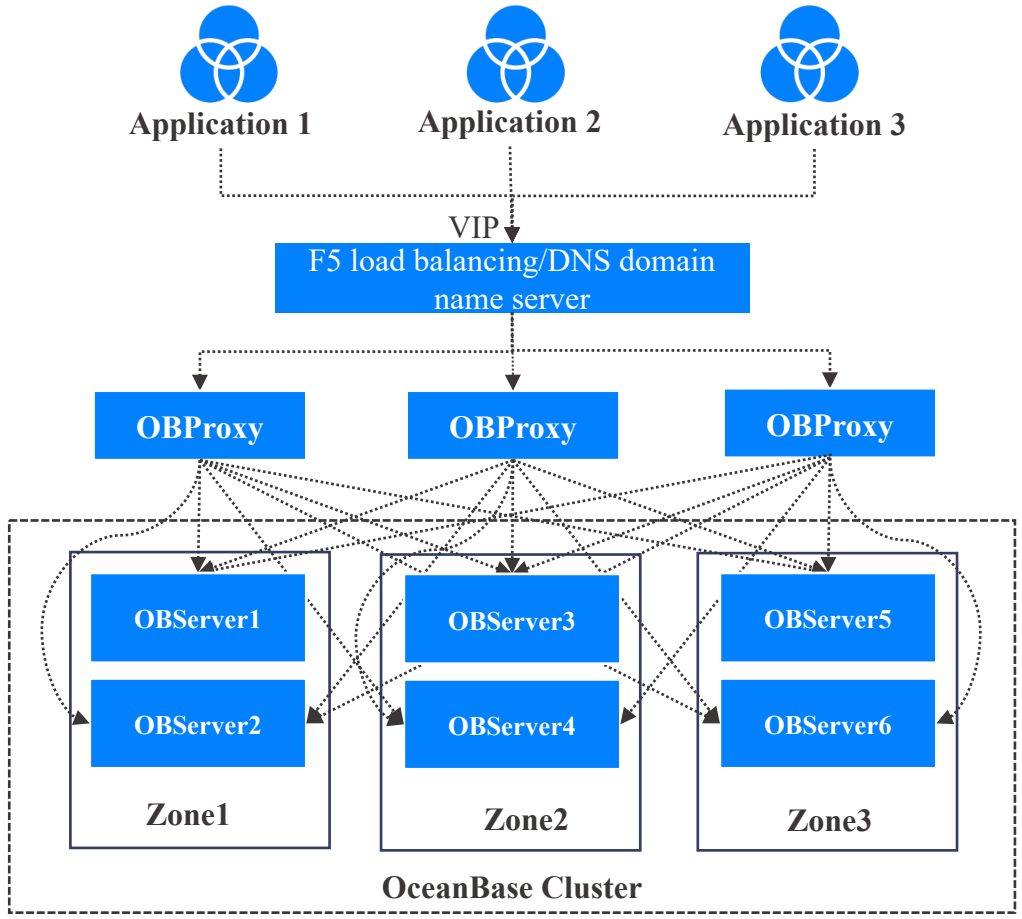
Installation and Deployment

04

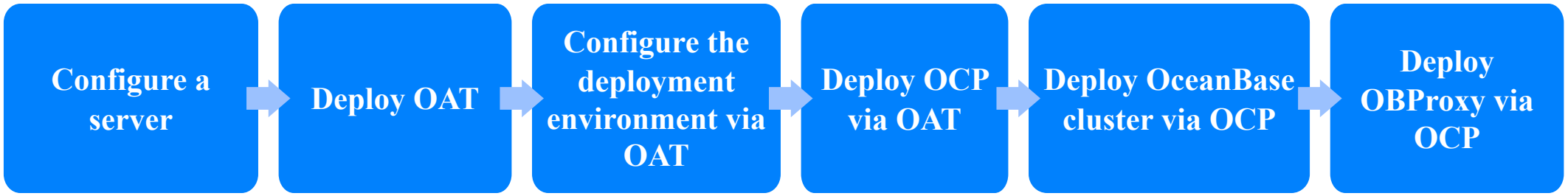
- Introduction to Deployment Form
- Resource Configuration Requirements
- Standalone Deployment of OceanBase
- **Cluster Deployment of OceanBase**
- Verification and Use

High-availability Cluster Deployment

- OceanBase runs a single-process program called observer on the server as a running instance of the database, utilizing local file storage for data and transaction Redo logs.
- Availability zone needs to be configured for the deployment of OceanBase cluster, which consists of several servers. The availability zone is a logical concept that represents a group of nodes with similar hardware availability in a cluster. It represents different meanings in different deployment methods. For example, when the entire cluster is deployed in the same internet data center (IDC), the nodes of an availability zone can belong to the same rack or the same switch, etc. When the cluster is distributed in multiple data centers, each availability zone can be aligned with a single data center.



Deployment Process (By Taking OceanBase Enterprise Edition as an Example)



- Modify the BIOS settings of server, plan the disk usage, create a directory and mount it to the disk.
- Configure the OAT server, and use the oat-all-in-one installation package to deploy OAT.
- Use OAT to complete the server configuration, including parameters, users, dependent packages and other standardized configurations.
- Support single-node cluster deployment and three-node cluster deployment .
- Deploy OceanBase cluster via OCP.
- Deploy OBProxy cluster via OCP to achieve load balancing.

Step 1: Configure the BIOS of Server

	Intel x86	AMD or ARM architecture
NUMA	Off	On
Configuration file	Set the parameter “vm.swappiness” as 0.	Set the parameters “kernel.numa_balancing”, “vm.zone_reclaim_mode” and “vm.swappiness” as 0.
Cstate Pstate EIST Power saving	Off	Off

- NUMA: The basic feature of NUMA（Non-Uniform Memory Access) server is that the hardware resources of the system are divided into multiple nodes by Linux and there are independent CPU, memory and I/O slot, etc. on each node. The speed at which CPU accesses its own node memory is far higher than that at which it accesses remote memory (memory of other nodes in the system). This is also the origin of NUMA.
- vm.swappiness: The kernel parameter “vm.swappiness” controls the relative weight of memory during the swap-out operation. The parameter size has a lot to do with how to use the swap partition. In case that swappiness=0, it means the maximum use of physical memory and then swap space.

Step 1: Prepare a Server (Mounting the Disk)

Suggestions for mounting the disk

- The server of OceanBase Database replies on the **data disk, transaction log disk and software disk of OceanBase Database**. If you are an individual user, you can put all data onto one disk and skip this step. If you are an enterprise user, it is recommended to separately mount the **data disk, transaction log disk and software disk of OceanBase Database**.
- It is recommended to use LVM or RAID for management and adopt the following scheme for partitioning:
  - Data disk: The data disk is used to store the baseline data.
  - Transaction log disk: It is recommended to set the size of transaction log disk to be 3 to 4 times as large as the memory of OceanBase Database or more.
  - Software disk of OceanBase Database: It is recommended to reserve a space of at least 200G for the installation disk of OceanBase Database to store the logs of seven days or more.

Step 1: Prepare a Server (Configure a Network Card)

Suggestions for configuring a network card

- It is recommended to configure two 10 gigabit network cards and perform configuration in the following way:
- The bond mode is named as bond0, mode1 (primary/standby) or mode4 (duplex). mode4 is recommended. Under mode4, the switch needs to be configured with 802.3ad.
- eth0 or eth1 is recommended as the name of network card.
- It is recommended to use the network service instead of NetworkManager.

Step 2: Prepare OAT Deployment Software

OceanBase Admin Toolkit (OAT) serves as a visualization platform for the installation and management of OceanBase ecosystem products and components.

Operation steps:

1. Use the command SCP to upload the oat-all-in-one installation package to the server.

```
scp <oat_directory/oat-all-in-one-xxx.tar> <oat_server_user>@<oat_server_ip>:oat_directory
```

The oat_server_ip is the IP address of the server for deploying OAT. The oat_directory is the directory where the installation package is stored for deployment. The oat_xxx.tar is the name of installation package.

2. Decompress the oat-all-in-one installation package.

The example is as follows: `tar -xvf oat-all-in-one-x86-420BP1.tar`

3. Run the installation script install.sh. Run the installation script install.sh in the decompressed directory oat-all-in-one-x86.

```
sh oat-all-in-one-x86/install.sh
```

Step 2: Deploy OAT

```
[root@OCP oat-all-in-one-x86]# ./install.sh
Before installation, please set the config below:
Input the docker root dir: /docker
Input the OAT data dir: /oat_data
Input the OAT HTTP listen port: 7000
Input the OAT database port: 3306
Input the OAT admin user password(login password): aaAA11__
Start prepare docker

● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; vendor preset: disabled)
   Active: active (running) since Sat 2024-03-30 13:48:22 CST; 2s ago
     Docs: http://docs.docker.com
 Main PID: 25749 (dockerd)
    Tasks: 46
   Memory: 39.8M
   CGroup: /system.slice/docker.service
           ├─25749 /usr/bin/dockerd --bip=172.17.0.1/16 -H unix:///var/run/docker.sock --selinux-enabled=false --log-opt max-size=1g --data-root=/docker --live-restore
           └─25768 containerd --config /var/run/docker/containerd/containerd.toml --log-level info

Mar 30 13:48:19 OCP dockerd[25749]: time="2024-03-30T13:48:19.118272402+08:00" level=info msg="pickfirstBalancer: HandleSubConnStateChange: 0xc000890550, READY" module=grpc
Mar 30 13:48:19 OCP dockerd[25749]: time="2024-03-30T13:48:19.118276691+08:00" level=info msg="pickfirstBalancer: HandleSubConnStateChange: 0xc000155440, CONNECTING" module=grpc
Mar 30 13:48:19 OCP dockerd[25749]: time="2024-03-30T13:48:19.119261667+08:00" level=info msg="pickfirstBalancer: HandleSubConnStateChange: 0xc000155440, READY" module=grpc
Mar 30 13:48:20 OCP dockerd[25749]: time="2024-03-30T13:48:20.831907588+08:00" level=info msg="Graph migration to content-addressability took 0.00 seconds"
Mar 30 13:48:20 OCP dockerd[25749]: time="2024-03-30T13:48:20.832428249+08:00" level=info msg="Loading containers: start."
Mar 30 13:48:22 OCP dockerd[25749]: time="2024-03-30T13:48:22.356785561+08:00" level=info msg="Loading containers: done."
Mar 30 13:48:22 OCP dockerd[25749]: time="2024-03-30T13:48:22.543005052+08:00" level=info msg="Docker daemon" commit=039a7df9ba graphdriver(s)=overlay2 version=18.09.9
Mar 30 13:48:22 OCP dockerd[25749]: time="2024-03-30T13:48:22.543108637+08:00" level=info msg="Daemon has completed initialization"
Mar 30 13:48:22 OCP dockerd[25749]: time="2024-03-30T13:48:22.560025800+08:00" level=info msg="API listen on /var/run/docker.sock"
Mar 30 13:48:22 OCP systemd[1]: Started Docker Application Container Engine.
Created symlink from /etc/systemd/system/multi-user.target.wants/docker.service to /usr/lib/systemd/system/docker.service.
Start prepare OAT
Loaded image: reg.docker.alibaba-inc.com/oceanbase/oat:4.2.0_20231211_x86
ca939534c83ce64f93a561e90ab8a23e3cb12d0e7935dcbbcbcec54592ef2771
check OAT url http://127.0.0.1:7000/hc
OAT API not ready, please wait, sleep 5s retry...
OAT API not ready, please wait, sleep 5s retry...
OAT API not ready, please wait, sleep 5s retry...
OAT API ready
Copy images and binary_packages to OAT data dir
Trigger OAT scan api to find images and binary_packages
Trigger scan task success, please visit OAT web site and wait for scan task finished
OAT is ready for visit
url is: http://172.20.178.49:7000
user/password is: admin/aaAA11
```

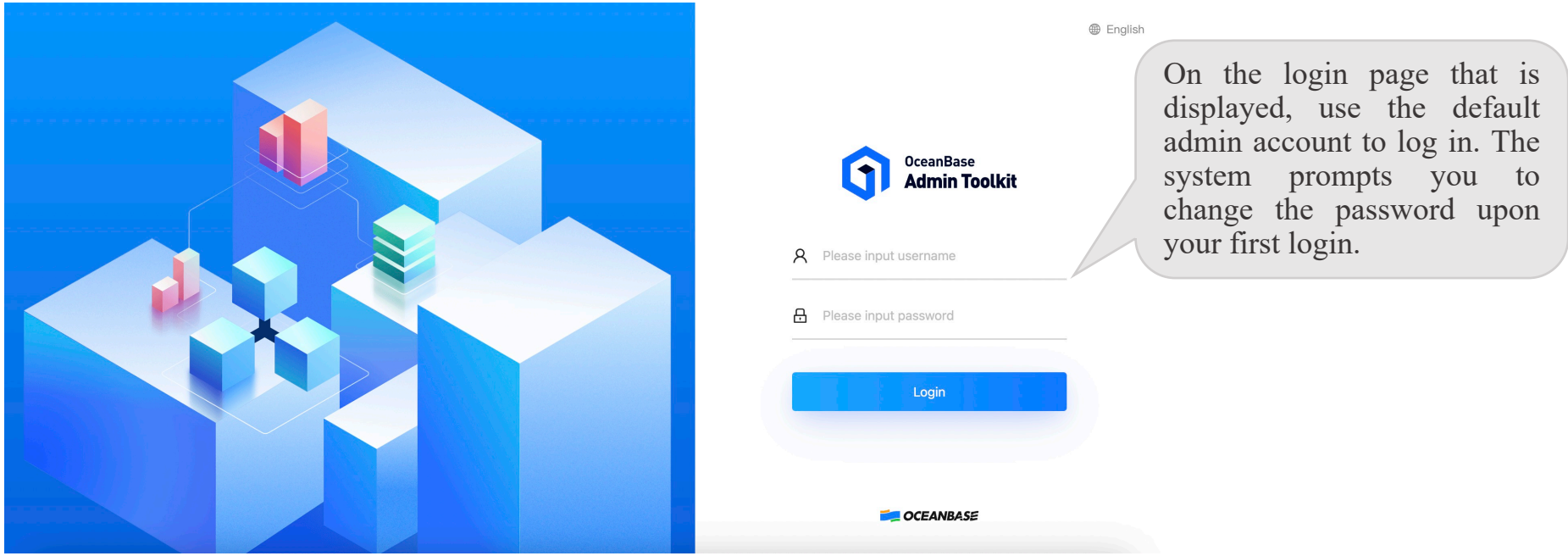
Step 2: Describe OAT Deployment

- By default, the root directory of Docker is /docker. It can be self-defined. If Docker has been installed, this step will be skipped.
- By default, the path of OAT data directory is /oat_data. It can be self-defined.
- By default, the monitoring port of OAT HTTP is 7000. It can be self-defined.
- By default, the database port of OAT is 3306. It can be self-defined.

If there is no Docker in an environment, run the command sh oat-all-in-one-x86/install.sh to automatically install Docker, and deploy OAT and scan the existing mirror and tool package.

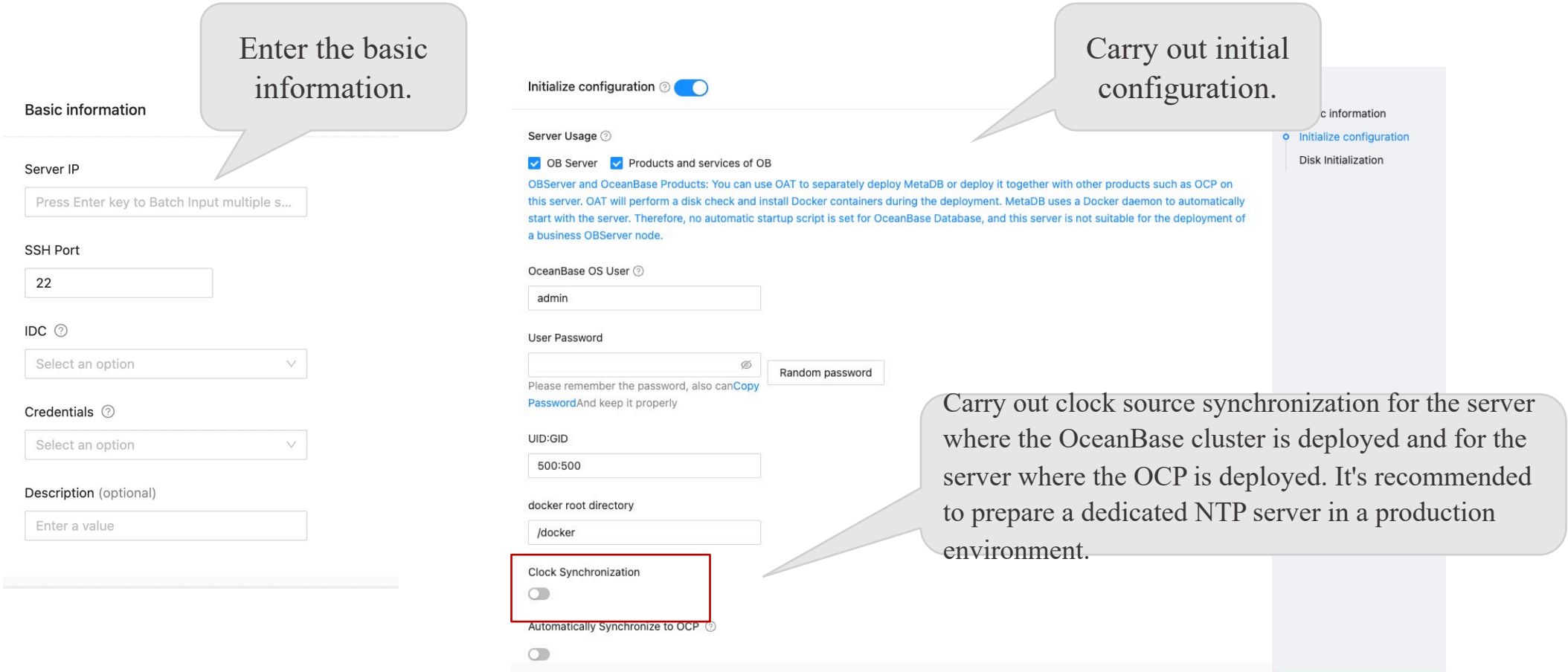
Step 2: Log in to OAT

- Enter the access address of OAT in a browser (http://<IP address of OAT server>:7000).



Step 3: Initialize the Server via OAT

In the left navigation bar, click Server to enter the Server Management page. Then click “Add a Server” in the top right corner of the page.

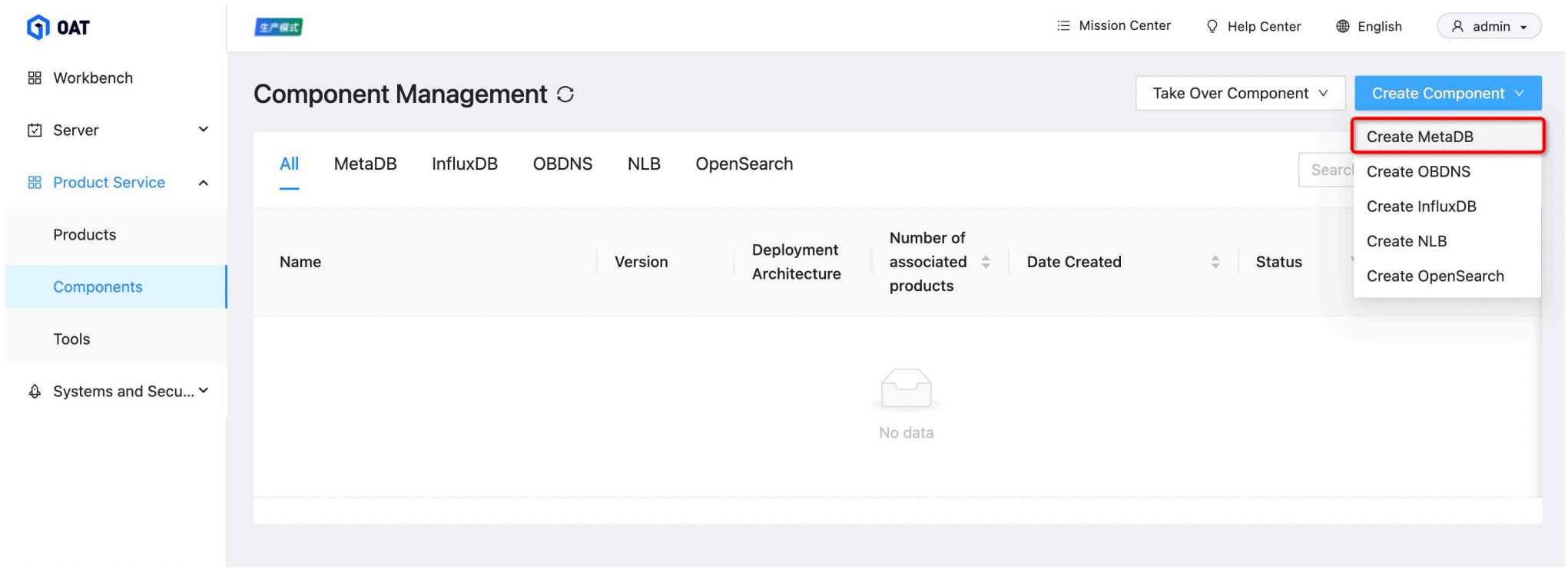


Step 4: Describe the Deployment of OCP via OAT

- OceanBase Cloud Platform (OCP) is an enterprise-level database management platform with OceanBase Database serving as its core.
- The OCP products run in Docker mode and have the same requirements on server structure and Linux operating system as the OceanBase database server. Currently, OCP supports single-node deployment and high-availability deployment. This course takes single-node deployment as an example.
- Single-node deployment means that all functions of OCP are deployed in the same node which provides all capabilities of OCP. including OCP-Server, MetaDB, MonitorDB and OBProxy.
- The MetaDB and MonitorDB of OCP belong to an OceanBase cluster that runs in a Docker container. They can be separately created via OAT, or created while OCP is created.

Step 4: Deploy MetaDB via OAT

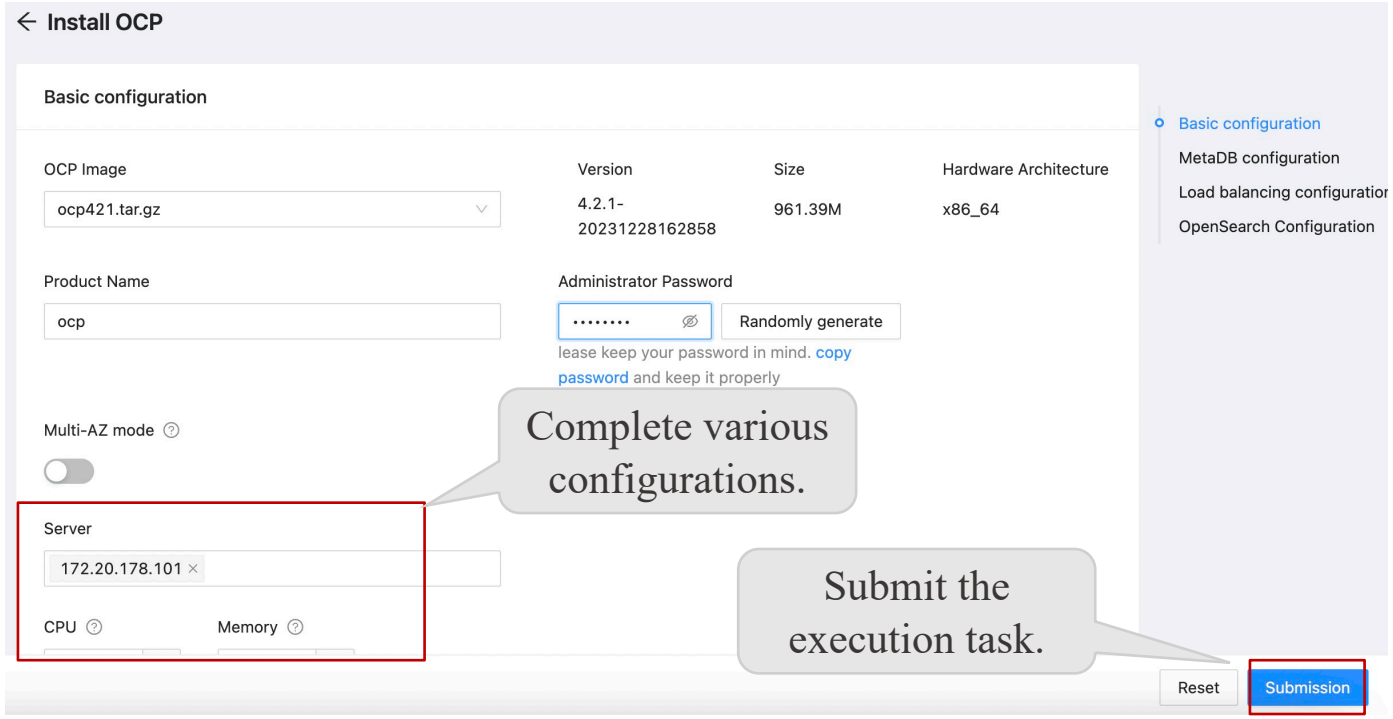
- MetaDB is a container-based OceanBase Database service with a built-in OceanBase Proxy for external direct access. It can be used for storing the metadata of such products as OCP, ODC and OMS.



Step 4: Deploy Single-node OCP via OAT

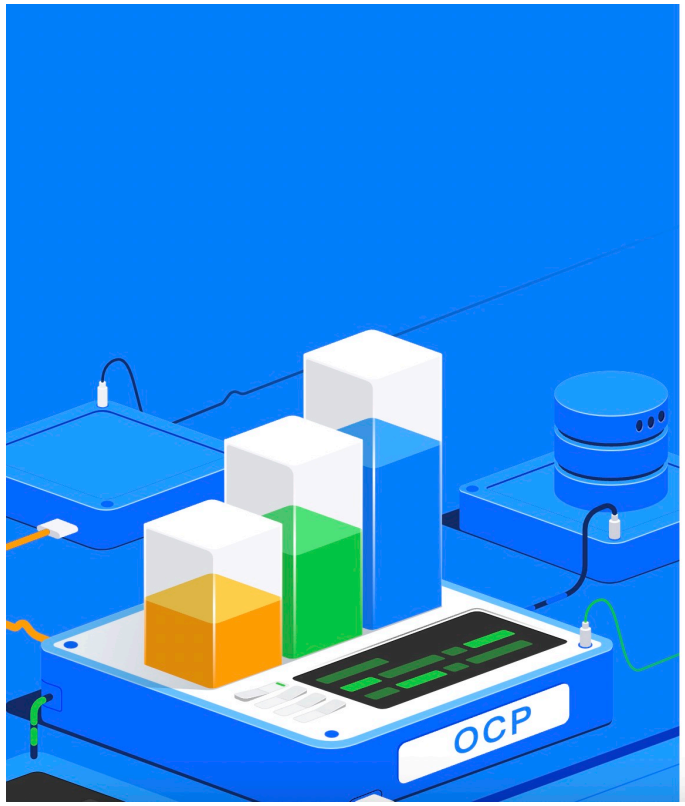
Operation steps

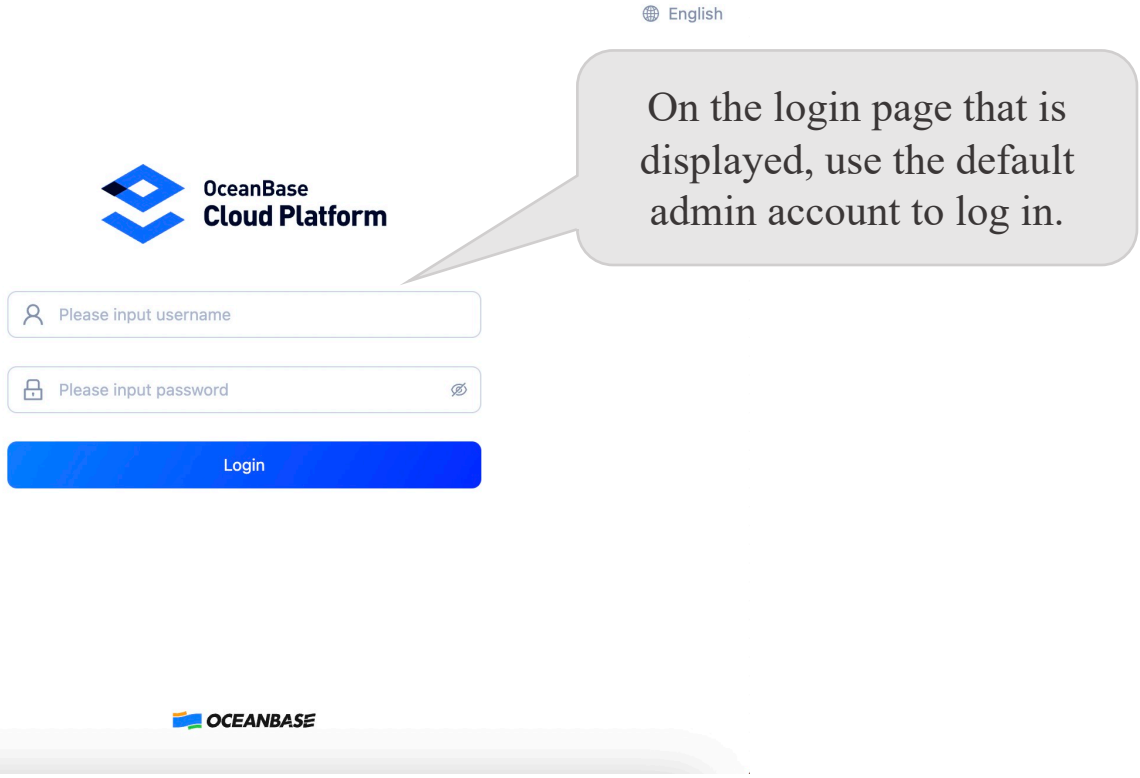
1. Complete underlying configuration on the OCP Installation page.
2. Complete the configuration of MetaDB.
3. Complete the configuration of load balancing.
4. Complete the configuration of OpenSearch.
5. After configuration, click “Submit”.



Step 4: Log in to the OCP Management Platform

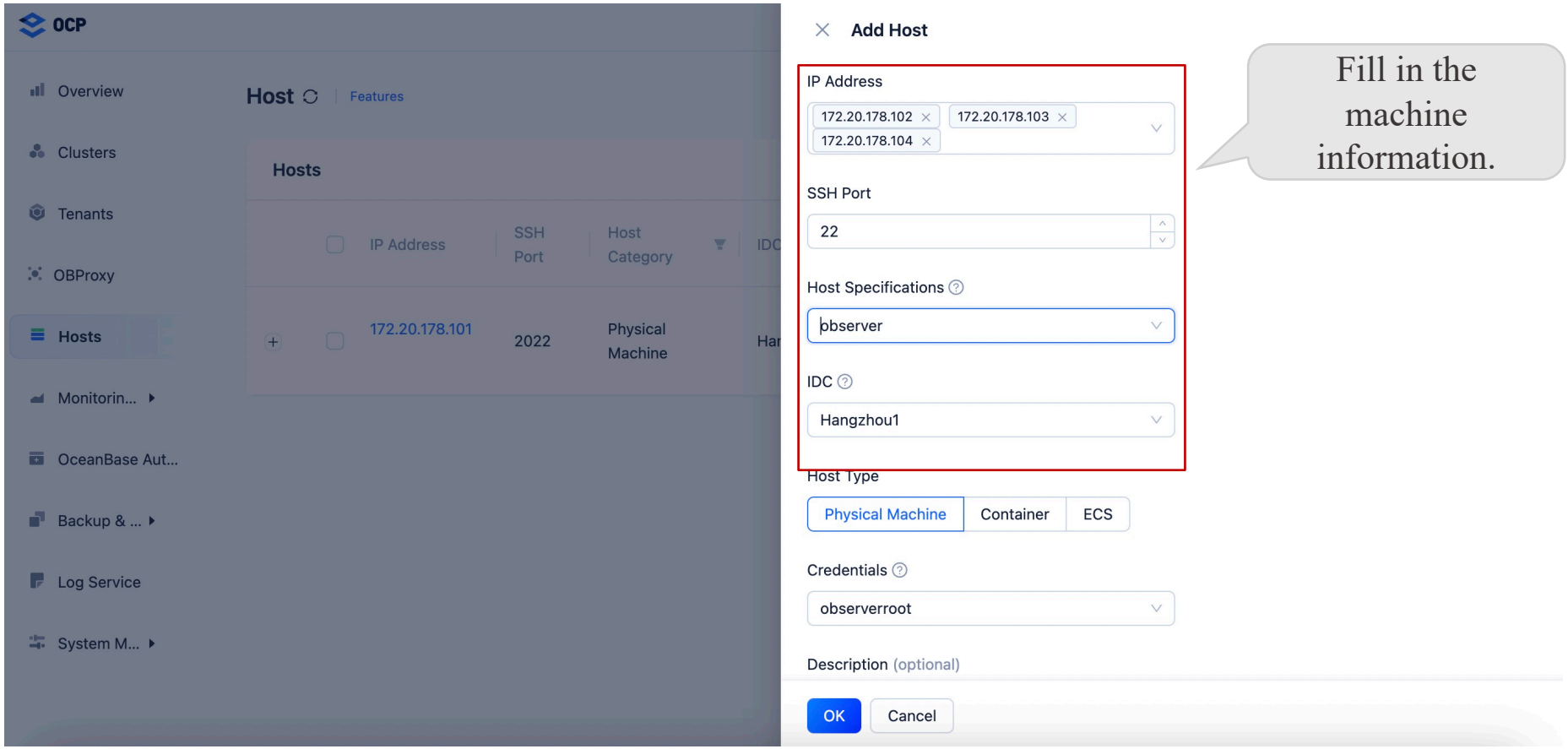
- Enter the access address of OCP in a browser (http://<IP address of OCP server>: **8080**).





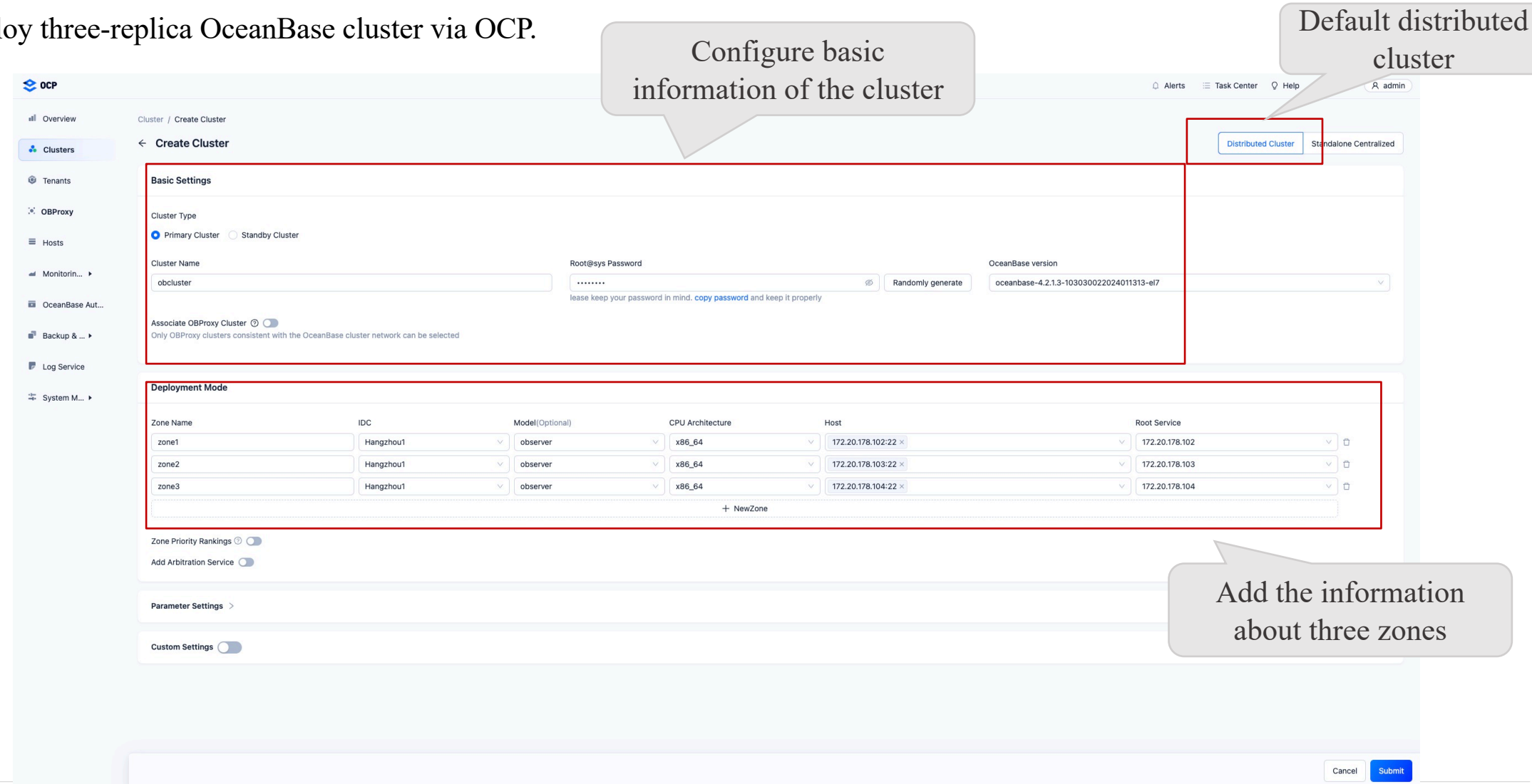
Step 5: Deploy OceanBase Cluster (Add a Server)

- Before deploying OceanBase cluster via OCP, add the servers(hosts) of the OceanBase cluster to be deployed to the OCP resource pool.

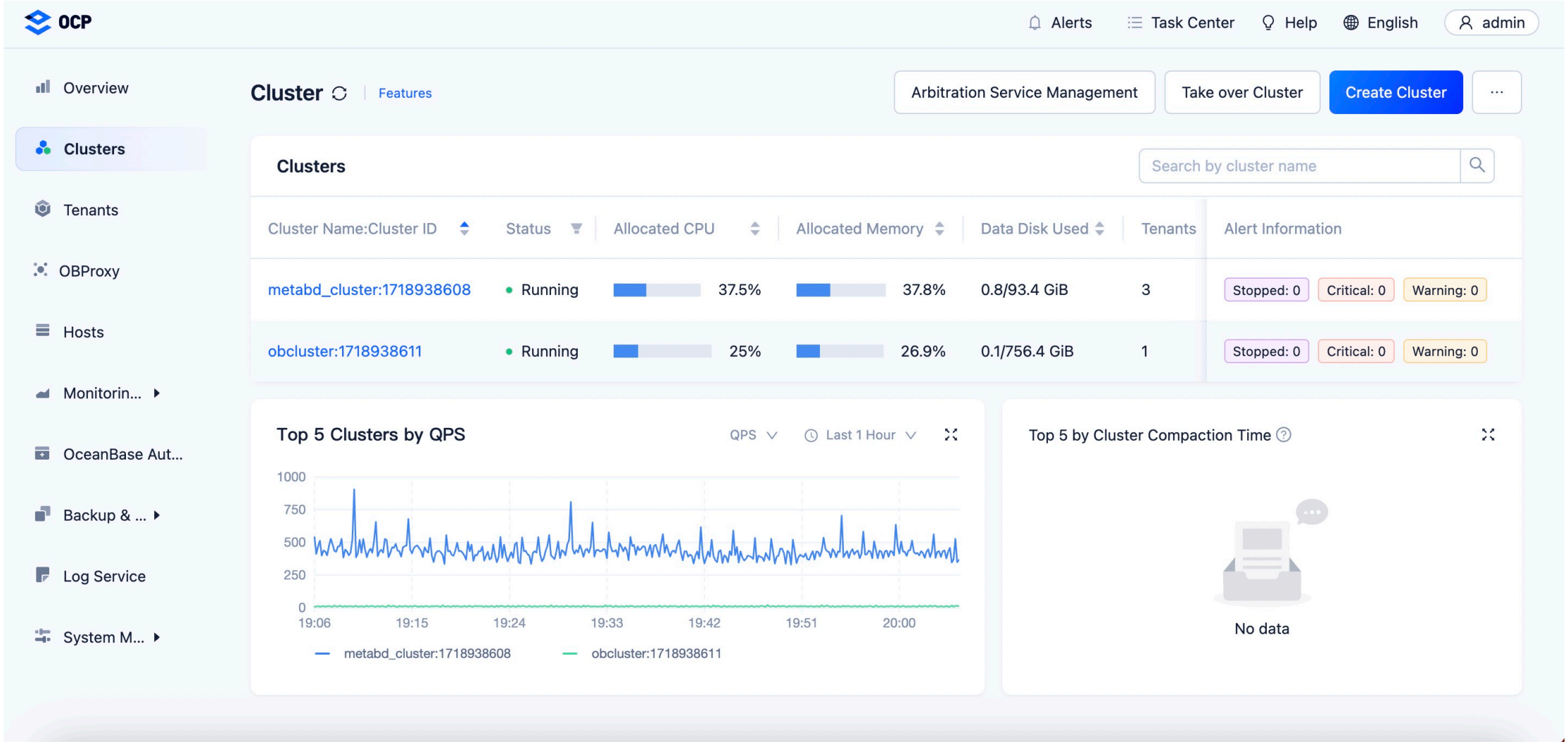


Step 5: Deploy OceanBase Cluster (Create a Cluster)

- Deploy three-replica OceanBase cluster via OCP.

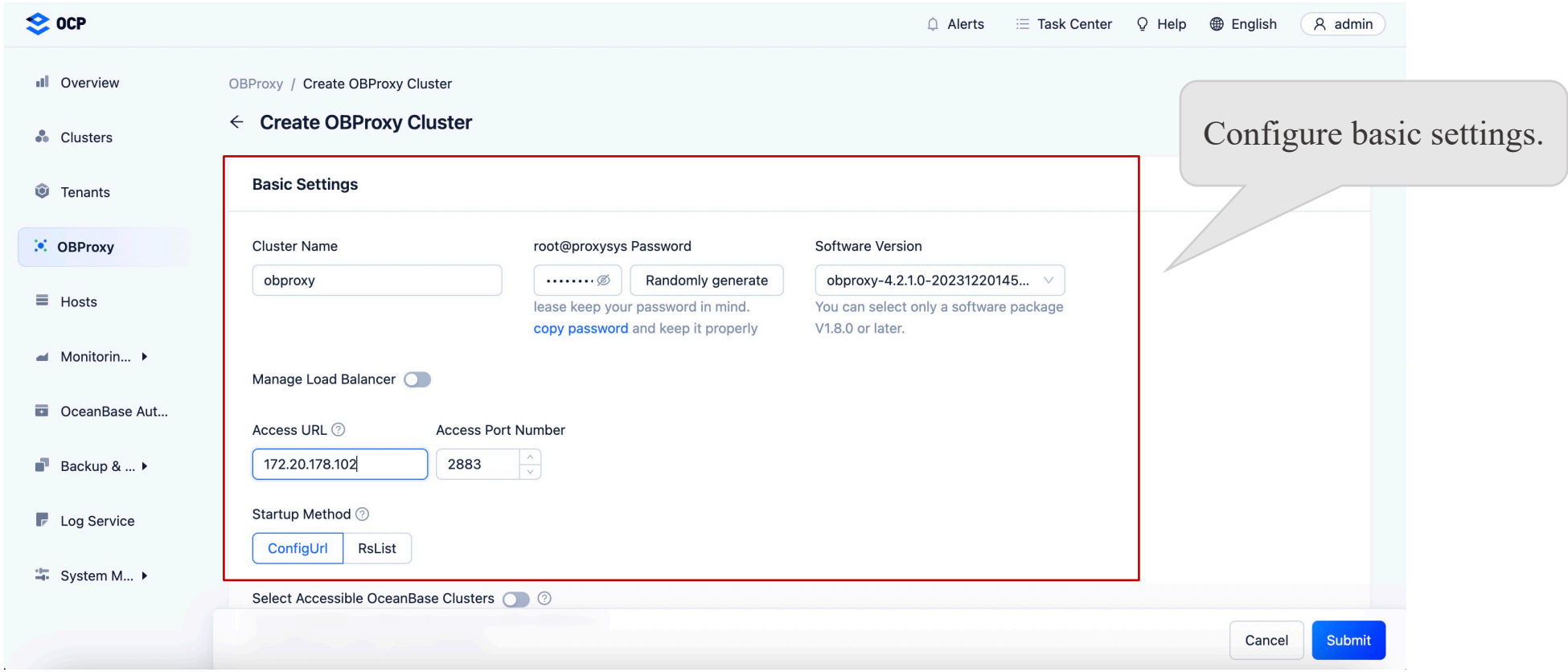


Step 5: Deploy OceanBase Cluster (View Cluster State)



Step 6: Deploy OBProxy

- OceanBase Database Proxy (ODP) is a database proxy middleware provided by OceanBase, which can be used for connection pool management, load balancing, failover, etc. OBProxy is an optional component. Its deployment depends on the application requirements and scenarios.



Installation and Deployment

04

- Introduction to Deployment Form
- Resource Configuration Requirements
- Standalone Deployment of OceanBase
- Cluster Deployment of OceanBase
- **Verification and Use**

View the File Directory Structure of OCP

OCP software directory:

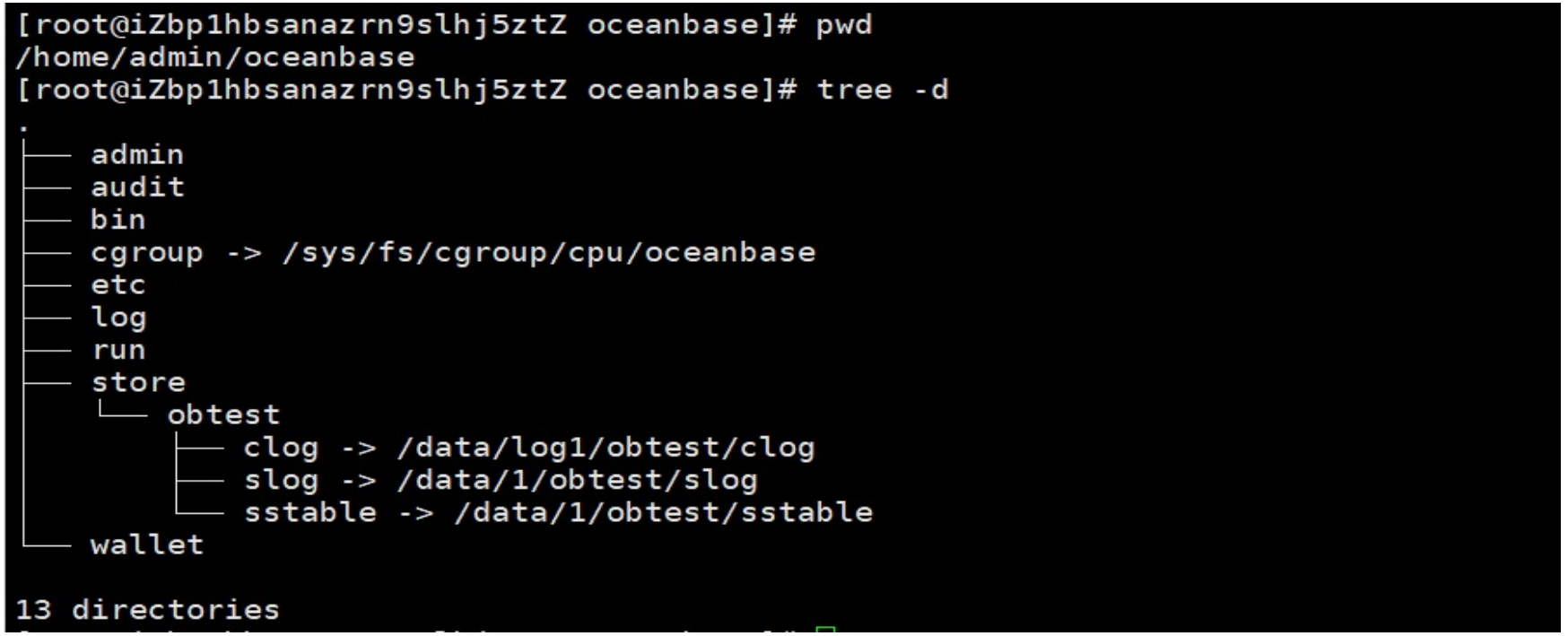
- OCP program: /home/admin/ocp-server/
- Main process log of OCP: /home/admin/logs/ocp/
- Task log of OCP: /home/admin/logs/task/
- Local file cache of OCP: /home/admin/data/files/

```
[root@iZbp18eoqcpusi7vqn1aklZ admin]# pwd
/home/admin
[root@iZbp18eoqcpusi7vqn1aklZ admin]# tree -d
.
├── data
│   └── files
│       ├── built-in
│       ├── oceanbase-upgrade-dep
│       └── sys-package
├── ic-server
│   ├── lib
│   │   └── ic_server_file
│   │       └── agents
│   └── script
├── logs
│   ├── obproxy
│   │   ├── log
│   │   └── minidump
│   ├── ocp
│   │   └── task
│   ├── sandbox
│   └── security
├── ocp-server
│   ├── alarm
│   ├── bin
│   ├── etc
│   ├── lib
│   ├── log -> /home/admin/logs/ocp
│   ├── run
│   └── utility
└── tools
    └── arthas
        ├── async-profiler
        └── lib
```

View the File Directory Structure of OBServer

OceanBase software directory:

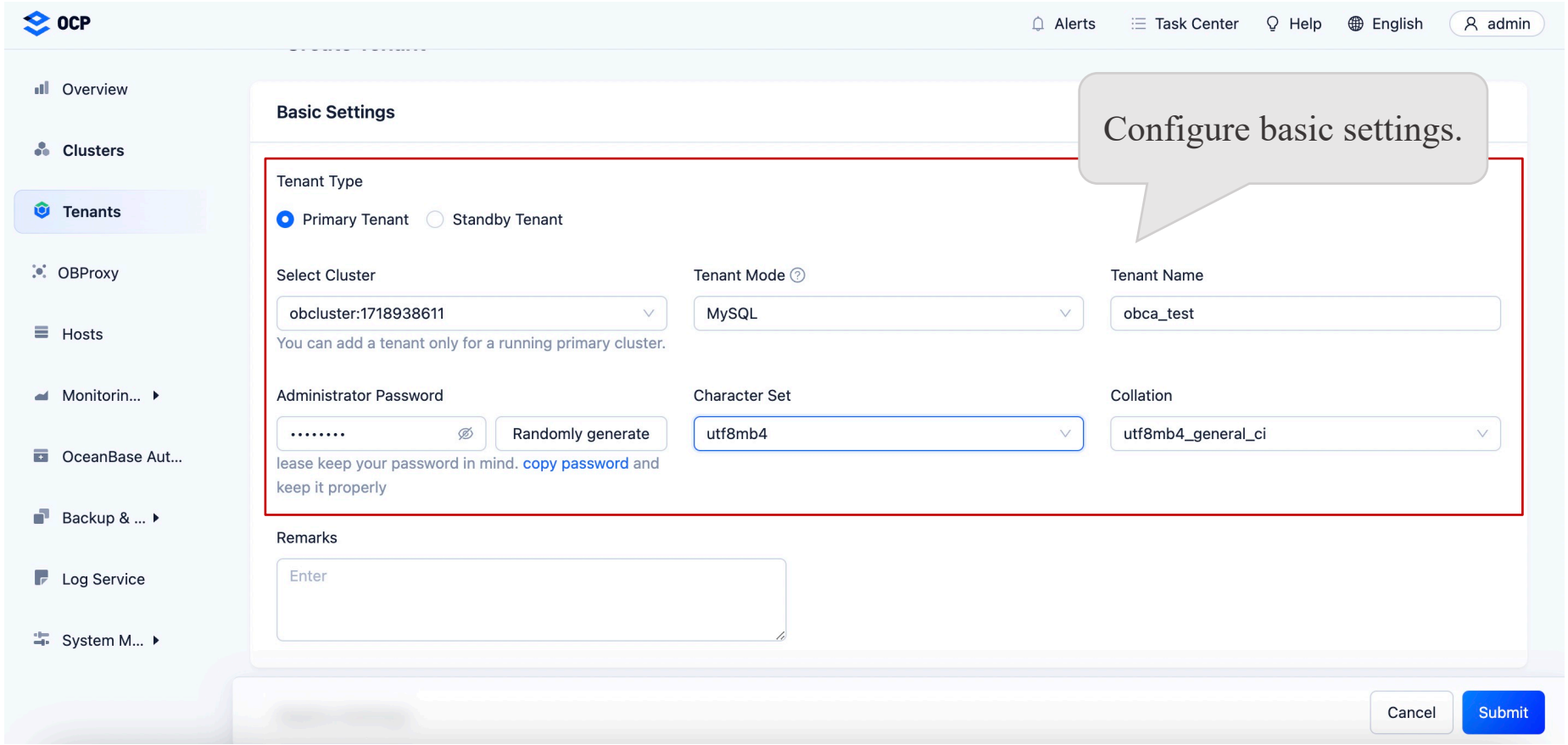
The working directories under the OBServer node usually contain audit, bin, etc, etc2, etc3, log, run and store, and there are three directories under the store directory, namely clog, slog and sstable. The directories etc2 and etc3 are used for backing up the configuration files. Audit logs are stored under the audit directory, the bin directory is used to store the observer binary files. The log directory is used to store the operation logs, including observer logs, RS logs and election logs.



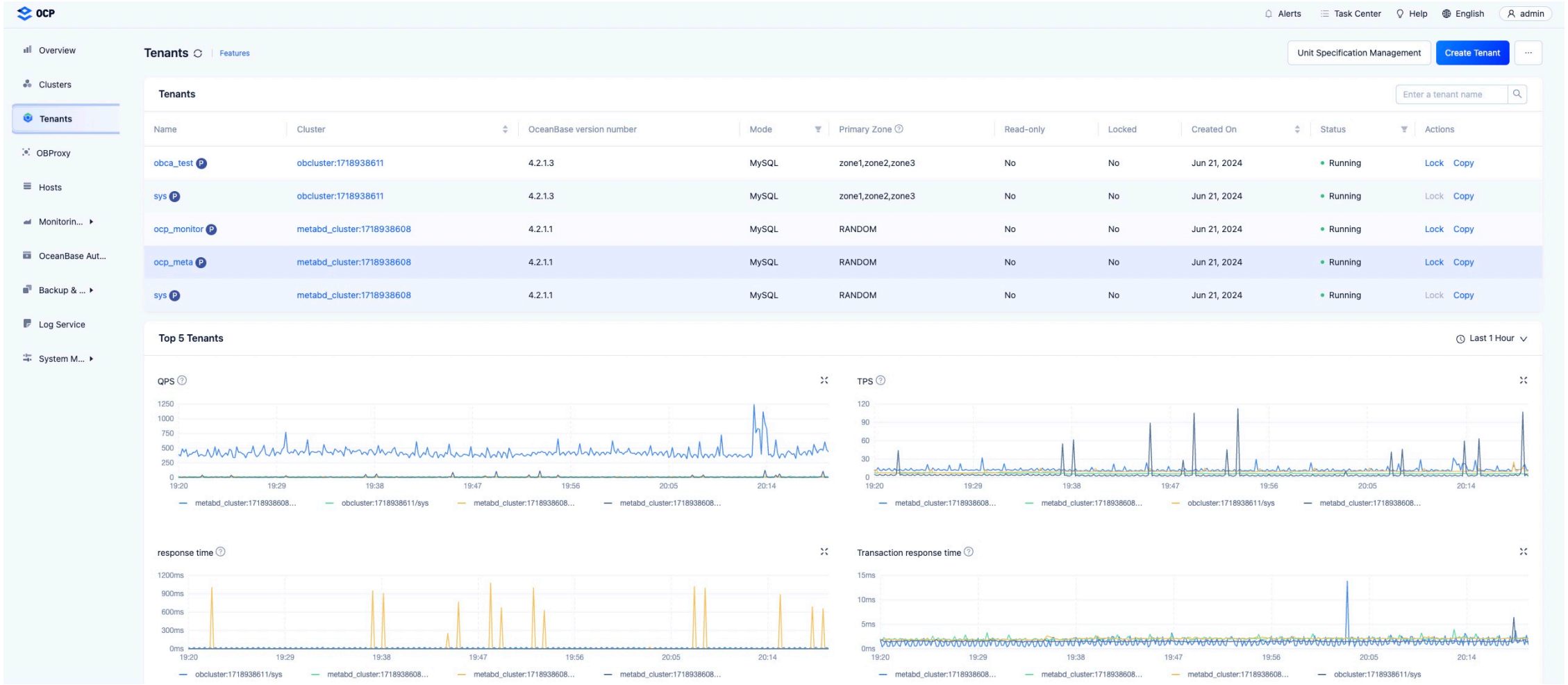
Create a Tenant by OCP

- Tenant is not only a container for various database objects but also a container for resources (CPU, Memory and IO, etc.).

  The primary tenant can be created in a cluster according to the service requirements.

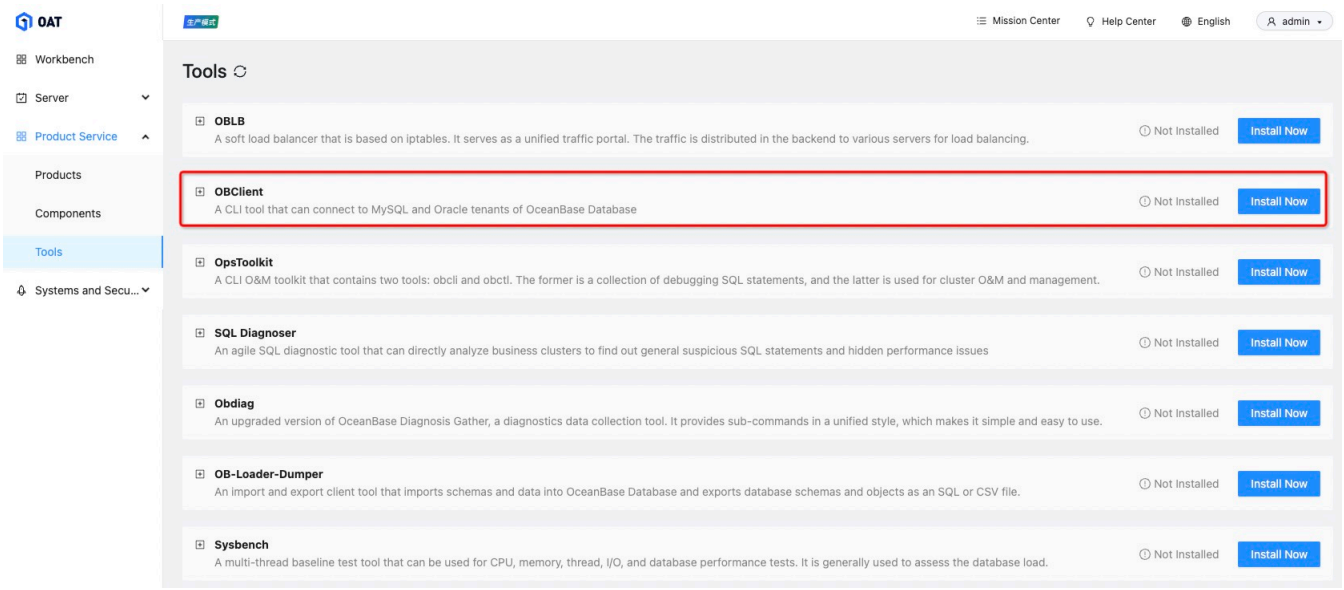


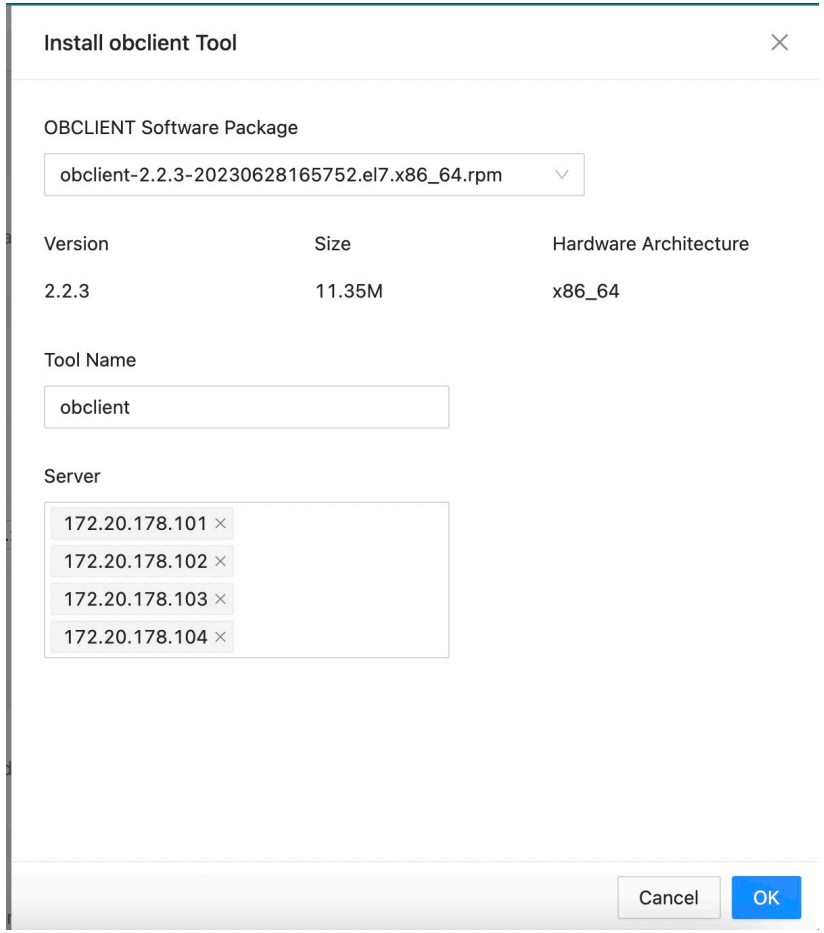
View the Tenant State by OCP



Install OBClient Tool

- OBClient is an interactive and batch query tool, which needs to be separately installed. It is a CLI. It acts as a client when being connected to the database, supporting the MySQL tenants and Oracle tenants in the OceanBase Database.
- Install the OBClient tool by OAT.





Connect the OceanBase User by OBClient

```
obclient -h172.20.178.68 -P2883 -uSYS@obca_test#obcluster -p -c -A
```

```
obclient -h172.20.178.68 -P2881 -uSYS@obca_test -p -c -A
```

Parameter description:

-h: Provides the connection IP for the OceanBase database, which is typically an ODP address, and for direct connection, use the address of a specific observer node.

-u: Provides the connection account for the tenant

-P: Provides the connection port for the OceanBase database, which is also the listening port for ODP, with the default being 2883, and can be customized during the deployment of ODP. For direct connection, the port is 2881, which can also be customized during the installation of OceanBase.

-c: Indicates that comments should not be ignored in the OBClient runtime environment.

-A: Indicates that statistics are not automatically obtained when OBClient connects to the database.

Connect the Tenant by OBClient (OBProxy)

- Connect by ODP:

```
obclient -h172.20.178.68 -P2883 -uSYS@obca_test#obcluster -p -c -A
```

```
[root@iZbp18eoqcpusi7vqn1aklZ soft]# obclient -h172.20.178.68 -P2883 -uSYS@obca_test#obcluster -p -c -A
Enter password:
Welcome to the OceanBase.  Commands end with ; or \g.
Your OceanBase connection id is 346906
Server version: OceanBase 4.2.1.3 (r103030022024011313-c6afef2be5748fc54b1b0159b5224ae288491424) (Built Jan 13 2024 13:36:43)

Copyright (c) 2000, 2018, OceanBase and/or its affiliates. All rights reserved.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

obclient [SYS]> show parameters where name ='memory_limit_percentage';
+-------+----------+--------------+----------+------------------------+-----------+-------+-----------------------------------------------------------------------------------------------------------------
+----------+---------+---------+------------------+
| ZONE  | SVR_TYPE | SVR_IP        | SVR_PORT | NAME                   | DATA_TYPE | VALUE | INFO
| SECTION  | SCOPE   | SOURCE  | EDIT_LEVEL       |
+-------+----------+--------------+----------+------------------------+-----------+-------+-----------------------------------------------------------------------------------------------------------------
+----------+---------+---------+------------------+
| zone3 | observer | 172.20.178.70 |     2882 | memory_limit_percentage | NULL      | 80    | the size of the memory reserved for internal use(for testing purpose). Range: [10, 95]
| OBSERVER | CLUSTER | DEFAULT | DYNAMIC_EFFECTIVE |
| zone2 | observer | 172.20.178.69 |     2882 | memory_limit_percentage | NULL      | 80    | the size of the memory reserved for internal use(for testing purpose). Range: [10, 95]
| OBSERVER | CLUSTER | DEFAULT | DYNAMIC_EFFECTIVE |
| zone1 | observer | 172.20.178.68 |     2882 | memory_limit_percentage | NULL      | 80    | the size of the memory reserved for internal use(for testing purpose). Range: [10, 95]
| OBSERVER | CLUSTER | DEFAULT | DYNAMIC_EFFECTIVE |
+-------+----------+--------------+----------+------------------------+-----------+-------+-----------------------------------------------------------------------------------------------------------------
+----------+---------+---------+------------------+
3 rows in set (0.008 sec)

obclient [SYS]> show variables like 'ob_query_timeout';
+-----------------+----------+
| VARIABLE_NAME    | VALUE    |
+-----------------+----------+
| ob_query_timeout | 10000000 |
+-----------------+----------+
1 row in set (0.007 sec)

obclient [SYS]>
```

Connect the Tenant by OBClient (OBServer)

- Directly connect to OBserver:

```
obclient -h172.20.178.68 -P2881 -uSYS@obca_test -p -c -A
```

```
[root@iZbp18eoqcpusi7vqn1aklZ soft]# obclient -h172.20.178.68 -P2881 -uSYS@obca_test -p -c -A
Enter password:
Welcome to the OceanBase.  Commands end with ; or \g.
Your OceanBase connection id is 3221687681
Server version: OceanBase 4.2.1.3 (r103030022024011313-c6afef2be5748fc54b1b0159b5224ae288491424) (Built Jan 13 2024 13:36:43)

Copyright (c) 2000, 2018, OceanBase and/or its affiliates. All rights reserved.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

obclient [SYS]> show parameters where name ='memory_limit_percentage';
+-------+----------+--------------+----------+------------------------+-----------+-------+-----------------------------
```

Summary

- The OceanBase Database provides two forms, namely Enterprise Edition and Community Edition, both of which support standalone centralized deployment and high-availability cluster deployment.
- The OceanBase Database Community Edition has an open-source kernel, while the Community Edition provides open interface and rich ecological capabilities to support enterprises or individuals to better achieve customized service requirements.
- For the OceanBase clusters of Enterprise Edition and Community Edition, the deployment methods are different. It is recommended to deploy the Enterprise Edition by OCP or CLI, and deploy the Community Edition by OBD or container.
- Only one zone is contained in standalone centralized deployment, and there is only one OBServer node in the Zone. It is suitable for running the low-importance systems and performing the personal development tests.
- The OceanBase deployment has resource requirements for the server. The Enterprise Edition has higher requirements than the Community Edition. In a functional test environment, the minimum configuration for Enterprise Edition is as follows: CPU: 4 cores, memory: 16 GB; and the minimum configuration for Community Edition is as follows: CPU: 2 cores, memory: 6 GB.
- The OceanBase Database cluster consists of at least three nodes in high availability and disaster recovery. Each node is corresponding to one OBSERVER process, and the OBSERVER processes on different nodes form a cluster to provide external services.

Summary

- The OceanBase product supports the deployment on multiple operating systems. The operating system needs to be configured with network and software manager (yum or zypper source).
- The installation package required for the deployment of OceanBase can be obtained from the official website. Different deployment methods require different installation packages.
- It is recommended to use SSD and RAID; RAID is not required for NVME SSD disk, and LVM is configured at the OS level; RAID 5 is suggested for SATA SSD disk. Cache WriteBack With BBU and auto_rebuild are enabled. It is recommended that the root directory should not be less than 50 GB.
- It is recommended to configure two 10 gigabit network cards. Each server is configured with dual optical port aggregation. It's recommended to configure the dual optical port in mode4 bonding mode. The switch is configured with 802.3ad protocol and fixed IP. The IP is accessible and the network is stable.
- Standalone deployment process: Prepare and configure a server, obtain the installation package, configure OBD, use the OBD GUI or CLI to deploy the OceanBase cluster, and carry out the connection test.
- It should be noted in standalone CLI deployment: carefully check the content and format of OBD configuration file.
- Distributed cluster deployment process: Configure a server, deploy OAT, configure the deployment environment via OAT, deploy OCP by OAT, and deploy OceanBase cluster and OBProxy by OCP.

Self-test

1. The OceanBase Database provides two forms, namely Enterprise Edition and Community Edition, among which the Community Edition only supports standalone centralized deployment. (False)
2. The OceanBase database can be deployed in various modes in different environments. For the deployment methods of Enterprise Edition and Community Edition, which one of the following is false (C)

   A. Deploy OceanBase cluster via OCP for the Enterprise Edition;

   B. Deploy OceanBase cluster via CLI for the Enterprise Edition;

   C. Unable to deploy OceanBase cluster via OCP for the Community Edition;

   D. Deploy OceanBase cluster via OBD for the Enterprise Edition.
3. The **standalone centralized deployment** of OceanBase Database in a small-size development mode is suitable for personal equipment. If the Community Edition is selected, the reserved memory of the cluster shall not be less than (B) GB.

   A. 2 B. 6 C. 8 D. 16
4. Both the OceanBase Community Edition and Enterprise Edition support deployment under windows (False).
5. The ports for direct connection and connecting the OceanBase Database via ODP are identical (False).

Contents

I. Product Overview

II. Cluster Architecture

III. Database Engines

IV. Installation and Deployment

V. Application Development

VI. O&M Management

Application Development

05

- **OceanBase Development Tools**
- OceanBase Database Design
- OceanBase DML/DQL Statements
- OceanBase SQL Tuning
- OceanBase Data Import/Export

Introduction to OceanBase Development Tools

Connection tools

In application development, connection tools are essential for creating and managing database objects, and the connection tools provided by OceanBase database include OBClient and ODC.

- **MySQL client (MySQL)**

  MySQL client is the official command-line tool of MySQL, which can be used to connect MySQL-compatible tenants in OceanBase, but cannot connect tenants in Oracle compatibility mode.

- **OceanBase client (OBClient)**

  OBClient is an interactive and batch processing query tool that requires separate installation. It features a command-line user interface, acting as a client when connected to a database, and supports MySQL and Oracle tenants of the OceanBase database.

- **OceanBase Developer Center (ODC)**

  ODC is an enterprise-grade database development platform customized for OceanBase database. ODC supports connections to both MySQL and Oracle tenants of the OceanBase database, offering database developers a suite of functionalities including routine development operations, WebSQL, SQL diagnostics, session management, and data import/export capabilities.

Introduction to OceanBase Development Tools

Drivers

OceanBase database also provides support for drivers in languages such as Java, C, and Python, allowing developers to choose the appropriate driver based on their preferences and needs for developing application systems. With the support of these tools and drivers, developers can construct application systems more flexibly and engage in efficient interactions and data manipulation with the OceanBase database.

- **Java drivers**
  - MySQL Connector/J: It is the official JDBC driver provided by MySQL, which only supports connections to MySQL tenants of the OceanBase database.
  - OceanBase Connector/J: Provides connections to OceanBase database for Java-based applications. Supports connections to both MySQL and Oracle tenants of the OceanBase database.
- **C driver (OceanBase Connector/C)**

  Allows C/C++ programs to access the OceanBase distributed database cluster in a more low-level manner for operations such as database connections, data access, error handling, and Prepared Statement processing. Supports both MySQL and Oracle tenants of the OceanBase database.
- **Python driver (PyMySQL)**

  PyMySQL is a library for connecting to MySQL server in Python 3.x version. Users can employ the PyMySQL driver to provide connections to MySQL tenants for Python applications.

Connection Tool - MySQL Client

Operation steps

- Opens a command-line terminal and ensure that the environment variable PATH contains the directory where MySQL client is installed
- Provides MySQL connection parameters in the following format

```
$mysql -h192.168.1.101 -uroot@obmysql#obdemo -
P2883 -pabcABC123 -c -D oceanbase -A
```

- After a successful connection, the default command-line prompt will appear: MySQL [oceanbase]>
- To exit the OceanBase command line, type exit and enter, or press ctrl + d

Parameter descriptions

- -h: Provides the connection IP for the OceanBase database, which is typically an OBProxy address.
- -u: Provides the connection account for the tenant, which comes in four formats: username@tenant_name#cluster_name,cluster_name: tenant_name: username, cluster_name-tenant_name-username, or cluster_name. tenant_name. username. The default administrator username for MySQL tenants is root
- -P: Provides the OceanBase database connection port, which is also the listening port of OBProxy. The default is 2883, but it can be customized.
- **-p**: Provides the account password. For security reasons, it may be omitted and instead, entered when prompted at the command line, with the password being invisible in the text.
- -c: Indicates that comments should not be ignored in the MySQL runtime environment, and Hint is a special comment that is not affected by -c.
- -A: Indicates that statistics are not automatically obtained when MySQL connects to a database.
- -D: The name of the database to access can be changed to the business database

Connection Tool - OBClient

Operation steps

- Opens a command-line terminal and ensure that the environment variable PATH contains the directory where OBClient is installed
- Provides OBClient connection parameters in the following format

```
$obclient -h192.168.1.101 -uroot@obmysql#obdemo
-P2883 -pabcABC123 -c -D oceanbase -A
```

- After a successful connection, the default command-line prompt: obclient> will appear
- To exit the OceanBase command line, type exit and enter, or press ctrl + d

Parameter description

- -h: Provides the connection IP for the OceanBase database, which is typically an OBProxy address.
- **-u:** Provides the connection account for the tenant, which comes in four formats: username@tenant_name#cluster_name,cluster_name: tenant_name: username, cluster_name-tenant_name-username, or cluster_name. tenant_name. Username. The default administrator username is root for MySQL tenants and SYS for Oracle tenants.
- -P: Provides the OceanBase database connection port, which is also the listening port of OBProxy. The default is 2883, but it can be customized.
- **-p**: Provides the account password. For security reasons, it may be omitted and instead, entered when prompted at the command line, with the password being invisible in the text.
- -c: Indicates that comments should not be ignored in the MySQL runtime environment, and Hint is a special comment that is not affected by -c.
- -A: Indicates that statistics are not automatically obtained when OBClient connects to a database
- **-D**: The name of the database to access can be changed to the business database

Connection Tool - OceanBase Developer Center (ODC)

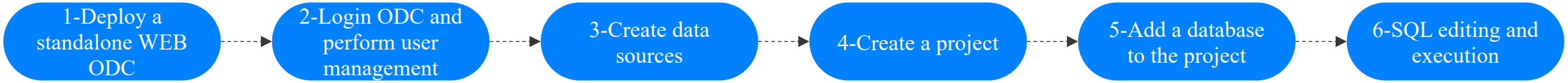
OceanBase Developer Center (ODC) is an enterprise-grade database development platform tailored for OceanBase database; ODC supports connections to databases in both MySQL and Oracle tenants within OceanBase, and provides database developers with features such as routine development operations, WebSQL, SQL diagnostics, session management, and data import/export capabilities.

- **ODC is available in two product forms: Desktop version and Web version.**
  - **Desktop version:** Focuses on database development tool capabilities, and supports Windows, Mac, and Linux operating systems, with lightweight and easy-to-deploy features
  - **Web version:** Provides control and collaboration capabilities in addition to the provision of tools, focuses on the security, compliance and efficiency of database changes, and offers two working modes: personal space and team space.
    - **Personal space:** Suitable for individual developers, it provides you with desktop-like experience on the browser side, you can freely access, create new data sources and use various windows and tools provided by the platform for database development.
    - **Team space:** Suitable for developers and DBAs to use in collaboration, which is both a development tool and a control and collaboration platform. ODC provides you with one-stop service for database development and control through a range of features such as project collaboration, change stabilization, data security, and separation of cold and hot data.

Connection Tool - OceanBase Developer Center (ODC)

Desktop ODC installation

WEB ODC installation



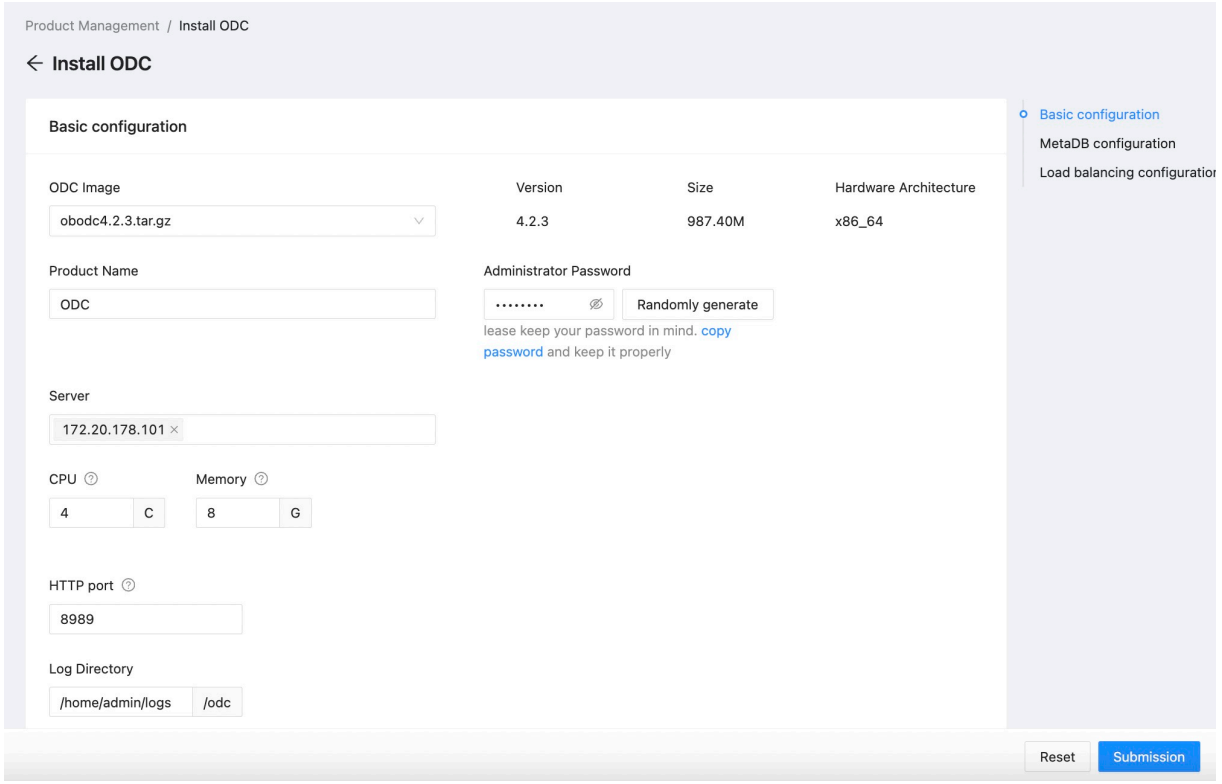
Connection Tool - OceanBase Developer Center (ODC)

The standalone web ODC deployment environment is configured with the following empirical values according to the number of ODC users:

Number of ODC users	Server types	Number of servers	Minimum resource requirements
20	ODC Docker deployment server	1	2C, 4GB RAM

Standalone Web ODC installation steps:

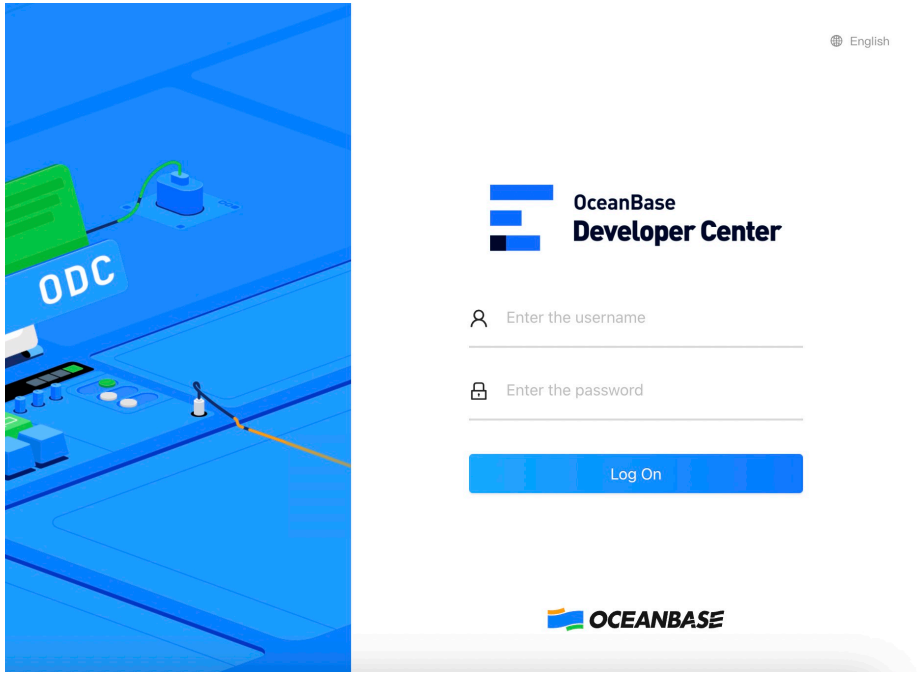
1. Open the OAT, and in the left navigation bar, click Product Services > Product Management.
2. On the Product Management page, click Install Product in the upper-right corner, and select Install ODC.
3. On the Install ODC page, configure the parameters.
4. When the ODC configuration is complete, click Submit.
5. After successful installation, enter http://<ip address>:8989/ in the browser interface to log in.



Connection Tool - OceanBase Developer Center (ODC)

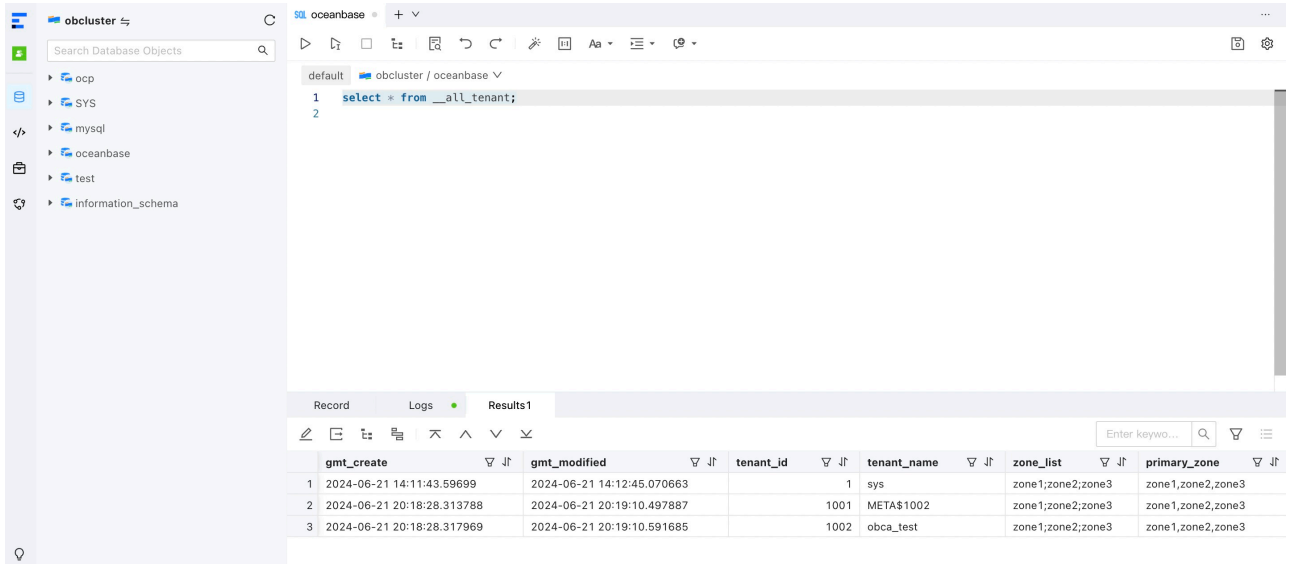
Login to ODC

After the deployment is completed, log in to OceanBase Developer Center, first create a data source, and then develop the database.



SQL development window

- SQL editing area: The SQL editing area allows you to edit and execute SQL statements.
- Execution Log tab: You can view the log of each SQL statement executed during the current connection process.
- Log tab: You can view the execution log of the current SQL window in the Log tab.
- Result tab: You can view the execution result of the current SQL statement in the Result tab.



Drivers - Java Driver

OceanBase Connector/J is a JDBC Type 4 driver, which can communicate directly with the database engine via the local protocol. OceanBase Connector/J is fully compatible with the usage of MySQL JDBC, capable of automatically recognizing whether the OceanBase database is operating in MySQL or Oracle mode, and simultaneously ensuring protocol-level compatibility with both modes.

The following is an example of creating a Java application:

1. Prerequisites: Ensure that the computer's Java environment is set to Java JDK version 8.
2. Get the database connection string, for example:

```
obclient -h172.20.178.58 -P2883 -uroot@ob_school#ob_school -p
```

3. Install OceanBase Connector/J driver

```
Extract the OceanBase Connector/J JAR package and put it into the local /usr/share/java path, and set the temporary environment
variable.
mv ./oceanbase-client-{version}.jar /usr/share/java
export CLASSPATH=/usr/share/java/oceanbase-client-{version}.jar: $CLASSPATH
```

4. Write the application Test.java

```
Class.forName("com.oceanbase.jdbc.Driver");
            Connection connection =
DriverManager.getConnection("jdbc:oceanbase://172.20.178.58:2883/school_db?pool=false&user=root@ob_school#ob_school&password=aaAA11__")
;
```

Driver - C Driver

The OceanBase Connector/C allows C/C++ programs to access the OceanBase distributed database cluster in a more low-level manner for operations such as database connections, data access, error handling, and Prepared Statement processing. OceanBase Connector/C, also known as LibOBClient, allows an application to run as an independent server process to communicate with OBServer nodes by using network connections. Supports both MySQL and Oracle tenants of the OceanBase database.

LibOBClient installation steps:

1. Prerequisites:
   - The GCC version shall be 3.4.6 or above, and 4.8.5 is recommended
   - The CMake version shall be 2.8.12 and above
2. Get the OceanBase Connector/C installation package from github. Download link: https://github.com/oceanbase/obconnector-c
3. Install libobclient

```
sudo rpm -ivh libobclient-xx.x86_64.rpm
```

4. Install obclient

```
sudo rpm -ivh obclient-xx.x86_64.rpm
```

Application Development

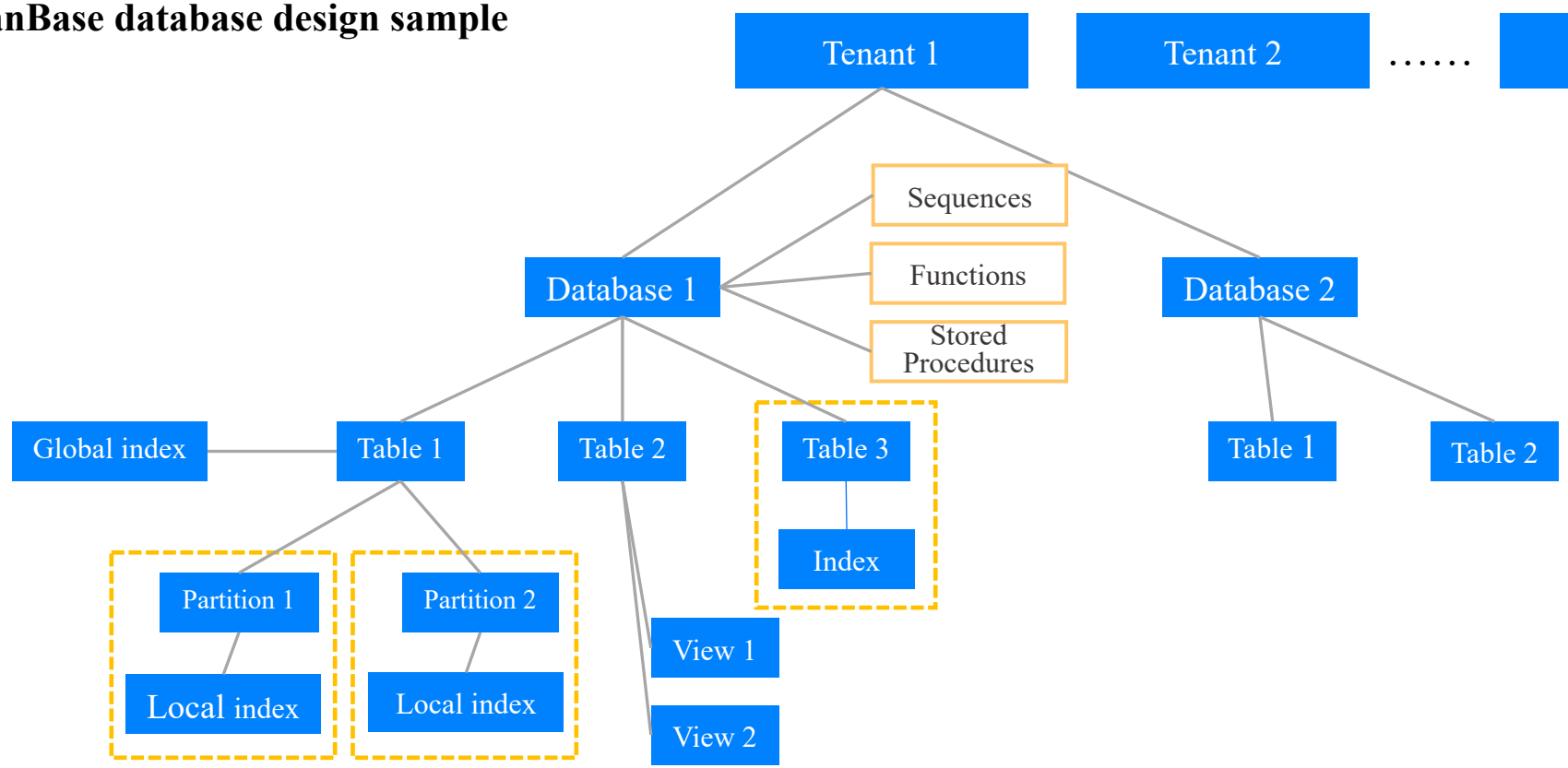
05

- OceanBase Development Tools
- **OceanBase Database Design**
- OceanBase DML/DQL Statements
- OceanBase SQL Tuning
- OceanBase Data Import/Export

Start Here

Good database design and planning are prerequisites for the stable operation of a database system. We need to design the database structure based on the characteristics of the application system operation.

OceanBase database design sample



Database Design and Development

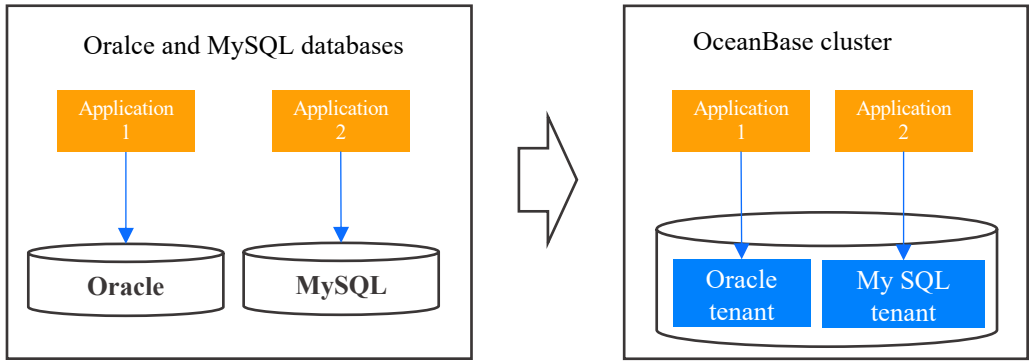
During the development of OceanBase database, it is necessary to plan tenants rationally and optimize the database structure and table relationships to enhance data access performance and efficiency. Based on this, we need to focus on the following aspects:

- **Tenant design:** Determine the type and resource quota of tenants required for the business system and ensure data isolation and security between tenants.
- **Database design:** Design the database structure according to project needs and functional requirements, including determining tables, views, stored procedures, and functions. Consider aspects such as data relationships, constraints, etc.
- **Table design:** Define columns and relationships for each table, and select appropriate data types and constraints. Focus on data integrity, consistency, and performance optimization, and design primary keys, foreign keys, and indexes to support queries and data manipulation.
- **Index design:** Design reasonable indexes based on query requirements and access patterns to enhance query efficiency and data access performance. Consider factors such as columns, types, uniqueness, and coverage of indexes.
- **Data integrity and consistency:** Ensure data integrity and consistency by defining appropriate constraints (such as primary keys, foreign keys, and uniqueness constraints) and mechanisms like triggers to avoid data conflicts or illegal operations.
- **Query optimization:** Design optimized queries and operations to improve query performance and data access efficiency based on database access patterns and requirements. Consider the application of technical means such as index design, data partition and query optimization.

Next, we start from the tenant design to embark on the development journey of the OceanBase database.

Tenant Design - SQL Engine Supports MySQL and Oracle Compatibility Modes

- Tenants currently have two compatibility modes: Oracle mode and MySQL mode, tenants with different compatibility modes organize their related business data in the appropriate manner. In the case of ORACLE compatibility mode tenants, this organization is called schema; in the case of MySQL compatibility mode tenants, this organization is called database.
- Utilizing the OceanBase database for business purposes first requires the creation of a user tenant. The creation of a tenant is usually done by the database operator through the “Create a New Instance" in the product OCP, or the direct creation of a tenant by the operator under the sys tenant (refer to the tenant creation related content in Chapter 4).



- A single cluster supports both MySQL and Oracle modes.
- When creating a tenant, it is necessary to configure the tenant with MySQL compatibility mode or Oracle compatibility mode.
- DBAs change from maintaining "multiple database products" to maintaining a "unified database product", DBAs can create tenants with different compatibility modes according to application requirements.

MySQL compatibility mode

- Compatible with most features and syntax of MySQL 5.7/8.0.
- Compatible with MySQL communication protocol, MySQL applications can be migrated to OceanBase directly.

Oracle compatibility mode

- Compatible with Oracle database in terms of data types, SQL functions, database objects and other basic features.
- Compatible with most of the development features in terms of procedural programming language (PL).
- Compatibility is also reflected in advanced features such as database security, backup and recovery, high availability and optimizer, with some features performing even better.

Tenant Design - MySQL Compatibility Overview

Before proceeding with application development or system migration, it is essential to first understand the compatibility features of the current version with Oracle and MySQL, in order to address potential differences and issues that may require adaptations. OceanBase Database is compatible with the vast majority of features and syntax of MySQL 5.7/8.0 versions, allowing existing MySQL applications to migrate smoothly to the MySQL tenant of the OceanBase Database.

Features	Description
Data types	**Integer types / Fixed-point types:** DECIMAL and NUMERIC / **Floating-point types:** FLOAT and DOUBLE / **Bit-Value type:** BIT. **Date and time types:** DATETIME, TIMESTAMP, DATE, TIME and YEAR / **Character types:** CHAR, VARCHAR, BINARY and VARBINARY / **Large object types:** TINYBLOB, BLOB, MEDIUMBLOB and LONGBLOB / **Text types:** TINYTEXT, TEXT, MEDIUMTEXT and LONGTEXT / **Enumeration types:** ENUM / **Collection types:** SET / **JSON data types / Spatial data types**
SQL syntax	SELECT/INSERT/UPDATE/DELETE/TRUNCATE
Procedural language	OceanBase database is compatible with most of the PL features of MySQL database, and the main supported PL features are as follows: Data types / Stored procedures / Custom functions / Triggers / Exception handling
System views	OceanBase database implements most of the views within the two internal databases information_schema and MySQL
Character sets	binary/utf8mb4/gbk/utf16/gb18030/latin1/gb18030_2022
Functions	The analytical (window) functions supported by the OceanBase Database are a superset of those in MySQL Database, meaning that all analytical (window) functions of MySQL Database are supported by OceanBase Database.
Partitioning types	Supports single-level partitioning, templated and non-templated subpartitioning; Supports Hash, Key, Range, Range Columns, List and List Columns partitioning;
Backup and recovery	Supports full backup/incremental backup/hot backup/table-level recovery.
Optimizer	View the execution plan command / View statistics / Query rewrite optimization / Optimizer Hint mechanism OceanBase database is compatible with the parallel execution capabilities of MySQL Database, including parallel query, parallel replication, and parallel write. Moreover, the OceanBase Database already supports parallel operators, including parallel aggregation, parallel join, parallel grouping, and parallel sorting. OceanBase database also supports plan cache and precompilation, which are not supported by the MySQL database.

Tenant Design - Oracle Compatibility Overview

OceanBase database is compatible with Oracle database in terms of data types, SQL functions, database objects and other basic features. In terms of Procedural Language (PL), it is essentially capable of being compatible with all development features. In terms of advanced features such as database security, backup and recovery, high availability, and optimizer, the OceanBase database offers excellent compatibility, and in some aspects, it even outperforms the Oracle database.

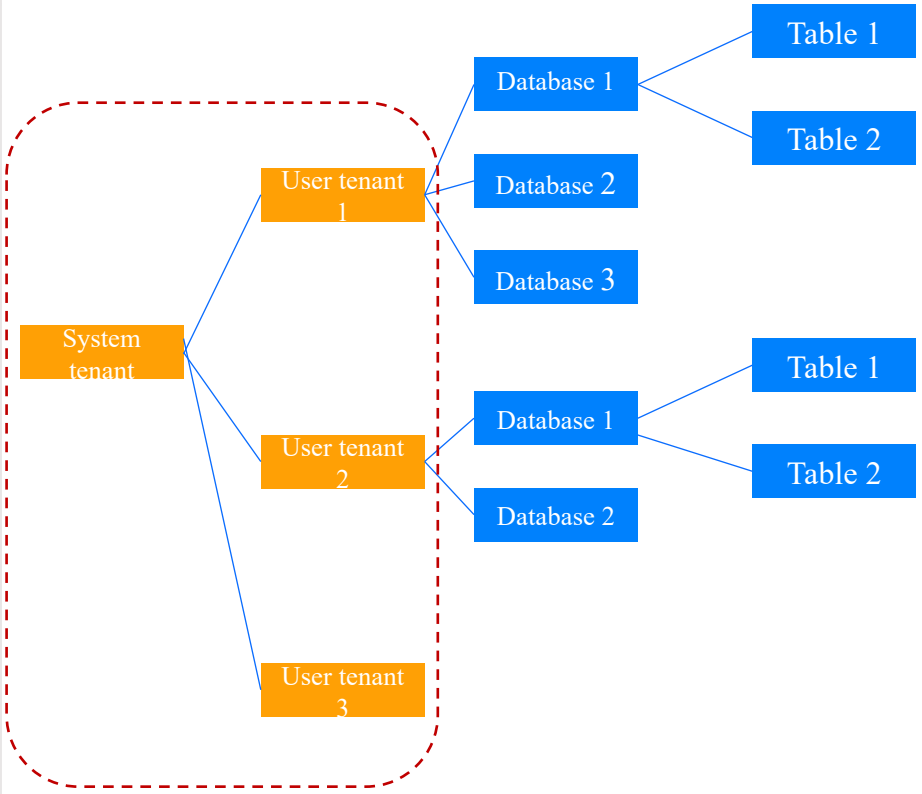
Features	Description
Data types	OceanBase database is compatible with the majority of data types supported in the Oracle database. For optimization purposes, the LONG and LONG RAW data types, being outdated, are not currently planned to be supported by the OceanBase database.
Built-in functions	OceanBase database is compatible with the majority of built-in functions supported in the Oracle database.
SQL syntax	OceanBase database supports the vast majority of SQL syntax from Oracle Database, and a few missing functionalities will result in a syntax not supported error. SELECT/INSERT/UPDATE/DELETE/TRUNCATE/Parallel query/Hint syntax
Procedural language	Process control/Collections and records/Static SQL/Dynamic SQL/Subroutines/Triggers/Exception handling/Program packages/User-defined data types/PL system packages/PL label security packages
System views	OceanBase database is compatible with some of the system views from Oracle Database, of which: Compatible with 195 dictionary views. Compatible with 20 performance views (starting with "v$").
Character sets	binary/utf8mb4/gbk/utf16/gb18030/latin1/gb18030_2022
Database object management	Table management/Constraints/Partition support/Index management/View management/Sequence management/Synonyms/Trigger management/Database links (DBLINKs)
Security features	Privilege management/Authentication/Transparent encryption/Auditing/Label security/Transport link encryption (SSL)
Backup and recovery	Supports NFS, Aliyun OSS, and Tencent Cloud COS as backup destinations/Supports tenant-level backup and recovery/Supports table-level recovery.
SQL engine	OceanBase Database's SQL engine is compatible with most features of the Oracle database: Supports query rewriting / Supports precompiled statements / Supports cost-based optimizer / Supports execution plan generation and presentation (EXPLAIN) / Supports execution plan caching / Supports execution plan fast parameterization / Supports execution plan binding / Supports Optimizer Hint / Supports adaptive cursor sharing ACS / Supports execution plan management SPM.

Tenant Design - Create a Tenant

Tenant design recommendations	
Key points	Description
Planning the tenant type	Once a tenant is created, the tenant type cannot be modified, so the tenant type shall be carefully planned before creation.
For operations with a large number of writes	It is necessary to allocate larger memory and log disk specifications.
For CPU-intensive operations	It is necessary to distribute opertions across different nodes to prevent imbalanced node loads, and to avoid creating hotspot nodes that could impact the whole database.
Migration from traditional database to OceanBase database	■ For smooth migration, a peer-to-peer architecture, i.e., a single unit tenant design, is often adopted to ensure that the business still needs to access only a standalone server when accessing the database. ■ When the business volume increases to the upper limit of a standalone unit's capacity, it can be expanded to a multi-unit tenant, where the business data is split into multiple groups according to the business scenarios. ■ Each group of business data is placed in a Unit to ensure that the database access of each business does not span across machines, thus achieving horizontal scaling with zero performance loss. ■ This is also one of the architectural advantages of OceanBase database over traditional databases: its easy horizontal scalability, which allows it to effortlessly break through the upper limits of standalone capacity.
The resource specification of a Unit cannot exceed the total allocated resources of a standalone server	■ For a single-unit tenant (resource pool UNIT_NUM=1), in the event of a capacity bottleneck, apart from upgrading to a server with a larger specification, the tenant can also be horizontally scaled to multiple units, thereby leveraging the resources of multiple nodes. ■ Multi-unit tenants require a careful design of the distribution of business data among units to prevent the performance degradation caused by distributed transactions. If the business volume is expected to exceed the upper limit of standalone unit capacity, it is essential to plan for data sharding in advance, and the design of data sharding should aim to minimize the occurrence of distributed transactions.

■ **Tenant creation**

For the tenant creation steps, please refer to: the fourth part of this course, which covers the relevant content on "Tenant Creation".



Database Design - Design Recommendations

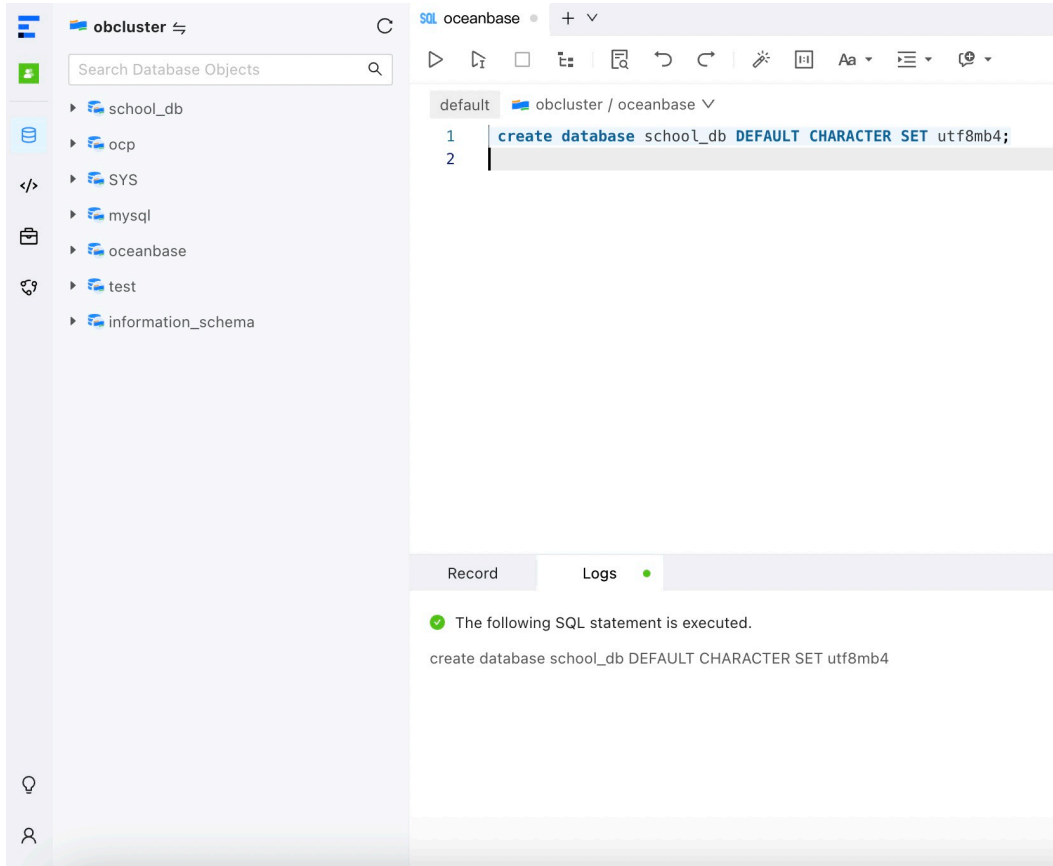
After creating the data source connection in ODC, we can use ODC to create the database.

The following principles and recommendations shall be followed when creating a database in OceanBase:

- It is recommended to assign a meaningful name to the database that ideally reflects its purpose and content. For example, use "Application Identifier_Subapplication Name (optional)_db" as the database name.
- When creating a database, ensure that you set the appropriate default character set and sorting rules to guarantee the correct storage and sorting of data. For long-term business development, it is recommended to use the utf8mb4 character set encoding when creating the database to ensure that the vast majority of characters can be stored.
- It is recommended to use the root user to create the database and associated user accounts, and to grant only the necessary permissions to ensure the security and controllability of the database.

For example, by using ODC, we can create a database named *school_db* for business use.

```
create database school_db DEFAULT CHARACTER SET utf8mb4;
```



Database Design - Basic Operations

OceanBase database provides many functions to manage databases, including creating, deleting and viewing databases. These operations facilitate developers in managing and maintaining databases more conveniently.

■ Create database

```
# MySQL schema #
CREATE {DATABASE | SCHEMA} [IF NOT EXISTS] database_name
    | [DEFAULT] {CHARACTER SET | CHARSET} charset_name
    | [DEFAULT] COLLATE collate_name
    | {READ ONLY | READ WRITE};

# ORACLE schema #
CREATE USER user_name
    IDENTIFIED BY [VALUES] password
    [REQUIRE {NONE | SSL | X509 | tls_option}]
    [PROFILE profile_name];
```

Example

```
# MySQL schema: Create a database, and specify the default
character set as utf-8mb4 #
CREATE DATABASE school_db DEFAULT CHARACTER SET utf8mb4;

# ORACLE schema: Create a user, specify a ciphertext
password and authenticate using an SSL connection #
CREATE USER school_user IDENTIFIED BY "abc123" REQUIRE SSL;
```

■ Delete database

```
# MySQL schema #
DROP DATABASE school_db;

# ORACLE schema #
DROP USER school_user
cascade;
```

■ Switch database

```
# MySQL schema #
USE school_db;

# ORACLE schema #
CONN school_user;
```

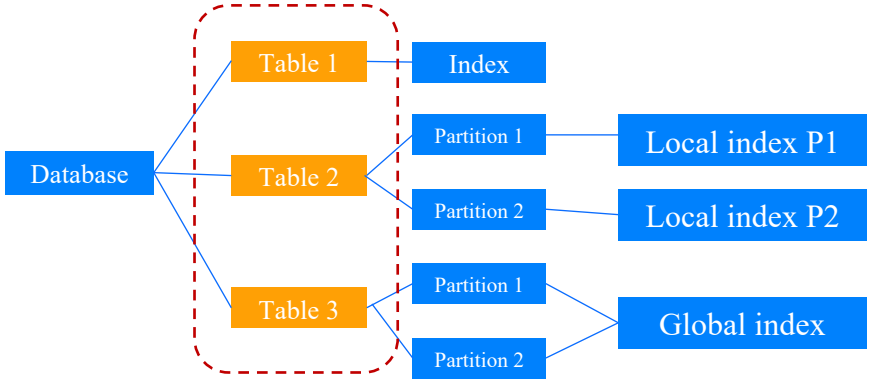
■ View defined database (MySQL schema only)

```
SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| oceanbase          |
| information_schema |
| mysql              |
| school_db          |
+--------------------+
```

Table Design - Basic Concepts

Model design of tables

- Logical model: Based on the rules of the relational model, each entity is represented as a table, and each column is represented as an attribute of the table, etc.
- Physical model: Based on the data volume and access characteristics of the business, combined with the features and advantages of OceanBase, perform the partition and index design of tables, etc.



Project example

Let's take an "Online Education Information System" as an example (the ER model is shown in the figure below) to demonstrate how to design tables, create, modify and query in the OceanBase database.

- Table Students: Student number (S_ID, primary key), Name (S_Name), Sex (Sex), Age (Age), Class number (CS_ID)
- Table Courses: Course number (C_ID, primary key), Course name (C_Name), Teacher (Teacher)
- Table Enrollment: Course number (C_ID), Student number (S_ID), Score (score)
- Table classes: Class number (CS_ID, primary key), Class name (CS_Name)

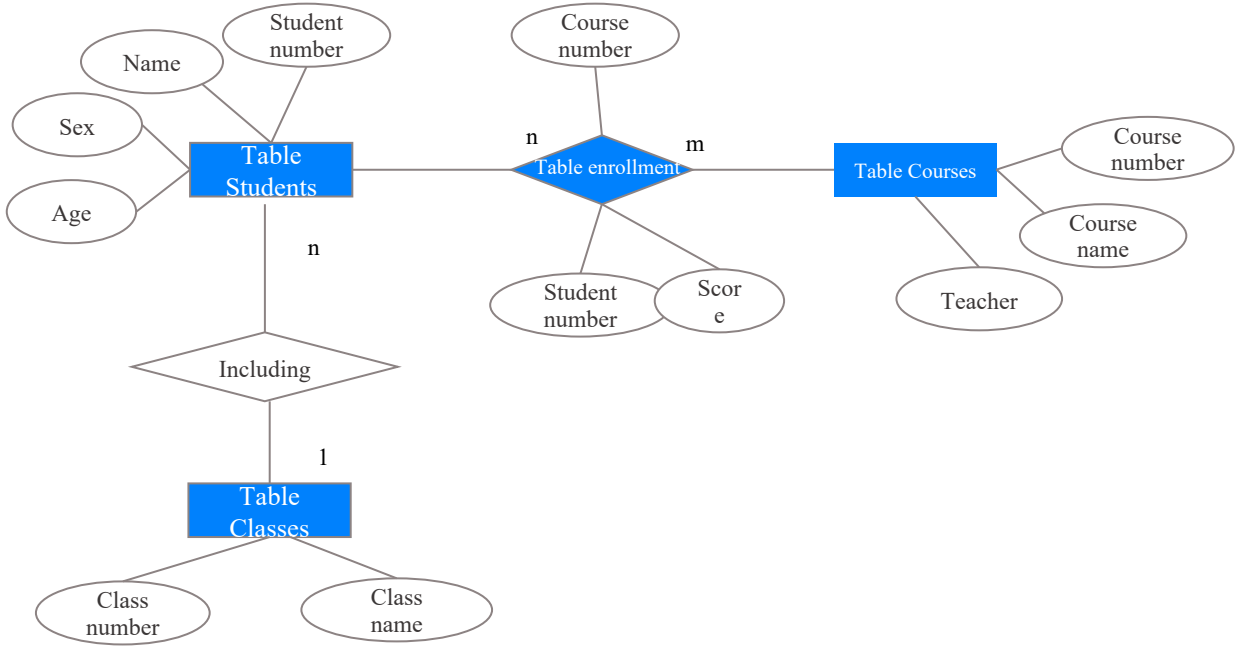


Table Design - Structure Design Recommendations

The fundamental principles of database table creation are to ensure the rationality, efficiency, and scalability of the database design. Additionally, it is important to select the appropriate data types, data lengths, and constraints in consideration of the actual business requirements and performance factors.

The following principles and recommendations shall be followed when creating tables in OceanBase database:

- **Select the appropriate data types:** Select the appropriate data types based on the characteristics of the data to be stored, such as numeric values, strings, dates/times, etc. OceanBase supports MySQL and Oracle mode data types, please refer to the related documents for details.
- **Use variable-length string types:** For string data, it is recommended to use variable-length string types (e.g., VARCHAR) and specify the maximum length. Ensure the length is sufficient to store the maximum value to prevent character truncation.
- **Define primary keys:** Consider whether to add a primary key to the table, and the primary key is used to uniquely identify each row in the table.
- **Other column constraints:** Add other constraints such as uniqueness constraints, foreign key constraints, check constraints, etc. as required.
- **NOT NULL constraints and default values:** If a column has a ``NOT NULL`` constraint, it is better to set a default value for that, especially for date/time types, where the default can be set to the current time.
- **Meaningful table and column names:** Table and column names shall be descriptive, avoiding meaningless names such as table1.
- **Select the partition scheme:** It is recommended to partition the table if the amount of data in the table is large. When creating a partition table, it is necessary to choose an appropriate partition method based on the data that will be stored in the table.

Example

```
-- Create Table courses
CREATE TABLE courses (
    C_ID BIGINT NOT NULL,
    C_Name VARCHAR(50) NOT NULL,
    Teacher VARCHAR(50) NOT NULL,
    PRIMARY KEY (C_ID)
);


-- Create Table classes
CREATE TABLE classes(
    CS_ID INT NOT NULL,
    CS_Name VARCHAR(50) NOT NULL,
    PRIMARY KEY (CS_ID)
);
```

Table Design - Basic Operations

- **Create a table**

```
CREATE TABLE [IF NOT EXISTS] table_name
(table_definition_list)
[table_option_list]
[partition_option]
[AS select];

CREATE TABLE [IF NOT EXISTS] table_name
LIKE table_name;
```

Example

```
CREATE TABLE courses (
    C_ID BIGINT NOT NULL,
    C_Name VARCHAR(50) NOT NULL,
    Teacher VARCHAR(50) NOT NULL,
    PRIMARY KEY (C_ID)
);
```

Delete a table

```
DROP TABLE courses ;
```

- **View a defined table (MySQL mode only)**

```
SHOW TABLES FROM school_db;
```

```
MySQL [school_db]> SHOW TABLES FROM school_db;
+---------------------+
| Tables_in_school_db |
+---------------------+
| courses             |
+---------------------+
1 row in set (0.00 sec)
```

- **View the table DDL**

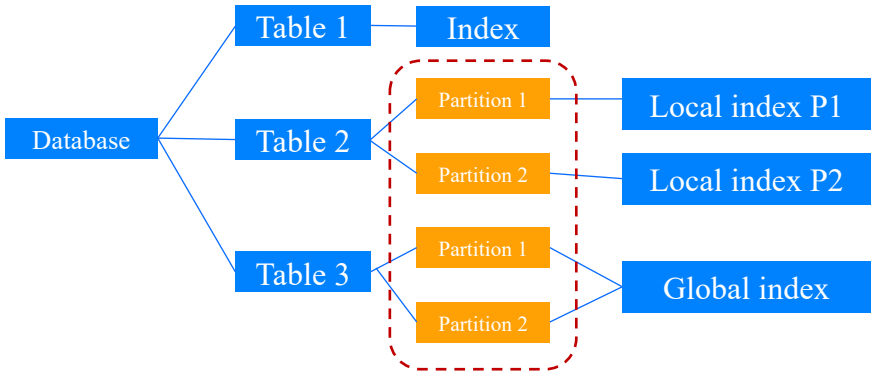
```
SHOW CREATE TABLE courses;
```

```
MySQL [school_db]> SHOW CREATE TABLE courses\G
*************************** 1. row ***************************
       Table: courses
Create Table: CREATE TABLE `courses` (
  `C_ID` bigint(20) NOT NULL,
  `C_Name` varchar(50) NOT NULL,
  `Teacher` varchar(50) NOT NULL,
  PRIMARY KEY (`C_ID`)
) DEFAULT CHARSET = utf8mb4 ROW_FORMAT = DYNAMIC COMPRESSION = 'zstd_1.3.8'
1 row in set (0.01 sec)
```

OceanBase Partition Design - Basic Concepts

When developing with the OceanBase database, if the volume of data in a table is very large and it leads to slow performance of the SQL statements in the application system, we can consider using partition tables. When developing partition tables, we need to consider the following key points:

- According to the business characteristics of the data table, select the appropriate partition type based on the design recommendations for partition tables.
- When performing join queries between two tables or cross-table transactions, the cross-machine communication may occur, which can be optimized for performance by means of table groups.



OceanBase database, in its MySQL mode and Oracle mode, currently supports the following partition types:

Partition methods	MySQL tenant	Oracle tenant
Range partition	✓	✓
Range Columns partition	✓	
List partition	✓	✓
List Columns partition	✓	
Hash partition	✓	✓
Key partition	✓	
Composite partition	✓	✓

- Range partition: Maps data to the corresponding partitions based on the partition key value ranges established for each partition during the definition of the partition table. It is a common partition type and is typically used in conjunction with date types.
- List partition: Specifies a list of discrete values for each partition's partition key. The advantage of the List partition is that it makes it easy to partition unordered or unrelated datasets.
- Hash partition: Records are hashed into different partitions based on the hash function values of the partition key
- Composite partition: Typically, a primary partition strategy is employed first, followed by a subpartition strategy on the sub-partitions, which is suitable when the business table contains an extremely large volume of data. Composite partition can take advantage of the benefits of multiple partition strategies

OceanBase Partition Design - Design Recommendations

If the amount of data is large and the access is centralized, you can use partition tables during the table creation. Here are some considerations to keep in mind when creating partition tables:

- **For the partition strategy**, it is recommended to design the table based on the actual use and application scenarios.
  - Actual use: History table, transaction table.
  - Application scenarios: Tables with obvious access hotspots.
- **For the selection of partition keys**, it is imperative to select the appropriate partition key and partition strategy when utilizing partition tables.
  - Hash partition: Select fields with high distinctiveness and the highest frequency in query conditions as the partition key for hash partition.
  - Range partition and List partition: Select appropriate fields as partition keys according to the business rules, but the number of partitions should not be too small. Example: If it is a large table of log type, perform Range partition according to the column of time type.
- **Notes on partition table constraints.**
  - When creating a partition table, at least one of the fields in every primary key or unique key on the table must be included in the table's partition key fields.
  - Global uniqueness in a partition table is recommended to be achieved through the primary key wherever possible.
  - The unique index of a partition table must contain the partition key of the table partitions.
- **Restrictions on the use of partitions.**
  - In the hash partition, it is not suitable to perform range queries based on the partition fields

Example

Assuming in the current "Online Education Information System," when the Table students has a large amount of data, then during the table design, it is possible to consider making both the Table students and the Table enrollment as partition tables.

```
-- Create Table students
CREATE TABLE students (
    S_ID BIGINT   NOT NULL,
    S_Name VARCHAR(50) NOT NULL,
    Sex CHAR(1),
    Age INT,
    CS_ID INT  NOT NULL,
    PRIMARY KEY (S_ID)
) partition by hash (S_ID) partitions 6;


-- Create Table enrollment
CREATE TABLE enrollment (
    S_ID BIGINT,
    C_ID BIGINT,
    score DECIMAL(5, 2),
    FOREIGN KEY (S_ID) REFERENCES
Students(S_ID),
    FOREIGN KEY (C_ID) REFERENCES
Courses(C_ID)
) partition by hash (S_ID) partitions 6;
```

OceanBase Partition Design - Basic Operation

■ Create a partition table

```
# Specify the partition method when creating the table #
CREATE TABLE table_name
PARTITION BY partition_type{(expression)
                  | COLUMNS(column_name_list)}
```

Notes: The partition types supported by OceanBase are: *HASH, KEY, LIST, RANGE.*

Example

```
# Create a partition table in MySQL schema
CREATE TABLE enrollment_his (
    s_id INT,
    c_id INT,
    score DECIMAL(5, 2),
    exam_date TIMESTAMP NOT NULL
)
PARTITION BY RANGE(UNIX_TIMESTAMP(exam_date))
(
  PARTITION p0 VALUES LESS THAN (UNIX_TIMESTAMP('2021-01-
01')) ,
  PARTITION p1 VALUES LESS THAN (UNIX_TIMESTAMP('2022-01-
01')) ,
  PARTITION p2 VALUES LESS THAN (UNIX_TIMESTAMP('2023-01-
01'))
);
```

■ Add a new partition

```
# MySQL schema #
ALTER TABLE enrollment_his
ADD PARTITION(PARTITION p3 VALUES LESS THAN
(UNIX_TIMESTAMP('2024-01-01')),
              PARTITION p4 VALUES LESS THAN
(UNIX_TIMESTAMP('2025-01-01'))
              );
# ORACLE schema #
ALTER TABLE enrollment_his
ADD PARTITION(PARTITION p3 VALUES LESS THAN
(UNIX_TIMESTAMP('2024-01-01')),
              PARTITION p4 VALUES LESS THAN
(UNIX_TIMESTAMP('2025-01-01'))
              );
```

■ Delete a partition

```
ALTER TABLE enrollment_his
DROP PARTITION p3,p4;
```

■ Empty a partition

```
ALTER TABLE enrollment_his
TRUNCATE PARTITION p3,p4;
```

Table Group Design - Table Group Requirements

In the previous example of "Partition Design Recommendations," we assumed that both the Table students and the Table enrollment have a large amount of data and have been divided into 6 data partitions each.

Assuming we want to query the test scores for various subjects of a student with the student ID 2024010102, as shown in the figure on the right.

- Student information and enrollment information are both located in the Partition P0 of the data table, and according to the settings of load balancing and Primary Zone strategies, these partitions have been scheduled onto two different OBServer hosts.
- When performing a join query on the two tables, we need to access the data partitions located on these two OBServer hosts to retrieve the required data. This cross-machine access will increase the query overhead and latency.

To solve this problem, OceanBase database provides the function of Table Group. By placing the Table students and the Table enrollment in the same table group, you can ensure that partitions with the same partition value are always located on the same OBServer host, thereby avoiding cross-machine access.

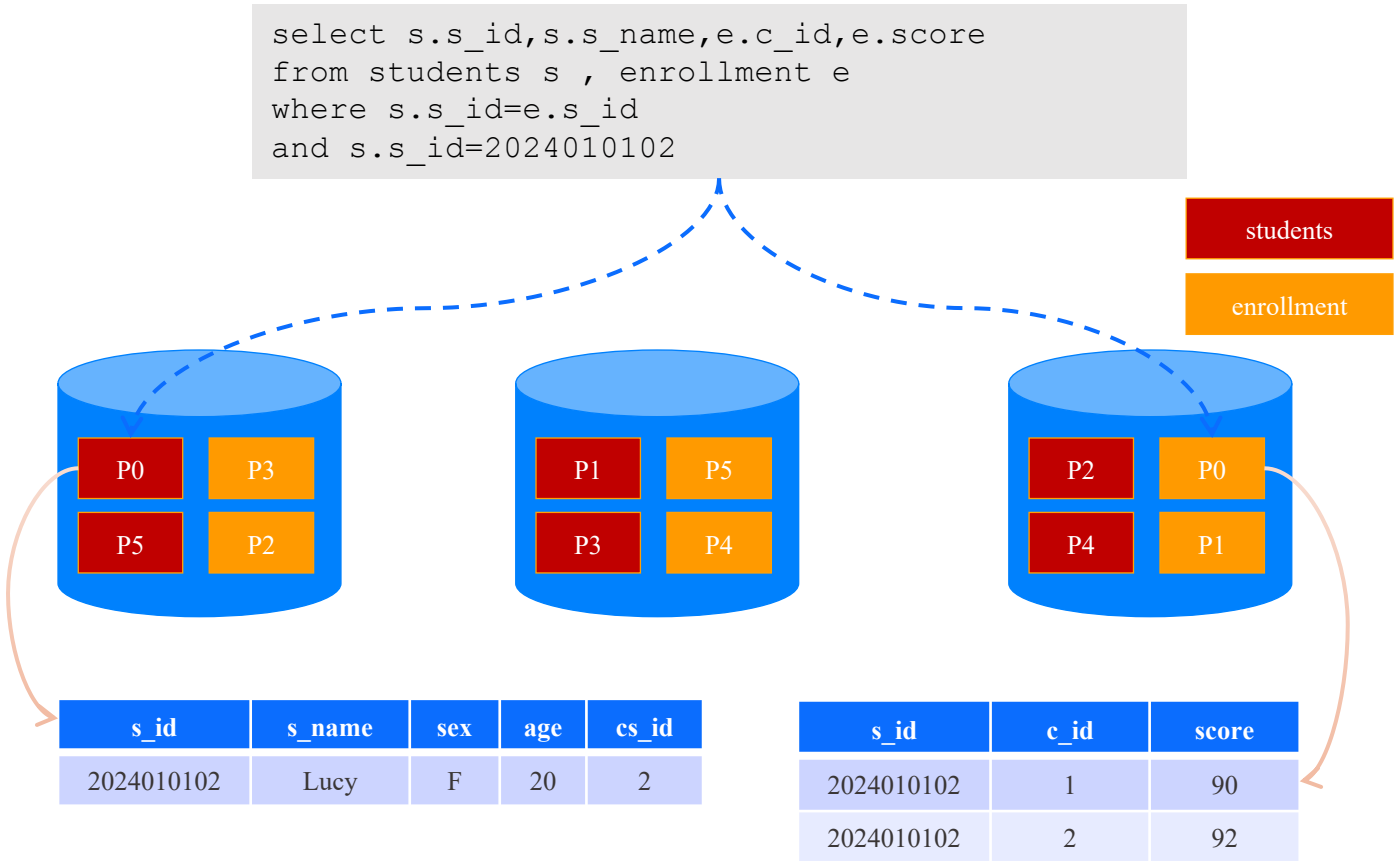
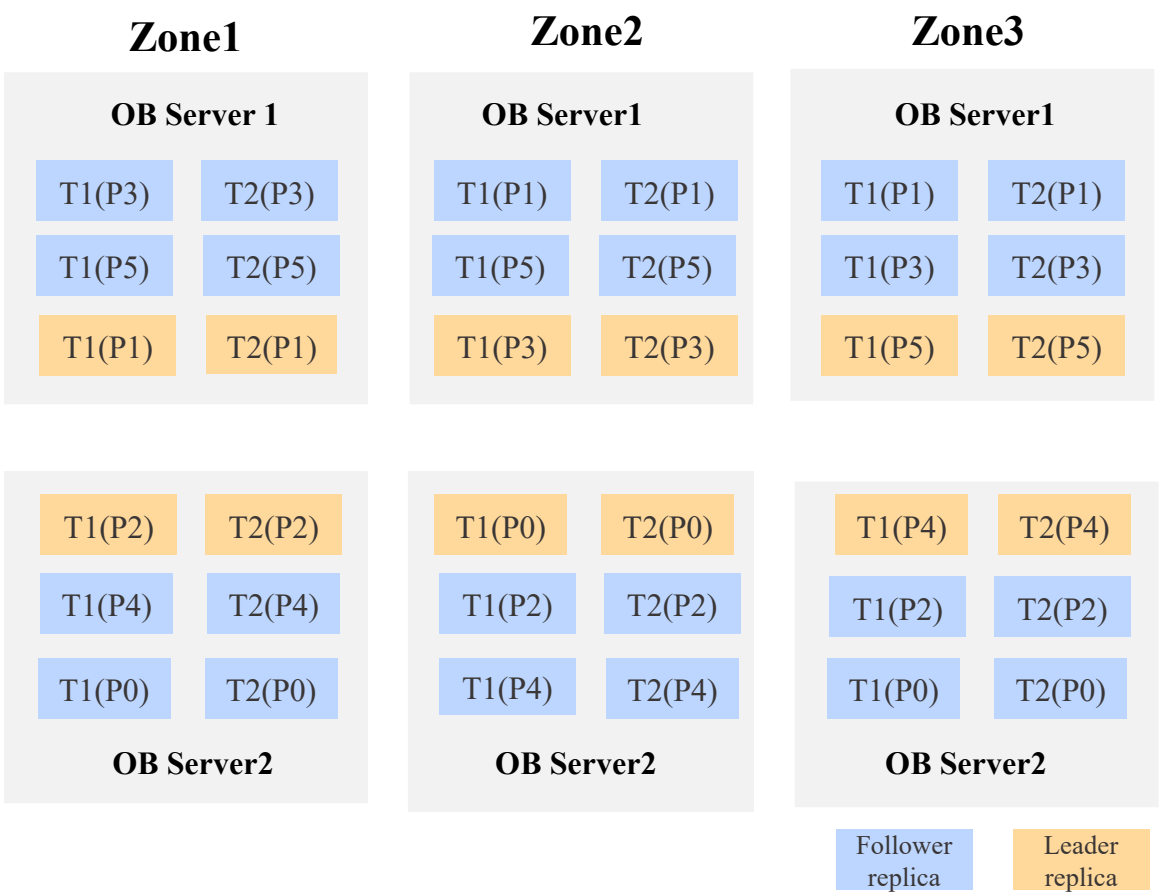


Table Group Design - Create a Table Group

T1 represents the Table students and T2 represents the Table enrollment.



Taking SHARDING = 'PARTITION' as an example:

- Suppose a cluster has 3 Zones, and the tenant's Unit Num is 2, with the Primary Zones of Table 1 and Table 2 being load-balanced
- The Table students and the Table enrollment form a Table Group.
- The Leader replicas with the same partition IDs in the Table Group are all on the same OB Server
- For a particular student, his/her personal information data and enrollment data are definitely served on the same OB Server, such as students (p0) and enrollment (p0)
- The overall load balance is not disrupted; each OBServer has both Leader and Follower replicas.
- If Zone2-OB Server2 fails, T1(P0) + T2(P0) will be migrated as a whole

```
CREATE TABLEGROUP tblgroup1 SHARDING = 'PARTITION';

CREATE TABLE students ( S_ID BIGINT AUTO_INCREMENT
NOT NULL,S_Name VARCHAR(50) NOT NULL,Sex CHAR(1),Age
INT,CS_ID INT  NOT NULL,PRIMARY KEY (S_ID))TABLEGROUP
= tblgroup1 partition by hash (S_ID) partitions 6;

CREATE TABLE enrollment (S_ID BIGINT,C_ID
BIGINT,score DECIMAL(5, 2), FOREIGN KEY (S_ID)
REFERENCES Students(S_ID),FOREIGN KEY (C_ID)
REFERENCES Courses(C_ID))TABLEGROUP = tblgroup1
partition by hash (S_ID) partitions 6;
```

Table Group Design - Creation Recommendations

Starting from V4.2.0, the table group no longer has the concept of partition, you can flexibly join tables with different partition methods only by defining the SHARDING attribute, if you want to restrict other tables to join the table group, you can modify the attributes of the table group.

SHARDING Attribute	Description
SHARDING = NONE	Partitioning requirements: There is no restriction on the partition types of the tables within the table group. Partition alignment rules: All partitions of all tables within such a table group are aggregated on the same machine.
SHARDING = PARTITION	■ Partitioning requirements: All tables must have the same partition definition. For subpartitioned tables, the partition definition, rather than the subpartition definition, is verified. Therefore, a table group can contain both partitioned and subpartitioned tables only if they have the same partition definition. ■ Partition alignment rules: Partitions with the same partition value are aggregated together, including partitions of partitioned tables and all subpartitions in the corresponding partitions of subpartitioned tables.
SHARDING = ADAPTIVE	■ Partitioning requirements: Tables in such a table group are all partitioned tables or all subpartitioned tables. If they are partitioned tables, they must have the same partition definition. If they are subpartitioned tables, they must have the same partition definition and subpartition definition. ■ Partition alignment rules: If all tables in such a table group are partitioned tables, partitions with the same partition value are aggregated together. If all tables in such a table group are subpartitioned tables, partitions with the same partition value and subpartition value are aggregated together.

Recommendations for creating table groups:

- It is recommended that the table group name be concise and clear, accurately reflecting its purpose and content for ease of later management and maintenance. For example, when naming table groups, you can name them according to business requirements, functional modules or data types, etc. You can name the order table group, user table group and product table group as order_tablegroup, user_tablegroup and product_tablegroup.
- It is recommended to perform good planning and design before creating table groups, considering factors such as business requirements, data relationships and query patterns to determine the necessity of grouping tables.
- It is recommended to place related tables in the same table group based on their join and query frequency to enhance query efficiency and data availability.
- It is recommended to reasonably authorize and manage the permissions of table groups to ensure that only authorized users can perform operations on the table groups to maintain data security and integrity.

Table Group Design - Basic Operations

■ Create a table group

```
CREATE TABLEGROUP tablegroup_name [tablegroup_option];
tablegroup_option:
SHARDING = 'NONE'
SHARDING = 'PARTITION'
SHARDING = 'ADAPTIVE'
```

Example

```
CREATE TABLEGROUP tblgroup1 SHARDING = 'NONE';

CREATE TABLEGROUP tblgroup2 SHARDING = 'PARTITION';

CREATE TABLEGROUP tblgroup3 SHARDING = 'ADAPTIVE';
```

■ Add tables to a table group

```
# Create a table and add the table to a table group #
CREATE TABLE students (
    S_ID BIGINT   NOT NULL,
    S_Name VARCHAR(50) NOT NULL,
    Sex CHAR(1),
    Age INT,
    CS_ID INT  NOT NULL,
    PRIMARY KEY (S_ID)
)TABLEGROUP = tblgroup1 partition by hash (S_ID)
partitions 6;

# Add an existing table to the table group #
ALTER TABLEGROUP tblgroup1 ADD classes;
```

■ View a table group

```
SHOW TABLEGROUPS WHERE tablegroup_name = 'tblgroup1';
```

■ Delete a table group

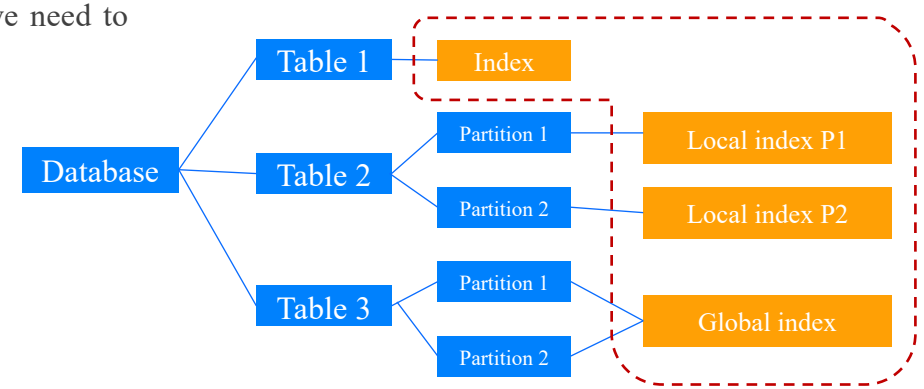
```
DROP TABLEGROUP IF EXISTS tblgroup1;
```

Index Design - Concepts and Design Recommendations

In order to improve database query performance and ensure data consistency and integrity, we need to create appropriate indexes on certain columns.

Index types:

- OceanBase database supports primary key indexes, unique indexes, and also secondary indexes, which constitute the above indexes can be a single column or multiple columns (composite indexes).
- Creating indexes on partition tables can be categorized into two types: Local indexes and global indexes. The difference between the two is: Local indexes share the same partition as the partition data, while global indexes have their own separate partition.



Based on the rules of index matching and the characteristics of OceanBase database, we recommend to follow the principles below to create and design indexes.

It is recommended to select **columns** that meet the following requirements to create indexes:

- **Most frequently queried columns:** Columns that frequently appear in WHERE clauses, and in join queries, are good candidates for indexing.
- **Highly selective columns:** The selectivity of an index is the ratio of unique values to the total number of rows, and a highly selective column means it can filter data better. For composite indexes, it is recommended to place highly selective columns in front.
- **Sorted and grouped columns:** Columns included in the ORDER BY, GROUP BY, and DISTINCT clauses can, if indexed, avoid sorting operations and thus improve query efficiency.

The following principles shall be observed to create indexes:

- **Covering index:** Utilize indexes that contain all necessary columns as much as possible, which can eliminate the need to access table data while meeting query requirements.
- **Equality priority:** For composite indexes, it is recommended to place columns with equality conditions in front.
- **Avoid indexing frequently updated columns:** This is because indexes need to be updated when updating data as well, which increases the overhead of write operations.
- **Limit the number of indexes:** More indexes are not better because each additional index requires maintenance time and increases the cost of write operations. Also indexes take up physical storage space.

Index Design - Difference between Primary Key and Unique Indexes

In OceanBase database, the primary key has the following characteristics

- **Non-null and unique:** The primary key cannot be NULL, and the primary key value shall be unique across the entire table. A primary key uniquely identifies a record, and the primary key value can be considered as the ID of the record.
- **Globally unique:** The primary key must be unique across the entire table, hence the primary key of a partition table must include the partition key, ensuring global uniqueness by maintaining uniqueness within the scope of different partitions.
- **Quantitatively unique:** In the OceanBase database, tables have one and only one primary key. If users do not specify a primary key for the table, OceanBase database will create a hidden primary key by default.
- **Primary key as a table:** In the OceanBase database, the primary key and table are two-in-one. The table data is stored in the order of the primary key, and no separate index is created specifically for the primary key.

Example

- When creating the Table students, specify S_ID as the primary key:

```
CREATE TABLE students (
    S_ID BIGINT   NOT NULL,
    S_Name VARCHAR(50) NOT NULL,
    Sex CHAR(1),
    Age INT,
    CS_ID INT  NOT NULL,
    PRIMARY KEY (S_ID)
) partition by hash (S_ID) partitions 6;
```

Unique indexes in an OceanBase database have the following characteristics

- **Nullable and Unique except Null:** Unique indexes are allowed to be NULL values and can contain duplicate NULL-valued records. Non-NULL key values must be unique across the entire table.
- **Globally unique:** Unique indexes must be unique across the entire table, so when defining unique local indexes in partition tables, the local indexes must include partition keys to ensure global uniqueness by maintaining uniqueness within the scope of different partitions.
- **Constraints as indexes:** OceanBase database automatically creates unique indexes when unique constraints are defined in the table.

Example

- Create a unique index on the NAME and S_ID columns in a Table students

```
CREATE UNIQUE INDEX students_ix2
ON students(s_name,s_id)
LOCAL;
```

Note: If the index key includes the partition key S_ID, the uniqueness within each partition implies global uniqueness, allowing the creation of a local index.

Index Design - Local Indexes and Global Indexes

Partition table index recommendation:

Select in the order of precedence of Local Indexes -> Global Partition Indexes -> Global Indexes, and use global indexes only when necessary, because global indexes will degrade the performance of DML and may result in distributed transactions.

Local indexes

- Local indices are created for data within a single partition, hence the index key values of a local index correspond one-to-one with the data in the table. That is, a partition of a local index always corresponds to a table partition, sharing the same partition rules.
- If you want to use a locally unique index to constrain data uniqueness, then the locally unique index must contain the table partition key.

Example

- Create a locally unique index on NAME,S_ID column of the Table

```
CREATE UNIQUE INDEX students_ix2
ON students(s_name,s_id)
LOCAL;
```

Global indexes:

- In business scenarios, in addition to the primary key, there are strong requirements for global uniqueness that need to be met by combinations of other columns, and this business requirement can only be achieved through the use of a globally unique index.
- In business scenarios where queries do not have access to the partition key as a conditional predicate, and the business table does not experience high concurrency in simultaneous writes, to avoid a full partition scan, a global index can be constructed based on the query conditions. If necessary, the global index can be partitioned according to a new partition key.

Example

- Create a globally unique index on the s_Name column of the Table students:

```
CREATE UNIQUE INDEX students_ix1
ON students(s_name)
GLOBAL;
```

Index Design - Basic Operations

■ Create indexes

```
CREATE [SPATIAL | UNIQUE] INDEX index_name
ON table_name
(sort_column_key [, sort_column_key... ])
[index_option...]
[partition_option];

sort_column_key:
    index_expr [ASC] opt_null_pos [ID id]

index_option: GLOBAL | LOCAL
```

Example

```
CREATE INDEX classes_ix1
ON classes(cs_name);
```

■ Delete indexes

```
DROP INDEX classes_ix1 on classes; # MySQL Schema
#
DROP INDEX classes_ix1 ; # Oracle Schema #
```

■ View index definitions

```
# MySQL schema #
SHOW INDEX FROM classes ;
```

```
obclient [school_db]> SHOW INDEX FROM classes;
+---------+------------+------------+--------------+-------------+-----------+-------------+----------+--------
| Table   | Non_unique | Key_name   | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed
+---------+------------+------------+--------------+-------------+-----------+-------------+----------+--------
| classes |          0 | PRIMARY    |            1 | CS_ID       | A         |        NULL | NULL     | NULL
| classes |          1 | classes_ix1 |            1 | CS_Name     | A         |        NULL | NULL     | NULL
+---------+------------+------------+--------------+-------------+-----------+-------------+----------+--------
2 rows in set (0.014 sec)
```

```
# ORACLE schema #
SELECT
DBMS_METADATA.GET_DDL('INDEX','CLASSES_IX1','SCHOOL
_DB')
FROM DUAL;
```

```
obclient [SCHOOL_DB]> SELECT DBMS_METADATA.GET_DDL('INDEX','CLASSES_IX1','SCHOOL_DB')  from dual;
+-------------------------------------------------------------------------------------------------+
| DBMS_METADATA.GET_DDL('INDEX','CLASSES_IX1','SCHOOL_DB')                                        |
+-------------------------------------------------------------------------------------------------+
| CREATE INDEX "SCHOOL_DB"."CLASSES_IX1" on "SCHOOL_DB"."CLASSES" (
 "CS_NAME"
) GLOBAL ;
 |
+-------------------------------------------------------------------------------------------------+
1 row in set (0.159 sec)
```

Database Design - Views

A view is a virtual table derived from one or more tables. Essentially, a view is a pre-stored query that the user accesses to get the data defined by that view. Unlike a table, a view has no allocated storage space and a view does not contain data; a view simply extracts or derives data from a referenced base table.

Advantages of views:

- Simplifies operations: Some of the SQL statements in a database can be quite complex, involving multiple table joins, displaying a large number of fields, etc. If such queries need to be executed frequently, creating a view can simplify the query operation.
- **Improves data security:** Important field information in the base table can be shielded and not displayed to users via the view.

Syntax of views:

```
create_view_stmt:
  CREATE [OR REPLACE] VIEW view_name
[(column_name_list)] AS select_stmt
[WHERE condition WITH {LOCAL | CASCADED}
CHECK OPTION];

column_name_list:
    column_name [, column_name ...]
```

In the "Online Education Information System," we often need to frequently query a student's personal information, enrollment information, and class information based on the student name. At this point, you can create a view to simplify the query operation.

Create views

```
create  VIEW  v_students_info AS
select s.s_name, c.c_name,e.score, cs.cs_name
from students s , enrollment e , courses c , classes cs
where s.s_id=e.s_id and e.c_id=c.c_id and
s.cs_id=cs.cs_id
```

Query views

```
select * from v_students_info where s_name='Lucy' ;
```

Database Design - Sequences

In the OceanBase database, a sequence is an auto-increment numeric sequence number generated by the database according to certain rules, which is typically a set of values at regular intervals (of numeric type). Due to their auto-incrementing nature, sequences are commonly used as primary keys and unique keys.

The syntax for creating a sequence is as follows:

```
CREATE SEQUENCE [IF NOT EXISTS]
[database_name. ]sequence_name
[MINVALUE int_value | NOMINVALUE]
[MAXVALUE int_value | NOMAXVALUE]
[START WITH int_value]
[INCREMENT BY int_value]
[CACHE int_value | NOCACHE]
[ORDER | NOORDER]
[CYCLE | NOCYCLE];
```

In the "Online Education Information System", the Table classes contains two columns: *Class ID and Class Name. To automatically generate class IDs when inserting class names, we can utilize a sequence.*

Create seq1 with an auto-increment step of 1

```
CREATE SEQUENCE seq1 START WITH 1 MINVALUE 1 MAXVALUE
1000000 INCREMENT BY 1 NOCYCLE NOORDER CACHE 30;
```

Refer to a sequence

```
/*The sequence number will increment with each execution*/
SELECT seq1.NEXTVAL FROM DUAL;
/*When inserting, reference the seq1*/
INSERT INTO classes(cs_id,cs_name)
VALUES(seq1.NEXTVAL,'obca');
```

Modify a sequence

```
 ALTER SEQUENCE seq1 RESTART START WITH 3;
```

Delete a sequence

```
 DROP SEQUENCE seq1;
```

Database Design - Stored Procedure

Stored procedures are a precompiled collection of SQL statements along with optional control-flow statements, which are treated as a single unit by PL. A stored procedure can refer to other stored procedures and can return multiple variables.

The syntax for creating a stored procedure is as follows (MySQL tenant):

```
CREATE PROCEDURE sp_name
([proc_parameter[,...]])[characteristic ...]
    BEGIN -- Start execution
      SQL statement; [ SQL statement; ]...
    END; -- End execution

proc_parameter:
    [ IN | OUT | INOUT ] param_name type

characteristic: {
    COMMENT 'string'
  | LANGUAGE SQL
  | [NOT] DETERMINISTIC
  | { CONTAINS SQL | NO SQL | READS SQL DATA |
MODIFIES SQL DATA }
  | SQL SECURITY { DEFINER | INVOKER }
}
```

In an "Online Education Information System," suppose we need to create a stored procedure to count the number of courses a student is enrolled in.

Create a stored procedure

```
DELIMITER //
CREATE PROCEDURE enroll_count(IN id INT,OUT enroll_count
INT)
   BEGIN
     SELECT COUNT(*) INTO enroll_count FROM enrollment
WHERE s_id=id;
   END //
DELIMITER;
```

Call the stored procedure

```
//Initialize parameters
SET @id=2024010102,@enroll_count=0;

//Call the stored procedure
CALL enroll_count(@id,@enroll_count);
```

Application Development

05

- OceanBase Development Tools
- OceanBase Database Design
- **OceanBase DML/DQL Statements**
- OceanBase SQL Tuning
- OceanBase Data Import/Export

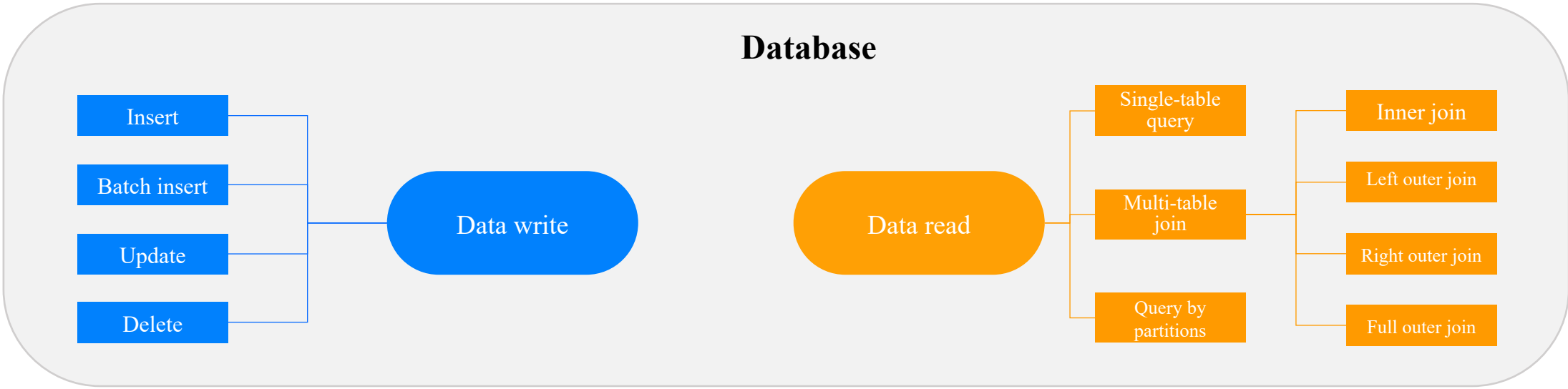
Database's Basic Services - Data Read/Write

Data write

- Write operations include insertion, updating, deletion, replacement, etc.
- Batch data insertion: In scenarios that require a large amount of simulated data for testing database performance or verifying features, ODC provides a data mocking feature. This feature can quickly generate data based on the field types in the table.

Data read

- Read operations include single-table queries and multi-table join queries.
- Multi-table join queries further possess the capabilities such as inner join, left outer join, right outer join, full outer join, etc., thereby meeting the needs for complex data query and data analysis.
- Query by partitions: In addition to partition pruning based on the query conditions of the SQL statement, OceanBase database also supports users to specify the partitions to access through the SQL statement.



INSERT

INSERT statement is used to insert new rows of data into a database table. With the INSERT statement, you can specify the columns to be inserted and the corresponding values, or insert an entire row of data.

■ Syntax

```
# Insert single row of data #
INSERT INTO table_name (column1, column2,
column3, ...)
VALUES (value1, value2, value3, ...) ;

# Insert multiple rows of data #
INSERT INTO table_name (column1, column2,
column3, ...)
VALUES
(value1a, value2a, value3a, ...) ,
(value1b, value2b, value3b, ...) ,
...;

# Omit the column names (specify the value of
each column in turn in the order of the columns
in the table) #
INSERT INTO table_name
VALUES (value1, value2, value3, ...) ;
```

Example

Insert a new record into the Table students:

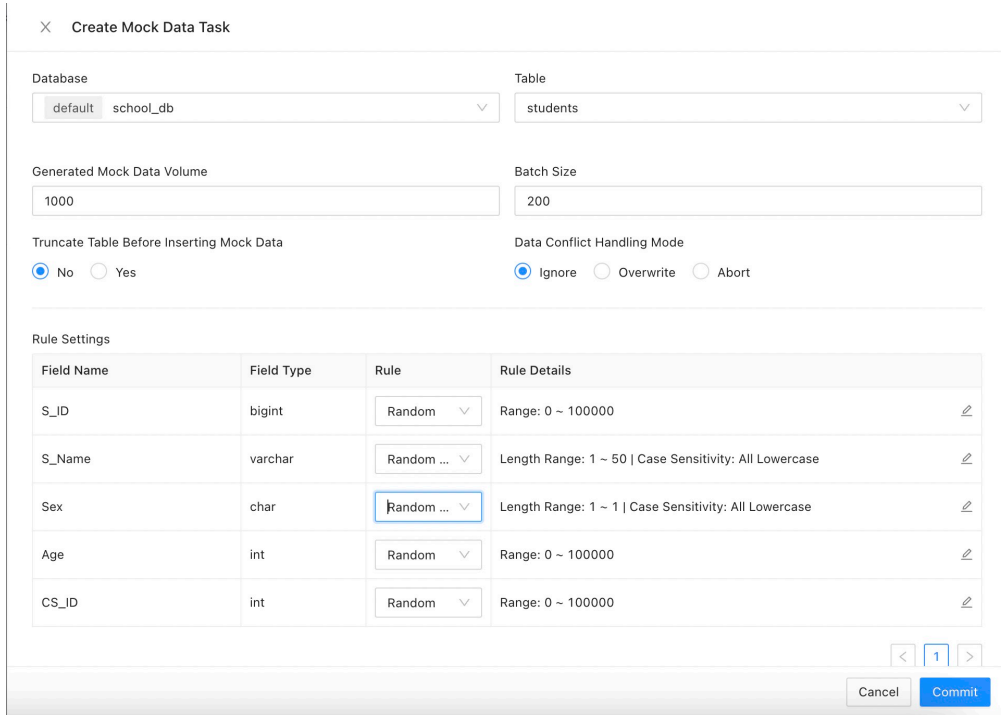
```
INSERT INTO students(s_id,name,sex,age)
VALUES(2024010101, 'GARY','M',23);
```

S_ID	Name	Sex	Age
2023010101	TOM	M	20
2022020202	AMY	F	21
2023030103	BETTY	F	20
2024040303	JACK	M	19
row 2024010101	GARY	M	23

Batch Insertion (ODC Data Mocking Feature)

In scenarios requiring a substantial amount of simulation data for database performance testing or feature verification, ODC offers a data mocking feature. This feature enables the rapid generation of data based on the field types of the table, assisting developers in quickly establishing realistic testing environments. This feature can save developer's time and effort, enhance the efficiency of testing, and ensure the accuracy and consistency of test data.

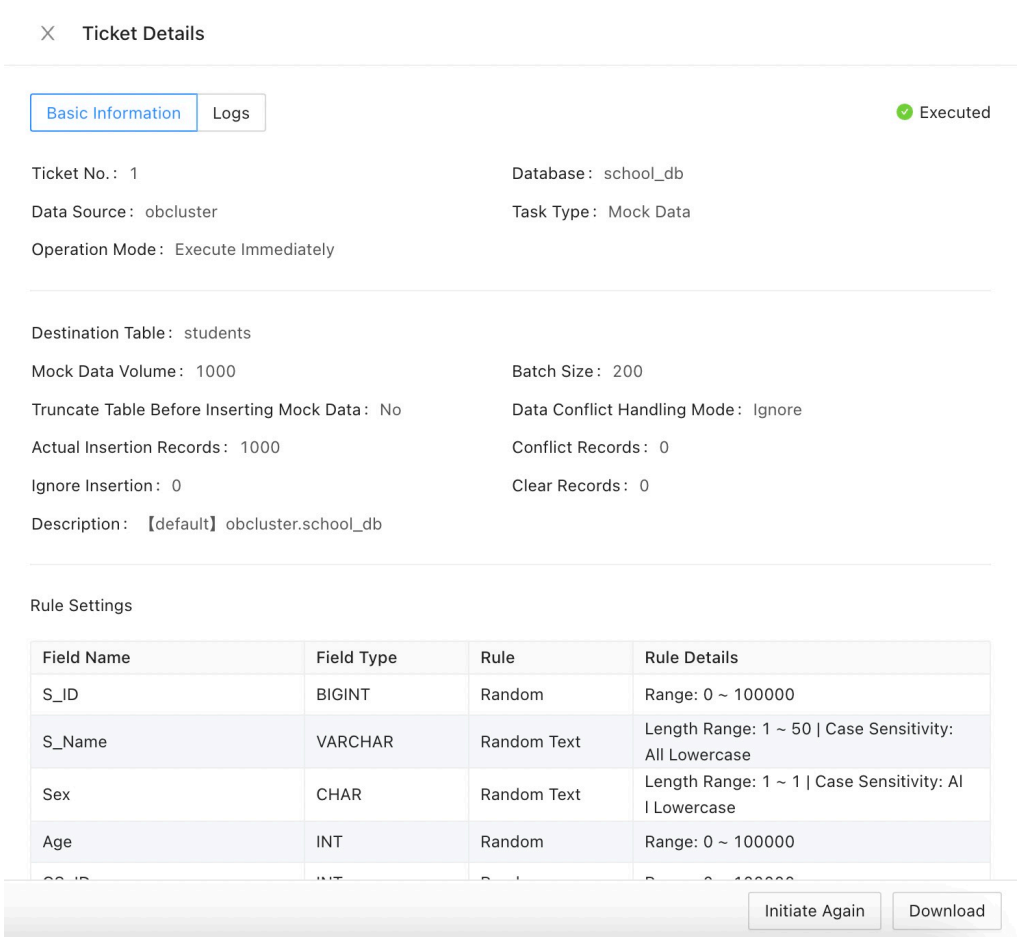
New simulated data

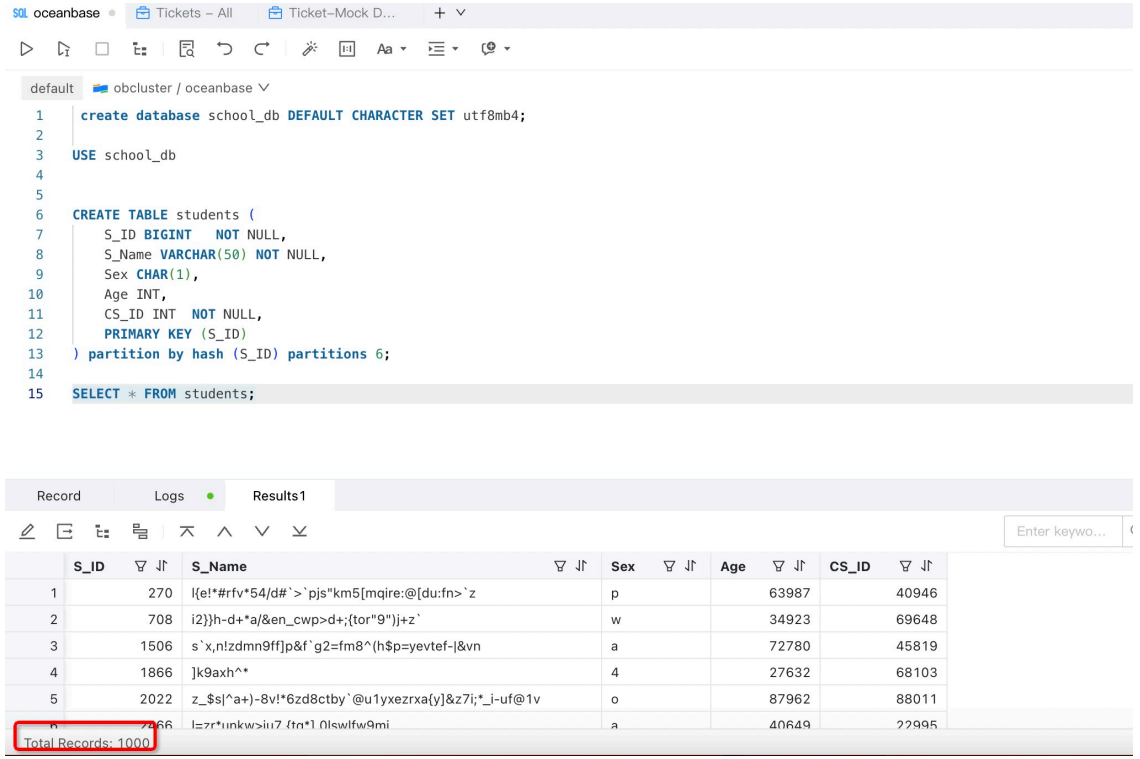


Information Item	Description
Database	Select the database to perform insertion, such as school_db.
Table	Select the table in the database to perform insertion, such as students.
Generation amount of simulated data	Specify the amount of data to be generated. Default value: Generate 1000 data entries, with a maximum limit of not exceeding 1 million rows of data.
Batch processing size	Specify the number of data entries to generate before performing a single commit (COMMIT command) operation. By default, a commit is made after generating 200 data entries.
Insert simulated data and clearing table	Select whether to clear the existing data in the table when inserting the simulated data.
Data conflict resolutions	Ignore: The default is Ignore, if there is a data conflict, the conflict data will not be inserted. Overwrite: Delete the existing data and insert new data if there is a data conflict. Terminate: If there is a data conflict, stop generating data and terminate the whole simulated data task.
Rule setting	Setting up data generation rules
Task setting	Select Immediate Execution or Scheduled Execution.

Batch Insertion (ODC Data Mocking Feature)

After submission, you can view the task in the task list on the Mock Data tab of the Task Center page. Click Task Information to view the basic task information and the data mocking settings.





UPDATE

UPDATE statement is used to modify records in an existing database table. With the UPDATE statement, you can update the values of certain fields in a table based on specific conditions.

- **Syntax**

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Example

Update a record in the Table students:

```
UPDATE students
SET age=22
WHERE s_id=2024010101 and name='GARY';
```

	S_ID	Name	Sex	Age
	2023010101	TOM	M	20
	2022020202	AMY	F	21
	2023030103	BETTY	F	20
	2024040303	JACK	M	19
row	2024010101	GARY	M	22

Note:

When the WHERE clause is omitted, all data in the table will be modified.

DELETE

DELETE statement is used to delete records from a table. With the DELETE statement, you can delete certain records in a table based on specific conditions.

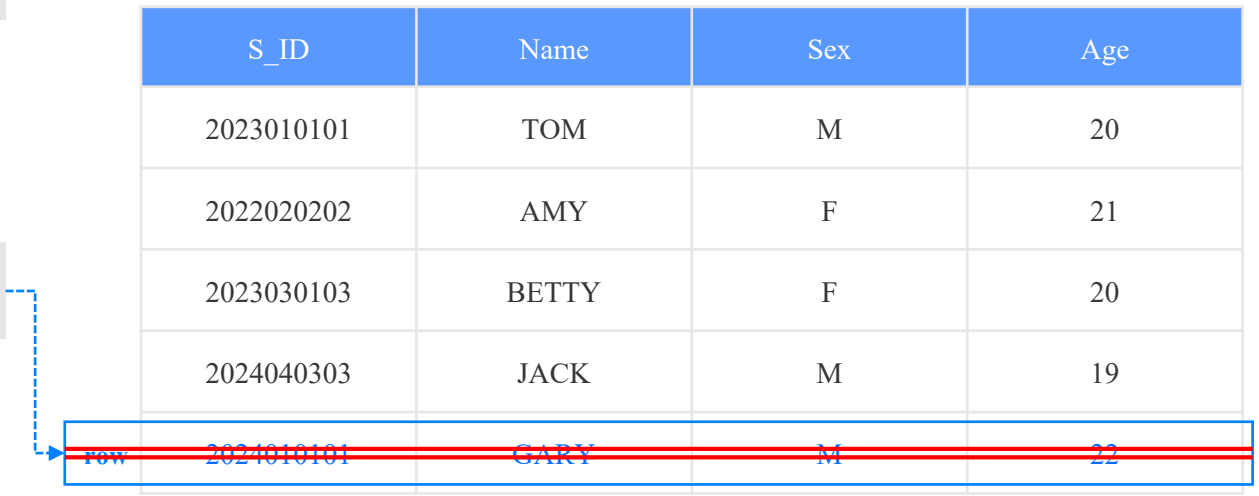
■ Syntax

```
DELETE FROM table_name
WHERE condition;
```

Example

Delete a record in the Table students:

```
DELETE FROM students
WHERE s_id=2024010101 and name='GARY';
```



S_ID	Name	Sex	Age
2023010101	TOM	M	20
2022020202	AMY	F	21
2023030103	BETTY	F	20
2024040303	JACK	M	19
row 2024010101	GARY	M	22

Note:

When the WHERE clause is omitted, all data in the table will be deleted.

SELECT

SELECT is the method used in databases to retrieve data. It can be combined with limiting clauses (such as WHERE) and ordering clauses (such as ORDER BY) to obtain the select results from the database.

■ Syntax

```
SELECT [/*+ hint statement */]
[ALL | DISTINCT] selectexpr [,selectexpr ...]
[FROM table_references]
[WHERE where_conditions]
[GROUP BY group_by_list]
[HAVING search_confitions]
[ORDER BY order_list]
[LIMIT [offset,] rowcount]|[FETCH FIRST rowcount ROWS]
[FOR UPDATE];
```

Example

Query the information about a student named BETTY:

```
SELECT *
FROM students
WHERE name = 'BETTY';
```

S_ID	Name	Sex	Age
2023010101	TOM	M	20
2022020202	AMY	F	21
2023030103	BETTY	F	20
2024040303	JACK	M	19

Notes: SQL syntax follows the tenant compatibility mode, in MySQL mode, you have to use SQL syntax of MySQL and in Oracle mode, you have to use SQL syntax of Oracle.

Basic Concepts of Joins

A join is a query that combines two or more tables, views or materialized views. When multiple tables appear in the FROM clause of a query, OceanBase database will perform a join, and the output columns of the query can be selected from any table in the FROM clause. If multiple tables have a column with the same name, then you must qualify all references to these columns with the table name during the query process.

Join type	Description
Inner join	The result is a join of the matching rows from the two joined tables.
Left (outer) join	The result includes all rows in the left table (on the leftmost side of the JOIN clause), excluding unmatched rows in the right table.
Right (outer) join	The result includes all rows in the right table (on the rightmost side of the JOIN clause), excluding any unmatched rows in the left table.
Full (outer) join	The result includes all rows from both left and right tables

Multi-table Join Query - Inner Join

INNER JOIN:

When joining two or more tables, the join operation is used to return data from multiple tables that meet the join conditions. The keyword INNER is usually omitted. The tables before and after the JOIN are respectively referred to as the left table and the right table. The conditions within the ON clause specify both the join conditions and the filtering conditions between the left table and right table.

Taking the Table students and Table classes in the "Online Education Information System" as an example, we will show how to perform a join query to obtain student information and their corresponding class information.

Table Students

Student ID (s_id)	Name (s_name)	Sex	Age	Class ID (cs_id)
2024010102	Lucy	F	20	2
2024010103	Jack	M	23	4
2024010101	Lily	M	22	1

Table Classes

Class ID (cs_id)	Class name (s_name)
1	obca
2	obcp
3	obce

```
select  s.*,c.cs_name from students s inner join
classes c  on  s.cs_id=c.cs_id order by s_id;
```

S_ID	Name	Sex	Age	cs_id	cs_name
2024010101	Lily	M	22	1	obca
2024010102	Lucy	F	20	2	obcp

Multi-table Join Query - Left Join

LEFT JOIN:

This statement enables the retrieval of all records from the left table, along with the records where the join conditions are met in both the left and right tables. When using a left join query, if a row in the left table is not found in the right table, it is automatically filled with NULL in the right table.

Taking the Table students and Table classes in the "Online Education Information System" as an example, we will show how to perform a join query to obtain student information and their corresponding class information.

Table Students

Student ID (s_id)	Name (s_name)	Sex	Age	Class ID (cs_id)
2024010102	Lucy	F	20	2
2024010103	Jack	M	23	4
2024010101	Lily	M	22	1

Table Classes

Class ID (cs_id)	Class name (s_name)
1	obca
2	obcp
3	obce

```
select s.*, c.cs_name  from students s left join
classes c  on  s.cs_id=c.cs_id order by s_id;
```

S_ID	Name	Sex	Age	cs_id	cs_name
2024010101	Lily	M	22	1	obca
2024010102	Lucy	F	20	2	obcp
2024010103	Jack	M	23	4	NULL

Multi-table Join Query - Right Join

RIGHT JOIN:

It represents a right outer join, this statement enables the retrieval of all records from the right table, along with the records where the join conditions are met in both the left and right tables. In the results returned by the right join, if data from the left table does not exist, the corresponding column returns to NULL.

Taking the Table students and Table classes in the "Online Education Information System" as an example, we will show how to perform a join query to obtain student information and their corresponding class information.

Table Students

Student ID (s_id)	Name (s_name)	Sex	Age	Class ID (cs_id)
2024010102	Lucy	F	20	2
2024010103	Jack	M	23	4
2024010101	Lily	M	22	1

Table Classes

Class ID (cs_id)	Class name (s_name)
1	obca
2	obcp
3	obce

```
select  s.*,c.cs_name from students s right join
classes c  on  s.cs_id=c.cs_id order by s_id;
```

S_ID	Name	Sex	Age	cs_id	cs_name
NULL	NULL	NULL	NULL	NULL	obce
2024010101	Lily	M	22	1	obca
2024010102	Lucy	F	20	2	obcp

Multi-table Join Query - Full Join

FULL JOIN:

This statement represents a full outer join, which retrieves all rows from both tables involved in the join, whether or not there is a match between them. When using a full outer join query, if a row from the left table or the right table does not find a match, the system will automatically fill in NULL for the unmatched rows.

Taking the Table students and Table classes in the "Online Education Information System" as an example, we will show how to perform a join query to obtain student information and their corresponding class information.

Table Students

Student ID (s_id)	Name (s_name)	Sex	Age	Class ID (cs_id)
2024010102	Lucy	F	20	2
2024010103	Jack	M	23	4
2024010101	Lily	M	22	1

Table Classes

Class ID (cs_id)	Class name (s_name)
1	obca
2	obcp
3	obce

```
select  s.*,c.cs_name from students s full join
classes c  on  s.cs_id=c.cs_id order by s_id;
```

S_ID	Name	Sex	Age	cs_id	cs_name
NULL	NULL	NULL	NULL	NULL	obce
2024010101	Lily	M	22	1	obca
2024010102	Lucy	F	20	2	obcp
2024010103	Jack	M	23	4	NULL

Query data by partition

OceanBase database supports querying data by partition, allowing users to directly specify the desired partitions in SQL statements. The database optimizer can also calculate the partitions to access based on the query range of the partition key in the WHERE condition and perform corresponding **partition pruning**, (excluding non-conforming partitions from the scanning range). The partition SQL eventually accesses is the intersection of both the specified partition and the partition pruning.

Example

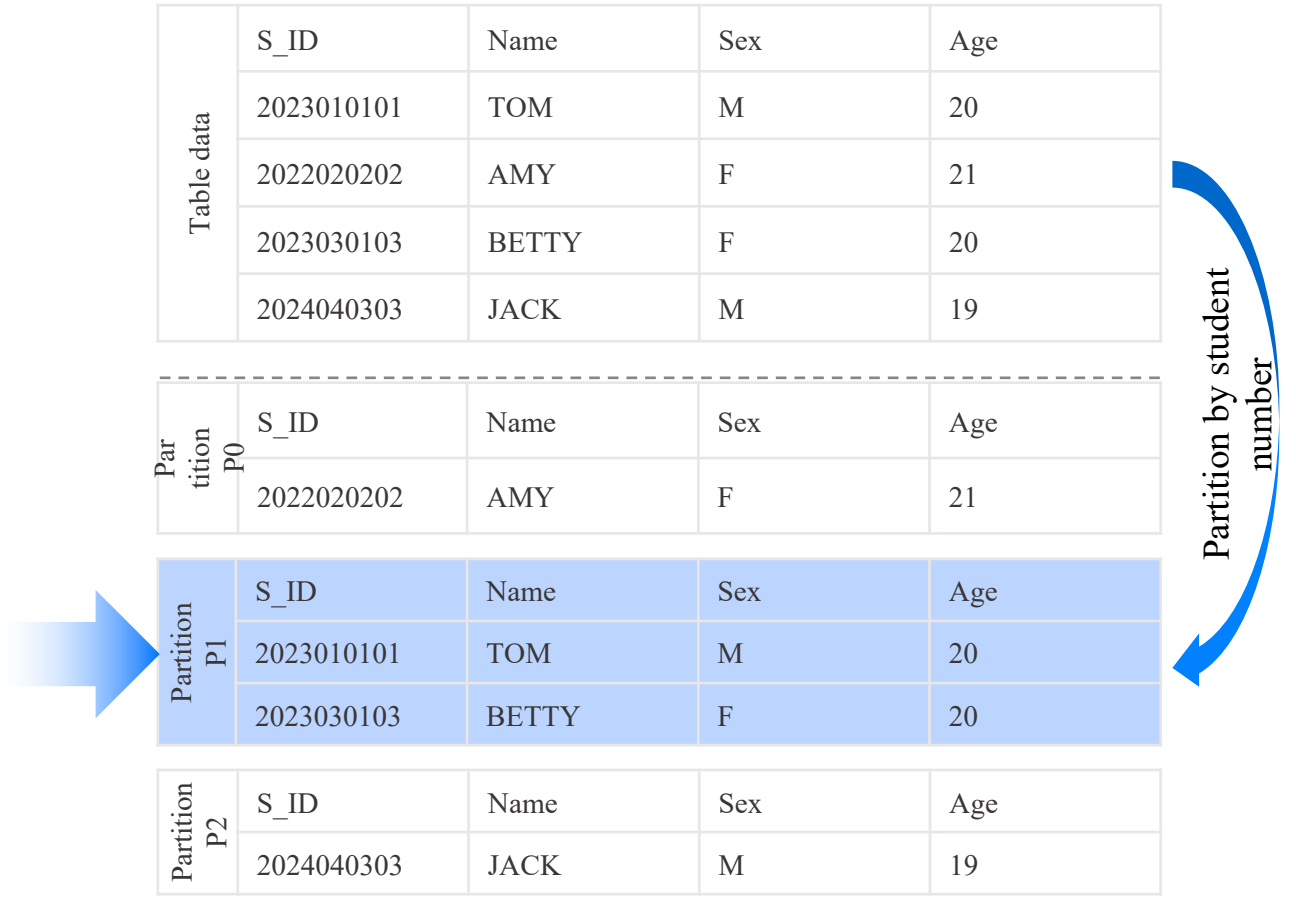
Partition the Table students by student number:

Partition	Student number range
P0	[,2022999999)
P1	[2022999999, 2023999999)
P2	[2023999999, 2024999999)

Query all data in Partition P1, in one of the two equal ways below

```
# Specify the partition directly in SQL #
SELECT * FROM students PARTITION(P1);

# Perform the partition pruning through the
database #
SELECT * FROM students;
WHERE s_id>2022999999 AND s_id<= 2023999999;
```



Application Development

05

- OceanBase Development Tools
- OceanBase Database Design
- OceanBase DML/DQL Statements
- **OceanBase SQL Tuning**
- OceanBase Data Import/Export

DBA's Advancement - Identifying and Resolving SQL Performance Issues

Performance issues:

Upon the initial launch of the "Online Education Information System," the data volume is limited and concurrency is low, so most SQL queries will not encounter performance issues. However, with the increase in user numbers and data volume, previously efficient SQL queries might become inadequate for the evolving environment.

Problem-solving approaches:

In order to resolve this problem, we need to identify and resolve potential performance issues in a timely manner. Performance may be improved by identifying slow queries, analyzing execution plans, optimizing indexes, or using Hint.

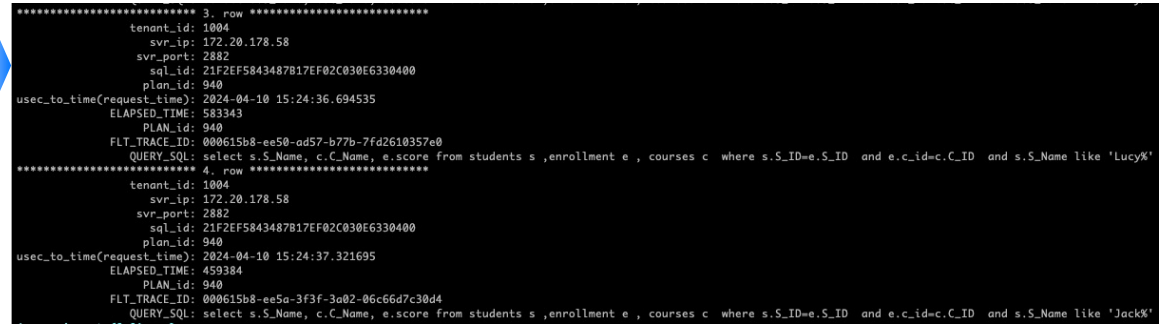
Identifying slow queries

For OceanBase database, you can use the OCP monitoring tools or GV$OB_SQL_AUDIT to pinpoint slow queries. GV$OB_SQL_AUDIT view can conveniently be used to investigate any SQL statement that has been executed in OceanBase, regardless of whether the execution was successful or failed, with detailed runtime information recorded. Moreover, OceanBase databases also cache the physical execution plans generated during the SQL execution process by default.

Identify slow queries (for example: Query SQL statements that took more than 100ms to execute in the last 10 minutes)

```
SELECT tenant_id,
   svr_ip,svr_port,sql_id,plan_id,
   usec_to_time(request_time),ELAPSED_TIME,
   PLAN_id,FLT_TRACE_ID,
   QUERY_SQL
FROM oceanbase.gv$OB_SQL_AUDIT
where
         TENANT_ID=1004
    and  request_time BETWEEN (TIME_TO_USEC(NOW())-
10*60*1000*1000)
    AND TIME_TO_USEC(NOW())
    and ELAPSED_TIME>100000
order by ELAPSED_TIME  desc
limit 10 ;
```

- **GV$OB_SQL_AUDIT:** A global SQL audit log table can be utilized to review various aspects of SQL execution, including the source of each client request, execution server information, execution status, waiting time, and the time spent in various stages of execution.



How to View Execution Plan

After finding a slow query, we can use EXPLAIN command to view the execution plan of the select statement in OceanBase database. In addition, we can also view the same execution plan in the SQL window through ODC.

Syntax of the EXPLAIN command:

```
EXPLAIN [BASIC | EXTENDED ] explainable_stmt;
```

- **EXPLAIN BASIC:** Utilized for the most basic plan display, it includes operator names, accessed table and index names, as well as filter information. BASIC is the default output mode.
- **EXPLAIN EXTENDED:** Used for the most detailed plan display, which can help you understand the optimizer's execution plan choices so that you can solve SQL execution plan problems.

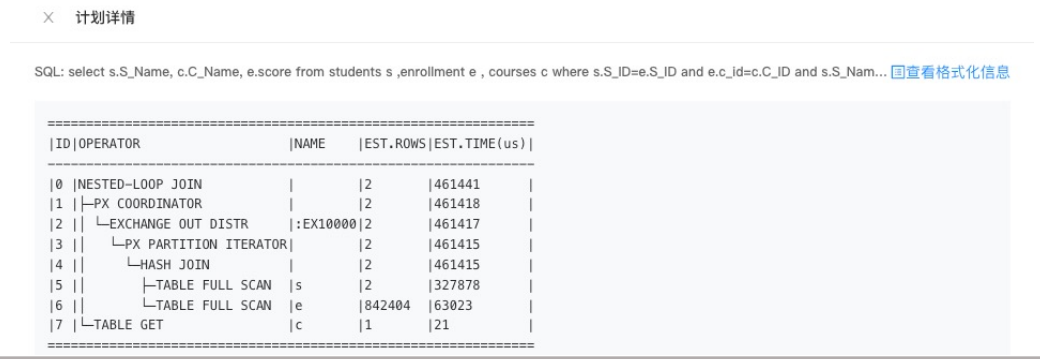
Example

```
select s.S_Name, c.C_Name, e.score from students s
,enrollment e , courses c  where s.S_ID=e.S_ID  and
e.c_id=c.C_ID  and s.S_Name like 'Lily%';
```

```
| ==========================================================
| |ID|OPERATOR                    |NAME    |EST.ROWS|EST.TIME(us)|
| ----------------------------------------------------------
| |0 |PX COORDINATOR              |        |2       |461443      |
| |1 | └EXCHANGE OUT DISTR        |:EX10000|2       |461441      |
| |2 |   └NESTED-LOOP JOIN        |        |2       |461438      |
| |3 |     ├PX PARTITION ITERATOR |        |2       |461415      |
| |4 |     │└HASH JOIN           |        |2       |461415      |
| |5 |     │  ├TABLE FULL SCAN   |s       |2       |327878      |
| |6 |     │  └TABLE FULL SCAN   |e       |842404  |63023       |
| |7 |     └DISTRIBUTED TABLE GET|c       |1       |21          |
| ==========================================================
```

Viewing the execution plan in ODC

1. In the SQL window, edit the SQL statement.
2. Select the SQL statement and click the execution plan icon.



Explanation of Execution Plan

Execution plan for table full scan

The table name (index name) of the corresponding table operation

Execution cost of the operation operator (in microseconds)

The number (starting from 0) obtained from a pre-order traversal of the execution tree

Name of operation operator

```
EXPLAIN SELECT * FROM students WHERE s_name='BETTY';
==========================================================
|ID|OPERATOR                   |NAME    |EST.ROWS|EST.TIME(us)|
----------------------------------------------------------
|0 |PX COORDINATOR             |        |1       |20          |
|1 |└─EXCHANGE OUT DISTR       |:EX10000|1       |19          |
|2 |  └─PX PARTITION ITERATOR  |        |1       |17          |
|3 |    └─TABLE FULL SCAN      |STUDENT |1       |17          |
==========================================================

Outputs & filters:
-------------------------------------
  0 - output ... , filter(nil)
  1 - output ... , dop=1
  2 - output ... , force partition granule, asc
  3 - output ... , partitions(p[0-2])
      range_key([STUDENT.S_ID]), range(MIN ; MAX)always true
```

- Named as TABLE FULL SCAN.
- NAME refers to the table name being "STUDENT".

Explanation of Execution Plan

Execution plan for primary key unique match

```
EXPLAIN SELECT * FROM students WHERE s_id=2023010101;
============================================
|ID|OPERATOR |NAME    |EST.ROWS|EST.TIME(us)|
--------------------------------------------
|0 |TABLE GET|STUDENT |1       |3           |
============================================
```

- The query condition is changed to: s_id=2023010101
- The operator changes to TABLE GET, meaning: Primary key unique match
- NAME is table_name, because the primary key is the table

Execution plan for index match

```
CREATE INDEX i_s_name ON students (s_name);
EXPLAIN SELECT * FROM students WHERE s_name='BETTY';
=============================================================
|ID|OPERATOR              |NAME            |EST.ROWS|EST.TIME(us)|
-------------------------------------------------------------
|0 |TABLE RANGE SCAN      |STUDENT(I_S_NAME)|1       |32          |
=============================================================
```

- The query condition is still s_name='BETTY', but the index i_s_name is added.
- The operator changes to TABLE RANGE SCAN, meaning: Index scan
- NAME is table_name(index_name)

Samples of SQL Tuning

The select statement is as follows:

```
select s.S_Name, c.C_Name, e.score from students s ,enrollment e , courses c  where s.S_ID=e.S_ID  and e.c_id=c.C_ID  and
s.S_Name like 'Lily%';
```

Before optimization

```
==========================================================
|ID|OPERATOR                  |NAME    |EST.ROWS|EST.TIME(us)|
----------------------------------------------------------
|0 |NESTED-LOOP JOIN          |        |2       |461441      |
|1 |├─PX COORDINATOR          |        |2       |461418      |
|2 |│ └─EXCHANGE OUT DISTR     |:EX10000|2       |461417      |
|3 |│   └─PX PARTITION ITERATOR|        |2       |461415      |
|4 |│     └─HASH JOIN          |        |2       |461415      |
|5 |│       ├─TABLE FULL SCAN  |s       |2       |327878      |
|6 |│       └─TABLE FULL SCAN  |e       |842404  |63023       |
|7 |└─TABLE GET               |c       |1       |21          |
==========================================================
```

Cost

TABLE FULL SCAN

Analyze:

1. The Table students is also referred to as 's' table, contains 1.5 million records,
2. Observing the select statement, the filter condition is **s.S_Name like 'Lily%'.**
3. Observing the execution plan, we know that a table full scan is done on the s table by the operator TABLE FULL SCAN.
4. Observing the execution plan, according to the filter condition, the final number of matching records is 1.

After optimization

Approach to solution:

- The table full scan of the 's' table is too costly and there is room for optimization.
- Build index for "s" table to improve query efficiency

```
create index i_s_name on students(s_name);
```

Perform validation:

```
=================================================================
|ID|OPERATOR                  |NAME       |EST.ROWS|EST.TIME(us)|
-----------------------------------------------------------------
|0 |NESTED-LOOP JOIN          |           |2       |133586      |
|1 |├─PX COORDINATOR          |           |2       |133563      |
|2 |│ └─EXCHANGE OUT DISTR     |:EX10000   |2       |133563      |
|3 |│   └─PX PARTITION ITERATOR|           |2       |133561      |
|4 |│     └─HASH JOIN          |           |2       |133561      |
|5 |│       ├─TABLE RANGE SCAN |s(i_s_name)|2       |23          |
|6 |│       └─TABLE FULL SCAN  |e          |842404  |63023       |
|7 |└─TABLE GET               |c          |1       |21          |
=================================================================
```

Cost

Index scan

Execution Plan Hint (HINT)

Hint is a type of SQL statement comments used to pass commands to the OceanBase database optimizer, through which the optimizer can generate the specified execution plan. In general, the optimizer will automatically select the best execution plan for user queries without the need for user-specified hints. However, in certain scenarios, the execution plan generated by the optimizer may not meet the user's requirements. In such cases, users may need to employ hints to proactively specify and generate specialized execution plans.

Hint is a special type of SQL comments, with `{DELETE|INSERT|SELECT|UPDATE|REPLACE} /*+ hint_text [,hint_text...] */`

- When the left marker of a comment is immediately followed by a plus sign (+), it indicates that what follows is the content of a Hint.
- A single statement can only contain one Hint comment, and this comment must immediately follow such keywords as SELECT, UPDATE, INSERT, REPLACE or DELETE.
- If the server is unable to recognize the Hint in the SQL statement, then the optimizer will choose to ignore the user-specified Hint and use the execution logic generated by the default plan.

1.INDEX Hint: Specify the index to be used for access

```
EXPLAIN SELECT /*+ INDEX(T1 I_S_NAME)*/ * FROM STUDENTS AS T1 WHERE S_NAME='BETTY';
```

```
=============================================================
|ID|OPERATOR                 |NAME          |EST.ROWS|EST.TIME(us)|
-------------------------------------------------------------
|0 |PX COORDINATOR           |              |1       |39          |
|1 | └EXCHANGE OUT DISTR     |:EX10000      |1       |38          |
|2 |   └PX PARTITION ITERATOR|              |1       |36          |
|3 |     └TABLE RANGE SCAN   |T1(i_s_name)  |1       |36          |
=============================================================
```

```
SELECT * FROM STUDENTS AS T1 USE INDEX(I_S_NAME) WHERE S_NAME='BETTY';
```

- The syntax of INDEX Hint supports both MySQL mode and Oracle mode.
- The MySQL mode also supports INDEX Hint in MySQL format:

2.FULL Hint: Specify access as either a table scan or a primary key

```
EXPLAIN SELECT /*+ FULL(T1)*/ * FROM STUDENTS AS T1 WHERE S_NAME='BETTY';
```

```
=============================================================
|ID|OPERATOR                 |NAME    |EST.ROWS|EST.TIME(us)|
-------------------------------------------------------------
|0 |PX COORDINATOR           |        |0       |96198       |
|1 | └EXCHANGE OUT DISTR     |:EX10000|0       |96198       |
|2 |   └PX PARTITION ITERATOR|        |0       |96198       |
|3 |     └TABLE FULL SCAN    |T1      |0       |96198       |
=============================================================
```

- In the OceanBase database, the primary key and table are two-in-one. Specifying the FULL hint can lead to a full table scan, or it might result in a primary key scan or the primary key unique match, depending on the query conditions. The specific execution details will be displayed in the detailed information of the EXPLAIN output.

In SQL statements, if a table name has an alias, i.e., table_name [AS] alias, the table alias must be used in order for the Hint to take effect.

Establishment of SQL Optimization System

R&D phase: Writing effective SQL

- In the R&D phase, formulating a clear SQL R&D specification can move the node of SQL issues to the R&D stage, solve low-level SQL errors in the code, and avoid potential SQL hazards.
- Schema design is a process of continuous design and optimization as service scenarios change. We must continuously **optimize the Schema design** based on the SQL load to ensure that SQL can maintain an optimal practice in line with service throughput.
- Conduct **end-to-end performance testing and benchmarking** to verify the system's behavior under load and stress.

Integration phase: Blocking risks

- Integration phase: When submitting SQL to the official runtime environment, you need to do SQL Review.
- Scanning SQL to ensure compliance with R&D specifications
- Identifying potential SQL risks in advance, with a focus on whether it can effectively utilize high-performance indexes, whether it requires Rewrite optimization, and whether it may lead to performance degradation

Operation and maintenance phase: Responding to challenges proactively

- During the operation and maintenance phase, SQL optimization is typically the most common scenario we deal with. Because SQL is initially deployed with a small amount of data and low concurrency, many SQL statements may not exhibit performance issues at the outset. As service volume grows and data accumulates, SQL queries may gradually become slower, or there may be a sudden performance drop due to changes in the execution plan. In the operation and maintenance stage, when the problematic SQL is identified, we need to:
  - **Exception diagnosis: Identifying Problematic SQL**
  - **Execution plan analysis:** Analyze the execution plan to find out the optimization method.
  - **SQL emergency response:** Conducting emergency measures for the root-cause SQL, for instance, creating appropriate indexes. Alternatively, in certain scenarios, the execution plan generated by the optimizer may not meet the user's requirements. In such cases, it is necessary to use hints to generate a specified execution plan.

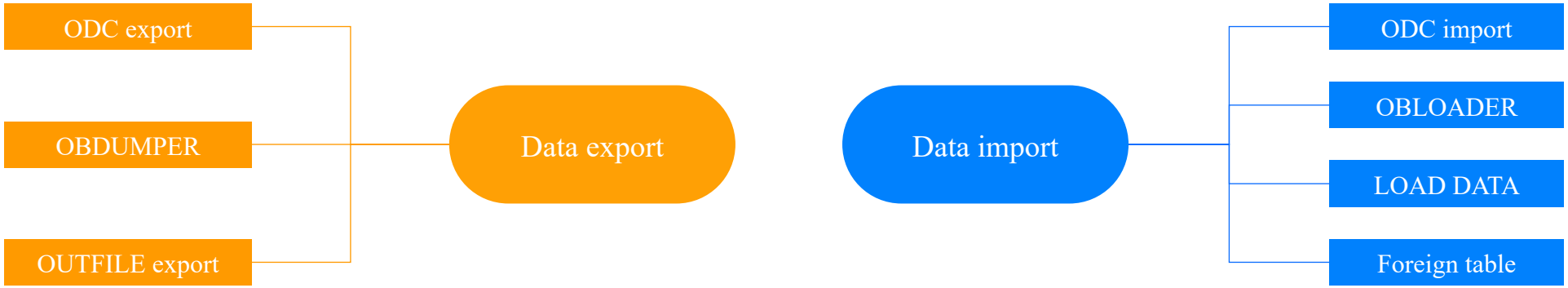
Application Development

05

- OceanBase Development Tools
- OceanBase Database Design
- OceanBase DML/DQL Statements
- OceanBase SQL Tuning
- **OceanBase Data Import/Export**

Overview of Data Import and Export

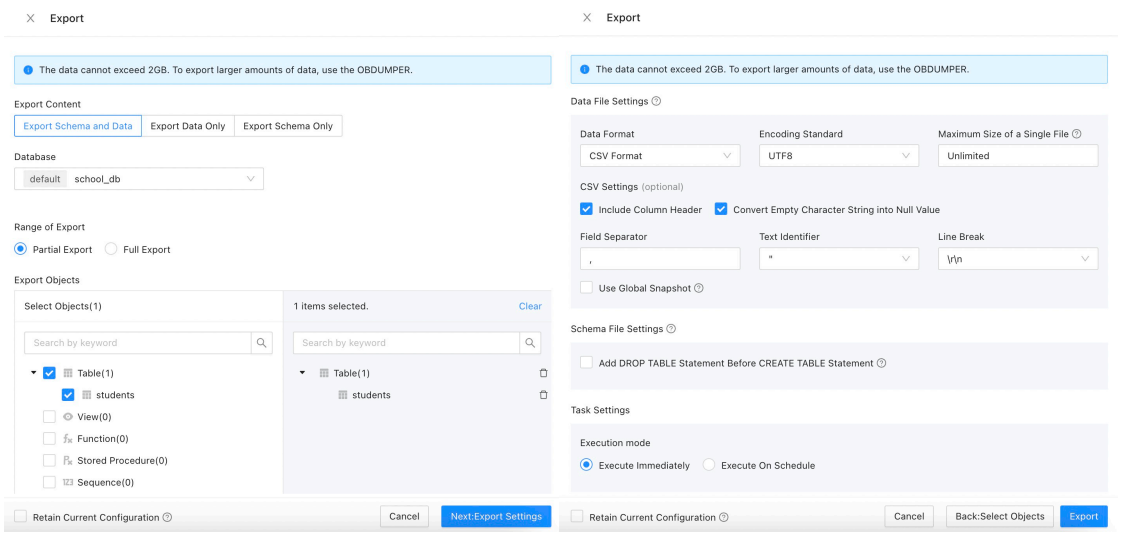
- In the daily development and maintenance process, data import and export is a common operation. OceanBase databases offer a variety of methods for data import and export, allowing users to conveniently back up, migrate, and share data, to meet diverse data management and interaction needs.



Data Export - ODC Data Export

ODC supports three export modes: Export the schema and data of database objects, export data only, and export schema only, as defined below:

- Export schema and data: Export both the definition statements of database objects and their data.
- Export data only: Export the data of table/view objects only.
- Export schema only: Export the definition statements of database objects only.



Considerations

- The Web version of ODC has limitations on the size of data that can be exported, supporting a maximum of 2GB of data (before compression), and any data exceeding this limit cannot be exported. If you need to export a large amount of data, use the derivative tool OBDUMPER.
- Support up to 5 export tasks running in parallel, with subsequent tasks waiting in a queue to run.
- The files involved in an export task are retained for 14 days by default.
- In ODC V4.1.0 and later versions, for OceanBase data sources, configuring the sys tenant account can improve the export speed. At the same time, it is no longer required to configure the sys tenant account when exporting objects other than tables and views.

Data Export - Introduction to OBDUMPER Tool

Introduction to the OBDUMPER tool:

OBDUMPER is a Java-based client tool that is specifically designed for use with OceanBase databases. Users can utilize this tool to export data from OceanBase into a file in either SQL or CSV format. Additionally, you can use this tool to export database objects defined within the database to a file. Compared with export tools such as MyDumper and SQLDeveloper, OBDUMPER has the following significant advantages in terms of functionality and performance:

- High performance: In scenarios such as tables without a primary key and partition tables, targeted performance optimizations have been provided.
- Feature-rich: Provide limited data export, data format diversity and globally consistent non-locking export and other capacities.

Core functions

- Support for exporting data definition language (DDL) of database objects
- Support for exporting data from tables to files in either CSV or SQL format
- For partition tables, you can specify the partition name to export data in some partitions.
- You can specify global filtering conditions to export only the data that meets the conditions.
- Support for configuring simple data processing rules.
- Support globally consistent non-locking read to ensure the global consistency of the exported data.
- Support SCN/TIMESTAMP flashback query to ensure global consistency of exported data.
- Support for exporting data through restricted mode, which does not rely on any account information under the system tenant.
- Support for using OBDUMPER to export data and upload it to Aliyun OSS

Data Export - OBDUMPER Environment Configuration

Runtime environment

Environment	Requirements
System version	Support for running on Linux and MacOSX systems. Personal PCs are for testing only. For exporting a large volume of data, it is recommended to use a high-specification machine.
Java environment	Install JDK 1.8+ and configure JAVA_HOME in the system environment variable.
Character sets	Please use UTF-8.
JVM parameters	You can adjust the JVM memory parameters in the script according to the actual amount of data.

Privileges

The following privileges are required to use OBDUMPER to connect to the OceanBase database to export data:

- The account connecting to the database needs to have execution privileges of CREATE, SELECT and other commands.
- When using OBDUMPER to connect to an OceanBase database Oracle tenant to export **DATABASE OBJECT DEFINITIONS**, it is recommended to connect to the database using an account with the Database Administrator (DBA) role; there is no role requirement for the account that connects to the database when you use OBDUMPER to connect to an OceanBase database Oracle tenant to **EXPORT DATA**.

Others

Foreign key definitions contain multiple columns and the order of the columns is not guaranteed when exporting. For example: FOREIGN KEY(c1,c2) REFERENCE (c1,c2)

When importing and exporting large volumes of data, please modify the memory parameters of the virtual machine in the running script (default): -Xms4G -Xmx4G）

When exporting data, it is advisable for users to initiate a merge first, which can enhance the performance of the export to a certain extent.

Data Export - OBDUMPER Data Export Syntax

```
[admin@localhost]>./obdumper -h <host IP> -P <port> -u <user> -p <password> --sys-password <user's password of sys tenant> -c <cluster> -t <tenant> -D <database name> [--ddl] [--csv|--sql|--cut]] [--all|--table 'table name'] -f   <data directory>
```

Parameters	Description
--ddl	Export DDL file, with the default file extension of -schema.sql
--sql	Export SQL file, with the default file extension of .sql.
--all	Exports all supported database object definitions and table data.
--sys-password	Specify the password for a user under the sys tenant. Null by default

Data Export - OBDUMPER Sample Demonstration

Cluster information

Database information	Sample value
Cluster name	Cluster A
OBProxy address	192.168.0.0
OBProxy port	2883
Tenant name	TenantA
Password for a user under the sys tenant (with at least read-only permissions)	Passroot123 (e.g. root@sys user's password)
Account of a user under the service tenant (with read/write permissions)	userA
Password for a user under a service tenant	Pass123
Schema name	USERA

Example

1. Export the definitions of all objects in Schema USERA to the /home/admin/DUMP-1/ directory.

```
[admin@localhost]> ./obdumper -h 192.168.0.0 -P 2883 -u userA -p Pass123 --sys-
password Passroot123 -c ClusterA -t tenantA -D USERA --ddl --all -f
/Users/admin/DUMP-1/
```

2. Export the data of all objects in Schema USERA to the /home/admin/DUMP-1/ directory in SQL format.

```
[admin@localhost]> ./obdumper -h 192.168.0.0 -P 2883 -u userA -p Pass123 --sys-
password Passroot123 -c ClusterA -t tenantA -D USERA --sql --all -f
/Users/admin/DUMP-1/
```

3. xport the definitions and data of all objects in Schema USERA to the /home/admin/DUMP-1/ directory in CSV format.

```
[admin@localhost]> ./obdumper -h 192.168.0.0 -P 2883 -u userA -p Pass123 --sys-
password Passroot123 -c ClusterA -t tenantA -D USERA --csv --ddl --all -f
/Users/admin/DUMP-1/
```

Data Export - Exporting Data with the OUTFILE Statement

The SELECT INTO OUTFILE statement allows for restrictions on the fields that need to be exported, which effectively meets scenarios where primary key fields are not required for export. Combined with the LOAD DATA INFILE statement, it is a convenient way to import and export data. Note: The client needs to be directly connected to the OceanBase database instance to do the import/export operation.

■ Syntax

```
select_stmt INTO
    OUTFILE string [ {CHARSET | CHARACTER SET} charset_name] [field_opt] [line_opt]

field_opt:
   {COLUMNS | FIELDS} field_term_list

line_opt:
    LINES line_term_list

field_term_list:
  field_term [, field_term ...]

field_term:
  {[OPTIONALLY] ENCLOSED | TERMINATED | ESCAPED} BY string

line_term_list:
  line_term [, line_term ...]

line_term:
  {STARTING | TERMINATED} BY string
```

TERMINATED: Set the terminator for the exported columns

OUTFILE: Store the query results in the specified file

Example

1. On the OBserver node, the tenant administrator connects to the MySQL tenant through a local Unix socket.

```
obclient -S /home/admin/oceanbase/run/sql.sock -uroot@ob_school -p******
```

2. Configure the path accessible by the database: /home/admin/

```
SET GLOBAL secure_file_priv = "/home/admin/";
```

3. After reconnecting to the database, use the SELECT INTO OUTFILE statement to export data in the Table students.

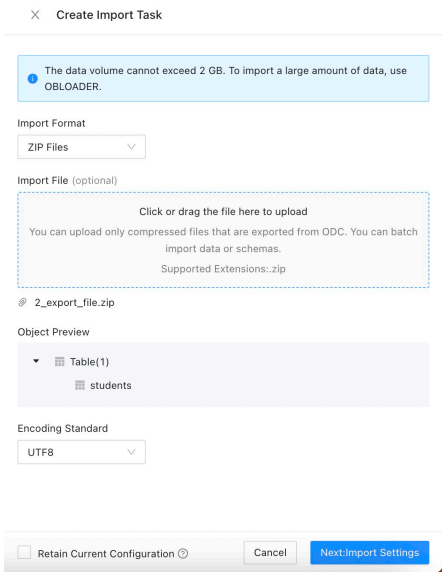
```
use school_db;
SELECT * INTO OUTFILE '/home/admin/students.sql'
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
FROM students;
```

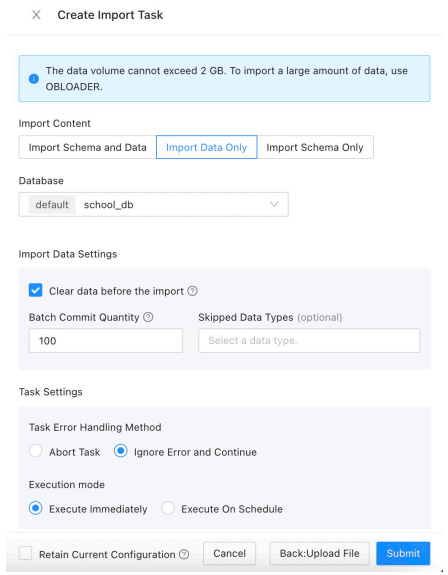
Data Import - ODC Import Data

ODC supports three import modes: Import schema and data of database objects, import data only, or import schema only. As defined below:

- Import schema and data: Import both the definition statements of database objects and their data.
- Import data only: Import the data of table/view objects only.
- Import schema only: Import the definition statements of database objects only

Currently, ODC supports batch importing of CSV files, SQL files, and ZIP compressed files.





Considerations

- The web version of ODC has a file size limitations for the uploaded files, supporting a maximum of 2GB for compressed files; files exceeding this size cannot be uploaded. If you need to import a large volume of data, use the derivative tool OBLOADER.
- Support up to 5 import tasks running in parallel, with subsequent tasks waiting in a queue to run.
- The files involved in an import task are retained for 14 days by default.
- In ODC V4.1.0 and later versions, for OceanBase data sources, configuring the sys tenant account can improve the import speed.

Data Import - Introduction to OBLOADER Import Tool

Introduction to the OBLOADER tool:

OBLOADER is a Java-based client tool that is specifically designed for use with OceanBase databases. OBLOADER tool provides very flexible command line options for importing structures and data into the OceanBase database in a variety of complex scenarios. OBLOADER is primarily used in conjunction with OBDUMPER. OBLOADER also supports SQL or CSV format files exported by tools such as Navicat, Mydumper and SQLDeveloper.

Key features of OBLOADER

- Support for importing Schema definition statements.
- Support for importing data files in CSV and SQL formats.
- Support for importing complex data formats such as byte offsets, string delimitation and mixed data files.
- Support for configuring simple data cleaning rules.
- Support field mapping relationships between profiles and tables.
- Support features such as flow limiting, anti-burst protection, breakpoint recovery and automatic retry
- Support for specifying a custom log directory, saving bad data, and handling conflict data
- Support for importing data through restricted mode, which does not rely on any account information under the system tenant.
- Support OBLOADER to import data from OSS to OceanBase database.

File formats supported by OBLOADER

- DDL file format: The file contains only DDL statements
- SQL file format: The file contains only INSERT statements, with records not spanning multiple lines
- MIX file format: The file contains a variety of mixed statements, including DDL and DML.
- POS file format: The data in the file is generated based on byte offsets, not character offsets.
- CUT file format: The data in the file is string-delimited, as opposed to CSV format where it is separated by individual characters.

Data Import - OBLOADER Import Tool Environment Configuration

Runtime environment

Environment	Requirements
System version	Support for running on Linux, MacOSX and Windows 7+ systems. Personal PCs are for testing only. For importing a large volume of data, it is recommended to use a high-specification machine.
Java environment	Install JDK 1.8+ and configure JAVA_HOME in the system environment variable.
Character sets	Please use UTF-8.
JVM parameters	You can adjust the JVM memory parameters in the script according to the actual amount of data.

Privileges

The following privileges are required to use OBLOADER to connect to the OceanBase database to import data:

- The account connecting to the database needs to have execution privileges of CREATE, SELECT, INSERT and UPDATE and other commands.
- When using OBLOADER to connect to an Oracle tenant to import database object definitions and data, it is recommended to connect to the database using an account of the Database Administrator (DBA).

Data Export - OBLOADER Data Import Syntax

```
[admin@localhost]>./obloader -h <host IP> -P <port> -u <user> -p <password> --sys-password <account password under sys tenant> -c <cluster> -t <tenant> -D <Schema database neme> [--ddl] [--csv|--sql] [--all|--table 'table name'] -f<data file or directory>
```

Parameters	Description
--ddl	Import DDL file, with the default file extension of -schema.sql
--sql	Import SQL file, with the default file extension of .sql
--all	Imports all supported database object definitions and table data
--sys-password	Specify the password for a user under the sys tenant. Null by default

Data Import - OBLOADER Sample Demonstration

Cluster information

Database information	Sample value
Cluster	Cluster A
OBProxy address	192.168.0.0
OBProxy port number	2883
Tenant name	TenantA
Password for a user under the sys tenant (with at least read-only permissions)	Passroot123 (e.g. root@sys user's password)
Account of a user under the service tenant (with read/write permissions)	userA
Password for a user under a service tenant	Pass123
Schema name	USERA

Example

1. Import all object definitions from the /home/admin/LOAD-1/ directory into Schema USERA.

```
[admin@localhost]> ./obloader -h 192.168.0.0 -P 2883 -u userA -p Pass123 --sys-password Passroot123 -c ClusterA -t tenantA -D USERA --ddl --all -f /Users/admin/LOAD-1/
```

2. Import CSV data files from the /home/admin/LOAD-1/ directory into Schema USERA.

```
[admin@localhost]> ./obloader -h 192.168.0.0 -P 2883 -u userA -p Pass123 --sys-password Passroot123 -c ClusterA -t tenantA -D USERA --csv --all -f /Users/admin/LOAD-1/
```

3. Import SQL data files from the /home/admin/LOAD-1/ directory into Schema USERA.

```
[admin@localhost]> ./obloader -h 192.168.0.0 -P 2883 -u userA -p Pass123 --sys-password Passroot123 -c ClusterA -t tenantA -D USERA --sql --all -f /Users/admin/LOAD-1/
```

Data Import - LOAD DATA

LOAD DATA statement supports loading data files on OSS and OBServer nodes.

LOAD DATA syntax

```
LOAD DATA
    [ [/*+ PARALLEL(N) load_batch_size(M) APPEND */]
      | [/*+ PARALLEL(N) direct(bool, int) */] ]
    [REMOTE_OSS] INFILE 'file_name'
    [REPLACE | IGNORE]
    INTO TABLE table_name
    [{FIELDS | COLUMNS}
        [TERMINATED BY 'string']
        [[OPTIONALLY] ENCLOSED BY 'char']
        [ESCAPED BY 'char']
    ]
    [LINES
        [STARTING BY 'string']
        [TERMINATED BY 'string']
    ]
    [IGNORE number {LINES | ROWS}]
    [(column_name_var
        [, column_name_var] ...) ]
```

Data import procedure

1.Parse files: OceanBase reads the data in the file based on the file name entered by the user, and decides whether to parse the data in the input file in parallel or serially based on the degree of parallelism input by the user.

2.Distribute the data: As OceanBase is a distributed database system, the data for various partitions may be distributed to different OBServers. The Load Data will calculate the data parsed out to determine which OBServer the data needs to be sent to.

3.Insert data: Upon receiving the data sent to it, the target OBServer performs a local INSERT operation to insert the data into the corresponding partition.

Note: For the versions after OceanBase 4.2.2, the features of the LOAD DATA have been enhanced to support the import of data from the client's local file system by specifying the local option.

Data Import - LOAD DATA General Import

Example

1. Log in to the machine where you want to connect to the OBServer node and create test data in the /home/admin directory.

```
[admin@observer01 /home/admin]# vi classes.csv
1,obca
2,obcp
3,obce
```

2. Set the system variable secure_file_priv to configure the path accessible for file import or export operations.

Note: For security reasons, if you set the system variable secure_file_priv, you can only connect to the database via a local socket to execute SQL statements that modify this global variable.

```
obclient -S /home/admin/oceanbase/run/sql.sock  -uroot@ob_school#ob_school -paaAA11__
obclient [(none)]> SET GLOBAL secure_file_priv = "/home/admin";
Query OK, 0 rows affected
```

3. Reconnect to the database and use the LOAD DATA statement

to

```
obclient -h172.20.178.58 -P2881 -uroot@ob_school -paaAA11__

obclient [school_db]> use school_db;
obclient [school_db]> LOAD DATA INFILE '/home/admin/classes.csv' INTO TABLE classes FIELDS TERMINATED BY ',';
Query OK, 3 rows affected
Records: 3  Deleted: 0  Skipped: 0  Warnings: 0

obclient [school_db]> select * from classes;
+-------+---------+
| CS_ID | CS_Name |
+-------+---------+
|     1 | obca    |
|     2 | obcp    |
|     3 | obce    |
+-------+---------+
3 rows in set (0.046 sec)
```

Data Import - LOAD DATA Bypass Import

OceanBase databases also support the bypass import method for inserting data into the database, which means that OceanBase databases allow for direct data writing into data files. Bypass import can circumvent the SQL layer interface, and directly allocate space and insert data within the data files, thereby enhancing the data import efficiency.

■ **Syntax:**

```
LOAD DATA /*+ direct(need_sort,max_error) parallel(N) */ INFILE
'file_name' ...
```

Parameters	Description
direct	Indicate the bypass import.
need_sort	Indicate whether the OceanBase database is required to sort the data. The value is of type bool: true: Indicate that sorting is required false： Indicate that sorting is not required
max_error	Indicate the maximum number of rows that can tolerate errors. The value is of type INT. If the value is exceeded, LOAD DATA will report failure.
parallel(N)	The parallelism of loading data, N is 4 by default.

Example:

```
obclient [school_db]> use school_db;
obclient [school_db]> LOAD DATA /*+ direct(true,1024)
parallel(16) */INFILE '/home/admin/classes.csv' INTO TABLE
classes FIELDS TERMINATED BY ',';
Query OK, 3 rows affected
Records: 3  Deleted: 0  Skipped: 0  Warnings: 0

obclient [school_db]> select * from classes;
+-------+---------+
| CS_ID | CS_Name |
+-------+---------+
|     1 | obca    |
|     2 | obcp    |
|     3 | obce    |
+-------+---------+
3 rows in set (0.046 sec)
```

Data Import - Foreign Table Import

Foreign table is a logical table object in a database that points to data stored in external storage service rather than within the database itself. A foreign table is created by the CREATE EXTERNAL TABLE statement. When creating a foreign table, it is necessary to specify the data file path and format to facilitate to read data from the external files. Here is a case example to demonstrate the foreign table import:

Example:

The current user has the CREATE privilege. Assume that a data.csv file is stored under /home/admin/ on the local machine, and the contents of the data.csv file are as follows:

1,"lin",98
2,"hei",90
3,"ali",95

1. On the OBserver node, the tenant administrator connects to the MySQL tenant through a local Unix socket.

```
obclient -S /home/admin/oceanbase/run/sql.sock -uroot@sys -p********
```

2. Configure the path accessible by the database: /home/admin/

```
SET GLOBAL secure_file_priv = "/home/admin/";
```

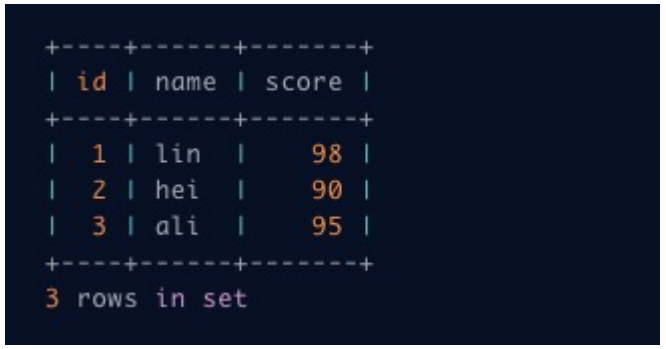
3. After reconnecting to the database, create the foreign table ext_t3.

```
CREATE EXTERNAL TABLE ext_t3(id int, name char(10),score int)
LOCATION = '/home/admin/'
FORMAT = (
TYPE = 'CSV'
FIELD_DELIMITER = ','
FIELD_OPTIONALLY_ENCLOSED_BY ='"'
)
PATTERN = 'data.csv';
```

Data Import - Foreign Table Import

Query foreign table

```
SELECT * FROM ext_t3;
```



```
+----+------+-------+
| id | name | score |
+----+------+-------+
|  1 | lin  |    98 |
|  2 | hei  |    90 |
|  3 | ali  |    95 |
+----+------+-------+
3 rows in set
```

Precautions for using external tables

- Foreign table can only be used for querying data, and do not support DML.
- When querying a foreign table, if the external file accessed by the foreign table has been deleted, the system does not report an error and returns to an empty row.
- Since the files accessed by the foreign table are managed by the external storage system, when the external storage system is not available, querying the foreign table will result in an error.
- Due to the data of foreign tables being stored in external data sources, querying them may involve factors such as cross-network and file systems, which could potentially impact query performance. Therefore, you need to choose the appropriate data source and optimization strategy when creating foreign tables to improve query efficiency.

Summary

- Supported client tools when connecting to OceanBase database tenants include: MySQL client, OBClient client, and ODC developer center. Supported drivers include: Java Driver, C Driver (OceanBase Connector/C) and Python Driver (PyMySQL).
- When developing with the OceanBase database, it is necessary to conduct a detailed planning and design of various database objects such as tenants, databases, tables, partitions, table groups, indexes (primary keys, unique indexes, secondary indexes, local indexes, global indexes), views, sequences, and stored procedures.
- OceanBase database provides a rich set of features to meet the data read and write requirements of applications. For example, query (single-table, multi-table join, partition), insert, update, delete and so on.
- When addressing performance issues in OceanBase database, you can approach the problem from three main aspects: Identifying slow SQL, execution plan analysis, and SQL tuning (optimizing indexes or using Hint, etc.).
- OceanBase database provides a variety of ways for data import and export. Data import methods include: ODC import, OBLOADER, LOAD DATA, and foreign table. Data export methods include: ODC export, OBDUMPER and OUTFILE.

Self-test

1. [True/false] MySQL client (mysql) can connect to both MySQL and Oracle tenants ( )

2. [True/false] OceanBase can support both MySQL and Oracle tenants in a cluster. ( )

3. [True/false] When using the Explain command to view the SQL execution plan, the SQL will be actually executed as well. ( )

4. [True/false] If you want to use a local unique index to enforce data uniqueness, the local unique index must include the table partition key. ( )

5. [Single answer] When connecting to an Oracle tenant using JDBC, which JDBC driver is required? ( )

   A. MySQL's standard JDBC driver B. Oracle's standard JDBC driver C. JDBC driver developed by OceanBase itself

6. [Single answer] For the optimization of slow queries, the incorrect description is _____.

   A. Locate slow queries through GV$OB_SQL_AUDIT

   B. The elapsed_time, execute_time and queue_time in GV$OB_SQL_AUDIT represent the total elapsed time, execution time and queuing time respectively.

   C. EXPLAIN commands shows the real plan ever executed by the SQL.

Self-test

7. [Multiple answers] What types of partitions does the OceanBase database support? ( )

A. Range partition B. List partition C. Hash partition

8. [Single answer] The correct statement regarding the SHARDING = PARTITION attribute of a table group is:

A. All partitions of all tables are aggregated on the same machine and there is no restriction on the type of partition of tables within the table group.

B. All tables have the same partition definition. If it is a subpartitioned table, the partition definition, rather than the subpartition definition, is verified.

C. All tables are either all partition tables or all subpartition tables. If they are partitioned tables, they must have the same partition definition. If they are subpartitioned tables, they must have the same partition definition and subpartition definition.

9. [Multiple answers] The import tools provided by OceanBase database include: ( )

A. OBLOADER B. ODC import C. LOAD DATA D. OBDUMPER

Contents

I. Product Overview

II. Cluster Architecture

III. Database Engines

IV. Installation and Deployment

V. Application Development

VI. Operations and Maintenance Management

Operations and Maintenance Management

06

- **Introduction to O&M System and Tools**
- Cluster Monitoring and Management
- Cluster and Tenant Scaling
- Disaster Recovery and High Availability Management
- Real-time Diagnosis and SQL Monitoring
- Alerting and Troubleshooting

Overview of Operations and Maintenance Management System

The daily operations and maintenance task of database administrator is to ensure the stable, safe and efficient operation of database system. The daily operations and maintenance task of OceanBase database administrator mainly includes the following:

Design planning stage

Architecture design of database

- **Architecture design:** Design the logical structure and high availability deployment scheme according to the service requirements and the high availability requirements, including primary/standby disaster recovery and resource isolation, etc.

Capacity planning and management

- **Capacity planning:** Predict the database capacity requirements according to the development trend of services, and reasonably allocate and adjust the hardware resources.
- **Capacity management:** Manage the storage space, number of connections and transaction capability to prevent resource bottlenecks.

Installation and deployment stage

Database installation and configuration

- **Database installation and configuration** Including the establishment of OceanBase database clusters, OBProxy clusters, OCP management tools, and data migration tools.
- **Parameter adjustment:** Adjust the parameters related to database clusters, OBProxy clusters and tenants according to the operating condition of database clusters to ensure the efficient operation of database.
- **Version upgrade:** Upgrade the software versions and patches according to the operating condition of database clusters to ensure system security and stability.

Development and design stage

Database design

- Sharding and partitioning strategy: Ensure high concurrency and throughput of services by planning tenant isolation, database sharding, table sharding, and partitioning according to the performance requirements and the architecture design considerations.
- **Index design:** Design reasonable indexes, including global index, local index, unique index and primary key, etc., to ensure the processing performance of services as requested by partitions and services.
- **Data distribution and aggregation design:** Reasonably design primary zone and table group according to the service forms, take into account both load balancing and data concentration to achieve the purpose of performance and high availability.

Operation stage

Performance monitoring and tuning

- Performance monitoring: Monitor the status of database systems through OCP and performance views, including the key indicators of such objects as hosts, clusters, sessions and SQL statements.
- **Performance tuning:** Analyze the cluster or tenant resource bottlenecks, number of sessions, transaction efficiency, SQL efficiency and other information, and carry out corresponding performance optimization.

Troubleshooting and emergency response

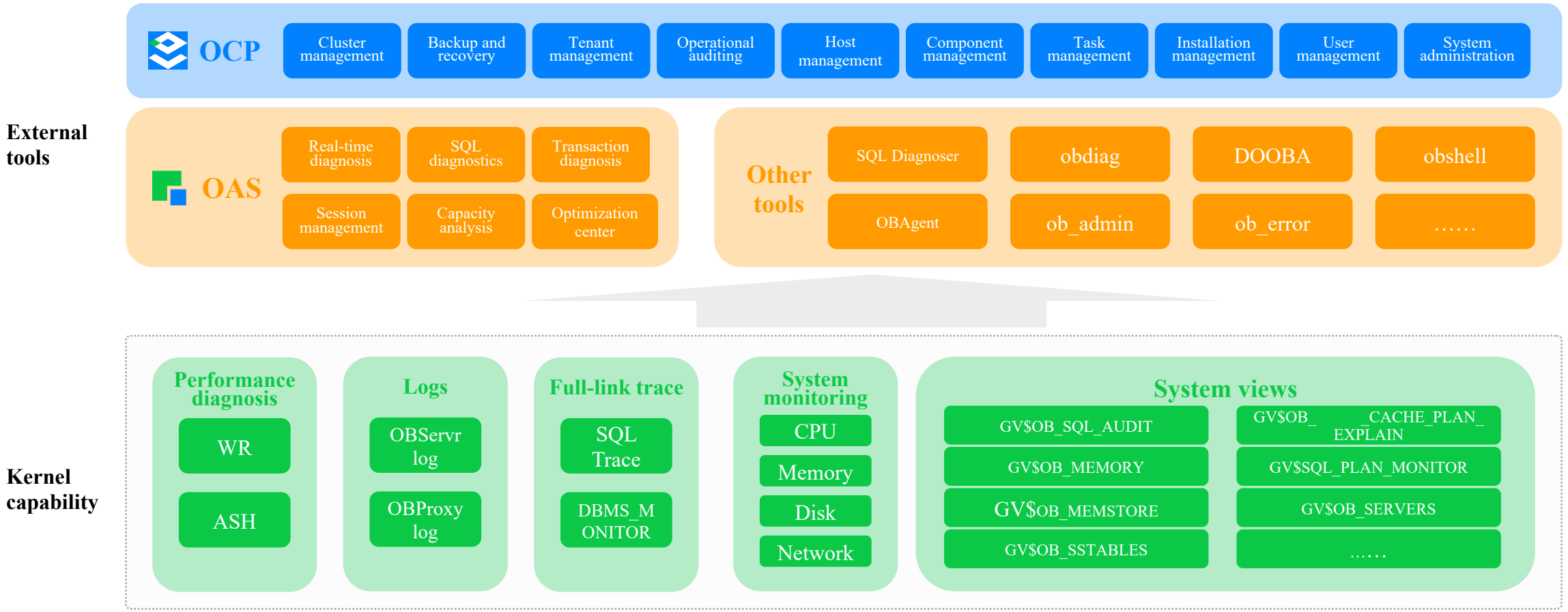
- **Fault diagnosis:** Find out the fault cause and develop an emergency plan by viewing the monitoring data of the database clusters and logs of related components.
- **Emergency response:** Perform resource isolation, primary/standby switchover, fault node isolation, reboot recovery, data recovery and other measures to ensure the overall availability of database.

Backup and recovery management

- **Backup strategy:** Develop a backup plan, configure backup media, and regularly complete the full or incremental backup of database tenants.
- **Periodic random inspection:** Carry out periodic random inspection on recovered backup data to ensure that the data can be rapidly recovered when they are lost or damaged.

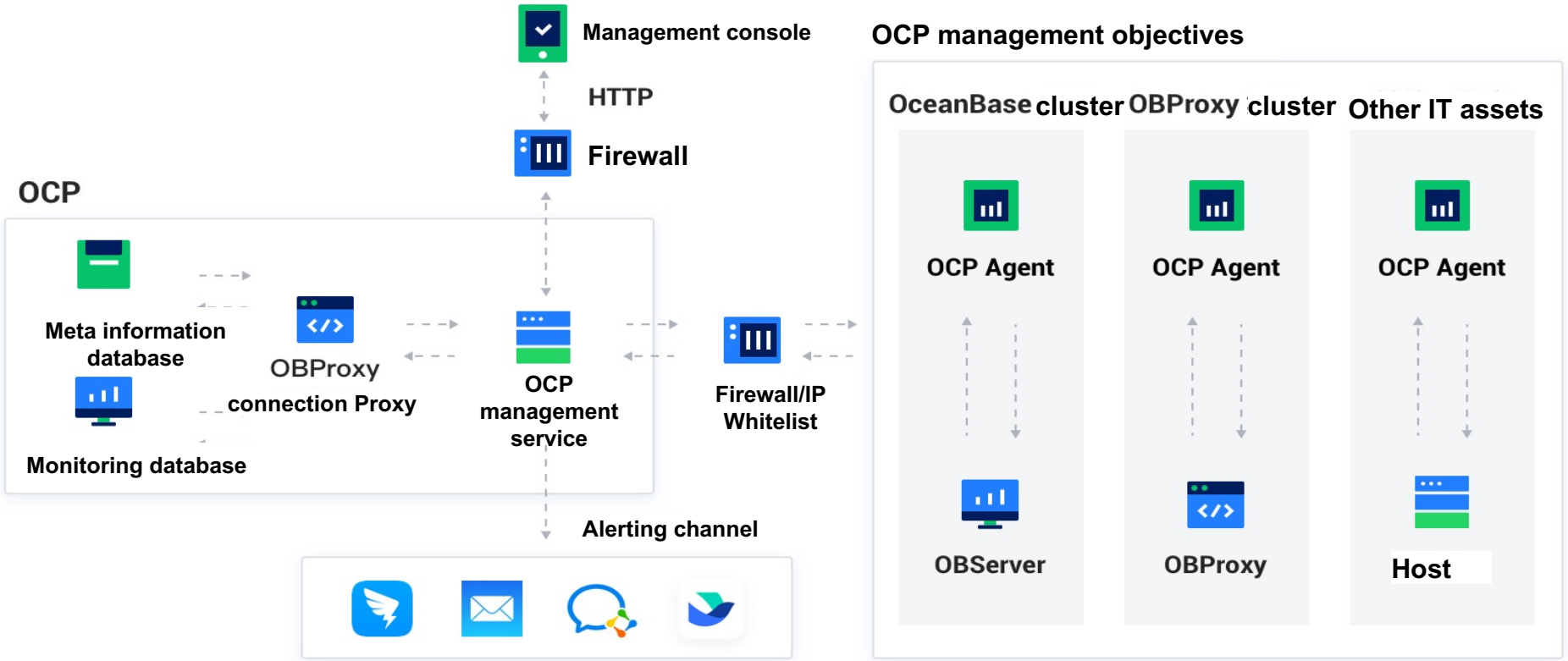
Overview of Operations and Maintenance Management Capability

Currently, the OceanBase database has offered rich O&M tools, including OCP for full life cycle management, OAS for SQL autonomy management, built-in views, and full-link analysis tools for easy problem diagnosis, etc.



OCP System Architecture

The OceanBase cloud platform includes six modules, namely Management Agent, Management Service, Metadata Repository, Monitor Repository, Management Console and OBProxy (OceanBase-specific reverse proxy).



OAS Diagnostic Tools

OceanBase Autonomy Service (OAS) is a comprehensive, intelligent diagnostic, and autonomic service designed for developers, operators, and DBAs. It offers users visual monitoring, performance optimization, troubleshooting, security management, and capacity management, facilitating the use of the OceanBase database in a simpler, and cost-effective and higher-performance manner. OAS is integrated in OCP as a built-in function module.

■ **SQL diagnostics**

SQL diagnosis shows the current overall situation of SQL execution in the cluster, including SQL statistics, performance monitoring, tenant-level request analysis, etc.

■ **Transaction diagnosis**

Large transactions in the database hold some resources for a long time, or occupy excessive system resources, so that the system is in an unstable state and cannot provide efficient services normally. OAS can view the details of SQL execution in a transaction.

■ **Session management**

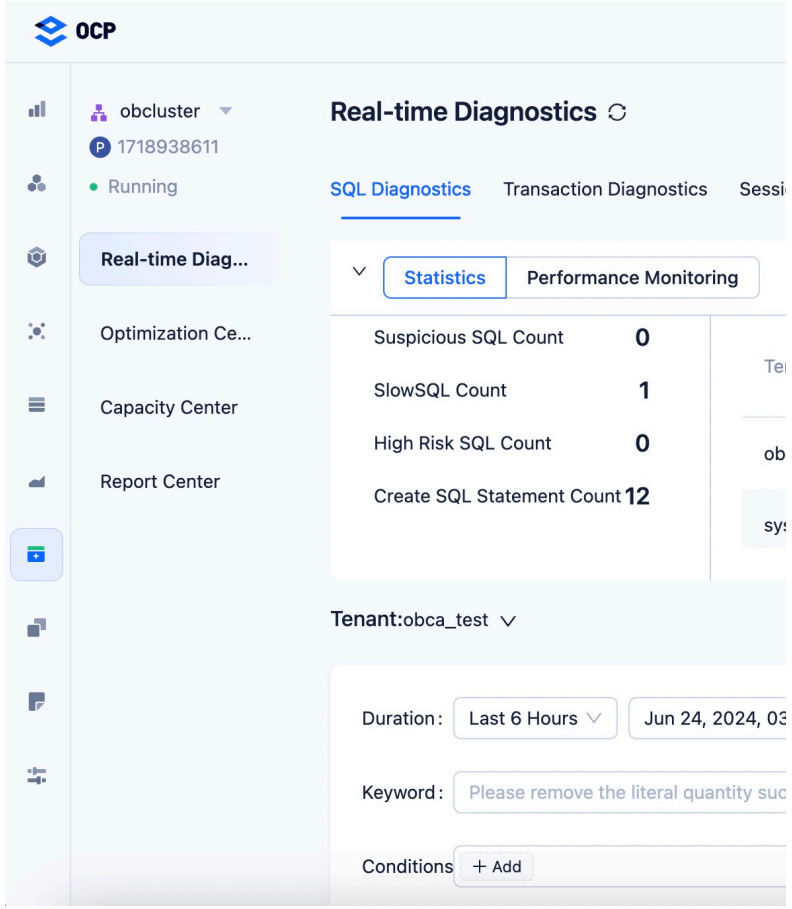
Include the session statistics and session details based on tenants or users, view the SQL text of a session, close the session or perform the full-link diagnosis of the session.

■ **Capacity management**

View the capacity overview and capacity risk of a cluster or tenant as well as the current and historical trends of CPU, memory, disk and number of connections of a tenant.

■ **Report management**

Include the historical report of active sessions (SQL that occupies more database resources within the historical diagnostic time, historical sessions and other information) and management performance report (cluster load, performance, SQL and other information within the specified period of time).



Report Analysis Tools

The OceanBase database offers two tools, namely WR (workload repository) and ASH (active session history), to summarize and sort out the system view data and assist in diagnosing the operating condition of database.

WR (Workload Repository)

WR (workload repository) can periodically collect the view data related to the performance of OceanBase Database. After a cluster is successfully deployed, WR will automatically collect snapshots on an hourly basis.

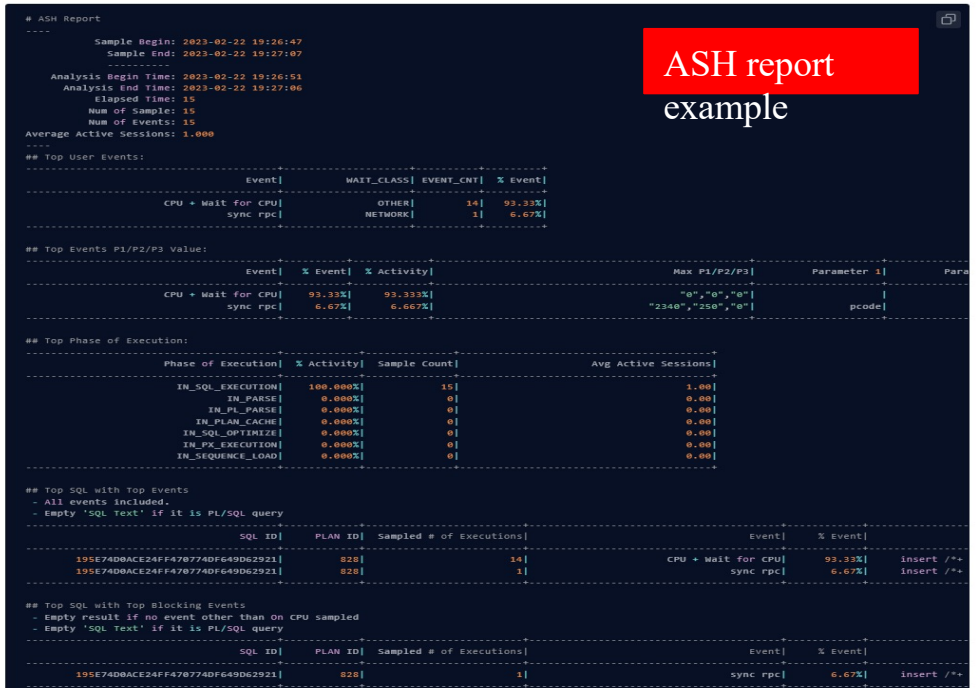
- **Data content:** Sampling information for SQL execution, including SQL text, wait event, pending execution state, waiting time, etc.; and database statistics, including memory, cache, RPC, lock, I/O and transaction, etc.
- **Data viewing:** Obtain the data by reading the WR view, includingWR_STATNAME, WR_SYSSTAT, WR_ACTIVE_SESSION_HISTORY, so as to achieve the analysis of TOP SQL and the viewing of statistical information.

ASH (Active Session History)

ASH (Active Session History) records the information about all active sessions in the database, and can provide the analysis reports for locating the transient anomalies.

- **Report generation:** Call the stored procedure dbms_workload_repository.ash_report to generate a report.

```
obclient [SYS]>call dbms_workload_repository.ash_report(
  BTIME => 'xxxx',     //sampling start time
  ETIME => 'xxxx',     //sampling end time
  SQL_ID => 'xxxx',  //[optional] SQL_ID of SQL to be sampled
  TRACE_ID => 'xxxx', //[optional] TRACE_ID of SQL to be sampled
  WAIT_CLASS => 'xxxx', //[optional] event type to be sampled
  REPORT_TYPE => 'TEXT', //[optional] REPORT_TYPE field only supports
the set text type
  );
```



Full-link Trace (FLT)

FLT can trace the related information about the execution of user's SQL requests on the client, OBProxy and OBServer components, and display such information to the user in a visual manner so as to help the user rapidly locate the problems.

Function introduction

- **Overview:** FLT is to record the related information such as components, modules, activities and SQL on the data access path via the trace logs, and to provide the ability to trace the precise location of each transaction or SQL execution time and other related information along the entire call link. The trace log file of OBProxy is named as obproxy_trace.log, while that of OBServer is named as trace.log.
- **Trace path:** The client supports the trace of obclient and JDBC components; OBProxy supports the trace of interactions with OBServer, self-processing time and overall processing time; and OBServer supports the trace of detailed SQL execution stages such as access path, execution plan generation, SQL compiling, and parallel task.

Operation mode

- **Enable or disable Trace**

1. The DBMS_MONITOR stored procedure is used to enable or disable FLT respectively according to the granularity of tenant, session, client, module or activity.
2. OceanBase provides SQL command to display the full-link trace of the last SQL execution at the interface of command line.

- **Result viewing**

1. OCP link query interface
2. View the last Trace result through the SQL command line.

System Views

System views are categorized into Performance Views and Dictionary Views.

- **Dictionary view:** It refers to a virtual table in the database, which is used for displaying the metadata information and system status information in the database, including table, column, index, user, role and permission, etc. The dictionary view is usually stored in a system tablespace, which is read-only and cannot be modified or deleted.
- **Performance view:** It refers to a virtual table built according to the database memory structure, which is used for displaying the performance statistics and monitoring information of database. As it is mainly related to performance, it is also called the dynamic performance view.

	Dictionary view	Performance view
SYS tenant	It refers to a virtual table which is used for displaying the metadata information and system status information in the database. You can query these views to obtain the structure and status information in the database. The dictionary view under the SYS tenant includes oceanbase.CDB_* view and oceanbase.DBA_* view.	It refers to a virtual table which is used for displaying the performance statistics and monitoring information in the database. You can query these views to obtain the performance status and monitoring information in the database. The name of performance view starts with GV$* and V$*, and all eligible V$ view information will be returned after you query the GV$ view.
MySQL tenant	The dictionary view under the MySQL tenant includes the information_schema.xxx view and the mysql.xxx view. The former provides database metadata for MySQL tenants, while the latter contains data dictionary tables that store database object metadata and system tables for other operational purposes.	The name of performance view under the MySQL tenant starts with GV$* and V$*, and each V$ view almost has corresponding GV$. The former is used for displaying the operating condition and performance indicators of the entire cluster, while the latter is used for displaying the operating condition and performance indicators of the node to which the client is connected.
Oracle tenant	The dictionary view under the Oracle tenant includes the views that start with DBA_, ALL_ and USER_. The view that starts with DBA_ displays all related information in the whole database, the view that starts with ALL_ is used for the overall overview of database for users, and the view that starts with USER_ is most commonly used by common database users. The sys user can access all views. However, authorization is required for common users to access DBA_ and ALL_ views.	The performance view under the Oracle tenant includes SYS.V$* view and SYS.GV$* that are placed under SYS schema.

View example

GV$OB_SQL_AUDIT displays the source, execution state and other statistical information of each SQL request on all OBServer nodes. This view is split by tenant. Except for system tenants, the other tenants cannot query across tenants.

The GV$OB_SQL_AUDIT view includes the server IP and port, trace_id, session connection ID, client IP and port, SQL ID, actual SQL sentence, execution plan ID, execution result, transaction ID, Hash value of execution plan, trace_id of full-link diagnosis and other details about SQL execution.

Log System

OBServer log

- **OBServer system log:** The system log is automatically printed during the operation of the OceanBase Database, which is used for monitoring alarm and diagnosis and is an effective complement to monitoring indices.
- **Log file:** The file is divided into startup operation log observer.log, election module log election.log and rootservice.log.
- **Log level:** It is classified into ERROR, WARN, INFO, EDIAG, WDIAG, TRACE and DEBUG.
- **Log format:** It includes time stamp, log level, module (such as Common, RPC, SQL), function, thread ID, and trace_id, etc.

OBServer log example

```
[2022-04-29 16:39:55.186527] WARN [COMMON] get_file_id_range (ob_log_file_group.cpp:127)
[103594] [0]     [Y0-0000000000000000-0-0] [lt=17] [dc=0] max file does not
exist(max_file_id=4, b_exist=false)

------------------------------------------

time: [2022-04-29 16:39:55.186527]
log_level: WARN
module: [COMMON]
function:get_file_id_range
file_name:line_number: (ob_log_file_group.cpp:127)
thread_id: [103594]
coroutine_id: [0]
trace_id: [Y0-0000000000000000-0-0]
log_used_time: [lt=17]
dropped_msg_count: [dc=0]
info:max file does not exist
parameter: (max_file_id=4, b_exist=false)
```

OBProxy log

- **Log file:**

File name	Description
obproxy_diagnosis.log	A diagnostic log recording Proxy login, startup, disconnection and route error, etc.
obproxy_digest.log	Record the information about SQL whose execution time exceeds the configuration item query_digest_time_threshold and about SQL whose execution fails.
obproxy_error.log	Record the information about SQL whose execution fails.
obproxy_slow.log	A log recording slow SQL request information.
obproxy_stat.log	A statistical log recording the SQL execution within a period of time.
obproxy_trace.log	A log recording the full-link diagnostic information.
obproxy.log	A log recording the Proxy details with the most comprehensive content.

- **Log level:** Consistent with the OBServer system log.

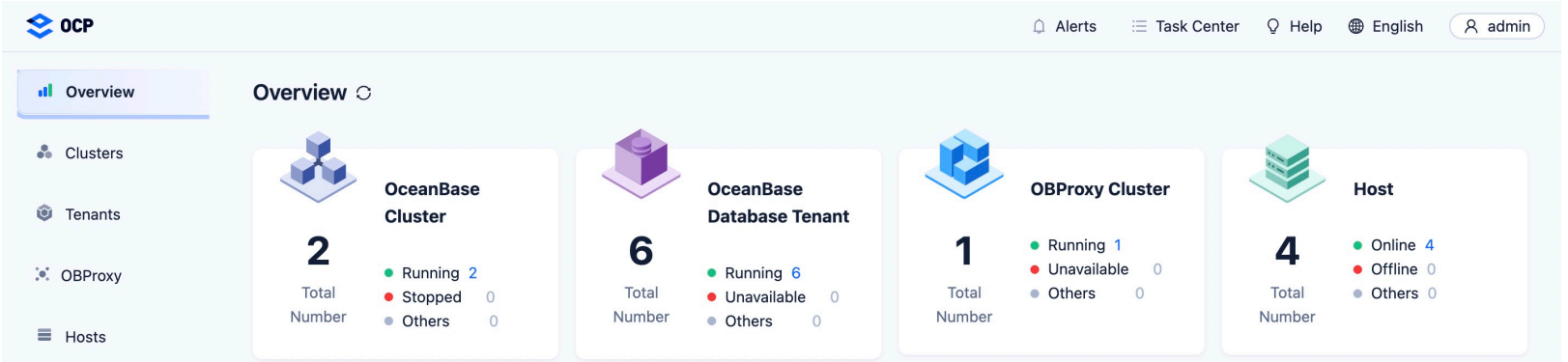
Operations and Maintenance Management

06

- Introduction to O&M System and Tools
- **Cluster Monitoring and Management**
- Cluster and Tenant Scaling
- Disaster Recovery and High Availability Management
- Real-time Diagnosis and SQL Monitoring
- Alerting and Troubleshooting

Management of OceanBase Clusters by OCP

OCP can manage the OBProxy clusters, OceanBase clusters, and hosts involved in the OceanBase database throughout their life cycle.



Cluster management by OCP

- **Cluster management:** Take over a cluster or migrate out of a cluster; create, delete, and expand primary and standby clusters; restart or stop a cluster; and support cluster monitoring, resource management, parameter configuration, and version upgrade.
- **Zone management:** Add, restart, stop, delete, etc.
- **OBServer management:** Add, restart, stop, replace, reinstall, delete a node; and stop the OBServer process, etc.

OBProxy management by OCP

- **Cluster management:** Create or delete a cluster; take over or migrate out of a cluster; connect an OB cluster, delete the connected OB cluster, stop, start and restart a cluster, etc.
- **OBProxy management:** Add, take over, delete, restart, stop, start, go offline, go online, upgrade, refresh configuration, etc.

Host management by OCP

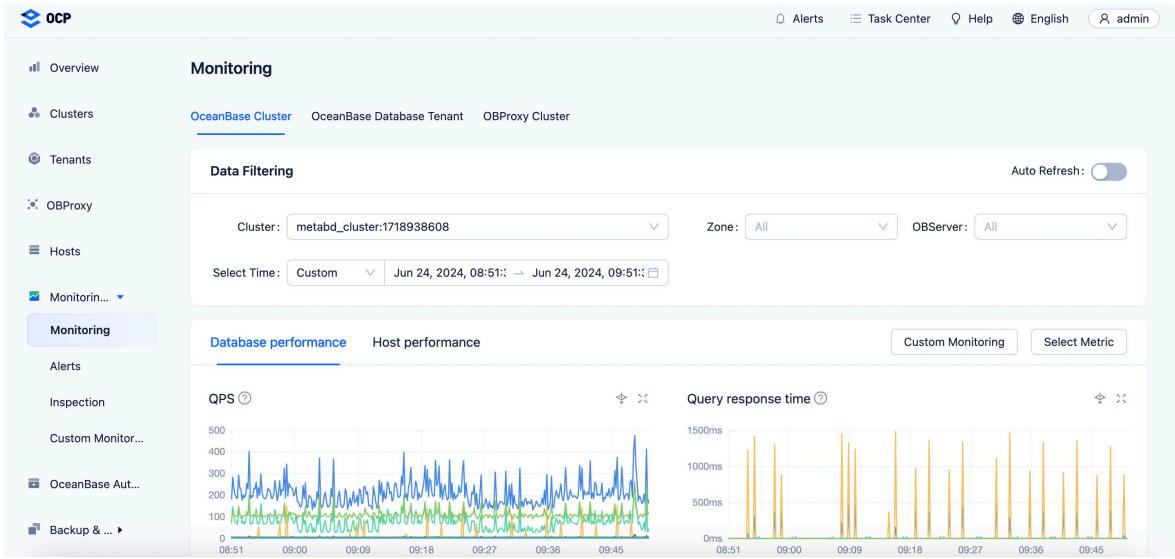
- **Host management:** Add, modify, delete hosts
- **Host details:** Include host status, host name, host alias, SSH port, disk usage, host model, data center, model, host type, subordinated service, clock offset, current time, time zone, operating system, CPU architecture, CPU model, number of CPU cores, label, total memory, and description; disk usage (capacity of disk partitions used by each path, used size, available size and utilization rate of the disk, and mount point).
- Agent management by OCP: The ocp-agent process includes ocp_agentd, ocp_mgragent, ocp_monagent; Agent can be restarted and reinstalled by OCP.

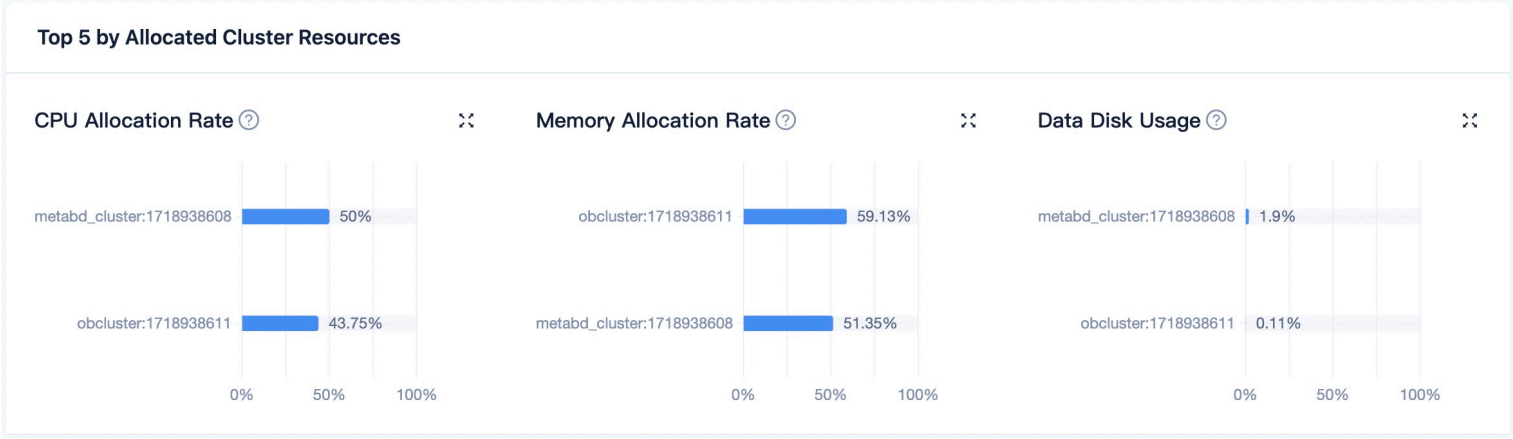
Monitoring of OceanBase Clusters by OCP

OCP can view the cluster topology and tenant topology. You can monitor the key indicators and performance indicators of database clusters, tenants, OBProxy clusters, hosts, SQL, resource allocation, and backup merging tasks.

Monitoring

- **Topology viewing:** The cluster topology includes zone, arbitration service, operating condition and operation list of hosts and objects; and the tenant topology is similar to the cluster topology, and the tenant relationship topology displays the information about the standby tenant associated with the primary tenant and log synchronization.
- **Performance monitoring:** Enter the performance monitoring page respectively from the database cluster, OBProxy cluster and tenant, and select the object range and time range to be monitored by cluster, zone, OBServer and time window to view indicators such as SQL, transaction, session, and storage, and respectively carry out the drill-down analysis of cluster, tenant and OBServer dimensions as the circumstances may require.
- **Resource monitoring:** Include host process, CPU, memory, load, file handle, number of threads and resource usage, etc.
- **Capacity monitoring:** Include CPU, memory, disk allocation, table group, schema, and data size, etc.

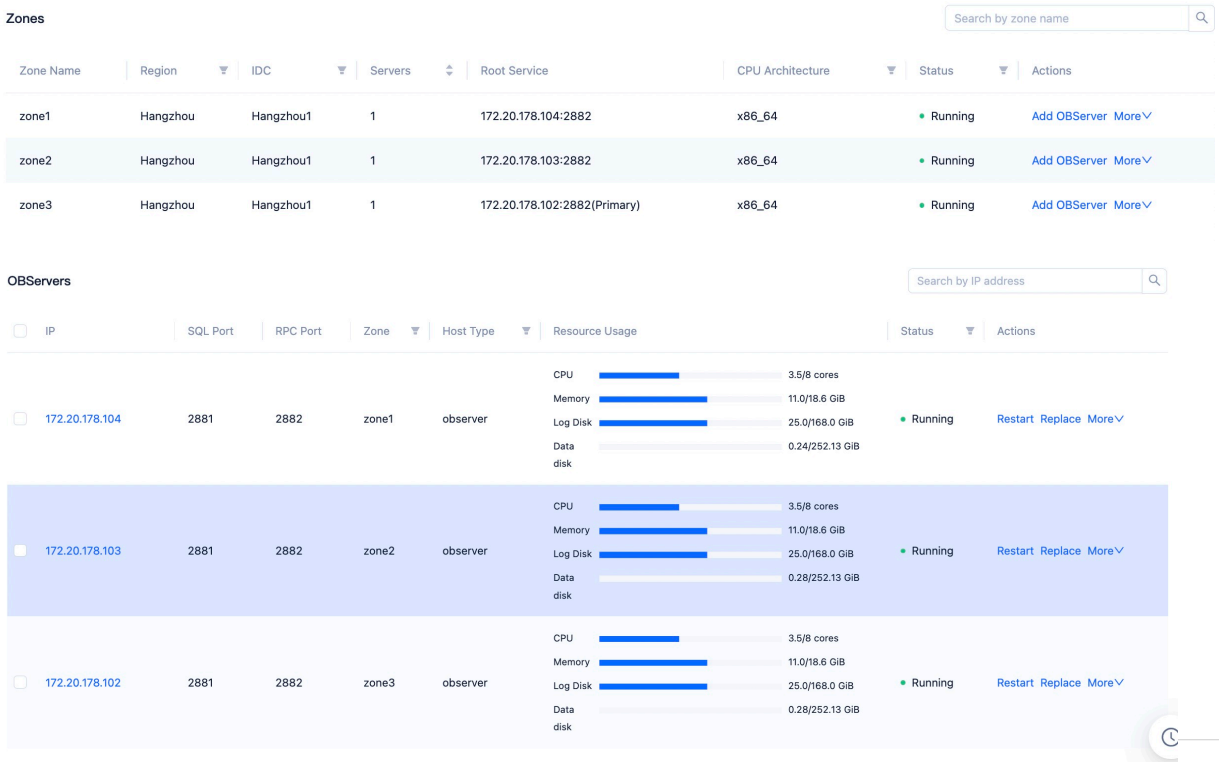


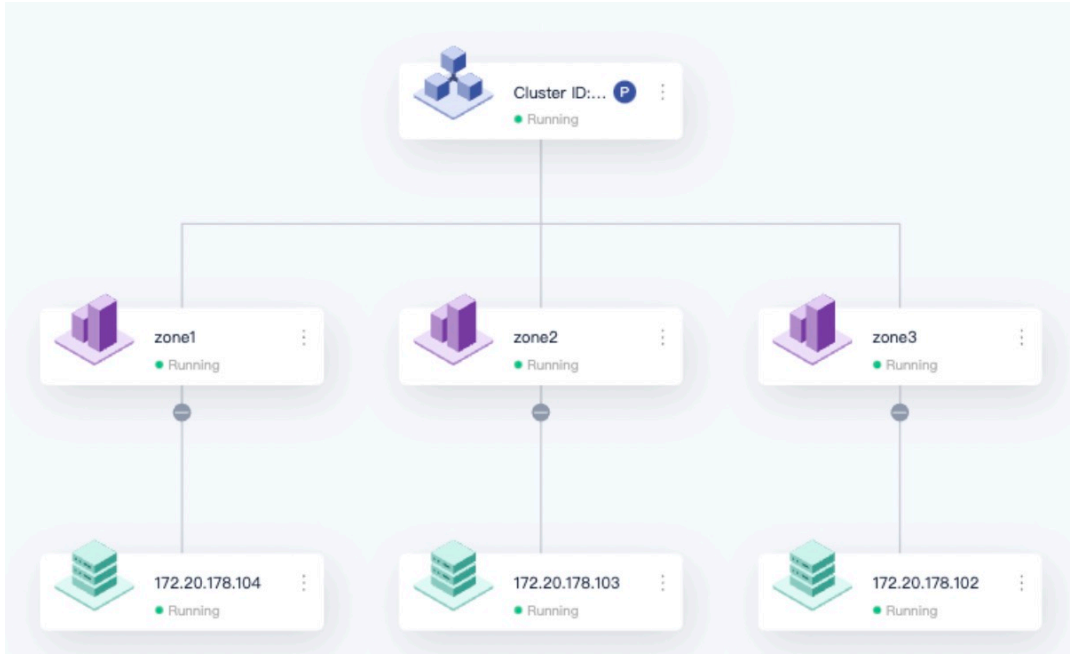


View Cluster Resources

View the cluster list by OCP, and view the associated configuration and operating condition of host, OBServer, Zone and OBProxy contained in the specified cluster.

- Cluster overview: After logging into the OCP cloud platform, click **Cluster** on the left navigation bar to enter the Cluster page. The cluster overview information includes: performance indicator Top5, cluster merging time Top5, machine resources, resource utilization rate and **cluster list**.
- Enter the details page of the specified cluster via the cluster name in the cluster list to view the basic information about this cluster, resource utilization rate, alarm, event, zone list, OBServer list, and associated **OBProxy cluster**, etc.
- In the details page of the specified cluster, click **Topology** on the left navigation bar to view the topology of this cluster.

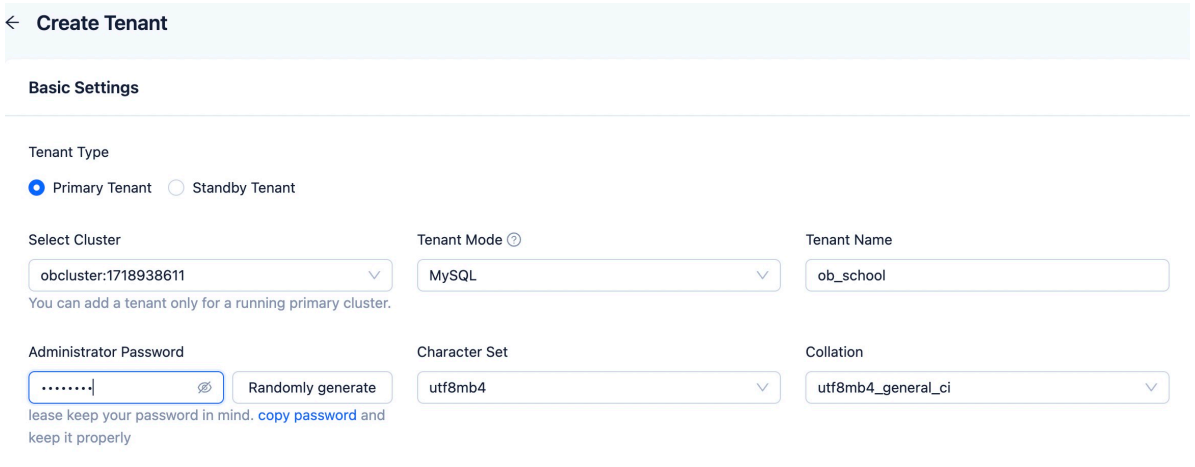


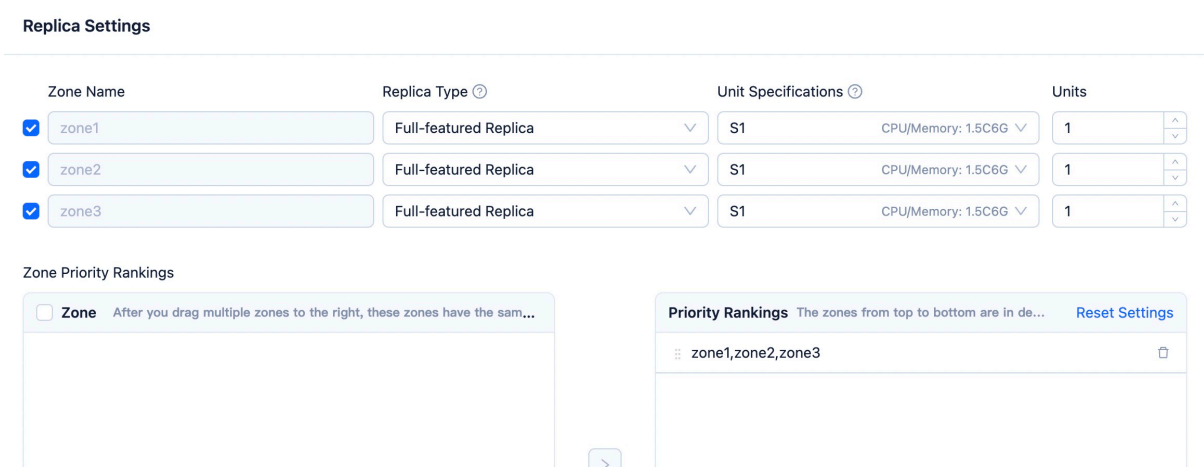


Create a Tenant by OCP

Tenant is not only a container for various database objects but also a container for resources (CPU, Memory and IO, etc.). Users can create tenants by OCP.

- After logging into OCP, click **Tenant** on the left navigation bar to enter the **Tenant** page, and click **Create a Tenant**.
- Select and fill in the basic information, mainly including cluster, tenant mode, tenant name, administrator password, character set and other information.
- Fill in the other settings, such as replica type, unit specification and unit quantity, and set the zone priority.

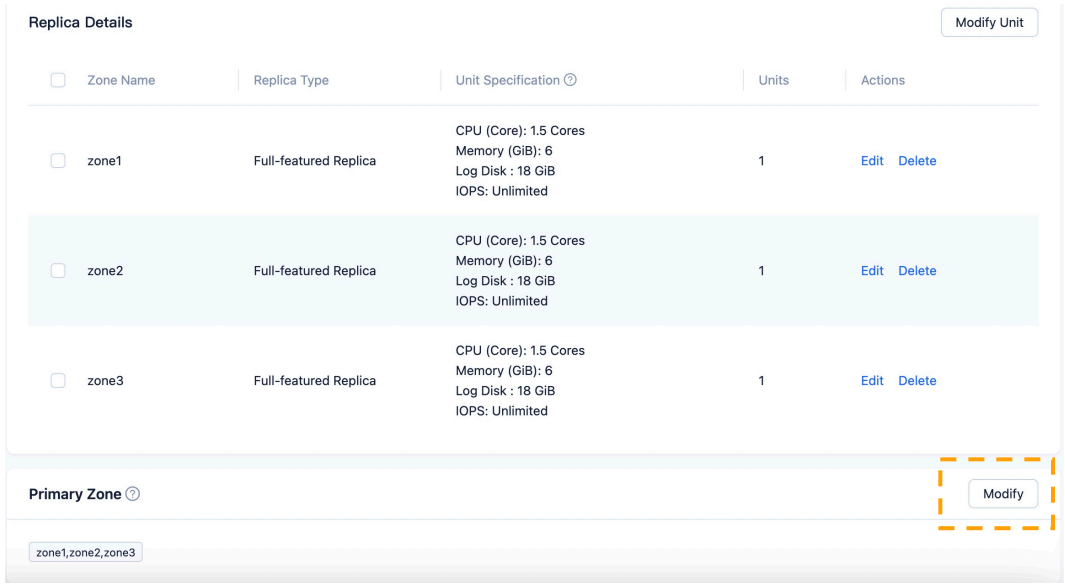


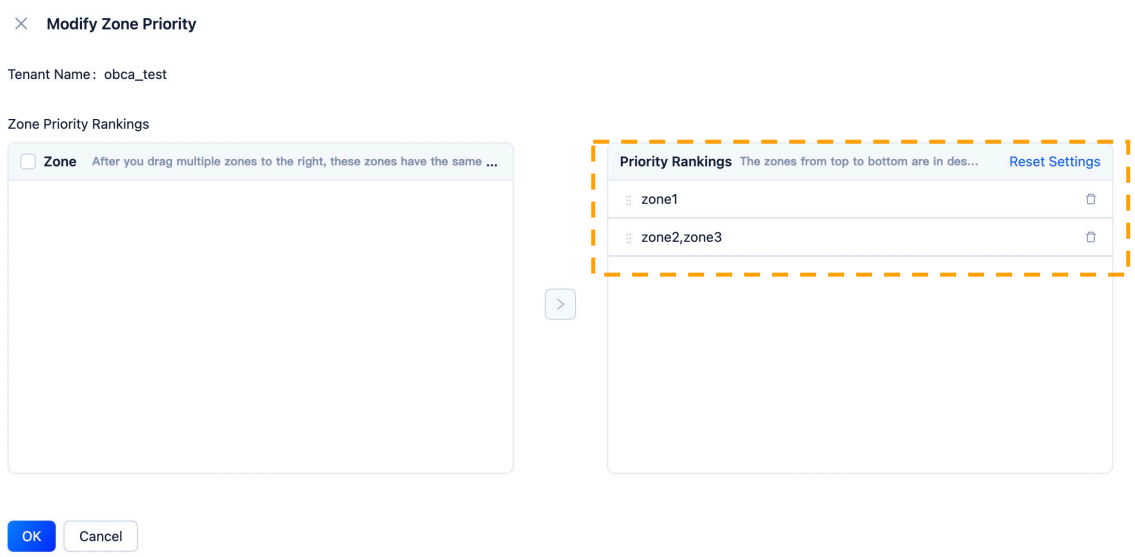


Set Primary Zone for Tenants

The Primary Zone of tenants determines the distribution of Leader replicas of the log streams among tenants. The adjustment of primary zone can achieve the effects of tenant resource adjustment and scaling.

- After logging into OCP, click **Tenant** on the left navigation bar to enter the **Tenant** page, select a tenant to be modified in the tenant list and enter the Tenant Overview page.
- In the Tenant Overview page, select Modify in the **Zone Priority** area, and select Zone priority ranking in the page **Modify Zone Priority**.



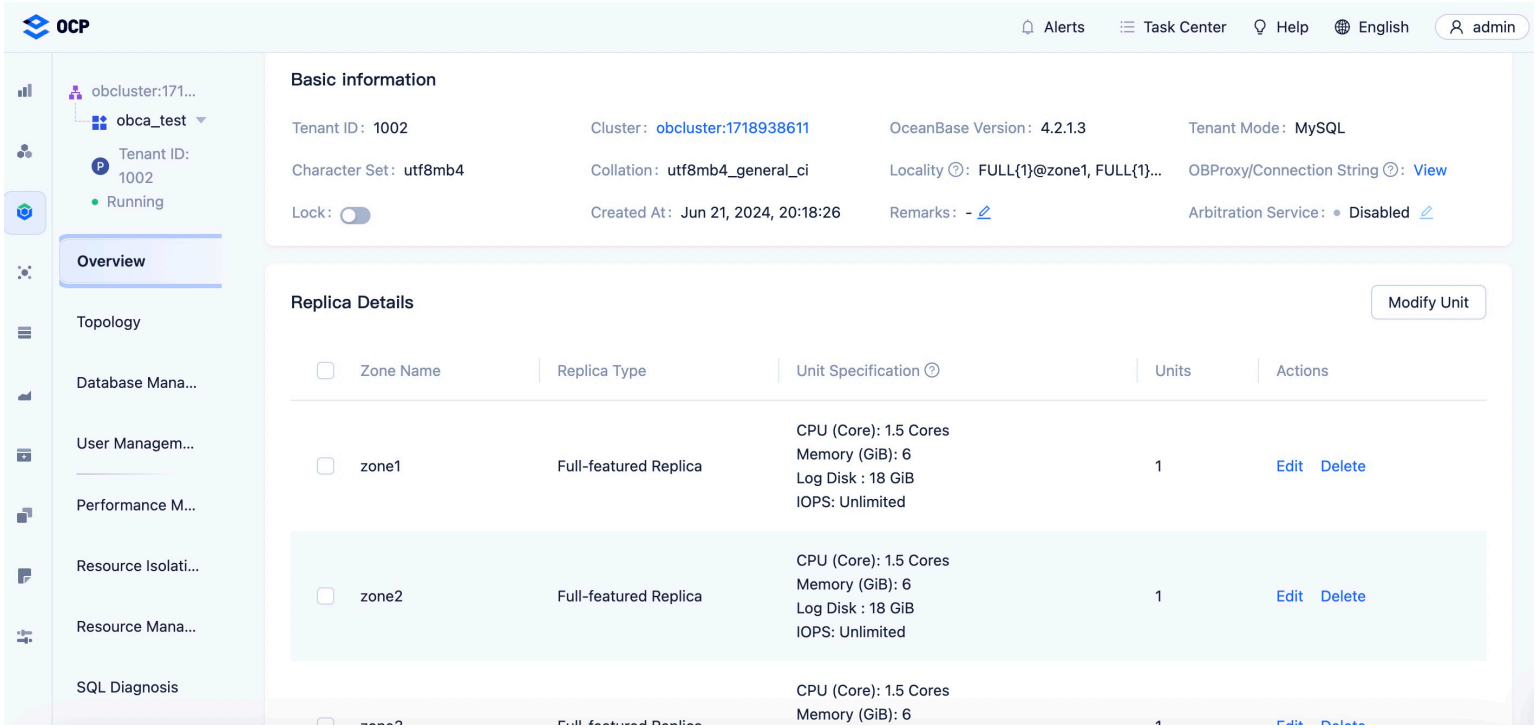


Management of OceanBase Tenants and Resources by OCP

OCP can manage the OceanBase tenants throughout the life cycle, involving resource specification, tenant management, database management, resource isolation, and other functions.

Management of Tenants and Resources

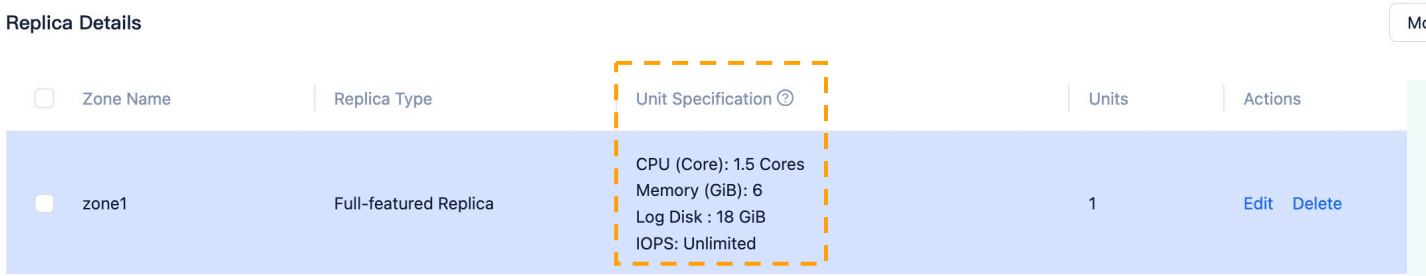
- **Management of resource specification:** OCP has some default resource specifications. You can create, modify, or delete the resource specification.
- **Tenant management:** Create, replicate, lock or delete a tenant, add a replica, modify the white list, Zone Priority and configuration of tenant parameters.
- **Database management:** This function applies only to MySQL mode tenants and supports the operations such as create, modify and delete.
- **User management:** The MySQL mode supports the functions such as create a new user, delete a user, grant user permission, cancel user permission; and the Oracle mode supports the management of users and roles.
- **Resource isolation:** Create, modify, and delete a resource group; and create, modify, delete, enable, and disable a resource isolation plan.

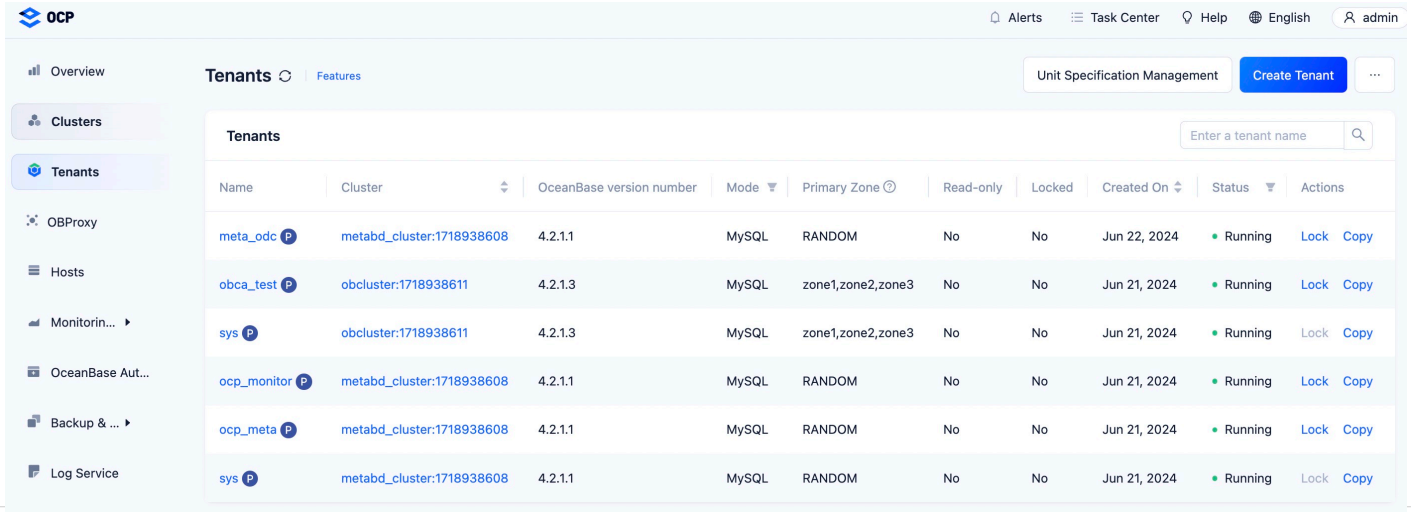


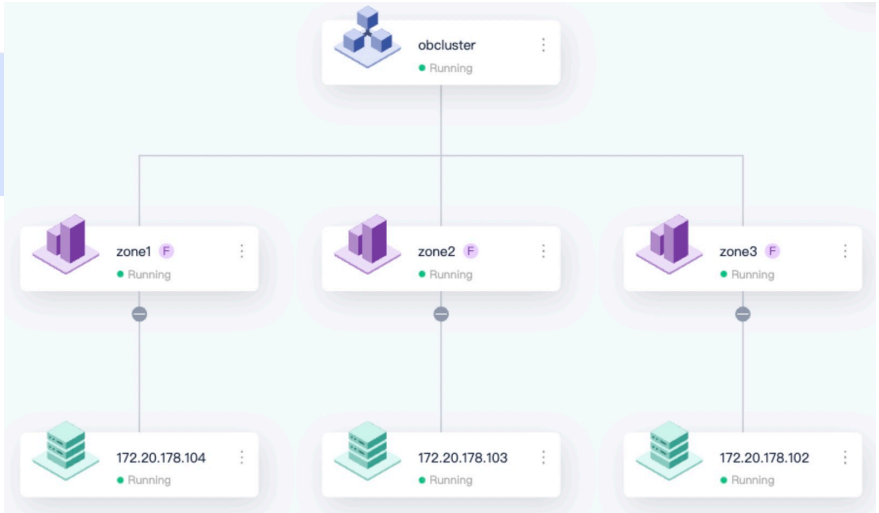
View the Operating State of Tenants

Through OCP, you can view the information about the cluster in which a tenant is located, tenant mode, and tenant resources.

- Tenant overview: After logging into OCP, click **Tenant** on the left navigation bar to enter the **Tenant** page, and view the information about **Tenant List** and **Top 5 Tenants**.
- The tenant list includes tenant name, subordinated cluster, OceanBase version number, tenant mode, Zone Priority, read-only, locked, status, creation time, and operation, etc.
- Click Tenant Name to enter the Tenant Overview page, including the basic information of a tenant, zone name, replica type, unit specification, unit quantity, zone priority, white list, etc.
- In the Tenant Overview page, click **Topology** on the left navigation bar to view the tenant topology and the tenant relationship topology.



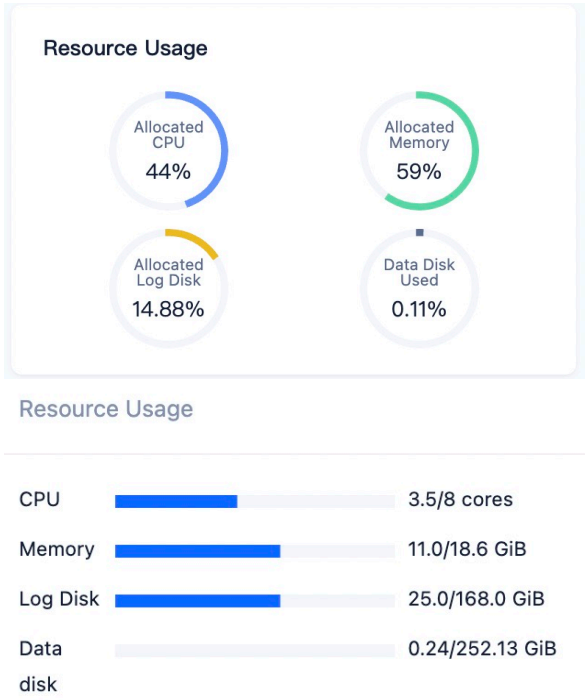


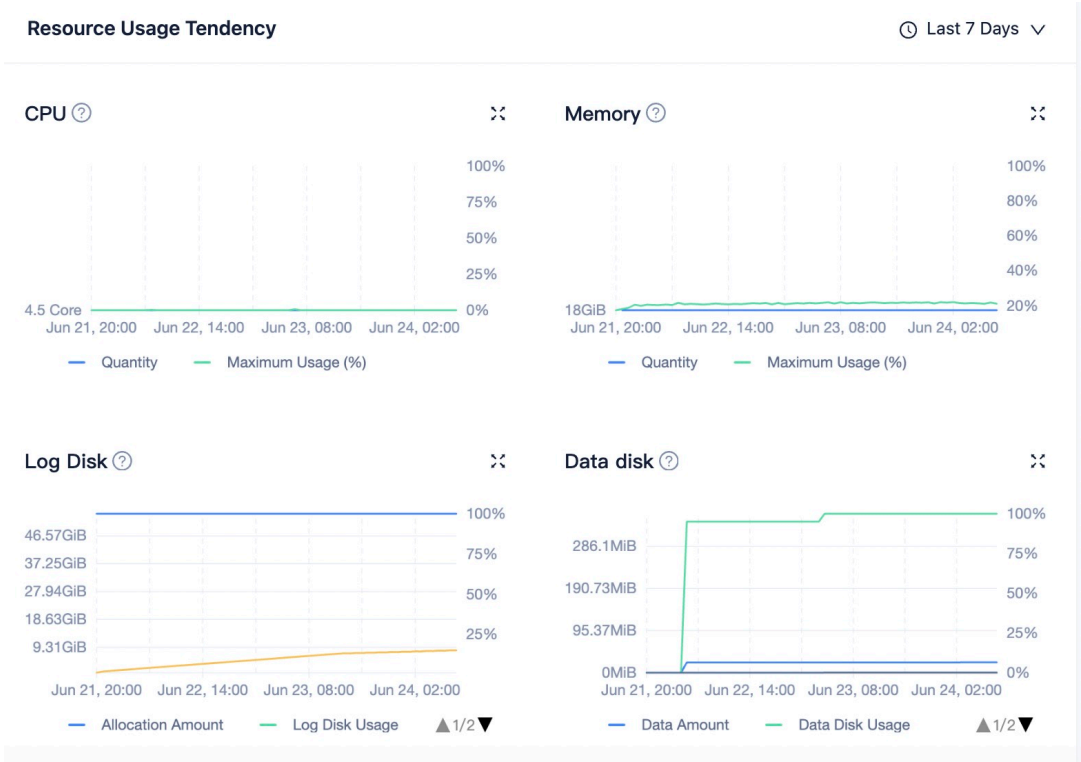


Tenant Capacity Monitoring

Through OCP, you can view the resource allocation of a cluster and the resource usage of a tenant, and diagnose whether scaling of the cluster or tenant is required.

- **Resource usage of clusters:** After logging into OCP, click Cluster on the left navigation bar to enter the Cluster page. In the Cluster List area, select a cluster to be operated and click its cluster name to enter the Cluster Overview page. In the Resource Usage section, you can view the resource usage about the current cluster; In the OBServer node list, you can view the resource usage of each OBServer. Click **Resource Management** on the left, the **Unit Distribution** page contains information about resource units, and the **Resource Usage** page contains trend charts of each kind of resources.
- **Resource usage of tenants:** After logging into OCP, click Tenant on the left navigation bar to enter the Tenant page. In the Tenant List area, select a tenant to be operated and click its tenant name to enter the Tenant Overview page. Click **Resource Management** on the left to view the resource usage trends of the tenant and determine if there are capacity issues.

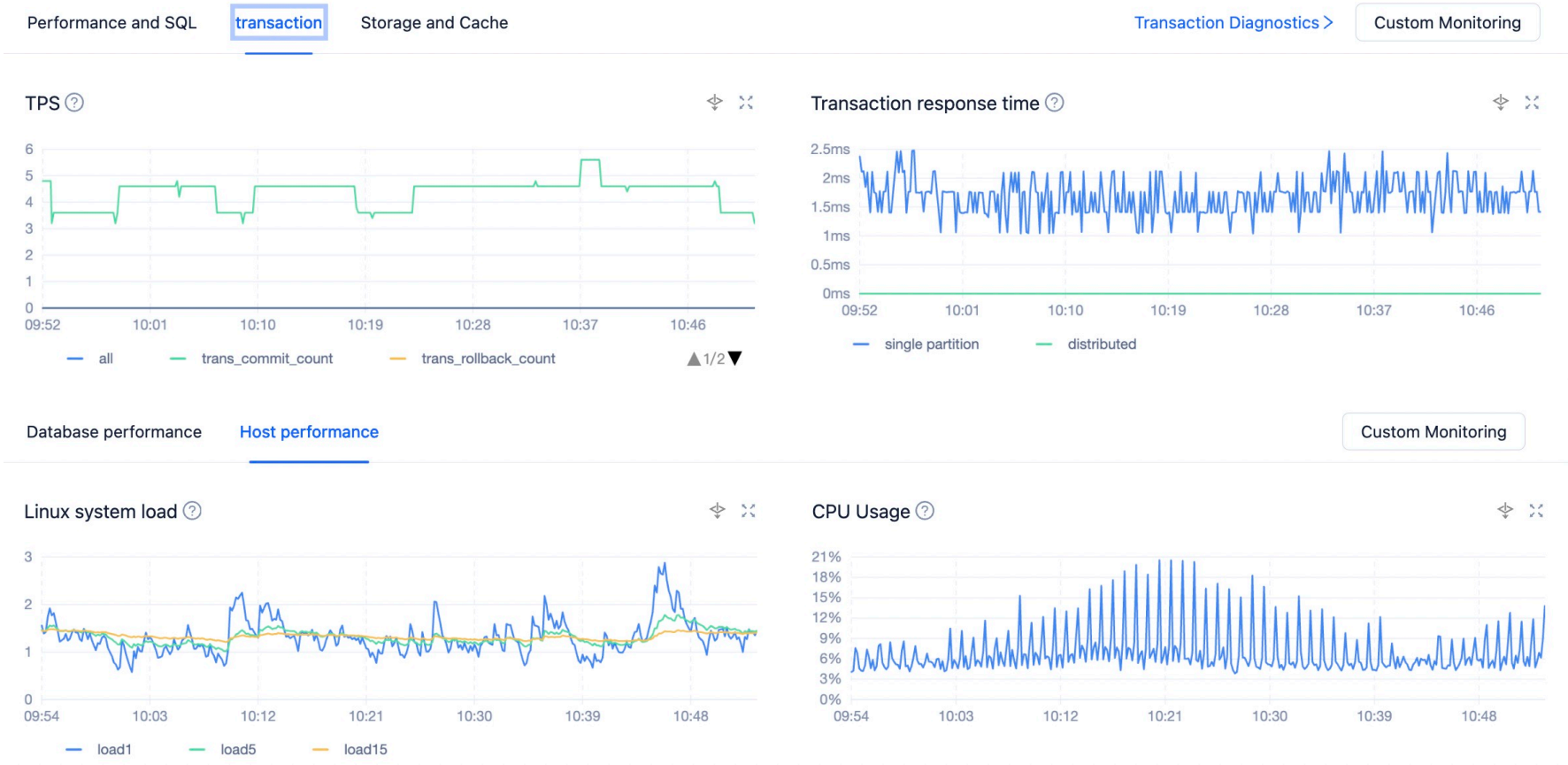




Tenant Performance Monitoring

You can use the tenant performance monitoring overview to analyze the overall operating state of a specific tenant, find possible suspects, and drill down to analyze the existing performance bottlenecks.

- In the Tenant Overview page, click Performance Monitoring on the left navigation bar to enter the Tenant Performance Monitoring page and view the label page containing performance and SQL, transaction, storage and caching. Focus on QPS, TPS, SQL and transaction response time, check whether there is volatility, and further analyze possible performance bottlenecks in combination with CPU load of the host and usage rate through drill-down function analysis.



Overview of Parameters and System Variables

The OceanBase Database offers parameters and system variables to adjust the system's default behavior.

Configuration Items (PARAMETER)

- **Cluster-level parameters:** They refer to the parameters applicable to the entire OceanBase Database cluster. They are global in nature and are used to configure basic information, performance parameters, security options, etc. more for the entire cluster. These parameters usually include data backup and recovery, load balancing and so on. The cluster-level parameters are usually configured when the cluster is started and cannot be easily modified after configuration.
- **Tenant-level parameters:** They refer to the parameters applicable to the tenant level (single tenant or multiple tenants). They are used for the specific configuration and optimization of single tenant or multiple tenants. These configuration items typically include options for storage engine parameters, SQL execution policies and access control. The tenant-level parameters are usually configured when a tenant is created and managed, and can be modified at any time if required.

System variables (VARIABLE)

- **Global variable:** Indicates Global-level settings, where global variables are shared among different users within the same tenant of the database. The modifications of global variables persist even after the session is terminated. In addition, the modification of global variables does not take effect on the currently opened session, and you need to re-create the session to take effect.
- **Session variable:** Indicates the session-level settings. When the client connects to the database, the database will replicate the global variables to automatically generate session variables. The modification of session variable only takes effect on the current session.

Difference

Scope of effect: Configuration items are categorized into cluster, zone, machine, and tenant; system variables are categorized into Global-level or Session-level for tenants.

Mode of effect: Configuration items are divided into two types: dynamic and restart-required; Session-level system variables are effective only for the current session; and Global-level system variables are effective for the newly connected session.

Parameters and Variables Management by OCP

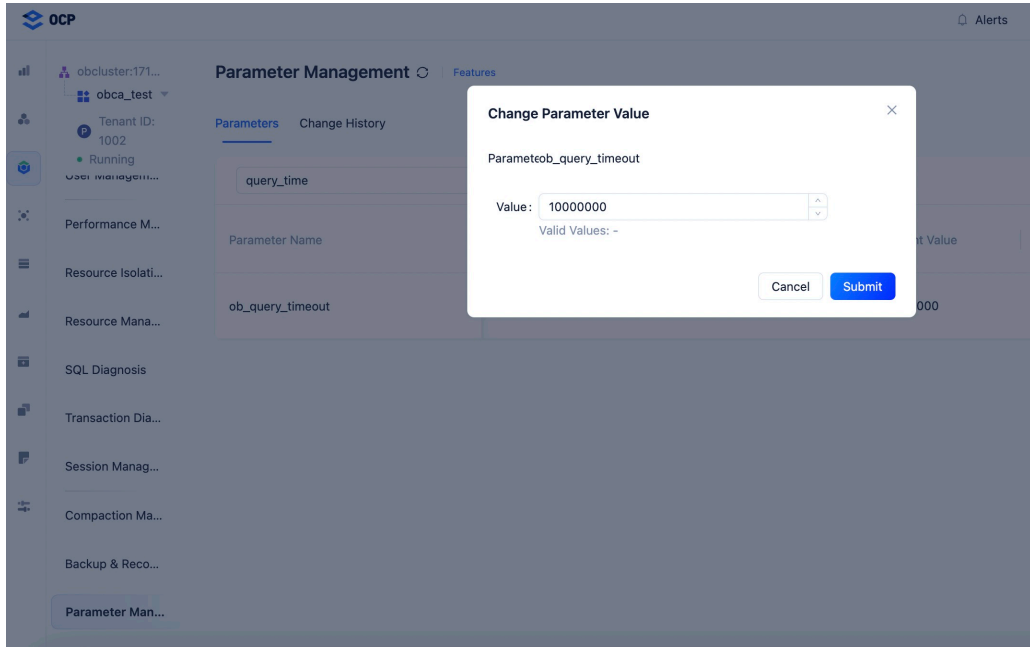
OCP can view and modify configuration items or system variables through the cluster or tenant's parameter management page.

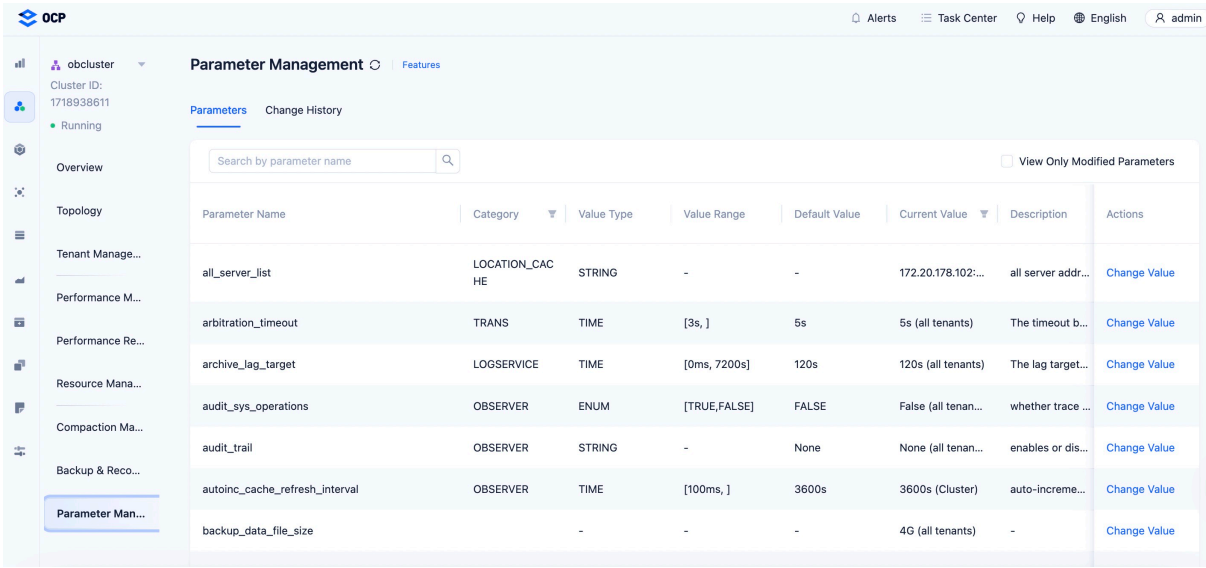
■ Cluster level

Enter the specified cluster page and click **Parameter Management** on the left navigation bar to view, search and modify the cluster-level configuration items and view the modification history.

■ Tenant level

Enter the specified tenant page and click **Parameter Management** on the left navigation bar to view, search and modify tenant-level configuration items and system variables and view the modification history.





Operations and Maintenance Management

06

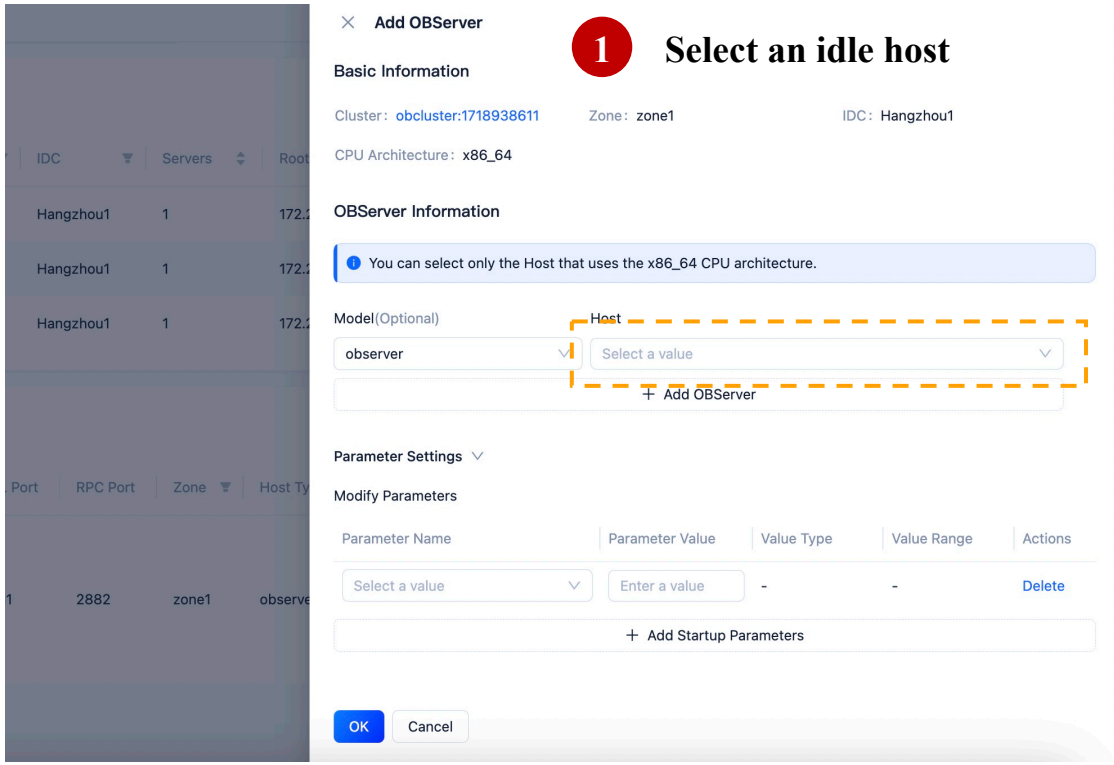
- Introduction to O&M System and Tools
- Cluster Monitoring and Management
- **Cluster and Tenant Scaling**
- Disaster Recovery and High Availability Management
- Real-time Diagnosis and SQL Monitoring
- Alerting and Troubleshooting

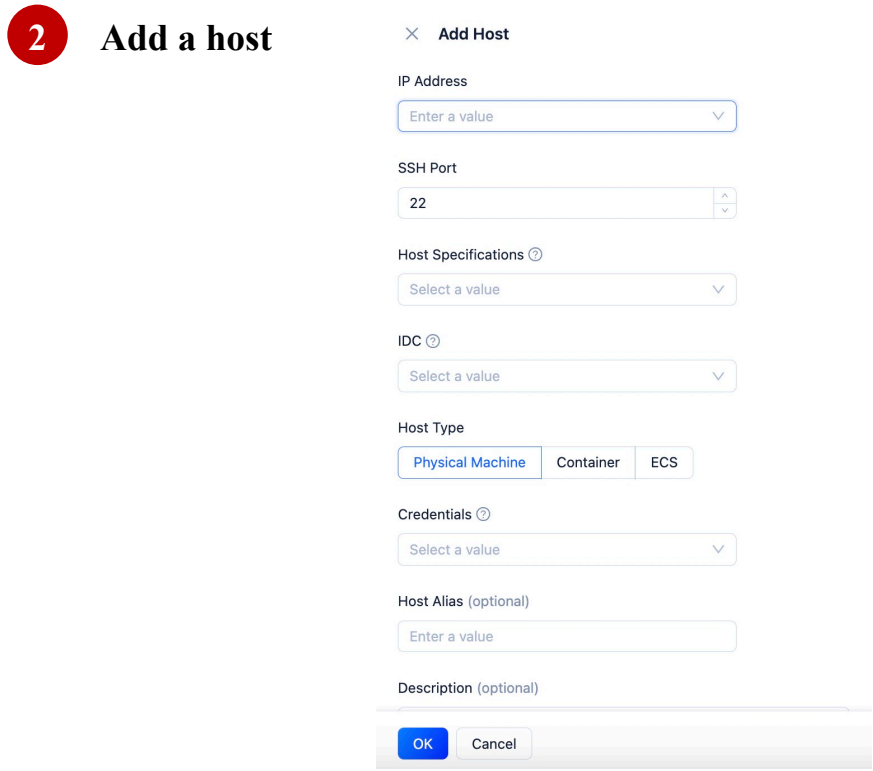
Cluster Scaling (I)

Increase the number of OB nodes in a database cluster by adding OBServer to the Zone of the cluster.

Add OBServer

1. In the upper right corner of the cluster overview page, click Add OBServer, specify the host type and host for the OBServer node, and configure the startup parameters for the OBServer.
2. For the Host option box, you can select an existing host or add a host.



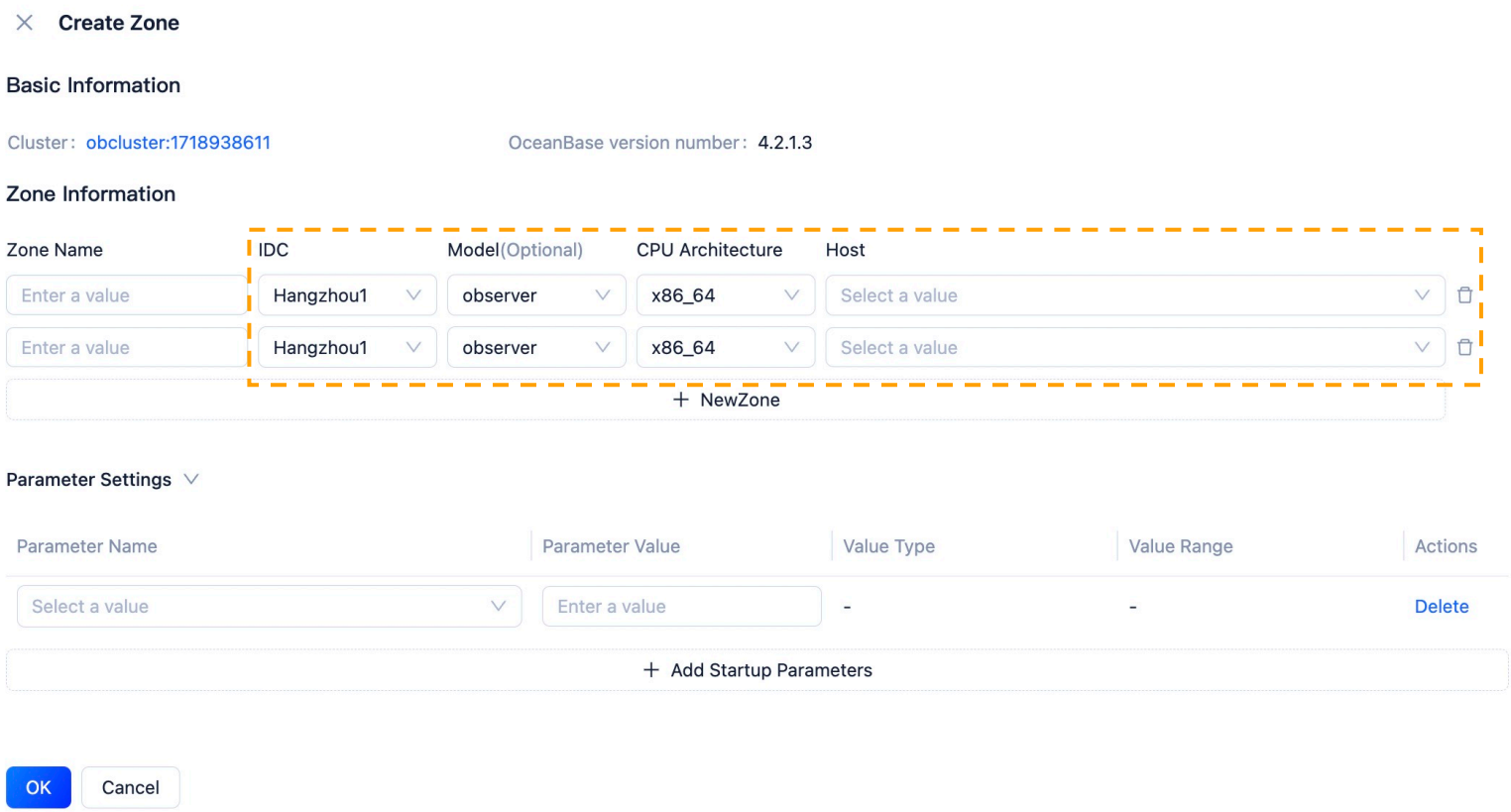


Cluster Scaling (II)

Increase the number of replicas of a database cluster by adding a Zone to the cluster.

Add Zone

1. In the upper right corner of the cluster overview page, click Add New Zone, specify the Zone name, data center, host type, CPU architecture and host, and configure the OBServer startup parameters.
2. For the Host option box, you can select an existing host or add a host.

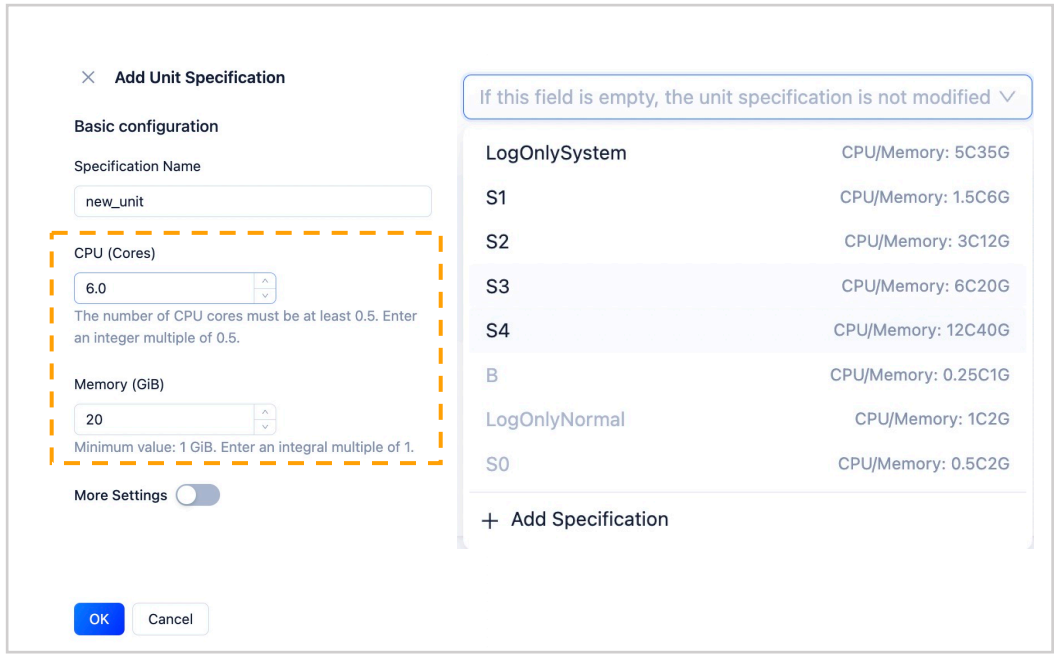


Tenant Resource Adjustment (I)

Adjust tenant resources by modifying the specifications of the Unit.

- In the Tenant Overview page, the replica details area displays the Zone name, replica type, Unit specification and the number of Units corresponding to that tenant.
- Select the **Edit** to select the corresponding Unit specification in the Unit Specification drop-down box, or you can add a new specification for configuration.

1 **Add a new specification**

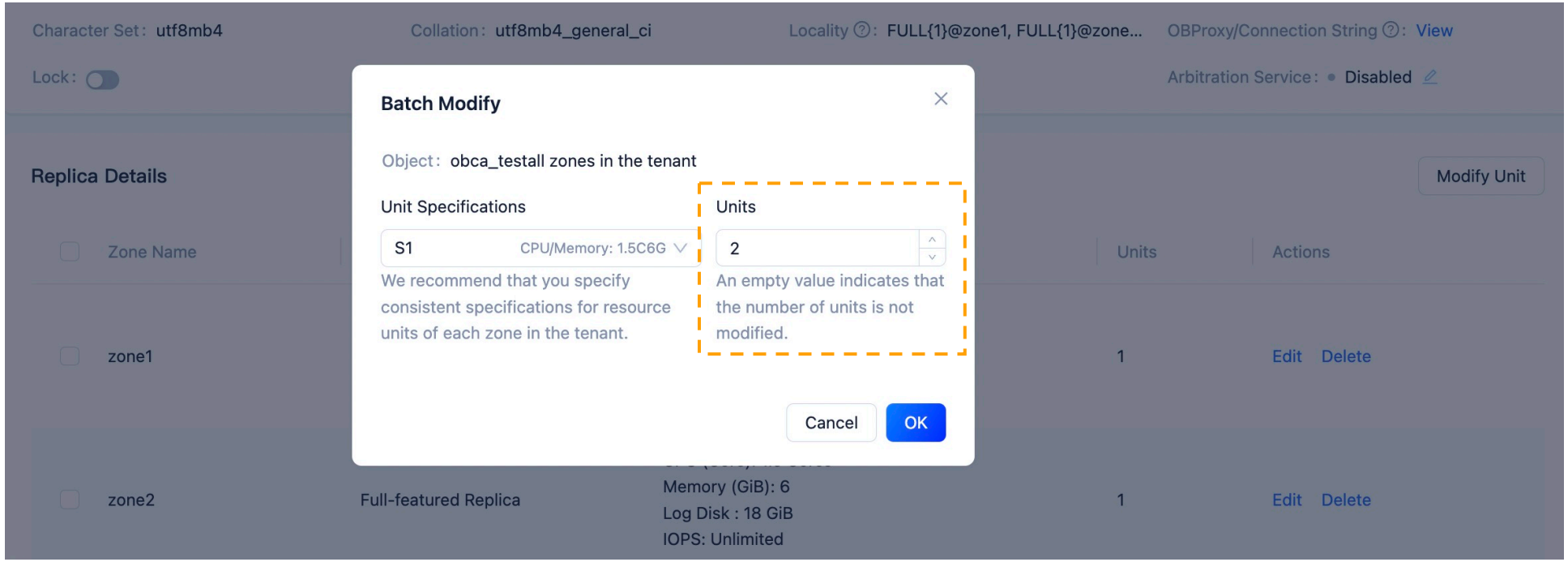


2 **Edit tenant's unit specification**

Tenant Resource Adjustment (II)

Adjust the tenant's resources by modifying the number of Units.

- In the Tenant Overview page, the replica details area displays the Zone name, replica type, Unit specification and the number of Units corresponding to that tenant.
- Select **Modify Unit** in the upper-right corner to adjust the Unit specifications and Unit quantities at the same time. If there is no suitable resource specification, you can add a new specification first and then configure it.



Operations and Maintenance Management

06

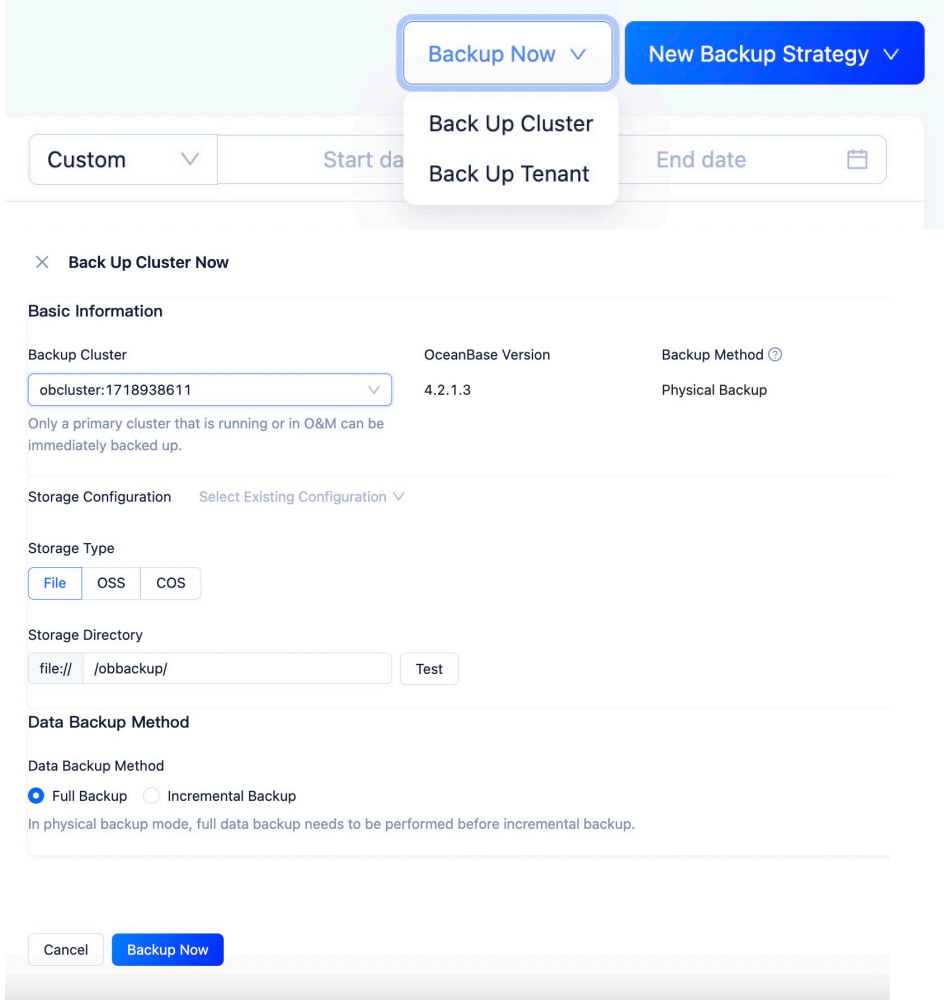
- Introduction to O&M System and Tools
- Cluster Monitoring and Management
- Cluster and Tenant Scaling
- Disaster Recovery and High Availability Management
- Real-time Diagnosis and SQL Monitoring
- Alerting and Troubleshooting

Data Backup and Recovery Management by OCP

OCP can perform backup and recovery of the OceanBase database, as well as manage the primary/standby tenants.

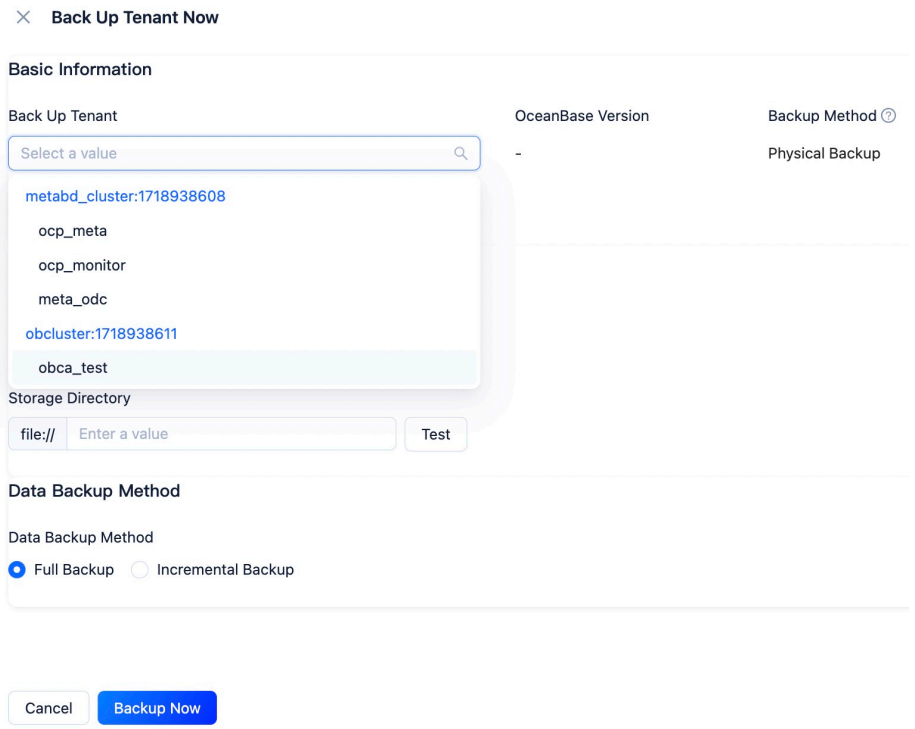
Backup and recovery

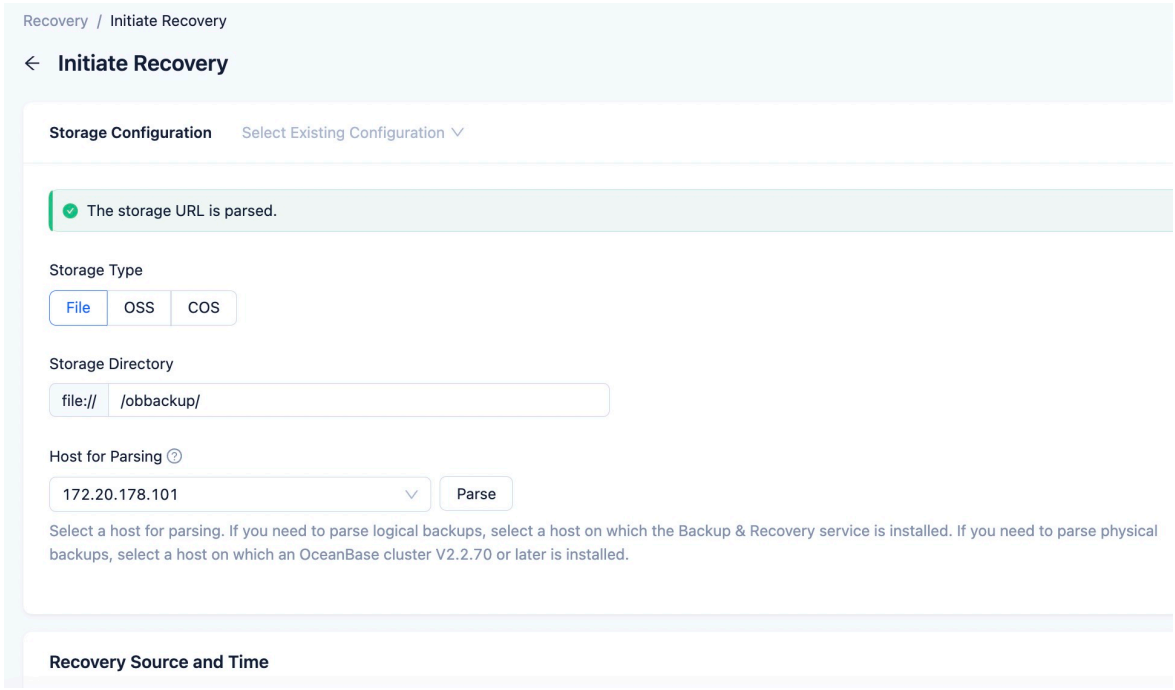
- Backup recovery is the core component of OceanBase database's high availability feature, which is mainly used to ensure data security, including preventing data loss caused by storage media damage and user misuse. In the event of data loss due to storage media damage or user misuse, the recovery feature may be utilized to restore user data.
- **Backup now:** V4.X supports tenant-level backups, allowing for the selection of full backups or incremental backups.
- **Backup strategy:** Supports the features of creating, editing, deleting, viewing backup policies, and suspending backup scheduling. Backup policy involves the configuration of storage type (supporting NFS, OSS and COS), storage directory, scheduling configuration, backup cleanup, alarm setting, secondary backup and other parameters.
- **Initiate recovery:** Supports full tenant recovery or table-level recovery.
- **Regular recovery:** OCP supports random recovery of tenants based on recovery sampling strategies, and automatically verifies the recoverability of backup data on a regular basis.



Immediate Backup or Recovery

- Log in to OCP, enter the Tenant Overview page, select **Backup & Recovery** in the left navigation bar, and click Backup Now. Fill in the storage parameters to perform Backup Now.
- On the Backup & Recovery page of the corresponding tenant, you need to have a backup policy to initiate a recovery, and you need to specify the storage configuration and backup scheduling configuration to create a tenant-level backup policy.
- Click Initiate Recovery on the Backup and Recovery page of the corresponding tenant, fill in the time to be recovered and the recovery information of the newly-created tenant, and then you can initiate the recovery operation.



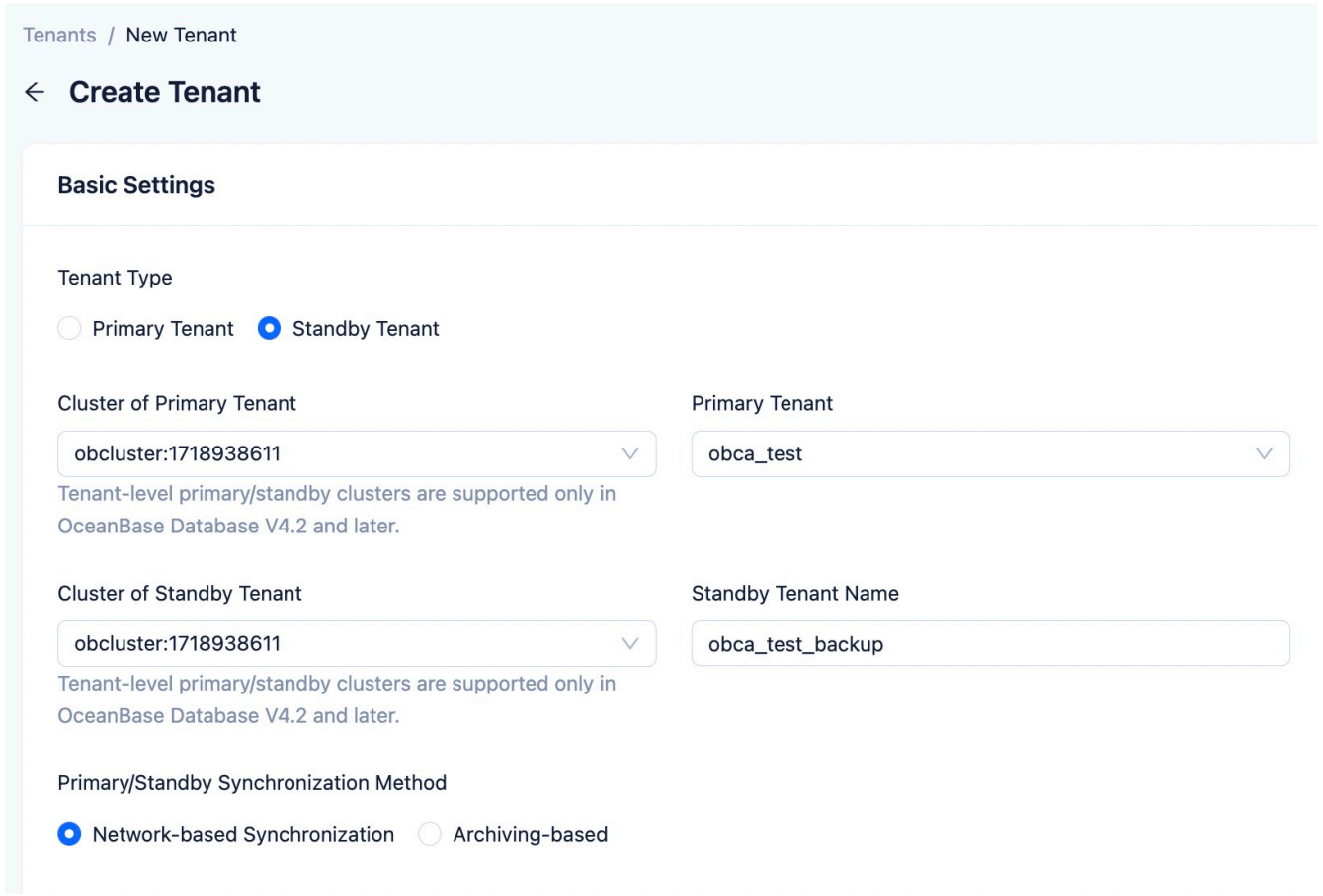


OCP Management over Primary/standby Tenants

OCP can perform backup and recovery of the OceanBase database, as well as manage the primary and standby tenants.

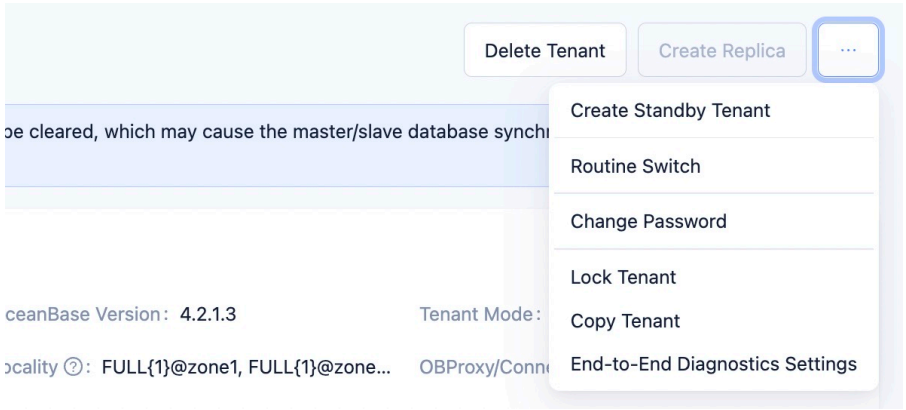
Primary/standby tenants

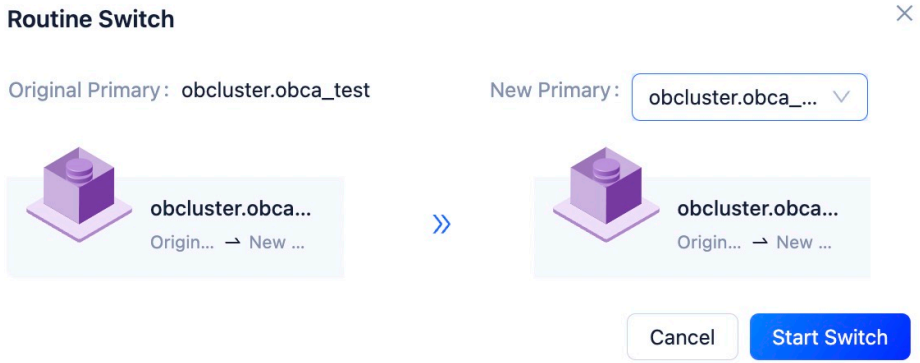
- **Concept:** The primary tenant is a user-created business tenant that supports full database service capabilities, including: Query, DML, DDL, etc. The standby tenant provides only disaster recovery and read-only service capabilities. A primary tenant and several standby tenants together form a tenant-level physical standby high availability solution. OCP supports such two scenarios as the creation of multiple standby tenants for a primary tenant (one-to-many), and the creation of standby tenants for standby tenants (cascade).
- **Routine switchover of primary and standby tenants:** After the switchover is completed, the primary and standby tenants exchange roles, and you can switch only specified tenants or all primary/standby tenants within a cluster.
- **Disaster recovery switchover of primary/standby tenants:** After the disaster recovery switchover is completed, the original standby tenant becomes a read-write independent primary tenant, and you can switch only specified tenants or all primary/standby tenants within a cluster.
- **Decoupling of primary/standby tenants:** The standby tenant is decoupled into an independent, read-write primary tenant.

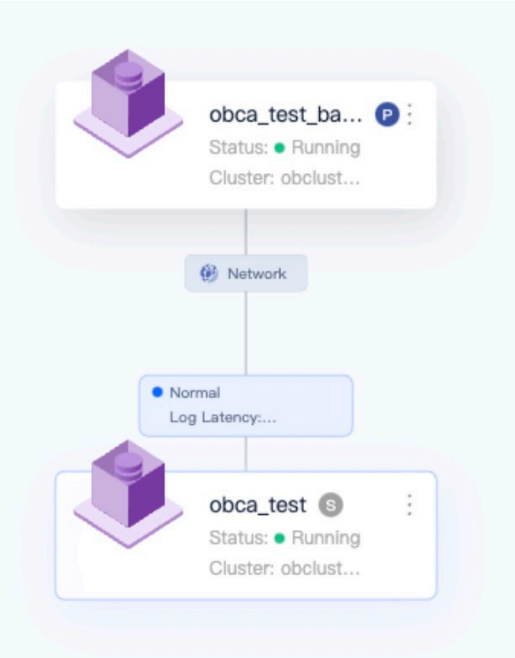


Routine Switchover of Primary/Standby Tenants

- Log in to OceanBase cloud platform, click Tenant in the left navigation bar, and enter the primary tenant in the Tenant List.
- Click the More icon in the upper right corner and select Routine Switchover.







Operations and Maintenance Management

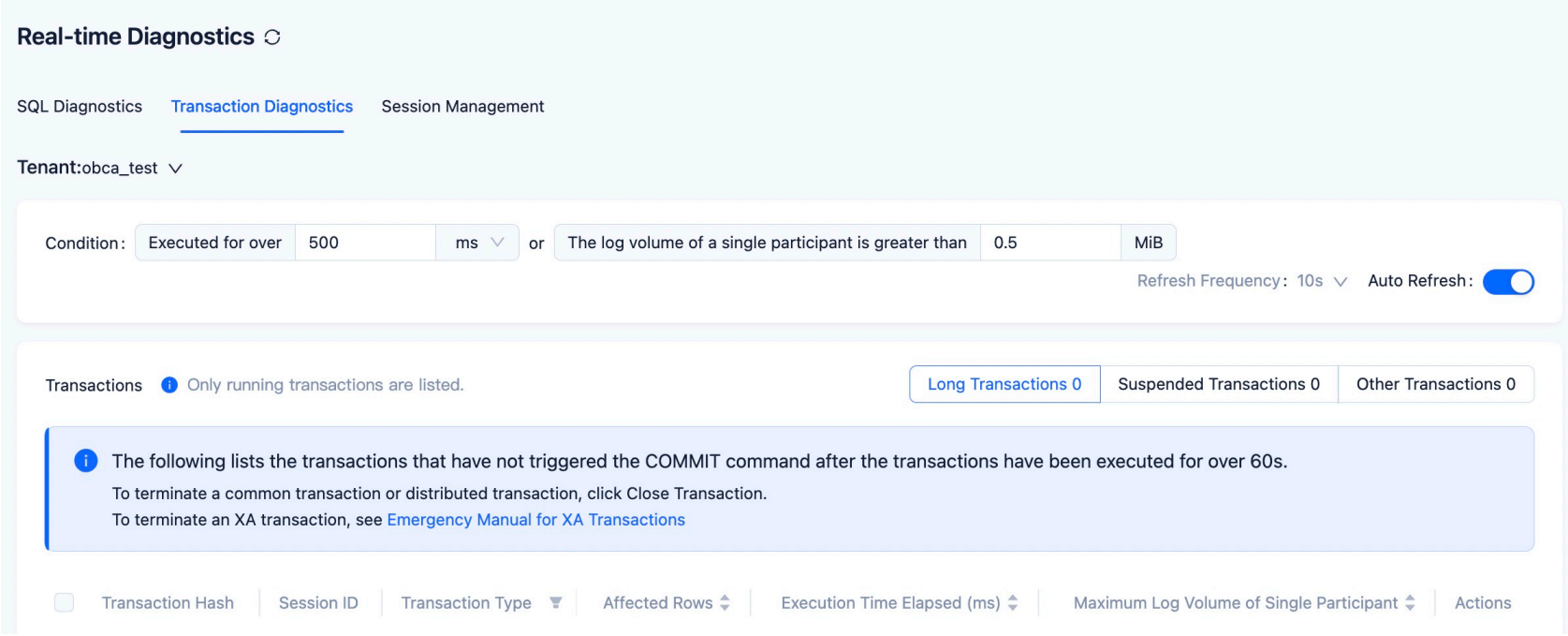
06

- Introduction to O&M System and Tools
- Cluster Monitoring and Management
- Cluster and Tenant Scaling
- Disaster Recovery and High Availability Management
- **Real-time Diagnosis and SQL Monitoring**
- Alerting and Troubleshooting

OAS Real-time Diagnosis - Transaction Diagnosis

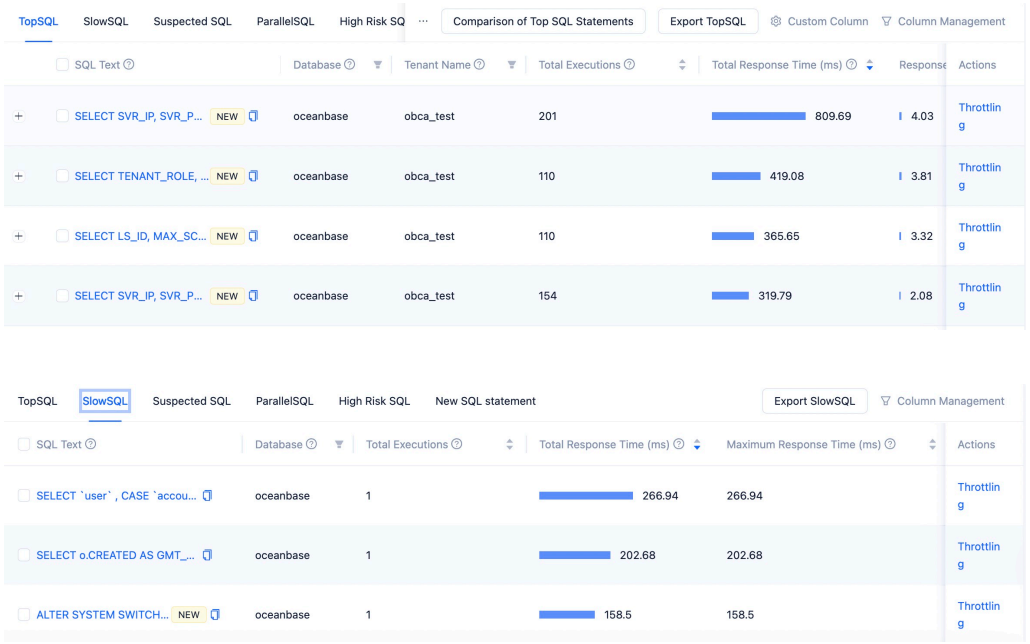
OceanBase Autonomous Service (OAS) provides transaction diagnosis functionality to help users quickly view transaction status and promptly handle abnormal situations such as long-term and pending transactions.

- **Transaction types: Local transactions** refer to single-node transactions, the **distributed transactions** refer to multi-node transactions, and **XA transactions** specifically refer to the distributed transactions that span across database clusters or tenants.
- **Transaction status: Long-term transactions** (transactions that start and execute for more than 60 seconds without committing are referred to as long-term transactions); **Pending transactions** (transactions that have entered the commit phase but, due to some reason, fail to commit successfully, are called pending transactions); **Other transactions** (any transactions that are not classified as long or pending transactions are categorized as other transactions).



OAS Real-time Diagnosis - SQL Diagnosis

OAS provides multi-dimensional features for SQL diagnosis, categorizing SQL statements based on their execution characteristics into Suspicious SQL, TopSQL, SlowSQL, ParallelSQL, High-risk SQL, and New SQL. When abnormalities occur during SQL execution, you can select different diagnostic types according to various application scenarios to pinpoint issues with the SQL statements.

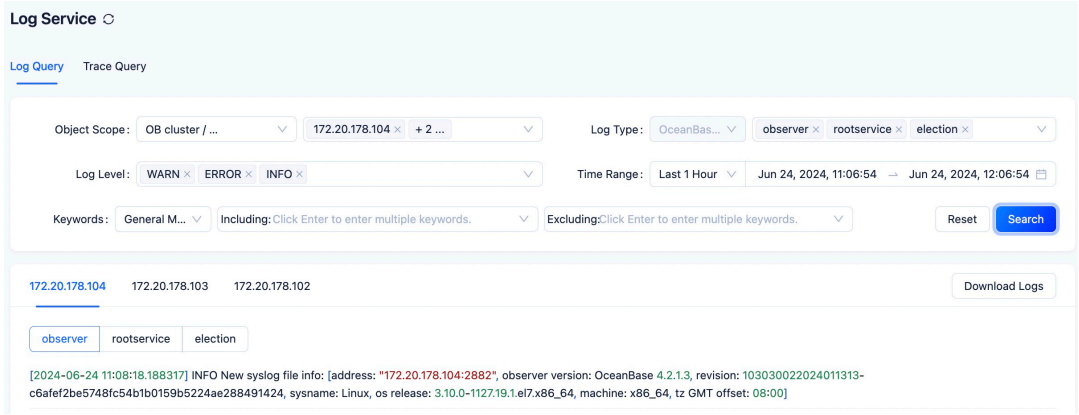


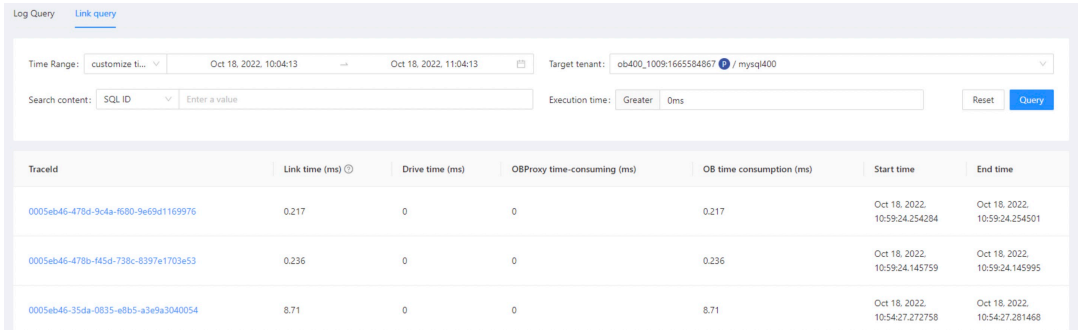
- **TopSQL:** TopSQL uses SQL IDs of SQL requests as the aggregation dimension to count the execution statistics of SQL of various shapes, sort SQL by different resource consumption, and show the historical trend of SQL execution.
- **SlowSQL**: Within OCP, the SQL that exceeds a certain execution time (configurable, with a default of 100ms) is referred to as SlowSQL.
- **Suspicious SQL:** Suspicious SQL diagnosis is derived from the existing TopSQL data collection within OCP, and leverage built-in expert experience to diagnose SQL statements. This process identifies SQL that may have potential risks, which are then termed as suspicious SQL.
- **ParallelSQL**: ParallelSQL refers to SQL that are executed using a parallel scheduling mechanism. By viewing the SQL execution profile, you can examine the operator-level details of SQL execution within the collection time frame.
- **High-risk SQL:** High-risk SQL refers to SQL that may pose a threat to system performance, security, and data integrity.
- **New SQL:** New SQL refers to SQL that were not executed during a period prior to the query interval (by default, 7 days), but have execution records within the query interval itself.

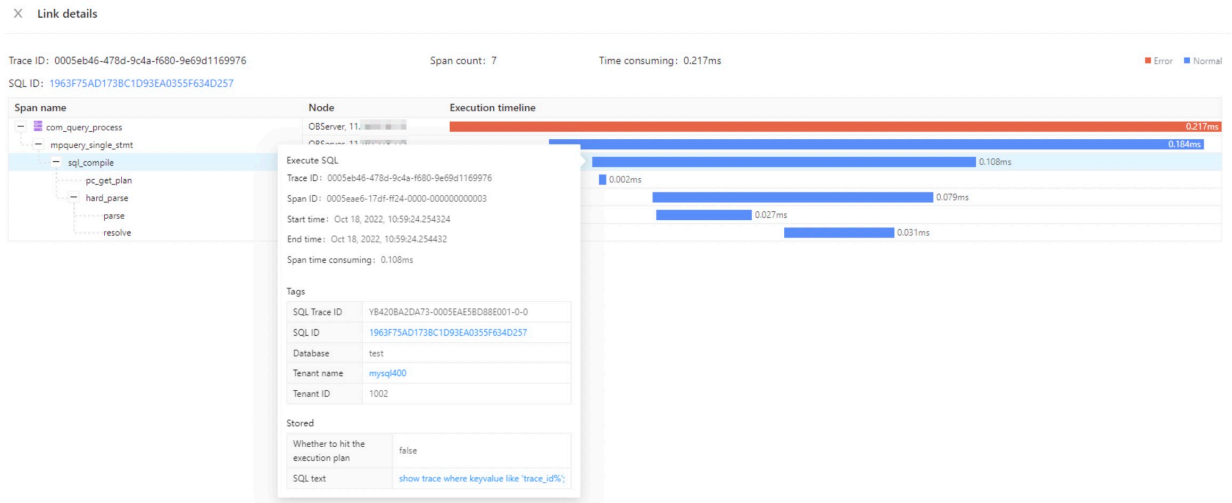
SQL Execution Link Trace

The OCP Link Query function can graphically display the full-link trace of SQL executions.

- Log in to OCP, select **Log Service** in the left navigation bar, and in the **Link Query** tab, set the parameters such as time range, tenant, SQL, etc. Click Query, and the Trace records that meet the query conditions will be displayed at the bottom of the interface.
- Click Link TraceId to enter the Link Details page to view the link data and span details.
- Pre-conditions: OpenSearch and its related parameters need to be configured.



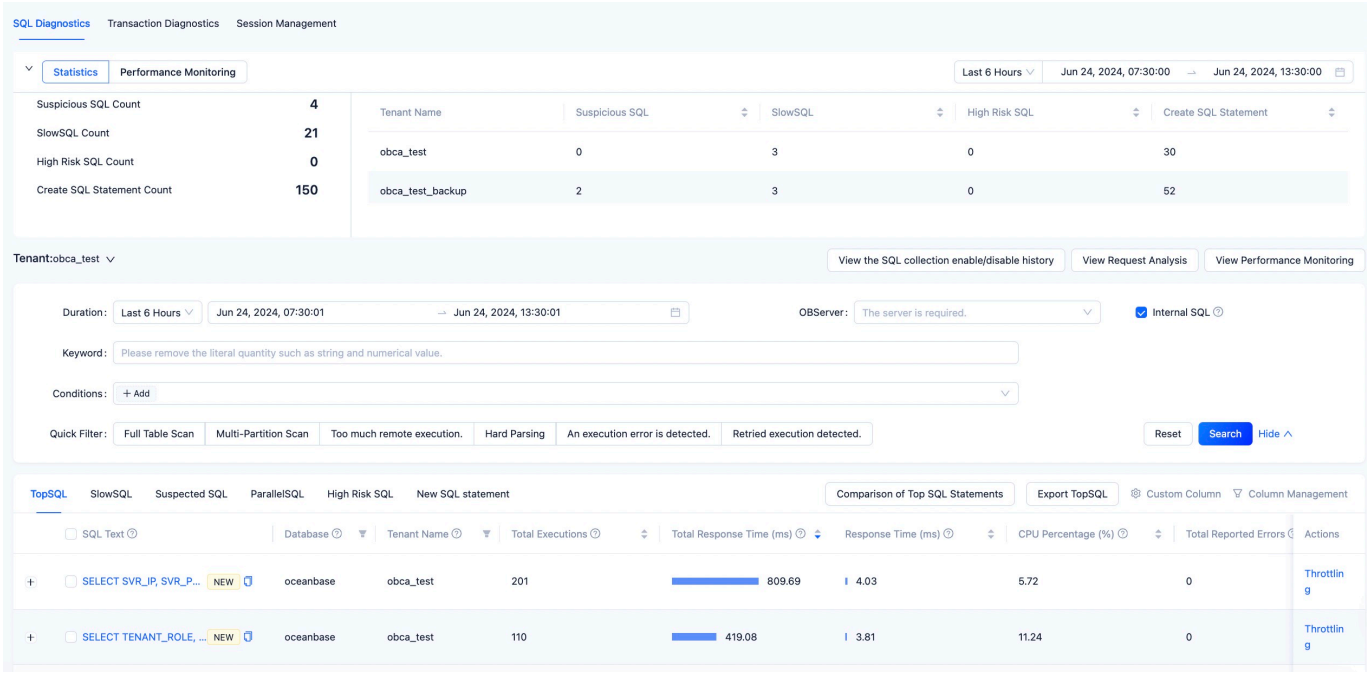


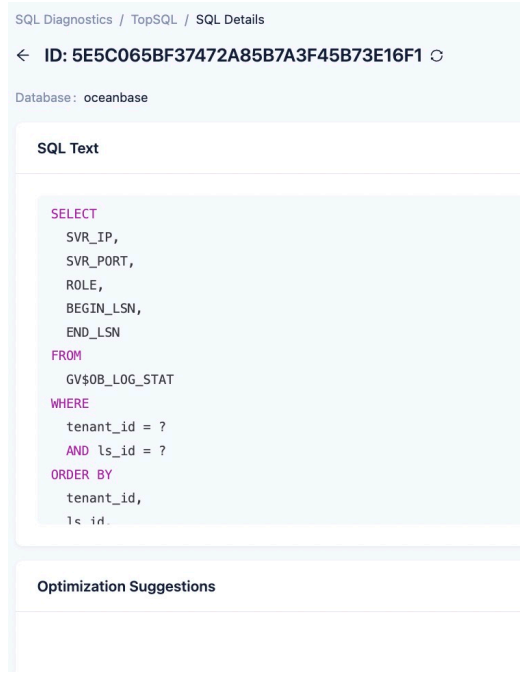


SQL Tuning

TOPSQL tuning example with OAS

- Log in to OCP, click **Autonomy Service** in the left navigation bar, select the cluster you want to view, and then go to the cluster's **Real-time Diagnosis** page. Alternatively, on the Tenant Overview page, click **SQL Diagnosis** in the left navigation bar, and then select the TopSQL tab.
- The TopSQL tab is sorted in reverse order by total response time by default, and the display supports filtering by time, OBServer, keywords etc., and the columns to be displayed can be also customized.
- Click on the SQL text to enter the SQL details page, where you can see optimization suggestions, historical runtime trends, execution plans, indexes, etc. for the SQL, and you can perform operations such as setting rate limiting, execution plan binding, index binding, etc. for the SQL. For ParallelSQL, you can view the profile information.





Operations and Maintenance Management

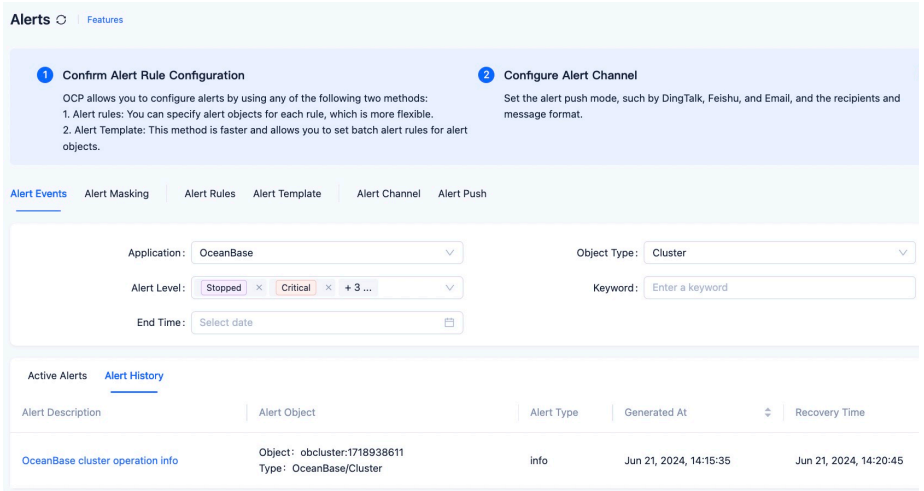
06

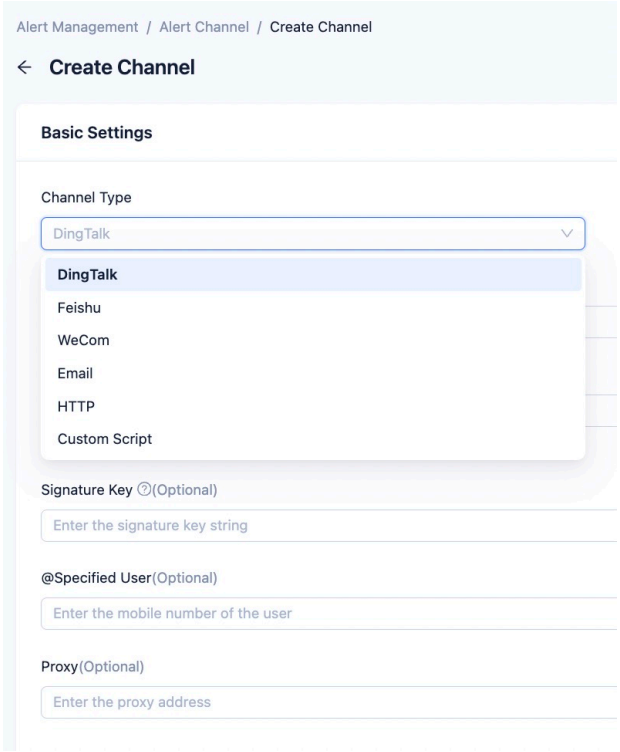
- Introduction to O&M System and Tools
- Cluster Monitoring and Management
- Cluster and Tenant Scaling
- Disaster Recovery and High Availability Management
- Real-time Diagnosis and SQL Monitoring
- **Alerting and Troubleshooting**

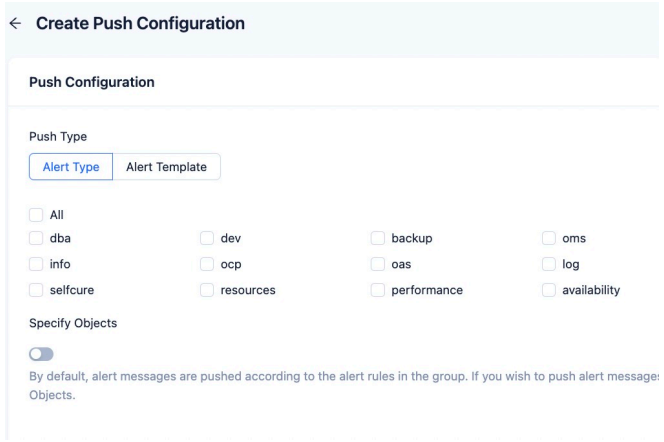
Alerting Configuration

OCP provides an alert monitoring service that keeps track of OceanBase clusters, tenants, and their dependent resources to ensure that users are promptly alerted in the event of business operation failures or resource shortages.

- **Alert rules:** OCP comes with a variety of built-in alert rules and templates, and it is also possible to customize alert rules and templates.
- **Alert channels:** Supported channels include DingTalk, Lark, WeCom, email, HTTP, and custom scripts (Python and Bash).
- **Alert notification:** Associate alert rules or alert templates with alert channels, configure the corresponding notification rules, and enable the push notifications for both alert and alert recovery.





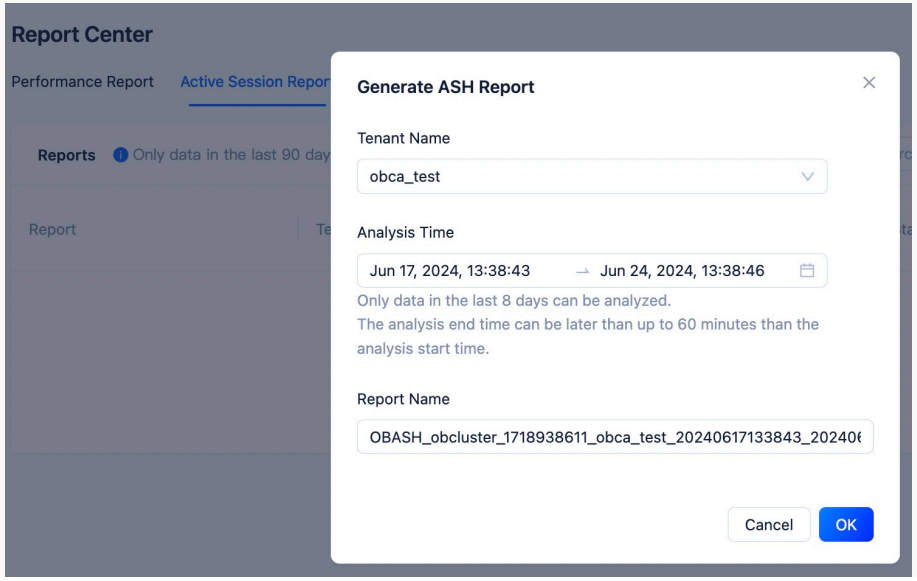


System Performance Diagnosis

Generate and view active session reports and management performance reports through OCP.

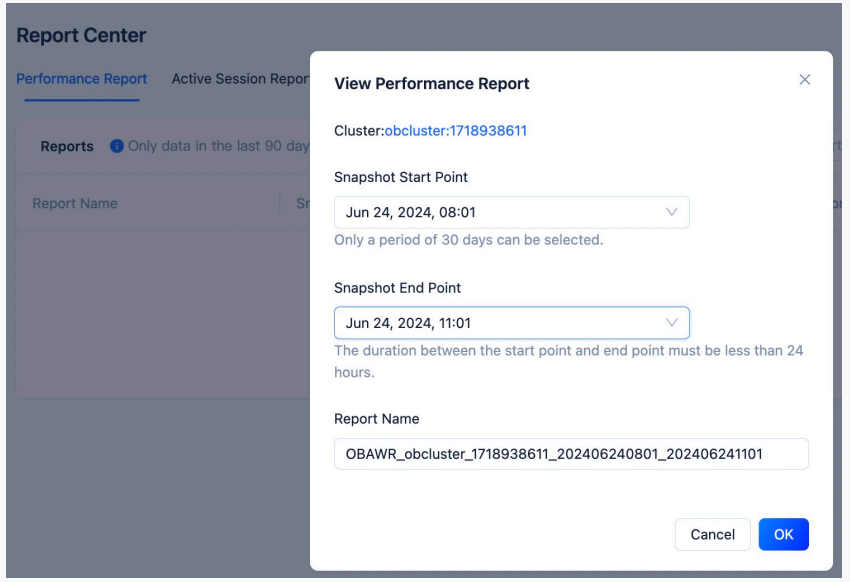
Active session report

- Log in to OCP, click **Autonomy Service** in the left navigation bar, and in the **Cluster Details** area, click the name of the cluster that you want to view, and the system will enter the **Real-time Diagnosis** page by default.
- Click **Report Center** in the left navigation bar, go to the **Active Session Report** tab, click **Generate Active Session Report**, and in the pop-up dialog box of **Generate Active Session History Report**, select the tenant and analysis time, enter a report name, and click OK to generate the report.
- After the report is generated, you can view the report online by clicking **View** in the **Report List**.



Management performance report

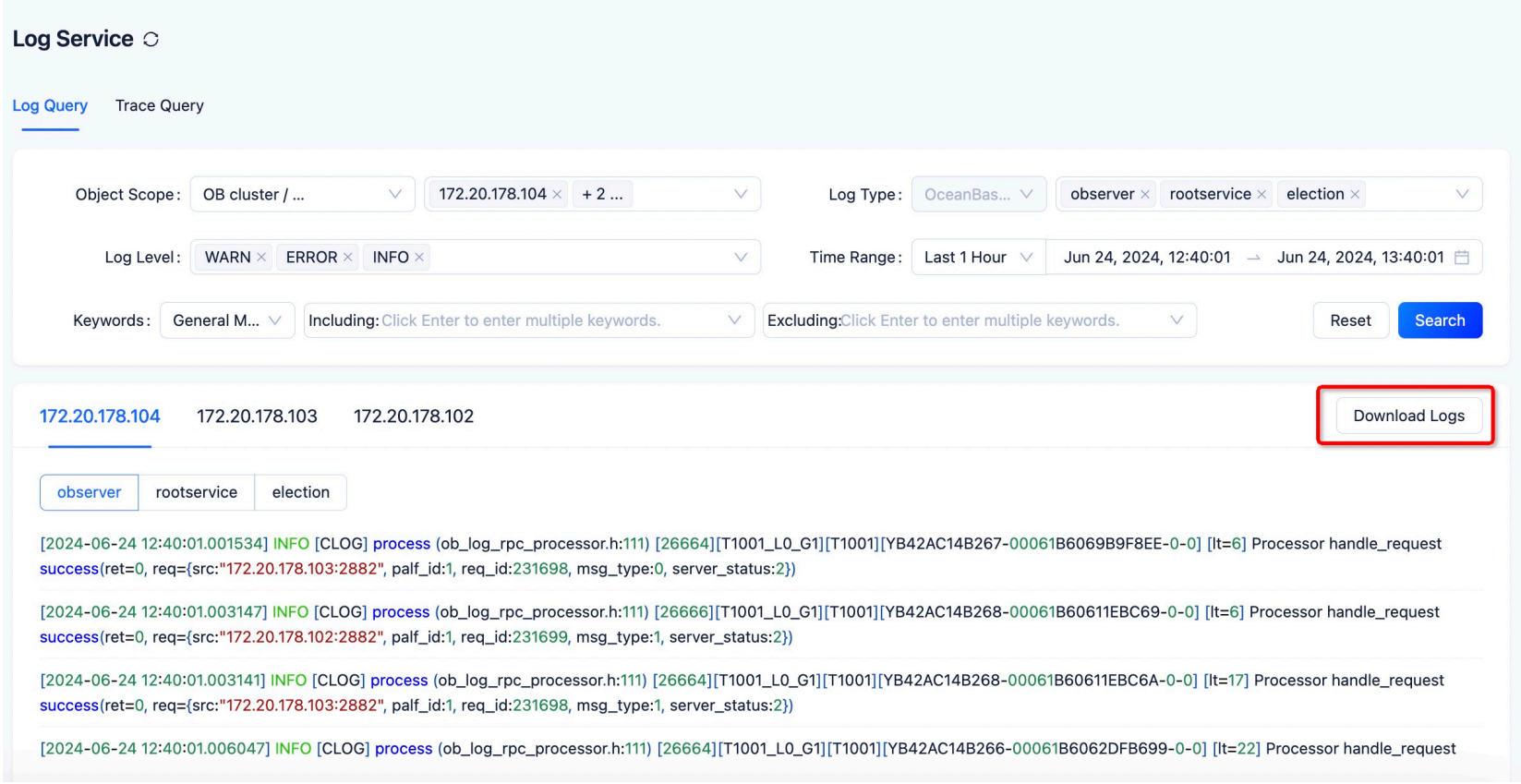
- Log in to OCP, click **Autonomy Service** in the left navigation bar, and in the **Cluster Details** area, click the name of the cluster that you want to view, and the system will enter the **Real-time Diagnosis** page by default.
- Click **Report Center** in the left navigation bar, go to the **Performance Report** tab, click **Generate Performance Report**, and in the pop-up dialog box of **Generate Performance Report**, select snapshot point, enter a report name, and click OK to generate the report.
- After the report is generated, you can view the report online by clicking **View** in the **Report List**.



Log Query

With the OCP log query function, you can view the logs for OBServer nodes, OBProxy, hosts and OCP Agents based on object range, log type, log level and time range.

- Log in to OCP, select **Log Service** in the left navigation bar, and in the **Log Query** tab, set the parameters such as query object, log type, log level, and time range, and then click Query to display the log records that meet the query conditions at the bottom of the interface.
- Click **Download Logs** to download all the logs that meet the conditions.



Summary

O&M system and tools

- DBA daily management tasks include: Database installation and configuration, database architecture design, capacity planning and management, performance monitoring and tuning, backup and recovery management, troubleshooting and emergency response.
- OceanBase kernel provides views, logs, PL/SQL, parameters and other functions as the database management interface. With the above functions along with operating system tools, a rich suite of tools, such as OCP, OAS, WR, ASH, ob_admin, etc., has been formed to assist in daily O&M management fault location.
- WR can periodically collect data from performance-related views of the OceanBase database to facilitate the comprehensive analysis of performance-related issues. ASH analyzes the information of active sessions to generate more detailed reports on SQL and sessions, which is suitable for diagnosing short-term performance fluctuations.
- OCP and OAS collect data related to OBServer, OBProxy, and hosts by installing Agents on the hosts and manage these data.
- FLT can trace the related information about the execution of user's SQL requests on the client, OBProxy and OBServer components, and display such information to the user in a visual manner so as to help the user rapidly locate the problems.

Summary

OCP O&M management

- OCP can take over the host via SSH and initialize the host, deploy OCP Agent, OBProxy, and OBServer on the host, and provides full lifecycle management for these components.
- OCP provides tenant management functions such as tenant creation, tenant resource adjustment, tenant resource isolation, tenant database management, tenant user permission settings, and monitoring of tenant operational metrics.
- Through OCP, you can also configure OceanBase database backup strategies, complete database backup, recovery, and verification.
- OceanBase 4.X version provides the function of primary and standby tenants. Through OCP, you can manage standby tenants, including creation, decoupling, and primary-standby switching.
- OCP can monitor the key indicators and performance indicators of database clusters, tenants, OBProxy clusters, hosts, SQL, resource allocation, and backup merging tasks.
- OAS provides SQL analysis and diagnosis functions for TopSQL, SlowSQL, ParallelSQL, etc., diagnosis functions for long-running and hanging transactions, as well as other functions such as session management, capacity management, and report management.

Self-test

1. [Single answer] Which of the following is not part of the DBA daily management tasks of the OceanBase database? ( )

A. Database installation B. SQL coding for applications C. Database performance optimization D. Data backup

2. [Single answer] Which of the following is not part of the functions or services provided by the OceanBase database kernel? ( )

A. Built-in views B. Stored procedure C. SQL D. OCP

3. [Single answer] Which of the following is a source of data for the ASH report? ( )

A. Logs B. OCP Agent C. Stored procedures D. ASH view

4. [Single answer] In which of the following components is the OBServer full-link trace data stored? ( )

A.observer.log B. election.log C. trace.log D.rootservice.log

5. [Single answer] Which of the following operations can be performed for OceanBase database SQL management via OCP? ( )

A. SQL coding B. View SQL execution plan C. SQL running D. View SQL execution results

6. [Multiple answers] Which of the following are included in the process of scaling out an OceanBase database cluster via OCP? ( )

A. Add Zone B. Add host CPU C. Add OBServer D. Modify primary Zone

Self-test

7. [Multiple answers] Which of the following are involved in tenant resource scaling via OCP? ( )

A. Adjust Unit specifications B. Add primary Zone C. Add OBServer D. Adjust the number of Units

8. [Multiple answers] Which of the following processes are included in the OCP Agent? ( )

A.ocp_agentd B.ocp_mgragent C. ocp_proxy D.ocp_monagent

9. [Multiple answers] What operations can be performed on a host via OCP? ( )

A. Install the OCP Agent B. Reboot the host C. Add a host D. Delete a host

10. [Multiple answers] What operations can be performed on tenants via OCP? ( )

A. Create a primary tenant B. Adjust tenant resources C. Lock tenants D. Create standby tenants

11. [True/false] The data source mechanism is the same for both WR and ASH, and no special processing is required as they both read the view data in real time. ( )

12. [True/false] Link Query and Log Query capabilities both retrieve log data from OBServer and OBProxy in real time. ( )

Thank You!